

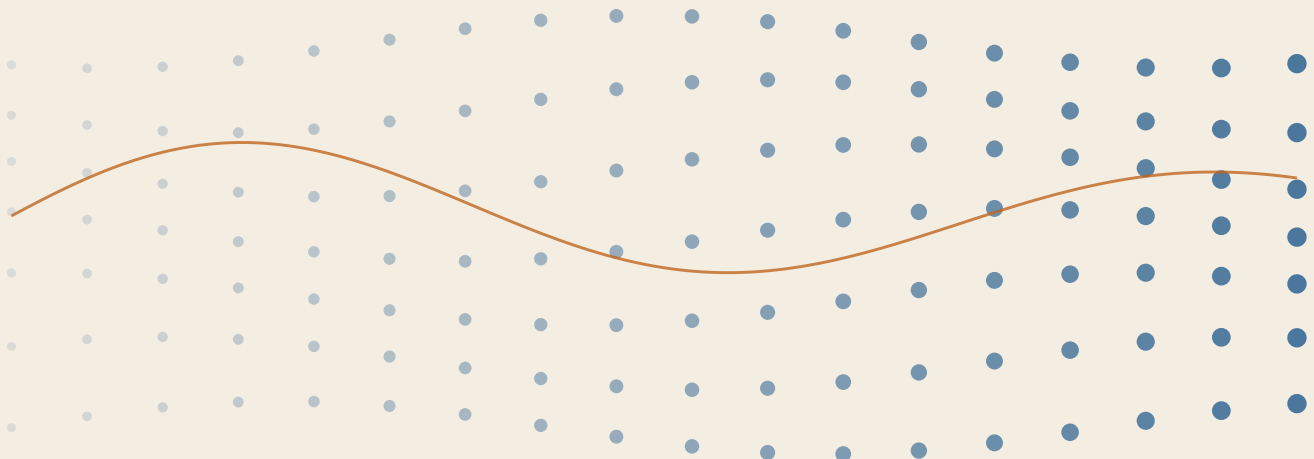
CATATAN KULIAH MASTER ARTIFICIAL INTELLIGENCE DI LINZ

Belajar Mesin Belajar

*dari neuron sampai difusi,
dengan analogi sehari-hari*

Eigenzayn

Edisi pertama, Oktober 2026



Belajar Mesin Belajar

Catatan kuliah Master Artificial Intelligence di Linz

Isi buku ini ditulis ulang dengan kata-kata sendiri dari kuliah yang diikuti, lalu diperkaya dengan rujukan dari literatur. Slide, naskah, dan soal ujian asli tetap milik para pengajarnya dan tidak disertakan. Semua rujukan di daftar pustaka diperiksa terhadap Crossref, arXiv, atau halaman hak cipta bukunya.

Disusun dengan $\text{\LaTeX}$.

Prakata

Pada semester musim dingin 2024/25 saya mulai belajar kecerdasan buatan di Linz. Semester pertama langsung padat: computer vision, jaringan saraf, dan LSTM yang diajarkan oleh penemunya sendiri. Setiap kuliah meninggalkan setumpuk slide. Dua tahun kemudian, map di laptop berisi ratusan PDF.

Slide bagus untuk mengikuti kuliah, tetapi kurang enak dibaca ulang. Kalimatnya terpotong, rumus muncul tanpa cerita, dan hubungan antar mata kuliah jarang terlihat. Padahal hubungan itulah yang paling menarik. Konvolusi di kuliah computer vision ternyata saudara dekat stensil beda hingga di kuliah mekanika fluida. Model difusi di kuliah machine learning lanjut ternyata bersandar pada persamaan Fokker–Planck dari fisika statistik. Kalman filter di kuliah model probabilistik ternyata cara yang sama yang dipakai ahli cuaca untuk menggabungkan simulasi dengan pengukuran.

Buku ini adalah catatan perjalanan itu. Saya menulis ulang isi kuliah dengan bahasa sendiri, menambahkan gambaran yang dulu membantu saya memahaminya, lalu merujuk literatur yang lebih luas dari sekadar slide. Harapannya, siapa pun yang membaca, entah mahasiswa, insinyur, atau orang yang sekadar penasaran, bisa mengikuti ceritanya dari satu neuron sampai model yang meniru aliran fluida.

Di tengah perjalanan, satu pertanyaan semakin sering muncul di kepala saya: bagaimana machine learning bisa membantu simulasi fisika, dan bagaimana fisika bisa membuat machine learning lebih bisa dipercaya? Pertanyaan itu menjadi benang merah buku ini. Hampir setiap bab ditutup dengan melihat materinya dari sudut simulasi, dan Bagian III membahas dunia *physics-informed machine learning* secara utuh: PINN, neural operator, simulator berbasis graf, penemuan persamaan dari data, dan model hibrida.

Gaya penulisannya sengaja dibuat seperti catatan yang dibaca, bukan buku latihan. Setiap gagasan besar diberi satu gambaran dari kehidupan sehari-hari sebelum rumusnya muncul. Analogi tidak pernah sempurna, dan kalau sebuah analogi mulai menyesatkan, teks akan mengatakannya. Contoh hitungan kecil ditulis di dalam paragraf, supaya pembaca bisa mengikutinya dengan pensil kalau mau. Istilah teknis dibiarkan dalam bahasa Inggris dan dicetak miring saat pertama muncul, karena istilah itulah yang dipakai di kuliah dan di makalah.

Setiap bab diakhiri catatan singkat tentang mata kuliah asal dan bacaan utamanya. Slide dan soal asli tetap milik para pengajarnya dan tidak disertakan. Yang ada di sini adalah pemahaman saya atas materi itu, jadi kekeliruan yang tersisa adalah kekeliruan saya.

Linz, Oktober 2026

Daftar Isi

Prakata	iii
0 Peta Perjalanan	1
I Fondasi	5
1 Belajar dari Data	6
2 Optimasi: Turun Gunung dalam Kabut	14
3 AI Klasik: Mencari, Merencanakan, Memutuskan	20
4 Belajar Tanpa Label	24
II Deep Learning dan Persepsi	29
5 Deep Learning: Teknik Dasar	30
6 Mengingat: dari RNN ke LSTM dan Transformer	37
7 Melihat: Computer Vision	43
8 Arsitektur Modern dan Model Generatif	50
9 Geometri dan Simetri	56
10 Bernalar dalam Ketidakpastian	65
11 Dari Estimasi ke Difusi	71
12 Belajar dari Ganjaran: Reinforcement Learning	80
13 Membuka Kotak Hitam: AI dan Visualisasi	86
III Sinyal, Sensor, dan Sistem	90
14 Menangkap Sinyal: Estimasi, Filter, dan Radar	91
15 Mengendalikan Sistem	95

16	Komputasi yang Menghilang	99
17	Robot dan Kendaraan Otonom	102
IV	AI untuk Ilmu Hayati	106
18	Membaca Kode Kehidupan	107
19	Molekul, Obat, dan Data Medis	111
20	Bentuk Protein	116
21	Melihat ke Dalam Tubuh	120
V	AI untuk Simulasi	123
22	Fluida yang Ingin Kita Tiru	124
23	Dari Persamaan ke Solver	127
24	Fisika Komputasi	131
25	Physics-Informed Neural Networks	135
26	Belajar Menyelesaikan: Neural Operator dan Simulator	138
27	Menemukan dan Memampatkan Dinamika	142
28	Model Hibrida: Fisika dan Data Bersama	146
29	Recurrence CFD: Simulasi dari Ingatan	149
VI	AI, Manusia, dan Masa Depan	153
30	Belajar dari Jejak Pengguna	154
31	AI, Hukum, dan Manusia	158
32	Ke Mana Arah AI	162
A	Glosarium	166
	Bibliografi	173

0 Peta Perjalanan

Sebelum masuk ke materi, ada baiknya melihat peta besarnya dulu: mata kuliah apa saja yang dilalui, dalam urutan apa, dan bagaimana semuanya tersusun menjadi buku ini.

0.1 Empat semester

Tabel 1 menyusun mata kuliah menurut semester ketika saya mengikutinya. Urutan bab di buku tidak persis mengikuti urutan waktu. Bab disusun menurut alur gagasan: fondasi lebih dulu, lalu deep learning, lalu simulasi. Tabel ini berguna bagi pembaca yang ingin membaca buku ini sambil mengikuti kuliah serupa.

Bab-bab tentang *scientific machine learning* (Bab 25 to 28) tidak berasal dari satu mata kuliah. Isinya dirangkai dari beberapa kuliah, terutama machine learning lanjut, metode numerik fluida, dan fisika komputasi, ditambah bacaan selama mengerjakan tugas akhir.

Buku ini terbagi menjadi enam bagian. Bagian I berisi fondasi: apa artinya belajar dari data, bagaimana mencari parameter terbaik, bagaimana AI klasik mencari dan merencanakan, dan bagaimana menemukan struktur tanpa label. Bagian II membahas deep learning dan persepsi, dari jaringan konvolusi sampai model generatif, reinforcement learning, dan explainable AI. Bagian III turun ke dunia sinyal dan sistem fisik: estimasi dan filter, radar, kendali, komputasi pervasif, dan robot. Bagian IV masuk ke ilmu hayati: genom, molekul dan obat, protein, dan citra medis. Bagian V membahas AI untuk simulasi fisika, bidang tugas akhir saya. Bagian VI menutup dengan sistem rekomendasi, sisi manusia dan hukum, dan perkembangan AI terbaru.

Beberapa mata kuliah di tabel berada di luar kebutuhan kurikulum. Semuanya tetap masuk ke buku, sebab cerita AI tidak lengkap tanpa robot yang bergerak, rekomendasi yang kita terima setiap hari, dan molekul yang dirancang dengan bantuan mesin.

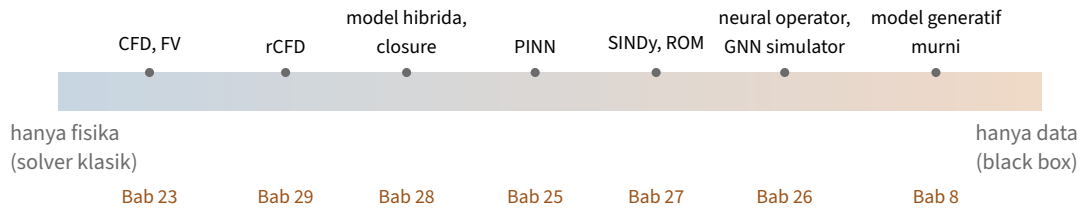
0.2 Mengapa simulasi

Simulasi fisika seperti *computational fluid dynamics* (CFD) menyelesaikan persamaan diferensial parsial di jutaan sel. Hasilnya akurat, tetapi mahal: satu kasus bisa berjalan sehari-hari di kluster komputer. Machine learning menawarkan jalan pintas. Ia belajar dari hasil simulasi sebelumnya, lalu menebak hasil baru dalam hitungan detik. Masalahnya, model yang hanya belajar dari data tidak tahu apa-apa tentang kekekalan massa atau energi. Ia bisa menghasilkan aliran yang terlihat meyakinkan tetapi melanggar fisika.

Saya suka membayangkan perbedaan ini seperti dua cara memasak. Juru masak pertama mengikuti resep secara persis: takaran, suhu, waktu. Hasilnya bisa diulang,

tetapi lambat, karena setiap langkah harus ditimbang. Juru masak kedua memasak dengan lidah. Ia mencicipi, mengingat rasa masakan sebelumnya, lalu menebak. Cepat, tetapi kadang meleset jauh. Juru masak terbaik memakai keduanya: resep menjaga agar masakan tidak gagal total, lidah membuatnya cepat dan luwes. Solver CFD adalah resep, machine learning adalah lidah, dan *physics-informed machine learning* adalah usaha memakai keduanya sekaligus.

Di antara dua ujung itu terbentang spektrum yang lebar. Gambar 1 menempatkan pendekatan-pendekatan yang akan kita temui di sepanjang buku.



GAMBAR 1. Spektrum pendekatan simulasi, dari solver berbasis persamaan sampai model berbasis data. Semakin ke kanan, semakin cepat, tetapi semakin sedikit jaminan fisiknya.

Literatur memakai beberapa nama untuk wilayah ini: *scientific machine learning*, *physics-informed machine learning* (Karniadakis et al., 2021), atau *physics-based deep learning* (Thuerey et al., 2021). Tinjauan yang khusus membahas mekanika fluida ditulis oleh Brunton et al. (2020), Vinuesa and Brunton (2022), dan Lino et al. (2023). Di buku ini saya memakai istilah yang lebih sederhana: AI untuk simulasi.

0.3 Tiga pintu masuk fisika

Karniadakis et al. (2021) membedakan tiga cara memasukkan pengetahuan fisika ke dalam model pembelajaran. Pembagian ini akan muncul berulang kali, jadi ada gunanya dikenal sejak awal.

Pintu pertama adalah data. Kalau sebuah model dilatih dengan cukup banyak hasil simulasi yang benar, ia ikut meniru pola fisika yang terkandung di dalamnya. Inilah yang disebut *observational bias*, dan contohnya adalah neural operator yang dilatih dari ribuan solusi persamaan (Bab 26). Pintu kedua adalah arsitektur. Model bisa dirancang agar otomatis mematuhi sifat tertentu, misalnya tetap berlaku ketika sistemnya diputar atau digeser. Konvolusi, yang dengan sendirinya menghormati pergeseran (Bab 5), dan jaringan yang menghormati rotasi (Bab 9) termasuk di sini. Ini *inductive bias*. Pintu ketiga adalah fungsi loss. Kita menambahkan suku yang menghukum setiap pelanggaran persamaan fisika, sehingga model belajar mematuhihinya selama pelatihan. Ini *learning bias*, dan contoh paling terkenalnya adalah PINN (Bab 25).

Setiap kali bertemu model baru di bab-bab berikutnya, ada tiga pertanyaan yang layak diajukan. Lewat pintu mana fisika masuk: data, arsitektur, atau loss? Apa yang terjadi kalau model dipakai di luar jangkauan data latihnya, misalnya untuk waktu yang lebih panjang atau geometri yang berbeda? Dan berapa biaya melatihnya dibanding biaya menjalankan solver klasik? Tiga pertanyaan ini akan menemani kita sampai akhir.

0.4 Notasi

Buku ini memakai notasi yang umum di kuliah. Vektor dicetak tebal ($\boldsymbol{x}$), matriks huruf besar tebal ($\boldsymbol{W}$), skalar huruf miring (x). Data latih ditulis $\{(\boldsymbol{x}_n, y_n)\}_{n=1}^N$, dan parameter model ditulis $\boldsymbol{w}$ atau $\boldsymbol{\theta}$. Ekspektasi ditulis $\mathbb{E}[\cdot]$, distribusi normal $\mathcal{N}(\mu, \sigma^2)$, dan divergensi Kullback–Leibler $\text{KL}(q \parallel p)$. Pada bab-bab fluida, $\boldsymbol{u}$ berarti kecepatan, p tekanan, ρ massa jenis, ν viskositas kinematik, dan T temperatur.

TABEL 1. Mata kuliah menurut semester dan bab yang membahasnya.

Semester	Mata kuliah	Bab
Musim dingin 2024/25	Deep Learning and Neural Nets I	5
	LSTM and Recurrent Neural Nets	6
	Computer Vision	7
	Machine Learning: Supervised Techniques	1
	Control Systems	15
	Production Automation Systems	15
	Optimum and Adaptive Signal Processing Systems	14
	AI in Society, AI and Law I	31
Musim panas 2025	Deep Learning and Neural Nets II	8
	Deep Reinforcement Learning	12
	Machine Learning: Unsupervised Techniques	4
	Machine Learning and Pattern Classification	4
	Statistical Learning Theory	1
	Introduction to Autonomous Vehicles	17
	Introduction to Robotic Systems	17
	Learning from User-generated Data	30
	Introduction to Genetics; Genome Analysis and Transcriptionomics	18
	Structural Bioinformatics	20
	AI in Life Sciences	19
	Deep Learning for Medical Imaging	21
	Robopsychology; Technology and Society; AI and Law II	31
Musim dingin 2025/26	Machine Learning: Advanced Techniques	11
	Probabilistic Models	10
	AI and Visualization	13
	Machine Learning: Basic Techniques	1
	Reinforcement Learning	12
	Numerical Methods in Fluid Mechanics	22, 23
	Computational Physics I	24
	Optimum and Adaptive Signal Processing Systems (latihan)	14
	Radar System Engineering (latihan)	14
	Pervasive Computing: Systems and Environments; Design and Development	16
	KS Operations Research	3
	Natural Language Processing	6
	Communicating AI	31
Musim panas 2026	Introduction to AI	3
	Deep Learning: Geometric Techniques	9
	Numerical Optimization for AI	2
	Tugas akhir: recurrence CFD	29

Bagian I

Fondasi

1 Belajar dari Data

Kerangka di bab ini tidak datang dari satu kuliah. Ia muncul pertama kali di kuliah supervised learning pada semester pertama, lalu kembali di kuliah unsupervised, di kuliah dasar-dasar machine learning, dan di bagian pembuka kuliah geometric deep learning. Karena dipakai oleh begitu banyak kuliah, saya menaruhnya di depan.

Semua yang ada di buku ini, dari LSTM sampai PINN, berdiri di atas satu gagasan sederhana: kita punya contoh, dan kita ingin aturan yang juga berlaku untuk contoh yang belum pernah dilihat. Bab ini membahas gagasan itu secara jujur. Apa yang sebenarnya kita minimalkan? Mengapa model yang sempurna di data latih bisa gagal di data baru? Dan mengapa dimensi tinggi membuat semuanya lebih sulit?

1.1 Tiga cara belajar

Bayangkan tiga murid. Murid pertama belajar dengan kunci jawaban: setiap soal latihan punya jawaban benar, dan ia menyesuaikan cara berpikirnya sampai jawabannya cocok. Murid kedua hanya diberi setumpuk soal tanpa jawaban. Ia mengelompokkan soal yang mirip dan menemukan polanya sendiri. Murid ketiga tidak diberi soal sama sekali. Ia mencoba sesuatu, lalu gurunya hanya berkata “bagus” atau “kurang”, kadang baru setelah banyak langkah.

Murid pertama mewakili *supervised learning*, di mana setiap input x datang bersama target y . Murid kedua mewakili *unsupervised learning*, di mana hanya ada x dan kita mencari strukturnya: kelompok, arah utama, atau bentuk distribusinya. Murid ketiga mewakili *reinforcement learning*, tempat sebuah agen bertindak, menerima *reward*, dan harus menebak tindakan mana yang pantas dipuji (Bab 12). Di antara ketiganya ada bentuk campuran. *Self-supervised learning*, misalnya, membuat target dari data itu sendiri, seperti menebak kata yang ditutup dalam sebuah kalimat. Hampir semua model bahasa besar dilatih dengan cara ini. Bab ini berfokus pada murid pertama, karena kerangka berpikirnya dipakai di mana-mana.

1.2 Kerangka supervised learning

Kita menganggap data berasal dari sebuah distribusi gabungan $p(x, y)$ yang tidak kita ketahui. Yang kita miliki hanyalah sampel $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$ yang diambil secara independen dari distribusi itu, asumsi yang biasa disebut *i.i.d.* Kita lalu memilih sebuah model $f(x; w)$ dari suatu *model class* $\mathcal{F}$, dan sebuah *loss function* $L(y, f(x))$ yang mengukur seberapa buruk tebakan kita. Dua besaran kemudian menjadi pusat

perhatian:

$$R(f) = \mathbb{E}_{(x,y) \sim p} [L(y, f(x))] = \int L(y, f(x)) p(x, y) dx dy, \quad (1.1)$$

$$\hat{R}_N(f) = \frac{1}{N} \sum_{n=1}^N L(y_n, f(x_n)). \quad (1.2)$$

Yang pertama disebut *generalization error* atau risiko sejati. Yang kedua disebut *empirical risk* atau *training error*. Yang sebenarnya kita inginkan adalah $R(f)$ kecil, tetapi yang bisa kita hitung hanya $\hat{R}_N(f)$. *Empirical risk minimization* (ERM) memilih $\hat{f} = \arg \min_{f \in \mathcal{F}} \hat{R}_N(f)$ dan berharap $R(\hat{f})$ ikut kecil. Hampir seluruh teori generalisasi pada dasarnya bertanya kapan harapan ini masuk akal.

Seandainya kita mengetahui $p(x, y)$ dengan sempurna, ada prediktor terbaik yang mungkin, yaitu *Bayes-optimal predictor* f^* , dengan risiko $R^* = R(f^*)$ yang disebut *Bayes risk*. Untuk loss kuadrat, prediktor itu adalah rata-rata bersyarat $f^*(x) = \mathbb{E}[y | x]$. Alasannya mudah dilihat. Tetapkan satu x dan cari bilangan c yang meminimalkan $\mathbb{E}[(y - c)^2 | x]$. Turunan terhadap c adalah $-2 \mathbb{E}[y - c | x]$, yang bernilai nol tepat ketika $c = \mathbb{E}[y | x]$. Kalau loss diganti dengan $|y - c|$, jawaban optimalnya berubah menjadi median bersyarat. Jadi pilihan loss menentukan statistik apa yang akan ditiru oleh model. Untuk klasifikasi dengan loss 0–1, prediktor Bayes memilih kelas dengan peluang bersyarat terbesar. Bayes risk umumnya tidak nol, karena data sendiri berisik: dua pasien dengan hasil laboratorium yang sama bisa saja punya diagnosis yang berbeda.

1.3 Dari asumsi noise ke fungsi loss

Kuliah dasar-dasar machine learning menekankan satu hal yang sering terlewat: fungsi loss bukan pilihan sembarang. Ia lahir dari asumsi tentang bagaimana data dibangkitkan. Misalkan $y = f(x; w) + \varepsilon$ dengan noise $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. Likelihood satu titik data adalah

$$p(y_n | x_n, w) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_n - f(x_n; w))^2}{2\sigma^2}\right).$$

Karena data i.i.d., likelihood seluruh data adalah hasil kali, dan negatif log-likelihoodnya menjadi

$$-\log p(\mathcal{D} | w) = \frac{1}{2\sigma^2} \sum_{n=1}^N (y_n - f(x_n; w))^2 + \frac{N}{2} \log(2\pi\sigma^2). \quad (1.3)$$

Suku kedua tidak bergantung pada w . Maka *maximum likelihood* dengan noise Gauss sama persis dengan meminimalkan *mean squared error*. Dengan jalan yang sama, noise Laplace menghasilkan loss mutlak $|y - f|$, keluaran Bernoulli dengan peluang $\sigma(f)$ menghasilkan *binary cross-entropy*, dan keluaran kategorikal dengan softmax menghasilkan *cross-entropy* (Tabel 1.1).

Kasus Bernoulli layak diturunkan juga, karena loss ini dipakai di hampir semua pengklasifikasi. Misalkan label $y \in \{0, 1\}$ dan model mengeluarkan peluang $p = \sigma(f(x; w))$ bahwa $y = 1$. Peluang satu label bisa ditulis dalam satu rumus, $p(y | x) =$

$p^y(1-p)^{1-y}$, karena untuk $y = 1$ rumus itu menjadi p dan untuk $y = 0$ menjadi $1 - p$. Negatif log-likelihood seluruh data adalah

$$-\log p(\mathcal{D} | \mathbf{w}) = -\sum_{n=1}^N \left[y_n \log p_n + (1 - y_n) \log(1 - p_n) \right], \quad (1.4)$$

dan itulah binary cross-entropy. Sebagai contoh, kalau model memberi peluang 0,9 untuk contoh yang labelnya satu, loss-nya $-\ln 0,9 \approx 0,105$. Kalau peluangnya hanya 0,1, loss-nya $-\ln 0,1 \approx 2,30$, lebih dari dua puluh kali lebih besar. Loss ini menghukum keyakinan yang salah dengan keras sekali, dan sifat itulah yang membuat model terdorong memberi peluang yang terkalibrasi.

TABEL 1.1. Asumsi noise dan loss yang dihasilkannya.

Asumsi pada $y \mathbf{x}$	Loss dari $-\log$ likelihood	Sifat
Gauss $\mathcal{N}(f, \sigma^2)$	kuadrat, $(y - f)^2$	peka terhadap pencilan
Laplace	mutlak, $ y - f $	lebih tahan pencilan
Bernoulli, $p = \sigma(f)$	binary cross-entropy	klasifikasi dua kelas
Kategorikal, $p = \text{softmax}(f)$	cross-entropy	klasifikasi banyak kelas

Timbangan dapur memberi gambaran yang pas. Kalau timbangan Anda kadang meleset sedikit ke atas atau ke bawah secara acak, rata-rata dari beberapa kali menimbang adalah tebakan yang baik, dan itulah loss kuadrat. Tetapi kalau sesekali seekor kucing melompat ke atas timbangan, satu hasil bisa meleset jauh sekali. Rata-rata akan ikut tertarik oleh angka itu, sedangkan median tidak. Noise berekor tebal seperti Laplace menghasilkan loss mutlak, yang tidak terlalu peduli pada satu pencilan.

1.4 Prior sebagai regularisasi

Maximum likelihood hanya mendengarkan data. Kalau datanya sedikit, model mudah percaya pada kebetulan. Pendekatan Bayesian menambahkan *prior* $p(\mathbf{w})$, yaitu keyakinan awal tentang parameter sebelum melihat data. Teorema Bayes kemudian memberi *posterior*

$$p(\mathbf{w} | \mathcal{D}) = \frac{p(\mathcal{D} | \mathbf{w}) p(\mathbf{w})}{p(\mathcal{D})},$$

dan estimasi *maximum a posteriori* (MAP) memaksimalkannya. Dalam bentuk logaritma,

$$\hat{\mathbf{w}}_{\text{MAP}} = \arg \min_{\mathbf{w}} \left[-\log p(\mathcal{D} | \mathbf{w}) - \log p(\mathbf{w}) \right]. \quad (1.5)$$

Dengan prior Gauss $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \alpha^{-1} \mathbf{I})$, suku kedua menjadi $\frac{\alpha}{2} \|\mathbf{w}\|_2^2$. Itulah *weight decay*, atau *ridge regression* dalam bahasa statistik. Dengan prior Laplace, suku kedua menjadi $\lambda \|\mathbf{w}\|_1$, yang dikenal sebagai lasso (Tibshirani, 1996). Lasso punya sifat yang menarik: ia cenderung membuat banyak bobot bernilai tepat nol. Sifat ini nanti menjadi inti metode SINDy untuk menemukan persamaan dari data (Bab 27).

Dalam simulasi, kita sering tahu banyak hal sebelum melihat data. Aliran harus kekal massa, temperatur dalam kelvin tidak boleh negatif, dan solusi harus simetris

kalau geometrinya simetris. Semua itu bisa dibaca sebagai prior yang kuat. PINN (Bab 25) memasukkan persamaan diferensial ke dalam loss dengan cara yang mirip suku $-\log p(w)$ di atas, hanya saja “prior”-nya menyangkut fungsi keluaran, bukan bobot.

1.5 Mengukur generalisasi

Risiko sejati (1.1) tidak bisa dihitung, jadi kita memperkirakannya. Cara paling sederhana adalah menyisihkan sebagian data yang tidak pernah disentuh selama pelatihan, lalu menghitung loss rata-rata di sana. Karena titik-titik uji itu independen dari model yang sudah jadi, rata-ratanya adalah estimator tak bias untuk $R(\hat{f})$.

Kesulitan muncul karena kita juga harus memilih *hyperparameter*: jumlah lapisan, laju belajar, kekuatan regularisasi. Kalau data uji dipakai untuk memilih, ia ikut bocor ke dalam model dan estimasinya menjadi terlalu optimistis. Karena itu data biasanya dibagi tiga: data latih, data validasi untuk memilih hyperparameter, dan data uji yang disentuh sekali saja di akhir. Kalau data sedikit, *k-fold cross-validation* membagi data menjadi k bagian dan melatih model k kali, setiap kali dengan satu bagian sebagai validasi. Rata-rata skornya menjadi estimasi generalisasi, dengan biaya k kali pelatihan. Perlu diingat bahwa cross-validation menghasilkan estimasi kinerja, bukan satu model. Model akhir biasanya dilatih ulang pada seluruh data latih.

Ada beberapa kebocoran yang sering terjadi tanpa disadari. Menormalisasi seluruh data, termasuk data uji, sebelum dibagi berarti statistik data uji ikut masuk ke pelatihan. Rata-rata dan simpangan baku seharusnya dihitung dari data latih saja. Untuk deret waktu atau hasil simulasi yang berurutan, membagi data secara acak juga menyesatkan. Titik-titik yang berdekatan dalam waktu hampir identik, sehingga ujian menjadi terlalu mudah. Data seperti ini sebaiknya dibagi menurut waktu atau menurut kasus simulasi.

1.6 Bias, varians, dan dua bentuk kurva

Ambil loss kuadrat dan anggap $y = g(x) + \varepsilon$ dengan $\mathbb{E}[\varepsilon] = 0$ dan $\text{Var}[\varepsilon] = \sigma^2$. Model $\hat{f}_{\mathcal{D}}$ bergantung pada data latih $\mathcal{D}$ yang acak. Untuk satu titik x , ekspektasi galatnya terhadap semua kemungkinan data latih dan noise terurai menjadi tiga suku (Geman et al., 1992):

$$\mathbb{E}[(y - \hat{f}_{\mathcal{D}}(x))^2] = \underbrace{(g(x) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)])^2}_{\text{bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)])^2]}_{\text{variens}} + \underbrace{\sigma^2}_{\text{noise}}. \quad (1.6)$$

Bias mengukur seberapa jauh rata-rata model dari kebenaran. Varians mengukur seberapa banyak model berubah kalau data latihnya diganti. Noise tidak bisa dikurangi oleh model apa pun.

Penurunannya hanya memakai satu trik: tambahkan dan kurangkan besaran yang sama. Tulis singkat $\hat{f} = \hat{f}_{\mathcal{D}}(x)$, $\bar{f} = \mathbb{E}_{\mathcal{D}}[\hat{f}]$, dan $g = g(x)$. Karena $y = g + \varepsilon$,

$$y - \hat{f} = \varepsilon + (g - \bar{f}) + (\bar{f} - \hat{f}).$$

Kuadratkan, lalu ambil ekspektasi. Noise ε berasal dari titik uji dan independen dari data latih, sehingga semua suku silang yang memuat ε hilang karena $\mathbb{E}[\varepsilon] = 0$. Suku silang $2(g - \bar{f})\mathbb{E}_{\mathcal{D}}[\bar{f} - \hat{f}]$ juga hilang, karena $g - \bar{f}$ tidak acak dan $\mathbb{E}_{\mathcal{D}}[\hat{f}] = \bar{f}$. Yang tersisa adalah tiga kuadrat: $\mathbb{E}[\varepsilon^2] = \sigma^2$, $(g - \bar{f})^2$, dan $\mathbb{E}_{\mathcal{D}}[(\hat{f} - \bar{f})^2]$, persis tiga suku di atas.

Contoh angka membuatnya konkret. Misalkan nilai sebenarnya di sebuah titik $g = 2$, noise bervarians $\sigma^2 = 0,25$, dan kita melatih model yang sama pada tiga himpunan data latih berbeda, yang memberi prediksi 1,6, 2,0, dan 2,1 di titik itu. Rata-rata prediksinya $\bar{f} = 1,9$, sehingga bias kuadratnya $(2 - 1,9)^2 = 0,01$. Variansnya $\frac{1}{3}[(1,6 - 1,9)^2 + (2,0 - 1,9)^2 + (2,1 - 1,9)^2] = \frac{1}{3}(0,09 + 0,01 + 0,04) \approx 0,047$. Galat kuadrat harapannya $0,01 + 0,047 + 0,25 \approx 0,31$, dan di sini noise adalah bagian terbesar. Model yang lebih rumit bisa saja menurunkan bias, tetapi tidak akan menyentuh 0,25 itu.

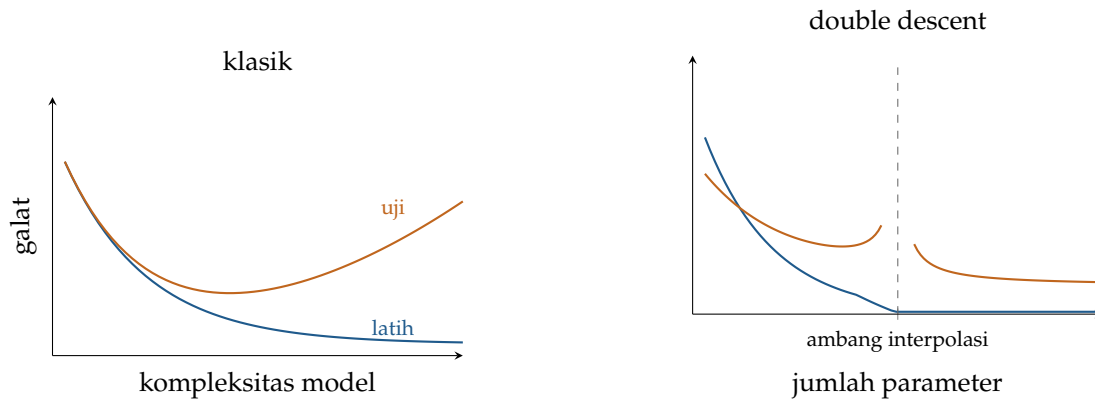
Papan dart membantu membayangkannya. Pemain dengan bias kecil dan varians kecil mengumpulkan semua anak panah di tengah. Pemain dengan bias besar tetapi varians kecil mengumpulkannya rapat di pojok kiri atas: konsisten, tetapi salah arah. Pemain dengan bias kecil tetapi varians besar menyebar ke mana-mana, meski rata-ratanya tepat di tengah. Model yang terlalu sederhana mirip pemain kedua. Model yang terlalu rumit mirip pemain ketiga, karena setiap set data latih membuatnya melempar ke tempat yang berbeda.

Gambaran klasik mengatakan bahwa ketika kompleksitas model bertambah, bias turun dan varians naik, sehingga galat uji membentuk huruf U dengan titik terbaik di tengah. Gambaran ini benar untuk banyak model klasik, tetapi jaringan saraf modern sering punya parameter jauh lebih banyak daripada jumlah data, dan tetap generalisasi dengan baik. Belkin et al. (2019) menunjukkan bahwa setelah *interpolation threshold*, yaitu titik ketika model baru saja mampu menghafal semua data latih, galat uji bisa turun lagi. Kurva lengkapnya disebut *double descent* (Gambar 1.1). Sebelumnya, Zhang et al. (2016) memperlihatkan bahwa jaringan yang sama bisa menghafal label acak dengan sempurna. Artinya, kapasitas mentah tidak menjelaskan generalisasi. Yang menentukan adalah solusi mana yang dipilih oleh algoritme pelatihan di antara banyak solusi yang sama-sama menghafal data. Stochastic gradient descent tampaknya cenderung memilih solusi yang halus, sebuah topik yang muncul lagi di Bab 2.

Kuliah geometric deep learning dan teori pembelajaran statistik memakai cara lain untuk memecah galat, yang lebih umum. Misalkan f^* prediktor Bayes, $f_{\mathcal{F}}$ prediktor terbaik di dalam model class, $\hat{f}$ hasil ERM, dan $\tilde{f}$ hasil algoritme optimasi yang sungguh kita jalankan. Maka

$$R(\tilde{f}) - R^* = \underbrace{R(f_{\mathcal{F}}) - R^*}_{\text{aproksimasi}} + \underbrace{R(\hat{f}) - R(f_{\mathcal{F}})}_{\text{estimasi}} + \underbrace{R(\tilde{f}) - R(\hat{f})}_{\text{optimasi}}. \quad (1.7)$$

Model class yang lebih besar mengecilkan galat aproksimasi, tetapi bisa membesarkan galat estimasi, karena semakin banyak fungsi yang kebetulan cocok dengan data. Galat optimasi muncul karena kita hampir tidak pernah benar-benar menemukan minimum global.



GAMBAR 1.1. Kiri, kurva U klasik. Kanan, double descent: setelah model mampu menghafal semua data latih, galat uji bisa turun lagi. Kedua kurva bersifat skematik.

1.7 Kutukan dimensi

Ada satu hal yang membuat machine learning di dimensi tinggi berbeda secara mendasar dari intuisi kita di dimensi rendah. Namanya *curse of dimensionality*. Ambil kubus satuan $[0, 1]^d$ dan kubus kecil di tengahnya dengan sisi 0,9. Fraksi volume di dalam kubus kecil adalah $0,9^d$. Untuk $d = 1$ hasilnya 0,9. Untuk $d = 10$ hasilnya sekitar 0,35. Untuk $d = 100$ hasilnya kira-kira $2,7 \times 10^{-5}$. Hampir seluruh volume kubus berdimensi seratus berada di kulit tipis dekat permukaannya. Titik acak hampir selalu dekat ke tepi, dan hampir semua pasangan titik punya jarak yang mirip.

Akibatnya, metode lokal seperti k -nearest neighbour kehilangan makna. Untuk menutup ruang berdimensi d dengan ketelitian ϵ , kita butuh sekitar ϵ^{-d} titik. Kuliah geometric deep learning menyatakannya secara formal: untuk kelas fungsi Lipschitz di d dimensi, galat estimasi terbaik turun hanya seperti $N^{-1/d}$. Dengan $d = 100$, menggandakan jumlah data hampir tidak membantu.

Mencari kunci yang jatuh di lorong panjang masih mudah. Di lantai sebuah aula, butuh waktu jauh lebih lama. Di gedung bertingkat yang penuh ruangan, kita mulai putus asa. Sekarang bayangkan gedung dengan seratus arah yang berbeda: sebanyak apa pun senter yang dibawa, sebagian besar ruang tetap gelap.

Lalu mengapa deep learning bisa bekerja pada gambar berukuran jutaan piksel? Ada dua jawaban yang akan muncul berkali-kali di buku ini. Jawaban pertama adalah *manifold hypothesis*: data nyata tidak memenuhi seluruh ruang. Gambar wajah manusia menempati semacam permukaan berdimensi jauh lebih rendah di dalam ruang piksel. Kalau model bisa menemukan permukaan itu, dimensi efektif masalahnya kecil. Jawaban kedua adalah simetri. Kita tahu bahwa kucing tetap kucing kalau digeser sedikit ke kiri. Kalau arsitektur dibuat invarian atau ekuivarian terhadap simetri seperti ini, ruang hipotesis menyusut drastis tanpa membuang solusi yang benar. Itulah gagasan pokok Bronstein et al. (2021), yang kita bahas di Bab 9.

1.8 Tidak ada makan siang gratis

Teorema *no free lunch* (Wolpert, 1996) menyatakan bahwa kalau dirata-ratakan atas semua kemungkinan masalah, tidak ada algoritme belajar yang lebih baik dari yang lain. Kedengarannya pesimistis, tetapi pesannya praktis: keberhasilan selalu datang dari asumsi yang cocok dengan masalahnya. Konvolusi bekerja untuk gambar karena gambar punya struktur lokal dan simetri geser. PINN bekerja untuk masalah yang persamaannya kita ketahui. Tanpa asumsi, tidak ada generalisasi.

1.9 Mengukur kinerja klasifikasi

Untuk klasifikasi dua kelas, hasil prediksi disusun dalam *confusion matrix* yang berisi *true positive* (TP), *false positive* (FP), *true negative* (TN), dan *false negative* (FN). Dari keempatnya diperoleh

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}, \quad \text{specificity} = \frac{TN}{TN + FP}.$$

Misalkan sebuah tes menandai 50 dari 1000 orang sebagai positif, dan 40 di antaranya memang sakit, sementara total orang yang sakit ada 60. Maka $TP = 40$, $FP = 10$, dan $FN = 20$, sehingga $\text{precision} = 40/50 = 0,80$ dan $\text{recall} = 40/60 \approx 0,67$. Precision menjawab “kalau tes bilang positif, seberapa bisa dipercaya?”, recall menjawab “dari semua yang sakit, berapa yang tertangkap?”. Keduanya sering dirangkum dengan rata-rata harmonik, $F_1 = 2 \cdot \frac{0,80 \cdot 0,67}{0,80 + 0,67} \approx 0,73$. Rata-rata harmonik dipakai karena ia tertarik ke nilai yang lebih kecil: model tidak bisa menutupi recall yang buruk dengan precision yang sempurna.

Akurasi bisa menipu kalau kelasnya tidak seimbang. Pada data dengan 99 persen kelas negatif, model yang selalu menjawab “negatif” punya akurasi 99 persen dan recall nol. Kurva ROC memplot recall terhadap $1 - \text{specificity}$ untuk semua ambang keputusan. Luas di bawahnya, AUC, sama dengan peluang model memberi skor lebih tinggi pada satu contoh positif acak dibanding satu contoh negatif acak.

1.10 Generalisasi di luar kelas

Pertanyaan-pertanyaan di bab ini akan muncul lagi di setiap bidang yang dibahas buku ini, dengan wajah yang berbeda-beda.

Yang pertama adalah *distribution shift*: data saat model dipakai tidak berasal dari distribusi yang sama dengan data latih. Model citra medis yang dilatih di satu rumah sakit bisa gagal di rumah sakit lain dengan pemindai berbeda (Bab 21). Model pengenalan aktivitas yang dilatih pada segelintir orang bisa gagal pada orang dengan gaya gerak lain (Bab 16). Model pengganti simulasi yang dilatih pada bilangan Reynolds antara 100 dan 1000 belum tentu benar pada 5000 (Bab 26).

Yang kedua adalah kejujuran pembagian data. Di kimia, molekul dari satu seri senyawa hampir identik, sehingga pembagian acak membuat model tampak jauh lebih baik dari kenyataannya (Bab 19). Di sensor yang dikenakan, jendela-jendela dari orang yang sama berkorelasi kuat. Di simulasi, potret berurutan dari satu kasus hampir sama.

Pola umumnya: bagilah data menurut satuan yang benar-benar baru saat model dipakai, entah itu pasien, orang, kerangka molekul, atau kasus simulasi.

Yang ketiga adalah mahalnnya data. Satu label ahli radiologi, satu pengukuran assay, atau satu simulasi CFD penuh bisa memakan waktu dan biaya besar. Di sinilah prior, dalam bentuk simetri, pengetahuan domain, atau hukum fisika, menjadi berharga: ia mengurangi data yang dibutuhkan dan menjaga model tetap masuk akal di luar daerah yang pernah dilihatnya.

CATATAN SUMBER. Bab ini merangkai materi dari Machine Learning: Basic Techniques, Supervised dan Unsupervised Techniques, Statistical Learning Theory, serta bagian pembuka Deep Learning: Geometric Techniques. Bacaan pendamping yang baik adalah Bishop (2006), Hastie et al. (2009), dan Bach (2024).

2 Optimasi: Turun Gunung dalam Kabut

Optimizer sudah saya pakai sejak tugas deep learning di semester pertama, tetapi teorinya baru dibahas secara sistematis di semester terakhir, di kuliah optimasi numerik: mengapa gradient descent bisa lambat, kapan ia pasti konvergen, dan apa yang dibeli metode Newton dengan biaya yang mahal. Bab ini menggabungkan kedua sisi itu.

Bab sebelumnya berakhir dengan perintah “minimalkan risiko empiris”. Sekarang kita membahas cara melakukannya. Hampir semua model di buku ini, dari regresi linear sampai PINN, dilatih dengan varian dari satu gagasan: ikuti arah turunan.

Bayangkan Anda berdiri di lereng gunung yang tertutup kabut tebal. Anda ingin turun ke lembah, tetapi hanya bisa melihat tanah di sekitar kaki. Strategi paling masuk akal adalah merasakan ke arah mana tanah paling curam menurun, melangkah ke sana, lalu mengulangnya. Inilah gradient descent. Persoalannya ada pada panjang langkah. Langkah yang terlalu kecil membuat Anda baru sampai besok pagi. Langkah yang terlalu besar membuat Anda melompati lembah dan mendarat di lereng seberang, bahkan lebih tinggi dari tempat semula.

2.1 Masalah dan syarat optimalitas

Kita mencari $\mathbf{x}^* = \arg \min_{\mathbf{x} \in \mathbb{R}^N} f(\mathbf{x})$ untuk fungsi f yang cukup mulus. Dua syarat klasik membantu mengenali minimum. Kalau $\mathbf{x}^*$ minimum lokal, gradiennya pasti nol, $\nabla f(\mathbf{x}^*) = \mathbf{0}$. Sebaliknya, kalau gradiennya nol dan Hessian $\nabla^2 f(\mathbf{x}^*)$ definit positif, maka $\mathbf{x}^*$ pasti minimum lokal yang tegas. Titik dengan gradien nol disebut *stationary point*, dan ia bisa berupa minimum, maksimum, atau *saddle point*. Di dimensi tinggi, saddle point jauh lebih banyak daripada minimum, karena untuk menjadi minimum semua nilai eigen Hessian harus positif sekaligus.

Fungsi f disebut *convex* kalau ruas garis yang menghubungkan dua titik di grafiknya selalu berada di atas grafik itu, yaitu $f(\lambda \mathbf{x} + (1 - \lambda)\mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{y})$ untuk setiap $\lambda \in [0, 1]$. Untuk fungsi konveks, setiap minimum lokal adalah minimum global. Regresi linear dengan loss kuadrat, regresi logistik, dan lasso semuanya konveks. Jaringan saraf tidak. Meski begitu, teori konveks tetap berguna, karena di sekitar sebuah minimum hampir semua fungsi mulus terlihat seperti mangkuk kuadrat.

Fungsi uji klasik yang tidak konveks adalah fungsi Rosenbrock (Rosenbrock, 1960), $f(x, y) = (1 - x)^2 + 100(y - x^2)^2$. Minimumnya di $(1, 1)$, tersembunyi di dasar lembah sempit berbentuk pisang. Menemukan lembahnya mudah. Yang sulit adalah berjalan menyusuri lembah itu sampai ke titik terendahnya.

2.2 Gradient descent

Aturan dasarnya adalah

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k), \quad (2.1)$$

dengan $\alpha_k > 0$ panjang langkah atau *learning rate*. Arah negatif gradien dipilih karena ekspansi Taylor orde satu, $f(\mathbf{x} + \mathbf{h}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \mathbf{h}$, paling cepat turun kalau $\mathbf{h}$ searah $-\nabla f(\mathbf{x})$.

Kapan langkah-langkah ini dijamin menuju minimum? Anggap gradien f bersifat Lipschitz dengan konstanta L , artinya $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|$. Konstanta L mengukur seberapa cepat kemiringan bisa berubah. Dengan langkah tetap $\alpha = 1/L$, setiap iterasi menjamin

$$f(\mathbf{x}_{k+1}) \leq f(\mathbf{x}_k) - \frac{1}{2L} \|\nabla f(\mathbf{x}_k)\|^2,$$

jadi selama gradien belum nol, nilai fungsi pasti turun.

Ketaksamaan ini, yang di naskah kuliah disebut *descent lemma*, bisa diturunkan dalam dua baris. Gradien yang Lipschitz berarti fungsi tidak bisa melengkung ke atas lebih cepat dari sebuah parabola dengan kelengkungan L , sehingga untuk setiap $\mathbf{h}$,

$$f(\mathbf{x} + \mathbf{h}) \leq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \mathbf{h} + \frac{L}{2} \|\mathbf{h}\|^2.$$

Ruas kanan adalah parabola yang selalu berada di atas fungsi. Masukkan langkah gradient descent, $\mathbf{h} = -\alpha \nabla f(\mathbf{x})$, sehingga ruas kanan menjadi $f(\mathbf{x}) - \alpha(1 - \frac{L\alpha}{2}) \|\nabla f(\mathbf{x})\|^2$. Penurunan terbesar didapat dengan memaksimalkan $\alpha(1 - L\alpha/2)$, yang terjadi pada $\alpha = 1/L$, dan nilainya $1/(2L)$. Dari sini juga terlihat batas atas langkah: kalau $\alpha \geq 2/L$, faktor $1 - L\alpha/2$ menjadi nol atau negatif, dan jaminan penurunannya hilang.

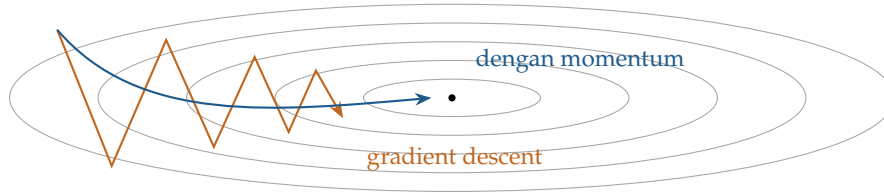
Kalau f juga *strongly convex* dengan konstanta μ , galat menyusut secara geometris dengan laju yang ditentukan oleh *condition number* $\kappa = L/\mu$.

Contoh kecil memperlihatkan arti condition number. Ambil $f(x_1, x_2) = \frac{1}{2}(x_1^2 + 100x_2^2)$. Gradiennya $(x_1, 100x_2)$, sehingga $L = 100$, $\mu = 1$, dan $\kappa = 100$. Dengan langkah tetap α , kedua koordinat berkembang terpisah: x_1 dikalikan $(1 - \alpha)$ dan x_2 dikalikan $(1 - 100\alpha)$ di setiap langkah. Supaya x_2 tidak meledak, kita butuh $|1 - 100\alpha| < 1$, yang berarti $\alpha < 0,02$. Tetapi dengan langkah sekecil itu, x_1 hanya menyusut dengan faktor sekitar 0,98. Bahkan dengan langkah terbaik, $\alpha = 2/(L + \mu)$, faktor terburuknya $(\kappa - 1)/(\kappa + 1) = 99/101$. Untuk mengecilkan galat sejuta kali, dibutuhkan kira-kira $\ln(10^{-6})/\ln(99/101) \approx 690$ langkah. Metode Newton menyelesaikan masalah kuadrat yang sama dalam satu langkah.

Itulah inti masalah optimasi dalam praktik. Gradient descent lambat bukan karena fungsinya rumit, melainkan karena skala arah-arahnya berbeda jauh. Pada lembah yang lonjong, lintasannya berzig-zag menyeberangi lembah alih-alih berjalan menyusurnya (Gambar 2.1).

2.3 Line search

Daripada menebak α sekali untuk selamanya, kita bisa mencarinya di setiap iterasi. *Line search* mencoba beberapa panjang langkah sepanjang arah turun $\mathbf{d}_k$ dan menerima yang



GAMBAR 2.1. Pada lembah lonjong, gradient descent berzig-zag. Momentum meredam gerak bolak-balik dan mempercepat gerak sepanjang lembah. Gambar skematik.

pertama dianggap cukup baik. Ukuran “cukup baik” yang paling umum adalah syarat Armijo (Armijo, 1966),

$$f(\mathbf{x}_k + \alpha \mathbf{d}_k) \leq f(\mathbf{x}_k) + c \alpha \nabla f(\mathbf{x}_k)^\top \mathbf{d}_k, \quad 0 < c < 1,$$

yang meminta agar penurunan nyata setidaknya sebagian kecil dari penurunan yang dijanjikan oleh garis singgung. Prosedur *backtracking* memulai dari $\alpha = 1$ lalu terus mengalikannya dengan sebuah faktor $\beta \in (0, 1)$ sampai syarat itu terpenuhi. Di deep learning, line search jarang dipakai, karena setiap evaluasi f berarti melewati seluruh data. Gagasannya muncul kembali dalam L-BFGS.

2.4 Metode Newton dan kerabatnya

Metode Newton memanfaatkan kelengkungan. Ia meminimalkan model kuadrat lokal $f(\mathbf{x}_k) + \nabla f^\top \mathbf{h} + \frac{1}{2} \mathbf{h}^\top \nabla^2 f \mathbf{h}$, yang menghasilkan

$$\mathbf{x}_{k+1} = \mathbf{x}_k - [\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k). \quad (2.2)$$

Di dekat minimum yang Hessian-nya definit positif, Newton konvergen secara kuadratik: jumlah digit yang benar kira-kira berlipat dua di setiap iterasi. Ada tiga hal yang membatasinya. Hessian berukuran $N \times N$, sehingga untuk jaringan dengan ratusan juta parameter, menyimpannya saja sudah mustahil. Jauh dari minimum, Hessian bisa tidak definit dan langkah Newton justru menanjak. Kuliah menyebut perbaikannya *Hessian modification*, yaitu menambahkan $\lambda \mathbf{I}$ sampai matriksnya definit positif. Terakhir, Newton tertarik ke titik mana pun yang gradiennya nol, termasuk saddle point.

Metode *quasi-Newton* seperti BFGS, dan versi hemat memorinya L-BFGS, membangun aproksimasi Hessian dari selisih gradien beberapa iterasi terakhir (Nocedal and Wright, 2006). L-BFGS populer untuk masalah deterministik berukuran sedang, termasuk tahap akhir pelatihan PINN.

2.5 Stochastic gradient descent

Dalam machine learning, f adalah rata-rata loss atas N contoh, jadi menghitung gradien penuh berarti melewati seluruh data. *Stochastic gradient descent* (SGD) menggantinya dengan gradien dari sebuah *mini-batch* acak berukuran B :

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \alpha_k \frac{1}{B} \sum_{n \in \mathcal{B}_k} \nabla_{\mathbf{w}} L(y_n, f(\mathbf{x}_n; \mathbf{w}_k)). \quad (2.3)$$

Gradien mini-batch adalah estimator tak bias dari gradien penuh, tetapi berisik. Teorinya berakar pada *stochastic approximation* (Robbins and Monro, 1951): agar iterasi konvergen, panjang langkah harus mengecil dengan syarat $\sum_k \alpha_k = \infty$ dan $\sum_k \alpha_k^2 < \infty$. Syarat pertama menjamin kita masih bisa berjalan sejauh apa pun, syarat kedua menjamin noise akhirnya teredam.

Untuk mengetahui harga rata-rata beras di seluruh kota, Anda bisa mendatangi semua warung, atau bertanya ke sepuluh warung yang dipilih acak. Jawaban sepuluh warung tidak tepat, tetapi didapat jauh lebih cepat. Kalau survei kecil itu diulang terus dan tebakan disesuaikan sedikit demi sedikit, Anda sampai ke angka yang benar lebih cepat daripada menunggu satu survei besar. SGD bekerja dengan cara yang sama.

Noise SGD ternyata bukan sekadar kekurangan. Ia membuat iterasi terus bergoyang dan sulit bertahan di minimum yang sempit. Hochreiter and Schmidhuber (1997b) berargumen bahwa minimum yang datar generalisasi lebih baik, karena perubahan kecil pada bobot tidak banyak mengubah loss, sehingga bobotnya bisa dideskripsikan dengan presisi lebih rendah dan modelnya efektif lebih sederhana. Keskar et al. (2016) mengamati bahwa batch yang besar cenderung berakhir di minimum tajam dengan generalisasi lebih buruk. Kuliah deep learning menyebut efek ini *self-regularization* oleh SGD.

2.6 Momentum dan metode adaptif

Metode *heavy ball* dari Polyak (1964) menambahkan ingatan tentang arah sebelumnya,

$$\mathbf{v}_{k+1} = \beta \mathbf{v}_k - \alpha \nabla f(\mathbf{x}_k), \quad \mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{v}_{k+1}, \quad (2.4)$$

dengan β biasanya sekitar 0,9. Komponen gradien yang berganti tanda setiap langkah saling meniadakan di dalam $\mathbf{v}$, sedangkan komponen yang konsisten menumpuk. Gradient descent biasa mirip orang yang berhenti total di setiap langkah lalu melihat lagi ke mana arah turun. Momentum mirip bola yang menggelinding: ia membawa kecepatannya, tidak langsung berbalik karena satu gundukan kecil, dan melaju lebih cepat di lereng yang panjang. Zig-zag di Gambar 2.1 teredam dengan cara ini.

Masalah condition number juga bisa diserang dengan memberi setiap parameter laju belajarnya sendiri. AdaGrad membagi langkah dengan akar jumlah kuadrat gradien masa lalu, RMSProp memakai rata-rata bergerak agar pembagi itu tidak terus membesar, dan Adam (Kingma and Ba, 2014) menggabungkan momentum dengan RMSProp:

$$\begin{aligned} \mathbf{m}_k &= \beta_1 \mathbf{m}_{k-1} + (1 - \beta_1) \mathbf{g}_k, & \mathbf{v}_k &= \beta_2 \mathbf{v}_{k-1} + (1 - \beta_2) \mathbf{g}_k^2, \\ \hat{\mathbf{m}}_k &= \frac{\mathbf{m}_k}{1 - \beta_1^k}, & \hat{\mathbf{v}}_k &= \frac{\mathbf{v}_k}{1 - \beta_2^k}, & \mathbf{w}_{k+1} &= \mathbf{w}_k - \alpha \frac{\hat{\mathbf{m}}_k}{\sqrt{\hat{\mathbf{v}}_k} + \epsilon}. \end{aligned} \quad (2.5)$$

Semua operasinya per elemen, dengan nilai bawaan $\beta_1 = 0,9$, $\beta_2 = 0,999$, dan $\epsilon = 10^{-8}$. Faktor $1 - \beta^k$ disebut *bias correction*, dan asalnya mudah ditelusuri. Membuka rekursi $\mathbf{m}_k$ dari $\mathbf{m}_0 = \mathbf{0}$ memberi $\mathbf{m}_k = (1 - \beta_1) \sum_{j=1}^k \beta_1^{k-j} \mathbf{g}_j$. Kalau gradien punya ekspektasi tetap $\mathbb{E}[\mathbf{g}]$, maka

$$\mathbb{E}[\mathbf{m}_k] = (1 - \beta_1)(1 + \beta_1 + \cdots + \beta_1^{k-1})\mathbb{E}[\mathbf{g}] = (1 - \beta_1^k)\mathbb{E}[\mathbf{g}],$$

karena deret geometri berhingga $\sum_{j=0}^{k-1} \beta_1^j = (1 - \beta_1^k)/(1 - \beta_1)$. Jadi m_k bias ke bawah dengan faktor tepat $1 - \beta_1^k$, dan membaginya dengan faktor itu menghapus bias. Pada langkah pertama dengan $\beta_1 = 0,9$, $m_1 = 0,1 g_1$, sepuluh kali terlalu kecil, dan koreksi mengembalikannya ke g_1 . Untuk $\beta_2 = 0,999$, faktornya pada langkah pertama 0,001, sehingga tanpa koreksi langkah awal Adam akan besar sekali dan tidak menentu. Setelah beberapa ribu langkah, kedua faktor mendekati satu dan koreksinya tidak berpengaruh lagi.

Beberapa salah paham tentang Adam cukup umum. Adam tidak selalu lebih baik dari SGD. Untuk beberapa tugas visi, SGD dengan momentum dan jadwal laju belajar yang baik masih menghasilkan generalisasi yang lebih baik. Menambahkan suku L2 ke loss lalu memakai Adam juga tidak sama dengan weight decay, karena suku itu ikut dibagi $\sqrt{\hat{v}}$ sehingga efeknya tidak seragam antar parameter. AdamW memisahkan weight decay dari langkah adaptif untuk mengatasi hal ini (Loshchilov and Hutter, 2017).

Laju belajar yang tetap pun jarang optimal. Pola yang umum adalah *warm-up*, yaitu mulai kecil dan naik selama beberapa ratus langkah, lalu menurun perlahan, misalnya mengikuti kurva kosinus. Warm-up mencegah langkah awal yang terlalu agresif ketika statistik Adam belum stabil.

2.7 Lanskap loss jaringan dalam

Lanskap loss jaringan dalam ternyata berbeda dari intuisi dimensi rendah. Untuk fungsi acak di dimensi tinggi, sebagian besar titik kritis dengan loss tinggi adalah saddle point, dan minimum lokal cenderung punya nilai loss yang mirip satu sama lain. Pada jaringan yang sangat lebar, ada banyak sekali solusi yang sama-sama mencapai loss latih hampir nol, dan solusi-solusi itu sering terhubung oleh lembah dengan loss rendah. Pertanyaan pentingnya pun bergeser, dari “apakah kita menemukan minimum global?” menjadi “minimum mana yang dipilih, dan apakah ia generalisasi?”.

2.8 Optimasi di luar kelas

Contoh mangkuk lonjong di atas bukan sekadar latihan. Masalah condition number yang sama muncul di banyak tempat dalam buku ini dengan nama yang berbeda. Filter adaptif LMS dalam pemrosesan sinyal adalah SGD dengan mini-batch satu sampel, dan kecepatan konvergensinya ditentukan oleh sebaran nilai eigen matriks autokorelasi input (Bab 14). Algoritme RLS, yang memperbarui invers matriks itu, adalah kerabat metode Newton. Faktorisasi matriks dalam sistem rekomendasi dilatih dengan SGD (Bab 30), dan model molekul dilatih dengan Adam seperti jaringan lain.

Di simulasi, loss sebuah PINN adalah jumlah beberapa suku: residu persamaan diferensial, syarat batas, syarat awal, dan data. Suku-suku ini punya skala dan kelengkungan yang jauh berbeda, sehingga condition number masalahnya buruk sekali. Wang et al. (2021) menunjukkan bahwa gradien suku residu bisa mendominasi suku batas, sehingga jaringan memenuhi persamaan di dalam domain tetapi melupakan syarat batasnya. Resep yang sering dipakai adalah memulai dengan Adam untuk

keluar dari daerah awal yang buruk, melanjutkan dengan L-BFGS untuk turun tajam ke minimum, dan menyeimbangkan bobot suku-suku loss secara adaptif (Bab 25).

CATATAN SUMBER. Teori konvergensi, line search, dan metode Newton mengikuti naskah kuliah Numerical Optimization for AI. Bagian SGD, momentum, dan optimizer adaptif mengikuti bab metode belajar di kuliah deep learning dasar. Bacaan lanjut: Nocedal and Wright (2006) untuk optimasi klasik dan Bottou et al. (2018) untuk optimasi skala besar dalam machine learning.

3 AI Klasik: Mencari, Merencanakan, Memutuskan

Kuliah pengantar AI baru saya ikuti di semester terakhir, pada musim panas 2026, setelah hampir semua kuliah deep learning selesai. Dengan urutan terbalik seperti itu, search, planning, game playing, dan representasi pengetahuan tidak tampil sebagai masa lalu yang usang, melainkan sebagai fondasi yang masih dipakai di dalam sistem modern, dari AlphaGo sampai agen berbasis model bahasa. Setahun sebelumnya, kuliah operations research memberi sisi lain dari cerita yang sama: mencari keputusan terbaik di antara pilihan yang terhitung tetapi banyak sekali.

Sebelum jaringan saraf mendominasi, AI berarti program yang bernalar secara eksplisit: mencari jalan di ruang kemungkinan, menyusun rencana langkah demi langkah, dan menarik kesimpulan dari fakta. Bab ini membahas gagasan-gagasan itu, karena tanpa mereka sulit memahami mengapa sistem AI paling kuat hari ini menggabungkan pembelajaran dengan pencarian.

3.1 Dari Turing ke Dartmouth

TURING (1950) mengganti pertanyaan “bisakah mesin berpikir?” dengan permainan tiruan: kalau seorang penanya tidak bisa membedakan mesin dari manusia lewat percakapan tertulis, apa alasan kita menolak menyebut mesin itu cerdas? Istilah *artificial intelligence* sendiri lahir dari proposal lokakarya musim panas di Dartmouth pada 1956. Sejak itu, sejarah AI bergerak dalam gelombang optimisme dan kekecewaan. Sistem pakar pada 1980-an menyimpan pengetahuan manusia dalam ribuan aturan jika-maka, lalu tersandung pada sulitnya menulis semua aturan itu dengan tangan. Gelombang statistik dan kemudian deep learning menggeser fokus dari aturan yang ditulis ke pola yang dipelajari (Russell and Norvig, 2021).

Kuliah juga menyinggung perdebatan filosofisnya. Searle (1980) mengajukan eksperimen pikiran ruang Cina. Seseorang yang tidak mengerti bahasa Cina duduk di dalam ruangan dan menjawab pesan dalam bahasa Cina dengan mengikuti buku aturan yang lengkap. Dari luar, jawabannya sempurna. Dari dalam, tidak ada pemahaman sama sekali. Searle berargumen bahwa memanipulasi simbol tidak sama dengan memahami. Perdebatan antara AI lemah, yang hanya meniru perilaku cerdas, dan AI kuat, yang sungguh memahami, belum selesai sampai sekarang, dan model bahasa besar membuatnya kembali hangat.

3.2 Pencarian

Banyak masalah bisa dirumuskan sebagai pencarian: ada keadaan awal, sekumpulan aksi, fungsi transisi yang mengatakan ke mana sebuah aksi membawa kita, uji tujuan, dan biaya setiap langkah. Teka-teki geser delapan angka, rute navigasi, dan penyusunan jadwal semuanya cocok dengan bentuk ini. Solusinya adalah jalur dari keadaan awal ke keadaan tujuan.

Pencarian tanpa informasi tidak tahu apa-apa tentang di mana tujuan berada. *Breadth-first search* menjelajah lapis demi lapis dan menjamin jalur terpendek dalam jumlah langkah, tetapi memorinya tumbuh eksponensial. *Depth-first search* masuk sedalam mungkin lebih dulu, hemat memori, tetapi bisa tersesat di cabang yang tak berujung. *Uniform-cost search* selalu memperluas simpul dengan biaya kumulatif terkecil, sehingga menemukan jalur termurah.

Pencarian dengan informasi memakai *heuristic* $h(n)$, perkiraan biaya dari simpul n ke tujuan. Algoritme A* (Hart et al., 1968) memperluas simpul dengan nilai

$$f(n) = g(n) + h(n) \quad (3.1)$$

terkecil, dengan $g(n)$ biaya yang sudah ditempuh. Kalau heuristik tidak pernah melebihi biaya sebenarnya, yaitu *admissible*, A* dijamin menemukan jalur optimal. Pada teka-teki delapan angka, jumlah ubin yang salah tempat adalah heuristik *admissible*, dan jumlah jarak Manhattan setiap ubin ke posisi seharusnya juga *admissible* tetapi lebih informatif, karena selalu sama atau lebih besar. Heuristik yang lebih informatif memangkas jauh lebih banyak simpul.

A* mirip pengemudi yang memilih belokan berdasarkan dua hal sekaligus: jarak yang sudah ditempuh di odometer, dan jarak garis lurus yang tersisa ke tujuan. Karena garis lurus tidak pernah lebih panjang dari jalan sebenarnya, ia tidak akan tertipu untuk mengabaikan rute yang sebenarnya baik.

Ketika jalurnya tidak penting dan yang dicari hanya keadaan terbaik, misalnya susunan jadwal yang paling sedikit bentrok, *local search* lebih cocok. Hill climbing terus bergerak ke tetangga yang lebih baik sampai tidak ada lagi, dan sering terjebak di puncak lokal. Simulated annealing (Kirkpatrick et al., 1983) kadang menerima langkah yang memburuk dengan peluang yang menurun perlahan, seperti logam yang didinginkan pelan-pelan agar atomnya menemukan susunan berenergi rendah. Tabu search (Glover, 1989) mengingat langkah-langkah terakhir dan melarang kembali ke sana, sehingga tidak berputar-putar di sekitar puncak yang sama.

3.3 Bermain melawan lawan

Dalam permainan dua pemain, keadaan berikutnya juga ditentukan oleh lawan yang berusaha mengalahkan kita. Algoritme *minimax* mengasumsikan lawan bermain sempurna: di giliran kita pilih langkah dengan nilai maksimum, di giliran lawan anggap ia memilih nilai minimum. Contoh dari Russell and Norvig (2021) memperlihatkan. Misalkan kita punya tiga langkah, dan lawan menjawab setiap langkah dengan tiga kemungkinan yang nilainya (3,12,8), (2,4,6), dan (14,5,2). Lawan akan memilih

minimum di setiap cabang, yaitu 3, 2, dan 2, sehingga kita memilih cabang pertama dengan nilai 3.

Alpha-beta pruning (Knuth and Moore, 1975) memperoleh jawaban yang sama tanpa melihat semua daun. Setelah cabang pertama memberi nilai 3, kita tahu kita bisa mendapat setidaknya 3. Di cabang kedua, begitu kita melihat daun bernilai 2, kita tahu lawan akan memberi paling banyak 2 di cabang itu, sehingga sisa daunnya tidak perlu diperiksa. Dengan urutan langkah yang baik, alpha-beta bisa menggandakan kedalaman pencarian dengan waktu yang sama.

Untuk permainan dengan cabang banyak seperti Go, minimax tidak mungkin dijalankan sampai akhir permainan. *Monte Carlo tree search* membangun pohon pencarian secara bertahap dengan memainkan banyak permainan acak sampai selesai, lalu memusatkan pencarian pada langkah yang menjanjikan. Aturan UCT (Kocsis and Szepesvári, 2006) menyeimbangkan eksploitasi langkah yang sejauh ini terbaik dengan eksplorasi langkah yang belum banyak dicoba, mengikuti teori bandit. AlphaGo (Silver et al., 2016) menggabungkan MCTS dengan jaringan saraf yang menilai posisi dan menyarankan langkah, lalu AlphaZero (Silver et al., 2018) mempelajari catur, shogi, dan Go hanya dari bermain melawan dirinya sendiri. Kuliah menekankan pelajaran yang lebih umum: pembelajaran memberi intuisi, pencarian memberi ketelitian, dan sistem terkuat memakai keduanya.

3.4 Perencanaan

Perencanaan adalah pencarian dengan bahasa yang lebih kaya. Alih-alih keadaan sebagai kotak hitam, keadaan dijelaskan dengan fakta-fakta logis, dan setiap aksi punya prasyarat dan efek. Bahasa STRIPS (Fikes and Nilsson, 1971), dan penerusnya PDDL, menulis aksi seperti “pindahkan balok A dari meja ke atas balok B” dengan prasyarat bahwa A dan B kosong di atasnya, dan efek bahwa A sekarang di atas B. Representasi ini memungkinkan heuristik umum yang tidak bergantung pada domain, misalnya dengan menyelesaikan versi masalah yang dilonggarkan, di mana efek negatif aksi diabaikan. Panjang rencana untuk masalah yang dilonggarkan memberi perkiraan biaya untuk masalah aslinya.

3.5 Pengetahuan dan logika

Representasi pengetahuan menyimpan fakta tentang dunia dalam bentuk yang bisa dipakai untuk bernalar. Logika proposisi bekerja dengan pernyataan benar atau salah, sedangkan logika predikat orde pertama menambahkan objek, relasi, dan kuantor, sehingga bisa menyatakan “setiap manusia fana”. Mesin inferensi menurunkan fakta baru dari fakta lama, misalnya dengan resolusi atau dengan rantai aturan maju dan mundur. Ontologi dan graf pengetahuan menyusun konsep ke dalam hierarki dan relasi, dan bentuk modernnya dipakai mesin pencari untuk menjawab pertanyaan faktual secara langsung. Kekuatan pendekatan simbolik adalah ketelitian dan kemampuan menjelaskan penalarannya. Kelemahannya adalah kerapuhan di hadapan dunia nyata yang berisik dan tidak lengkap, persis di tempat machine learning unggul.

3.6 Keputusan optimal: operations research

Kuliah operations research membahas keputusan yang bisa dirumuskan sebagai optimasi dengan kendala. Bentuk paling penting adalah *linear programming*: maksimalkan fungsi linear dengan kendala pertidaksamaan linear. Contoh klasik dari Hillier and Lieberman (2015) adalah pabrik yang membuat dua produk dengan keuntungan 3 dan 5 per unit, dan tiga pabrik pendukung dengan kapasitas terbatas:

$$\max 3x + 5y \quad \text{dengan} \quad x \leq 4, \quad 2y \leq 12, \quad 3x + 2y \leq 18, \quad x, y \geq 0. \quad (3.2)$$

Daerah yang memenuhi semua kendala adalah sebuah poligon, dan karena fungsi tujuannya linear, optimumnya selalu berada di salah satu titik sudut. Di sini titik terbaiknya $(x, y) = (2, 6)$ dengan keuntungan 36. Metode simpleks berjalan dari satu titik sudut ke titik sudut tetangga yang lebih baik sampai tidak ada lagi perbaikan.

Setiap program linear punya pasangan, yaitu program dual, yang variabelnya bisa dibaca sebagai harga bayangan setiap sumber daya: berapa keuntungan bertambah kalau kapasitas satu pabrik dinaikkan satu unit. Pada optimum, nilai primal dan dual sama, dan teori dualitas ini menjadi dasar banyak algoritme optimasi modern.

Kalau variabel harus bilangan bulat, misalnya jumlah truk, masalahnya menjadi *integer programming*, yang jauh lebih sulit. *Branch and bound* memecah masalah menjadi cabang dengan menambahkan kendala seperti $x \leq 2$ atau $x \geq 3$, memakai solusi relaksasi linear sebagai batas, dan membuang cabang yang batasnya sudah lebih buruk dari solusi terbaik yang ditemukan. *Dynamic programming* (Bellman, 1957) memecah keputusan berurutan menjadi submasalah dengan prinsip optimalitas: bagian akhir dari kebijakan optimal juga optimal untuk keadaan tempat bagian itu dimulai. Prinsip yang sama menjadi persamaan Bellman di reinforcement learning (Bab 12). Untuk masalah yang terlalu besar, heuristik seperti simulated annealing dan tabu search kembali menjadi pilihan, dan analisis keputusan menambahkan ketidakpastian dengan pohon keputusan dan nilai harapan.

3.7 Mengapa semua ini masih penting

Sistem AI modern jarang murni simbolik atau murni statistik. AlphaZero adalah pencarian pohon yang dipandu jaringan saraf. Model bahasa yang menyelesaikan soal matematika sering menghasilkan beberapa kemungkinan langkah lalu memilih yang terbaik, sebuah bentuk pencarian. Robot gudang memakai A* untuk rute dan pembelajaran untuk persepsi. Dalam simulasi teknik, keputusan tentang desain dan jadwal operasi sering diselesaikan dengan program linear atau integer yang dibangun di atas prediksi dari model pengganti. Pembelajaran dan penalaran tidak bersaing. Keduanya saling melengkapi.

CATATAN SUMBER. Bab ini mengikuti kuliah Introduction to AI (sejarah, search, game playing, planning, representasi pengetahuan, machine learning untuk permainan, filsafat AI) dan kuliah KS Operations Research (linear programming dan dualitas, integer programming, dynamic programming, heuristik, analisis keputusan). Buku rujukan utamanya adalah Russell and Norvig (2021) dan Hillier and Lieberman (2015).

4 Belajar Tanpa Label

Pada musim panas 2025 saya mengikuti dua kuliah yang saling melengkapi. Kuliah unsupervised learning dari Sepp Hochreiter memakai naskah panjang yang bergerak dari PCA dan ICA sampai biclustering, metode yang lahir dari analisis data ekspresi gen. Kuliah machine learning and pattern classification memberi sisi praktisnya: pengklasifikasi dasar, seleksi fitur, dan cara membangun sistem pengenalan pola dari awal sampai akhir.

Sebagian besar data di dunia tidak berlabel. Rekaman sensor, foto di ponsel, dan hasil pengukuran ekspresi ribuan gen datang tanpa keterangan tentang apa isinya. Unsupervised learning mencari struktur di dalam data seperti itu: arah-arahan yang penting, sumber-sumber tersembunyi, kelompok-kelompok alami, atau peta berdimensi rendah yang mempertahankan tetangga. Hasilnya sering menjadi langkah pertama sebelum model supervised dibangun.

4.1 Dua cara melihat PCA

PCA sudah muncul di Bab 7 sebagai eigenfaces. Kuliah unsupervised menekankan bahwa ada dua cara setara untuk menurunkannya. Cara pertama mencari arah v dengan varians proyeksi terbesar, $\max_{\|v\|=1} v^T \Sigma v$, yang jawabannya vektor eigen kovarians dengan nilai eigen terbesar. Cara kedua mencari subruang berdimensi q yang meminimalkan galat rekonstruksi kuadrat rata-rata ketika data diproyeksikan ke subruang itu lalu dikembalikan. Kedua masalah punya jawaban yang sama, karena varians total tetap, dan varians yang tidak tertangkap oleh proyeksi adalah tepat galat rekonstruksinya.

PCA unik hanya sampai tanda setiap komponen, dan kalau dua nilai eigen sama, arah-arahnya bisa diputar bebas di dalam subruang mereka. Komponen utama tidak berkorelasi satu sama lain, tetapi tidak berarti independen. PCA juga linear. Untuk struktur yang melengkung, *kernel PCA* (Schölkopf et al., 1998) memetakan data secara implisit ke ruang fitur berdimensi tinggi lewat sebuah kernel, lalu menjalankan PCA di sana. Yang dibutuhkan hanya matriks kernel antar titik data, bukan pemetaannya secara eksplisit.

4.2 Memisahkan sumber: ICA

Di sebuah pesta, dua orang berbicara bersamaan, dan dua mikrofon di sudut ruangan merekam campuran suara keduanya dengan proporsi yang berbeda. Bisakah kita mendapatkan kembali suara masing-masing orang tanpa tahu proporsi campurannya? Inilah *cocktail party problem*, dan *independent component analysis* (Hyvärinen and Oja, 2000) menjawabnya. Modelnya $x = As$, dengan s sumber yang saling independen dan A

matriks campuran yang tidak diketahui. Kita mencari matriks pemisah W sehingga $y = Wx$ sedekat mungkin dengan independen.

Kuncinya adalah teorema limit pusat. Campuran dari beberapa sumber independen cenderung lebih mirip Gauss daripada sumber-sumber aslinya. Jadi kita bisa membalik logikanya: cari arah proyeksi yang hasilnya paling *tidak* Gauss. Ketidak-Gaussian diukur dengan kurtosis atau negentropi. Langkah praktisnya dimulai dengan *whitening*, yaitu mentransformasi data agar kovariansnya menjadi matriks identitas. Setelah itu, yang tersisa hanya mencari rotasi, dan algoritme seperti FastICA atau Infomax menemukannya. ICA tidak bisa memulihkan urutan dan skala sumber, dan ia gagal kalau lebih dari satu sumber berdistribusi Gauss, karena campuran Gauss yang diputar tetap Gauss.

Perbedaan dengan PCA mudah dilihat pada sinyal suara. PCA menemukan arah varians terbesar, yang biasanya masih campuran kedua pembicara. ICA menemukan arah yang membuat setiap keluaran terdengar seperti satu orang.

4.3 Analisis faktor

Analisis faktor mirip probabilistic PCA (Bab 11), $x = Wz + \mu + \varepsilon$, dengan satu perbedaan penting: noise ε punya kovarians diagonal Ψ yang boleh berbeda untuk setiap variabel. Setiap variabel teramati punya “noise pribadinya” sendiri, dan faktor laten z menjelaskan korelasi di antara mereka. Model ini tidak punya solusi tertutup, sehingga dilatih dengan EM. Dalam data biologis, di mana setiap gen atau setiap probe punya tingkat noise yang berbeda, asumsi ini jauh lebih masuk akal daripada noise yang sama untuk semua.

4.4 Peta berdimensi rendah

Kadang yang dicari bukan sumber, melainkan peta: gambaran dua dimensi dari data berdimensi tinggi yang mempertahankan sebanyak mungkin hubungan kedekatan. Kuliah membahas sederet metode dengan filosofi yang berbeda.

Multidimensional scaling mencari titik-titik di bidang yang jarak-jaraknya sedekat mungkin dengan jarak asli. *Non-negative matrix factorization* menguraikan data menjadi bagian-bagian yang hanya boleh dijumlahkan, tidak boleh dikurangkan, sehingga komponennya sering bisa ditafsirkan sebagai bagian-bagian nyata, misalnya bagian wajah atau topik dalam teks. *Locally linear embedding* (Roweis and Saul, 2000) menganggap setiap titik bisa direkonstruksi secara linear dari tetangganya, lalu mencari koordinat rendah dimensi yang mempertahankan bobot rekonstruksi itu. Isomap (Tenenbaum et al., 2000) mengganti jarak lurus dengan jarak di sepanjang graf tetangga, sebuah pendekatan jarak geodesik, sehingga data yang tergulung seperti gulungan kue bisa dibentangkan. *Self-organizing map* (Kohonen, 1982) menempatkan grid neuron di ruang data dan menariknya ke arah data sambil menjaga neuron yang bertetangga di grid tetap berdekatan. Metode yang paling populer untuk visualisasi saat ini adalah t-SNE dan UMAP (McInnes et al., 2018), yang mencocokkan peluang ketetanggaan di ruang asli dengan di ruang peta.

Peta seperti ini menyenangkan dilihat, tetapi mudah disalahbaca. Jarak antar kelompok di peta t-SNE tidak bermakna kuantitatif, dan ukuran kelompok bisa diperbesar atau diperkecil oleh algoritmenya. Kuliah AI and Visualization kembali ke masalah ini di Bab 13.

4.5 Pengelompokan

4.5.1 *k-means*

Algoritme *k-means* (Lloyd, 1982) bergantian antara dua langkah: tetapkan setiap titik ke pusat terdekat, lalu pindahkan setiap pusat ke rata-rata titik-titik anggotanya. Setiap langkah tidak menaikkan jumlah kuadrat jarak titik ke pusatnya, sehingga algoritme pasti berhenti, meski belum tentu di solusi terbaik. Ambil empat titik di garis bilangan, 1, 2, 10, dan 11, dengan pusat awal 1 dan 2. Pada langkah pertama, titik 1 masuk ke pusat pertama, sedangkan 2, 10, dan 11 masuk ke pusat kedua, yang lalu pindah ke rata-ratanya, $23/3 \approx 7,67$. Pada langkah kedua, titik 2 sekarang lebih dekat ke pusat pertama, sehingga kelompoknya menjadi $\{1, 2\}$ dan $\{10, 11\}$, dengan pusat 1,5 dan 10,5. Langkah berikutnya tidak mengubah apa pun. Kalau pusat awalnya buruk, hasil akhirnya bisa buruk juga, sehingga *k-means* biasanya dijalankan beberapa kali dari awal acak yang berbeda.

4.5.2 *Model campuran dan EM*

k-means adalah versi keras dari model campuran Gauss. Pada model campuran, setiap titik punya peluang menjadi anggota setiap kelompok, dan kelompoknya boleh lonjong dengan kovarians sendiri. Algoritme EM bergantian antara menghitung peluang keanggotaan (langkah E) dan memperbarui rata-rata, kovarians, dan bobot setiap komponen dengan rata-rata berbobot (langkah M). Ketika kovarians semua komponen dibuat sama dan kecil sekali, keanggotaan menjadi nol atau satu, dan EM berubah menjadi *k-means*.

4.5.3 *Hierarki dan kemiripan*

Pengelompokan hierarkis tidak memilih jumlah kelompok di awal. Versi aglomeratif mulai dengan setiap titik sebagai kelompoknya sendiri, lalu berulang kali menggabungkan dua kelompok terdekat. Arti “terdekat” menentukan bentuk hasilnya. *Single linkage* memakai pasangan titik terdekat dan cenderung menghasilkan rantai panjang, *complete linkage* memakai pasangan terjauh dan menghasilkan kelompok yang padat, dan *average linkage* berada di antaranya. Hasilnya adalah dendrogram yang bisa dipotong di ketinggian mana pun. Metode berbasis kemiripan, termasuk pengelompokan spektral, memandang data sebagai graf kemiripan dan memotongnya di tempat ikatan paling lemah, sebuah ide yang sudah kita temui pada segmentasi gambar.

4.5.4 *Biclustering*

Data ekspresi gen berbentuk matriks: baris adalah gen, kolom adalah sampel jaringan atau kondisi. Sekelompok gen sering hanya bekerja sama di sebagian sampel, misalnya

hanya di jaringan tumor tertentu. Mengelompokkan seluruh baris atau seluruh kolom akan melewati pola ini. *Biclustering* mencari submatriks, yaitu sekelompok baris yang berperilaku mirip pada sekelompok kolom. FABIA (Hochreiter et al., 2010), yang dikembangkan di Linz, merumuskannya sebagai model faktor dengan prior yang jarang sekali (*sparse*), sehingga setiap faktor hanya melibatkan sedikit gen dan sedikit sampel. Inilah salah satu titik pertemuan pertama antara machine learning dan ilmu hayati di buku ini, dan kita akan kembali ke data seperti ini di Bab 19.

4.6 Dari fitur ke pengklasifikasi

Kuliah pattern classification membahas jalur lengkap sebuah sistem pengenalan pola: ambil sinyal mentah, ubah menjadi fitur, pilih fitur yang berguna, latih pengklasifikasi, lalu evaluasi dengan jujur. Contoh yang dipakai banyak berasal dari audio, seperti mendeteksi kejadian akustik dalam rekaman.

Pengklasifikasi dasar yang dibahas punya karakter yang berbeda-beda. k -nearest neighbour tidak belajar apa pun dan hanya mengingat data latih, lalu meminjam label tetangga terdekat. Naive Bayes menganggap semua fitur independen bersyarat terhadap kelas, asumsi yang hampir selalu salah tetapi sering bekerja dengan baik. Analisis diskriminan linear (FISHER, 1936) mencari arah proyeksi yang memisahkan rata-rata kelas sejauh mungkin relatif terhadap sebaran di dalam kelas. Pohon keputusan memotong ruang fitur dengan pertanyaan ya atau tidak yang berurutan, dan random forest (Breiman, 2001) merata-ratakan banyak pohon yang dilatih pada sampel dan fitur acak. Support vector machine (Cortes and Vapnik, 1995) mencari bidang pemisah dengan margin terlebar, dan dengan kernel ia bisa memisahkan kelas yang tidak terpisah secara linear. Ide marginnya adalah regularisasi dalam bentuk geometris: di antara semua bidang yang memisahkan data latih, pilih yang paling jauh dari titik mana pun.

Seleksi fitur bisa dilakukan dengan tiga cara. Metode filter menilai setiap fitur sendiri-sendiri, misalnya dari korelasinya dengan label. Metode wrapper mencoba berbagai himpunan fitur dengan melatih model sungguhan. Metode embedded memilih fitur sebagai bagian dari pelatihan, seperti lasso yang mendorong bobot fitur yang tidak berguna ke nol. Kuliah juga memperkenalkan jaringan saraf dengan cara pandang praktis: kapan lapisan dense cukup, kapan konvolusi lebih cocok, dan bagaimana transfer learning serta pretraining self-supervised mengurangi kebutuhan data berlabel.

4.7 Di luar kelas

Metode tanpa label muncul di hampir setiap bidang yang disentuh buku ini. Dalam ilmu hayati, pengelompokan profil ekspresi gen memisahkan subtype kanker yang tidak terlihat di bawah mikroskop, dan *biclustering* menemukan modul gen yang aktif bersama. Dalam pemrosesan sinyal, ICA memisahkan sinyal jantung ibu dan janin dari elektroda yang sama, dan PCA memampatkan data sensor sebelum dikirim. Dalam simulasi, PCA dengan nama proper orthogonal decomposition mereduksi medan aliran berjuta sel menjadi beberapa puluh mode (Bab 27). Dalam sistem rekomendasi, faktorisasi matriks yang mirip analisis faktor menemukan selera tersembunyi pengguna

(Bab 30). Di semua tempat itu, pertanyaannya sama: struktur apa yang tersembunyi di balik angka-angka ini?

CATATAN SUMBER. Bab ini mengikuti naskah kuliah Machine Learning: Unsupervised Techniques (PCA dan kernel PCA, ICA, analisis faktor, metode proyeksi, clustering, biclustering) dan kuliah Machine Learning and Pattern Classification (pengklasifikasi dasar, seleksi dan konstruksi fitur, jaringan saraf dalam praktik). Bacaan pendamping: Bishop (2006), Hastie et al. (2009), dan Duda et al. (2001).

Bagian II

Deep Learning dan Persepsi

5 Deep Learning: Teknik Dasar

Ini kuliah deep learning pertama saya di Linz, pada semester musim dingin 2024/25. Naskah kuliahnya lebih mirip buku daripada slide, lengkap dengan sejarah, notasi yang rapi, dan bab khusus tentang inisialisasi dan normalisasi.

Bab ini membangun jaringan saraf dari satu neuron sampai jaringan konvolusi. Empat pertanyaan menjadi benang merahnya: apa yang bisa dihitung oleh sebuah jaringan, bagaimana gradiennya dihitung, bagaimana sinyal dijaga agar tidak mati atau meledak di jaringan yang dalam, dan bagaimana mencegah jaringan sekadar menghafal.

5.1 Sejarah singkat

McCulloch and Pitts (1943) memodelkan neuron sebagai unit ambang: jumlahkan input yang berbobot, lalu keluarkan satu kalau jumlahnya melewati ambang. Rosenblatt (1958) membuat *perceptron*, yang bisa mempelajari bobotnya sendiri. Antusiasme sempat surut ketika menjadi jelas bahwa satu lapisan perceptron tidak bisa menyelesaikan masalah sesederhana XOR. Algoritme backpropagation untuk jaringan berlapis kemudian dipopulerkan oleh Rumelhart et al. (1986), meski gagasannya sudah muncul lebih awal. Lecun et al. (1998) memakai jaringan konvolusi untuk membaca tulisan tangan, dan ledakan modern dimulai ketika jaringan konvolusi besar yang dilatih di GPU memenangkan kompetisi ImageNet (Krizhevsky et al., 2017). Sejarah yang lebih lengkap, termasuk peran Linz melalui LSTM, ditulis oleh Schmidhuber (2015).

5.2 Dari satu neuron ke jaringan berlapis

Satu neuron menghitung *pre-activation* $s = \mathbf{w}^\top \mathbf{x} + b$, lalu *activation* $a = f(s)$. Kalau f fungsi tangga, kita mendapat perceptron. Kalau f sigmoid $\sigma(s) = 1/(1 + e^{-s})$, keluarannya bisa dibaca sebagai peluang, dan model ini tidak lain adalah regresi logistik. Dalam kedua kasus, batas keputusannya, $\mathbf{w}^\top \mathbf{x} + b = 0$, selalu berupa bidang datar.

Mengapa satu bidang datar tidak cukup untuk XOR bisa dibuktikan dalam beberapa baris. Titik (0,0) dan (1,1) berlabel nol, sedangkan (0,1) dan (1,0) berlabel satu. Misalkan ada garis $w_1x_1 + w_2x_2 + b = 0$ yang memisahkannya, dengan label satu di sisi positif. Titik (0,0) memberi $b < 0$. Titik (0,1) dan (1,0) memberi $w_2 + b > 0$ dan $w_1 + b > 0$. Kalau keduanya dijumlahkan, diperoleh $w_1 + w_2 + b > -b > 0$. Padahal (1,1) berlabel nol, sehingga seharusnya $w_1 + w_2 + b < 0$. Kontradiksi. Satu lapisan tersembunyi dengan dua neuron sudah cukup untuk menyelesaikannya.

Jaringan *feed-forward* dengan L lapisan menumpuk transformasi yang sama berkali-kali. Dengan notasi kuliah,

$$\mathbf{s}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = f(\mathbf{s}^{(l)}), \quad \mathbf{a}^{(0)} = \mathbf{x}, \quad \hat{\mathbf{y}} = \mathbf{a}^{(L)}. \quad (5.1)$$

Tanpa fungsi aktivasi yang nonlinear, tumpukan lapisan ini hanyalah satu perkalian matriks besar. Dengan nonlinearitas, *universal approximation theorem* (Cybenko, 1989; Hornik et al., 1989) menjamin bahwa satu lapisan tersembunyi dengan cukup banyak neuron bisa mendekati fungsi kontinu apa pun pada domain yang kompak. Teorema ini tidak mengatakan berapa banyak neuron yang dibutuhkan, dan juga tidak menjamin gradient descent akan menemukannya. Dalam praktik, jaringan yang dalam memakai parameter jauh lebih hemat daripada jaringan yang lebar tetapi dangkal.

Cara saya membayangkannya adalah origami. Lapisan dengan aktivasi ReLU melipat ruang input sepanjang beberapa garis lipatan. Lapisan berikutnya melipat hasil lipatan itu sekali lagi. Dengan beberapa kali lipatan, selembar kertas bisa berubah menjadi bentuk yang rumit, dan jumlah bagian datar yang terbentuk bertambah jauh lebih cepat dengan menambah lipatan baru daripada dengan memperlebar kertas.

5.3 Backpropagation

Untuk melatih jaringan dengan gradient descent, kita membutuhkan $\partial \mathcal{L} / \partial \mathbf{W}^{(l)}$ untuk semua lapisan. Backpropagation adalah aturan rantai yang dihitung dengan urutan yang cerdas: dari belakang ke depan, sambil menyimpan hasil antara. Definisikan *delta* lapisan l sebagai $\delta^{(l)} = \partial \mathcal{L} / \partial \mathbf{s}^{(l)}$. Maka

$$\delta^{(L)} = f'(\mathbf{s}^{(L)}) \odot \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}}, \quad (5.2)$$

$$\delta^{(l)} = f'(\mathbf{s}^{(l)}) \odot (\mathbf{W}^{(l+1)})^\top \delta^{(l+1)}, \quad (5.3)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}. \quad (5.4)$$

Bayangkan sebuah pabrik yang menghasilkan produk cacat. Manajer di ujung jalur produksi melihat seberapa besar cacatnya, lalu membagi kesalahan itu ke para pekerja di stasiun sebelumnya, sebanding dengan seberapa besar sumbangan mereka pada produk akhir (bobot $\mathbf{W}^\top$) dan seberapa peka mereka saat itu (f'). Setiap pekerja meneruskan bagian kesalahannya ke stasiun sebelumnya dengan cara yang sama. Tidak ada yang dihitung dua kali, dan itulah sebabnya backprop murah.

Contoh dengan angka membuatnya konkret. Ambil input $x = 1$, satu neuron tersembunyi $a_1 = \sigma(w_1 x)$ dengan $w_1 = 0,5$, keluaran linear $\hat{y} = w_2 a_1$ dengan $w_2 = 2$, target $y = 1$, dan loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$. Lintasan maju memberi $s_1 = 0,5$, $a_1 = \sigma(0,5) = 0,6225$, $\hat{y} = 1,2449$, dan $\mathcal{L} = 0,0300$. Delta keluaran adalah $\delta_2 = \hat{y} - y = 0,2449$, sehingga $\partial \mathcal{L} / \partial w_2 = \delta_2 a_1 = 0,1525$. Untuk lapisan tersembunyi, $\sigma'(s_1) = a_1(1 - a_1) = 0,2350$, sehingga $\delta_1 = 0,2350 \times w_2 \times \delta_2 = 0,1151$ dan $\partial \mathcal{L} / \partial w_1 = \delta_1 x = 0,1151$. Pengecekan dengan beda hingga selalu baik dilakukan saat menulis backprop sendiri: naikkan w_1 sebesar 0,001, dan loss seharusnya naik kira-kira $0,001 \times 0,1151$.

Kerangka kerja modern tidak menurunkan rumus delta secara manual. Mereka memakai *automatic differentiation* (Baydin et al., 2015). Program dipecah menjadi operasi dasar yang turunannya diketahui, lalu turunan dirangkai dengan salah satu dari dua cara. *Forward mode* membawa turunan terhadap satu input maju bersama perhitungan, cocok kalau input sedikit dan output banyak. *Reverse mode* mencatat semua operasi lalu

berjalan mundur, cocok kalau outputnya sedikit, misalnya satu nilai loss, dan inputnya banyak, misalnya jutaan bobot. Backpropagation adalah reverse mode yang diterapkan pada jaringan saraf. Biayanya hanya beberapa kali biaya satu lintasan maju, berapa pun jumlah parameternya.

Autodiff tidak hanya bisa menghitung turunan loss terhadap bobot. Ia juga bisa menghitung turunan keluaran jaringan terhadap *input*. Kalau sebuah jaringan $u_\theta(x, t)$ memodelkan temperatur, autodiff memberi $\partial u_\theta / \partial t$ dan $\partial^2 u_\theta / \partial x^2$ secara eksak, tanpa grid dan tanpa beda hingga. Kemampuan sederhana inilah yang membuat PINN mungkin: residu persamaan panas $\partial_t u - \kappa \partial_{xx} u$ bisa dihitung langsung di titik mana pun (Bab 25).

Kuliah juga memperkenalkan dua objek turunan yang lebih umum. Jacobian $J = \partial \hat{\mathbf{y}} / \partial \mathbf{x}$ mengukur kepekaan setiap keluaran terhadap setiap input, dan backprop pada dasarnya mengalikan Jacobian lapisan-lapisan dari kanan ke kiri dengan sebuah vektor. Hessian $\mathbf{H} = \partial^2 \mathcal{L} / \partial \mathbf{w} \partial \mathbf{w}^\top$ mengukur kelengkungan loss dan menentukan condition number masalah optimasi (Bab 2). Hasil kali Hessian dengan sebuah vektor bisa dihitung tanpa pernah membentuk Hessian, cukup dengan dua lintasan autodiff.

5.4 Fungsi aktivasi

Sigmoid dan tanh adalah fungsi aktivasi klasik, tetapi keduanya jenuh di kedua sisi: untuk input yang besar, turunannya hampir nol, dan turunan sigmoid tidak pernah melebihi seperempat. ReLU, $\max(0, s)$, murah dan tidak jenuh untuk input positif, tetapi neuron bisa “mati” kalau pre-activation-nya selalu negatif. ELU memperhalus bagian negatif dengan $\alpha(e^s - 1)$ sehingga rata-rata keluarannya lebih dekat ke nol. SELU adalah ELU yang diskalakan dengan konstanta tertentu, $\lambda \approx 1,0507$ dan $\alpha \approx 1,6733$, dan kita akan melihat sebentar lagi mengapa angka-angka itu istimewa. Transformer umumnya memakai GELU atau SiLU, yang mulus di sekitar nol.

Pasangan aktivasi keluaran dan loss ditentukan oleh jenis tugas, sesuai Tabel 1.1: keluaran linear dengan MSE untuk regresi, sigmoid dengan binary cross-entropy untuk dua kelas, dan softmax dengan cross-entropy untuk banyak kelas. Pasangan ini dipilih karena delta keluarannya menjadi sederhana sekali. Untuk softmax dengan cross-entropy, misalnya, $\delta^{(L)} = \hat{\mathbf{y}} - \mathbf{y}$.

Hasil ini layak diturunkan sekali, karena ia muncul di setiap pengklasifikasi. Dengan $\hat{y}_k = e^{s_k} / \sum_j e^{s_j}$ dan label one-hot $\mathbf{y}$, loss-nya $\mathcal{L} = -\sum_k y_k \log \hat{y}_k = -\sum_k y_k s_k + \log \sum_j e^{s_j}$, karena $\sum_k y_k = 1$. Turunan suku pertama terhadap s_i adalah $-y_i$. Turunan suku kedua adalah $e^{s_i} / \sum_j e^{s_j} = \hat{y}_i$. Jadi $\partial \mathcal{L} / \partial s_i = \hat{y}_i - y_i$. Turunan softmax yang rumit, dengan semua suku silangnya, saling menghapus dengan logaritma dari cross-entropy. Sebagai contoh, kalau tiga kelas memberi $\hat{\mathbf{y}} = (0,7; 0,2; 0,1)$ dan kelas yang benar adalah yang pertama, deltanya $(-0,3; 0,2; 0,1)$: skor kelas yang benar didorong naik, skor kelas lain didorong turun sebanding dengan peluang yang keliru diberikan kepadanya.

5.5 Menjaga sinyal tetap hidup

Di jaringan yang dalam, sinyal maju dan gradien mundur melewati banyak perkalian matriks. Kalau setiap lapisan sedikit mengecilkan sinyal, setelah lima puluh lapisan sinyalnya hilang. Kalau setiap lapisan sedikit membesarkannya, sinyal meledak. Permainan bisik berantai memberi gambaran yang tepat. Kalau setiap orang berbisik sedikit lebih pelan dari yang ia dengar, pesan lenyap setelah dua puluh orang. Kalau setiap orang berbicara sedikit lebih keras, orang di ujung barisan sudah berteriak. Inisialisasi yang baik membuat setiap orang mengulang pesan dengan volume yang sama.

Kuliah membahas hal ini dengan *mean field theory*, yaitu melacak rata-rata dan varians aktivasi dari lapisan ke lapisan. Ambil satu pre-activation, $s_i = \sum_{j=1}^J w_{ij} a_j$. Kalau bobot diambil i.i.d. dengan rata-rata nol dan varians $\text{Var}(w)$, independen dari aktivasi, maka setiap suku punya rata-rata nol dan varians $\mathbb{E}[w^2 a^2] = \text{Var}(w) \mathbb{E}[a^2]$. Varians jumlah J suku independen adalah jumlah variansnya, sehingga

$$\text{Var}(s_i) = J \text{Var}(w) \mathbb{E}[a^2]. \quad (5.5)$$

Agar varians tidak berubah dari lapisan ke lapisan, kita butuh $J \text{Var}(w) \approx 1$ untuk aktivasi yang hampir linear di sekitar nol, yaitu inisialisasi LeCun, $\text{Var}(w) = 1/J$. Glorot meratakan kebutuhan arah maju dan mundur menjadi $\text{Var}(w) = 2/(J + I)$, dengan I jumlah output. Untuk ReLU, $a = \max(0, s)$. Kalau s simetris di sekitar nol, separuh nilainya dipotong menjadi nol dan separuh lainnya dibiarkan, sehingga $\mathbb{E}[a^2] = \frac{1}{2} \mathbb{E}[s^2] = \frac{1}{2} \text{Var}(s)$. Masukkan ke rumus di atas: varians terjaga kalau $J \text{Var}(w) \cdot \frac{1}{2} = 1$, yaitu $\text{Var}(w) = 2/J$ (He et al., 2015). Untuk lapisan dengan 512 input, simpangan baku bobotnya $\sqrt{2/512} = 0,0625$. Dengan $\sqrt{1/512} \approx 0,044$ yang cocok untuk tanh, varians sinyal di jaringan ReLU akan menyusut separuh di setiap lapisan, dan setelah dua puluh lapisan tinggal sekitar sepersejuta.

Inisialisasi menjaga sinyal di awal pelatihan, sedangkan normalisasi menjaganya terus-menerus. *Batch normalization* (Ioffe and Szegedy, 2015) menormalkan setiap pre-activation dengan rata-rata dan simpangan baku di dalam mini-batch, lalu memberi skala γ dan geser β yang bisa dipelajari:

$$\hat{s} = \frac{s - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \quad y = \gamma \hat{s} + \beta. \quad (5.6)$$

Saat inferensi, statistik batch diganti rata-rata bergerak yang dikumpulkan selama pelatihan. *Layer normalization* melakukan hal yang serupa, tetapi rata-ratanya dihitung atas neuron dalam satu contoh, bukan atas contoh dalam satu batch. Karena tidak bergantung pada ukuran batch, layer norm menjadi standar di jaringan rekuren dan transformer.

Pendekatan yang khas Linz adalah *self-normalizing networks* (Klambauer et al., 2017). Dengan aktivasi SELU dan inisialisasi yang tepat, pemetaan pasangan (rata-rata, varians) dari satu lapisan ke lapisan berikutnya punya titik tetap yang stabil dan menarik di $(0, 1)$. Kalau aktivasi menyimpang dari titik itu, lapisan berikutnya menariknya kembali. Jaringan menormalkan dirinya sendiri tanpa lapisan normalisasi tambahan, dan konstanta λ serta α pada SELU dipilih tepat agar titik tetap itu ada.

5.6 Regularisasi

Jaringan modern punya parameter jauh lebih banyak daripada jumlah contoh. Tanpa pengekanan, ia bisa menghafal. Kuliah membahas beberapa cara mengekangnya. Weight decay, suku $\frac{\lambda}{2}\|w\|^2$ pada loss, setara dengan prior Gauss pada bobot (Bab 1). Menambahkan noise pada neuron atau bobot selama pelatihan memaksa jaringan tahan terhadap gangguan kecil, dan untuk model linear, noise Gauss pada input setara dengan regularisasi L2. Dropout mematikan setiap neuron dengan peluang p selama pelatihan (Hinton et al., 2012), sehingga jaringan tidak bisa bergantung pada satu neuron tertentu. Saat inferensi semua neuron aktif dan keluaran diskalakan, dan dropout bisa dibaca sebagai melatih ensemble amat besar dari sub-jaringan yang berbagi bobot. Early stopping menghentikan pelatihan ketika galat validasi mulai naik. Data augmentation memperbanyak data dengan transformasi yang tidak mengubah label, seperti menggeser, memutar, atau mengubah kecerahan. Ada juga cara yang mengubah ukuran jaringan selama pelatihan, yaitu menambah neuron (*growing*) atau membuang bobot yang kurang penting (*pruning*), serta pencarian eksplisit akan minimum yang datar (Hochreiter and Schmidhuber, 1997b).

Dua kesalahan praktis sering terjadi di sini. Yang pertama adalah lupa mematikan dropout dan lupa memakai statistik batch norm hasil pelatihan saat evaluasi. Di PyTorch, inilah beda antara `model.train()` dan `model.eval()`. Yang kedua adalah augmentasi yang diam-diam mengubah label. Membalik angka 6 bisa menjadikannya 9. Pada data fisika, memutar medan kecepatan tanpa ikut memutar arah vektor-vektor kecepatannya juga menghasilkan data yang salah.

5.7 Jaringan konvolusi

Gambar berukuran 256×256 dengan tiga kanal punya hampir dua ratus ribu input. Lapisan fully-connected ke seribu neuron akan membutuhkan dua ratus juta bobot. Jaringan konvolusi memangkas angka ini dengan dua asumsi: pola penting bersifat lokal, dan pola yang sama bisa muncul di mana saja. Satu *kernel* kecil, misalnya 3×3 , digeser ke seluruh gambar,

$$(a * k)_{ij} = \sum_{u,v} a_{i+u, j+v} k_{uv}, \quad (5.7)$$

dan bobot yang sama dipakai di semua posisi (*weight sharing*). Secara matematis, operasi ini adalah *cross-correlation*, karena konvolusi sejati membalik kernelnya lebih dulu. Karena kernel dipelajari, perbedaan itu tidak penting.

Bayangkan Anda mencari satu tanda tangan di sebuah dokumen besar dengan kaca pembesar kecil. Anda menggeser kaca pembesar ke seluruh halaman dan mencatat seberapa mirip isi di bawahnya dengan tanda tangan yang dicari. Anda tidak butuh kaca pembesar yang berbeda untuk setiap posisi. Setiap kernel konvolusi adalah satu kaca pembesar yang mencari satu jenis pola.

Untuk input berukuran I , kernel K , padding P , dan stride S , ukuran keluarannya adalah

$$O = \left\lfloor \frac{I + 2P - K}{S} \right\rfloor + 1, \quad (5.8)$$

dan satu lapisan dengan C_{in} kanal masuk serta C_{out} kanal keluar punya $K^2 C_{in} C_{out} + C_{out}$ parameter, tidak bergantung pada ukuran gambar. Rumus ukuran keluaran mudah diturunkan dengan menghitung posisi. Setelah padding, panjang input menjadi $I + 2P$. Kernel sepanjang K pertama kali diletakkan di posisi paling kiri, dan ruang gerakanya sampai mentok di kanan adalah $I + 2P - K$ piksel. Dengan langkah S , banyaknya langkah yang muat adalah $\lfloor (I + 2P - K)/S \rfloor$, ditambah satu untuk posisi awal. Jumlah parameternya berasal dari satu kernel $K \times K$ untuk setiap pasangan kanal masuk dan kanal keluar, ditambah satu bias per kanal keluar. LeNet memberi contoh klasik (Lecun et al., 1998). Input 32×32 dengan kernel 5×5 tanpa padding menghasilkan keluaran 28×28 . Dengan enam kernel, lapisan pertama punya $5 \cdot 5 \cdot 6 + 6 = 156$ parameter, dan pooling 2×2 dengan stride dua mengecilkan peta fitur menjadi 14×14 .

Pooling mengambil maksimum atau rata-rata di jendela kecil. Ia mengecilkan peta fitur dan memberi sedikit ketahanan terhadap pergeseran kecil. *Receptive field* sebuah neuron adalah daerah input yang memengaruhinya, dan setiap lapisan konvolusi atau pooling memperluasnya. Lapisan awal melihat tepi dan tekstur, lapisan tengah melihat bagian objek, lapisan akhir melihat objek utuh. Kalau kernel lapisan pertama sebuah jaringan terlatih divisualisasikan, hasilnya hampir selalu detektor tepi dan bercak warna, mirip sel-sel di korteks visual primer.

Karena konvolusi linear, backprop melaluinya juga berupa konvolusi. Gradien terhadap input adalah konvolusi delta dengan kernel yang dibalik, operasi yang disebut *transposed convolution*. Gradien terhadap kernel adalah korelasi antara input dan delta yang dijumlahkan atas semua posisi, karena kernel yang sama dipakai di mana-mana. Transposed convolution juga dipakai sebagai lapisan untuk memperbesar resolusi, misalnya di bagian decoder U-Net (Bab 8).

Sifat terpenting konvolusi adalah ini: kalau input digeser, keluarannya ikut bergeser dengan cara yang sama. Untuk operator geser T , berlaku $(Ta) * k = T(a * k)$. Sifat ini disebut *translation equivariance*. Pooling global di ujung jaringan mengubahnya menjadi *invariance*: label tidak berubah kalau objeknya digeser. Bab 9 akan menunjukkan bahwa konvolusi adalah satu-satunya operator linear yang punya sifat ini, lalu menggeneralisasikannya ke rotasi, permutasi, dan graf.

5.8 Konvolusi di mana-mana

Konvolusi jauh lebih tua dari jaringan saraf, dan hampir setiap bidang di buku ini punya versinya sendiri. Dalam pemrosesan sinyal, filter FIR yang meratakan atau menajamkan sinyal adalah konvolusi satu dimensi dengan kernel yang dirancang insinyur (Bab 14). Dalam pengolahan citra klasik, filter Gauss dan Sobel adalah kernel yang dirancang dengan tangan (Bab 7). Dalam genomik, jaringan konvolusi satu dimensi yang digeser sepanjang urutan DNA mempelajari motif pendek tempat protein menempel, persis seperti kaca pembesar yang mencari tanda tangan (Bab 18). Dalam sensor yang dikenakan, konvolusi pada sinyal akselerometer mempelajari pola gerak (Bab 16).

Dalam simulasi, hubungannya langsung terasa bagi siapa pun yang pernah menulis

solver numerik. Kernel

$$\mathbf{k} = \frac{1}{h^2} \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

adalah stensil lima titik untuk Laplacian $\nabla^2 u$ pada grid berjarak h . Kernel $\frac{1}{2h}(-1, 0, 1)$ adalah turunan pusat. Dengan kata lain, solver beda hingga pada grid teratur adalah jaringan konvolusi dengan kernel yang sudah ditentukan oleh fisika. Jaringan konvolusi yang dilatih pada data simulasi bisa dipandang sebagai solver yang mempelajari stensilnya sendiri. Pandangan ini menjelaskan mengapa CNN dan U-Net menjadi pilihan pertama untuk model pengganti pada grid teratur, dan mengapa mesh yang tidak teratur membutuhkan jaringan graf (Bab 9 and 26).

CATATAN SUMBER. Bab ini mengikuti naskah kuliah Deep Learning and Neural Nets I, yang kini bernama Deep Learning: Basic Techniques: sejarah, notasi, arsitektur dasar, generalisasi, jaringan konvolusi, metode belajar, regularisasi, inisialisasi, dan normalisasi. Bacaan pendamping: Goodfellow et al. (2016), Prince (2023), dan ulasan LeCun et al. (2015).

6 Mengingat: dari RNN ke LSTM dan Transformer

Di semester pertama, kuliah LSTM diberikan langsung oleh Sepp Hochreiter, yang menganalisis masalah vanishing gradient pada 1991 dan bersama Jürgen Schmidhuber menerbitkan LSTM pada 1997. Kuliahnya tidak berhenti di sejarah. Bagian akhirnya membahas xLSTM, versi modern yang dikembangkan di Linz untuk bersaing dengan transformer. Setahun kemudian, kuliah NLP melengkapi sisi lain ceritanya: embedding kata dan model bahasa besar.

Banyak data datang berurutan: kata dalam kalimat, detak jantung, harga saham, langkah waktu sebuah simulasi. Untuk data seperti ini, model harus punya ingatan. Bab ini mengikuti evolusi ingatan buatan, dari jaringan rekuren sederhana yang cepat lupa, ke LSTM yang menjaga ingatan dengan gerbang, ke transformer yang melihat seluruh masa lalu sekaligus, lalu kembali lagi ke ingatan rekuren versi modern.

6.1 Jaringan rekuren

Jaringan feed-forward memproses setiap input sendiri-sendiri. *Recurrent neural network* (RNN) membawa sebuah keadaan tersembunyi h_t dari satu langkah ke langkah berikutnya:

$$h_t = f(Wx_t + Rh_{t-1} + b), \quad \hat{y}_t = g(Vh_t). \quad (6.1)$$

Matriks R adalah bobot rekuren. Bobot yang sama dipakai di setiap langkah waktu, persis seperti kernel konvolusi dipakai di setiap posisi gambar. Saat membaca novel, kita tidak mengingat setiap kata. Kita membawa ringkasan berjalan di kepala: siapa tokohnya, apa masalahnya, apa yang baru saja terjadi. Setiap kalimat baru memperbarui ringkasan itu. Keadaan tersembunyi h_t adalah ringkasan berjalan tersebut.

Kuliah membahas beberapa varian klasik. Jaringan Jordan mengumpankan kembali keluarannya, jaringan Elman mengumpankan kembali lapisan tersembunyinya, jaringan rekuren penuh menghubungkan semua unit, jaringan NARX memakai input dan keluaran yang tertunda, dan *time-delay neural network* hanya melihat jendela waktu yang tetap.

Untuk melatih RNN, *backpropagation through time* (BPTT) membuka jaringan menjadi jaringan feed-forward yang dalam, satu lapisan untuk setiap langkah waktu, lalu menjalankan backprop biasa. Karena bobotnya dipakai bersama, gradien untuk R adalah jumlah kontribusi dari semua langkah. Untuk deret yang panjang, BPTT dipotong setelah k langkah supaya memori dan waktu tetap terbatas. Alternatifnya adalah *real-time recurrent learning* (RTRL), yang membawa turunan keadaan terhadap semua bobot maju bersama waktu. Gradien tersedia di setiap langkah tanpa menyimpan masa lalu,

tetapi harganya mahal: untuk jaringan rekuren penuh dengan n unit, biayanya tumbuh seperti n^4 per langkah. Dalam bahasa Bab 5, RTRL adalah forward mode, sedangkan BPTT adalah reverse mode.

6.2 Gradien yang hilang dan meledak

Masalah berikut mengubah sejarah jaringan saraf. Delta di langkah $t - k$ bergantung pada delta di langkah t melalui perkalian k buah Jacobian:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-k}} = \prod_{\tau=t-k+1}^t \frac{\partial \mathbf{h}_\tau}{\partial \mathbf{h}_{\tau-1}} = \prod_{\tau} \text{diag}(f'(s_\tau)) \mathbf{R}. \quad (6.2)$$

Kalau norma setiap faktor lebih kecil dari satu, hasil kalinya menyusut secara eksponensial. Inilah *vanishing gradient*. Kalau lebih besar dari satu, ia meledak. Hochreiter menganalisis gejala ini pada 1991 dalam tesis diplamanya (Hochreiter, 1991), dan Bengio et al. (1994) menunjukkan secara independen bahwa ketergantungan jangka panjang memang sulit dipelajari dengan gradient descent.

Seberapa cepat gradien hilang? Ambil RNN skalar $h_t = \tanh(r h_{t-1} + w x_t)$ dengan $r = 0,5$. Karena turunan tanh tidak pernah melebihi satu, setiap langkah mengalikan gradien dengan paling banyak 0,5. Setelah dua puluh langkah, faktornya paling besar $0,5^{20}$, kurang dari satu per sejuta. Informasi dari dua puluh langkah sebelumnya praktis tidak lagi memengaruhi perubahan bobot. Dengan $r = 1,5$ dan unit yang bekerja di daerah linearnya, faktor yang sama bisa mencapai sekitar 3300.

Gradien yang meledak relatif mudah ditangani dengan *gradient clipping*: kalau norma gradien melewati batas tertentu, skalakan turun. Gradien yang hilang lebih sulit, karena tidak ada sinyal yang bisa diperbesar. Solusinya harus datang dari arsitektur.

6.3 LSTM

Hochreiter and Schmidhuber (1997a) memecahkan masalah ini dengan dua gagasan yang di kuliah disebut sebagai inti LSTM: *constant error carousel* dan *gating*.

Gagasan pertama sederhana. Bayangkan satu unit ingatan c_t yang diperbarui dengan penjumlahan, bukan perkalian, yaitu $c_t = c_{t-1} + (\text{sesuatu yang baru})$. Turunannya $\partial c_t / \partial c_{t-1}$ bernilai tepat satu. Error yang mengalir mundur lewat c tidak menyusut dan tidak membesar. Ia berputar tanpa berubah, seperti penumpang di komidi putar.

Penjumlahan tanpa kendali akan mengisi ingatan dengan sampah. Karena itu LSTM memberi gerbang yang dipelajari. Gerbang input memutuskan apa yang boleh masuk, dan gerbang output memutuskan apa yang boleh dibaca. Gerbang lupa (*forget gate*) ditambahkan kemudian oleh Gers et al. (2000) agar sel bisa mengosongkan dirinya. Versi yang kini dipakai di mana-mana berbentuk

$$\begin{aligned} z_t &= \tanh(\mathbf{W}_z \mathbf{x}_t + \mathbf{R}_z \mathbf{h}_{t-1} + \mathbf{b}_z), & i_t &= \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{R}_i \mathbf{h}_{t-1} + \mathbf{b}_i), \\ f_t &= \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{R}_f \mathbf{h}_{t-1} + \mathbf{b}_f), & o_t &= \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{R}_o \mathbf{h}_{t-1} + \mathbf{b}_o), \\ c_t &= f_t \odot c_{t-1} + i_t \odot z_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned} \quad (6.3)$$

Di sini z_t adalah input sel, i_t , f_t , dan o_t adalah gerbang input, lupa, dan output, dan c_t adalah keadaan sel. Sepanjang jalur sel, $\partial c_t / \partial c_{t-1} = f_t$. Selama gerbang lupa dekat satu, error mengalir hampir tanpa hambatan, dan jaringan sendiri yang memutuskan kapan harus melupakan.

Saya membayangkan keadaan sel sebagai buku catatan. Gerbang input adalah stabilo: hanya hal yang ditandai yang ditulis ke buku. Gerbang lupa adalah penghapus, karena kadang halaman lama perlu dibersihkan. Gerbang output adalah keputusan tentang halaman mana yang dibacakan sekarang. Tulisan di buku tidak memudar dengan sendirinya. Ia hanya berubah kalau ditulis atau dihapus.

Gambaran ini juga membantu menghindari beberapa salah paham. LSTM tidak menghilangkan semua masalah gradien. Jalur sel memang aman, tetapi jalur yang melewati gerbang dan h masih bisa mengalami vanishing. LSTM membuat belajar jangka panjang menjadi mungkin, bukan otomatis. Bias gerbang lupa juga sering diinisialisasi positif, misalnya satu, supaya di awal pelatihan sel cenderung mengingat. Dan pembagian peran tanh dan sigmoid bukan kebetulan: gerbang memakai sigmoid karena nilainya harus berada di antara nol dan satu, seperti keran, sedangkan input sel memakai tanh karena isinya boleh positif atau negatif.

LSTM mendominasi pemrosesan deret selama dua dekade, dari pengenalan suara di ponsel sampai terjemahan mesin. Kuliah juga menampilkan contoh di luar bahasa, seperti klasifikasi protein dari urutan asam amino dan analisis interval detak jantung.

6.4 Attention dan transformer

LSTM memampatkan seluruh masa lalu ke dalam vektor berukuran tetap. Untuk menerjemahkan kalimat yang panjang, pemampatan ini menjadi leher botol. Bahdanau et al. (2014) membiarkan dekoder melihat kembali semua keadaan enkoder dan memilih yang relevan. Mekanisme ini disebut *attention*. Transformer (Vaswani et al., 2017) kemudian membuang rekurensi sama sekali dan hanya memakai attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \quad Q = XW_Q, K = XW_K, V = XW_V. \quad (6.4)$$

Baris ke- i hasilnya adalah rata-rata berbobot dari semua *value*, dengan bobot dari kemiripan *query* ke- i dengan setiap *key*.

Perpustakaan adalah gambaran yang pas. Anda datang dengan sebuah pertanyaan, yaitu query. Setiap buku punya label di punggungnya, yaitu key, dan isi, yaitu value. Anda membandingkan pertanyaan dengan semua label, lalu membaca isi buku dengan perhatian yang sebanding dengan kecocokannya. Buku yang labelnya cocok sekali dibaca serius, yang tidak cocok hanya dilirik. Jawaban Anda adalah campuran dari isi buku-buku itu.

Sebagai contoh kecil, ambil query $q = (1, 0)$, dua key $k_1 = (1, 0)$ dan $k_2 = (0, 1)$, value $v_1 = 10$ dan $v_2 = 0$, serta $d_k = 2$. Skornya $q \cdot k_1 / \sqrt{2} \approx 0,707$ dan 0. Softmax mengubahnya menjadi bobot 0,670 dan 0,330, karena $e^{0,707} \approx 2,028$ dibagi $2,028 + 1$. Keluarannya 6,70. Pembagian dengan $\sqrt{d_k}$ menjaga skor tidak terlalu besar saat dimensinya tinggi, supaya softmax tidak jenuh menjadi pilihan tunggal yang keras.

Mengapa tepat $\sqrt{d_k}$? Anggap komponen q_i dan k_i saling independen dengan rata-rata nol dan varians satu. Hasil kali titik $\mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$ adalah jumlah d_k suku yang masing-masing punya rata-rata nol dan varians $\mathbb{E}[q_i^2]\mathbb{E}[k_i^2] = 1$, sehingga variansnya d_k dan simpangan bakunya $\sqrt{d_k}$. Untuk $d_k = 64$, skor tipikal berukuran sekitar delapan. Selisih delapan pada input softmax berarti rasio bobot $e^8 \approx 3000$, sehingga hampir semua bobot jatuh ke satu kunci dan gradien ke kunci-kunci lain praktis nol. Membagi dengan $\sqrt{d_k}$ mengembalikan simpangan baku skor ke satu, berapa pun dimensinya.

Satu blok transformer berisi *multi-head attention*, yaitu beberapa attention paralel dengan proyeksi yang berbeda yang hasilnya digabung, lalu jaringan feed-forward yang bekerja per posisi. Keduanya dibungkus koneksi residual dan layer normalization. Karena attention tidak peduli urutan, informasi posisi ditambahkan lewat *positional encoding*. Untuk model bahasa yang menulis dari kiri ke kanan, sebuah *causal mask* mencegah setiap posisi melihat masa depan.

Kelebihan transformer adalah semua posisi diproses paralel selama pelatihan, dan dua token mana pun hanya berjarak satu langkah attention. Kekurangannya, yang ditekankan di kuliah, adalah biaya attention yang tumbuh kuadratik terhadap panjang deret, serta keharusan menyimpan key dan value dari semua token sebelumnya ketika model membangkitkan teks.

Ramsauer et al. (2020), juga dari Linz, menunjukkan bahwa aturan update jaringan Hopfield modern dengan keadaan kontinu,

$$\xi^{\text{baru}} = \mathbf{X} \text{softmax}(\beta \mathbf{X}^\top \xi), \quad (6.5)$$

sama dengan attention. Kolom-kolom $\mathbf{X}$ adalah pola yang disimpan dan ξ adalah query. Biasanya satu langkah update sudah cukup untuk mengambil pola yang paling mirip. Dari sudut pandang ini, attention adalah cara membaca dari sebuah ingatan asosiatif yang kapasitasnya sangat besar.

6.5 Dari bahasa ke vektor

Kuliah NLP mengisi sisi datanya. Kata harus diubah menjadi angka sebelum masuk ke jaringan. Cara paling sederhana adalah *bag of words*, yang menghitung frekuensi kata dalam dokumen dan mengabaikan urutannya. Model *n-gram* memprediksi kata berikutnya dari $n - 1$ kata sebelumnya dengan menghitung frekuensi di sebuah korpus, tetapi jumlah konteks yang mungkin meledak seiring bertambahnya n .

Word embedding memetakan setiap kata ke vektor padat berdimensi beberapa ratus, sedemikian sehingga kata-kata yang muncul dalam konteks serupa punya vektor yang berdekatan (Mikolov et al., 2013). Hubungan seperti “raja dikurangi pria ditambah wanita kira-kira sama dengan ratu” muncul dengan sendirinya. Embedding juga menyerap bias dari teks pelatihannya (Bolukbasi et al., 2016), tema yang kembali di Bab 31.

Model bahasa besar menggabungkan semua ini. BERT (Devlin et al., 2018) dilatih menebak kata yang ditutup, sehingga ia melihat konteks kiri dan kanan. GPT-3 (Brown et al., 2020) dilatih menebak kata berikutnya, dan menunjukkan bahwa dengan skala

yang cukup, sebuah model bisa mengerjakan tugas baru hanya dari beberapa contoh di dalam prompt.

6.6 Rekurensi yang kembali

Biaya kuadratik transformer mendorong pencarian arsitektur dengan biaya linear. *State space model* memakai rekurensi linear

$$\mathbf{h}_t = \bar{\mathbf{A}} \mathbf{h}_{t-1} + \bar{\mathbf{B}} x_t, \quad y_t = \mathbf{C} \mathbf{h}_t, \quad (6.6)$$

yang bisa dihitung paralel justru karena linear. Mamba (Gu and Dao, 2023) membuat $\bar{\mathbf{B}}, \mathbf{C}$, dan langkah diskretisasinya bergantung pada input, sehingga model bisa memutuskan apa yang disimpan dan apa yang diabaikan, mirip gerbang LSTM. Rekurensi ini tidak lain adalah diskretisasi persamaan diferensial biasa yang linear, $\dot{\mathbf{h}} = \mathbf{A}\mathbf{h} + \mathbf{B}x$, sebuah objek yang akan kita temui lagi di Bab 11 and 27.

Kuliah ditutup dengan xLSTM (Beck et al., 2024), yang berangkat dari tiga keterbatasan LSTM yang disebut terang-terangan di slide. Keterbatasan pertama adalah ketidakmampuan merevisi keputusan penyimpanan. Kalau vektor yang lebih cocok datang belakangan, LSTM sulit menimpa isi lama. Jawabannya adalah *exponential gating*. Gerbang input $i_t = \exp(\tilde{i}_t)$ bisa bernilai besar sehingga isi baru mendominasi, sementara sebuah keadaan normalisasi $\mathbf{n}_t = \mathbf{f}_t \odot \mathbf{n}_{t-1} + i_t$ menjaga skalanya, dan keluaran membaca $\mathbf{c}_t/\mathbf{n}_t$. Agar eksponensial tidak meluap, dipakai keadaan penstabil $m_t = \max(\log f_t + m_{t-1}, \log i_t)$. Varian ini disebut sLSTM.

Keterbatasan kedua adalah kapasitas penyimpanan, karena satu sel LSTM hanya menyimpan skalar. mLSTM mengganti sel dengan sebuah matriks dan memakai aturan update kovarians:

$$\mathbf{C}_t = f_t \mathbf{C}_{t-1} + i_t \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{n}_t = f_t \mathbf{n}_{t-1} + i_t \mathbf{k}_t, \quad \tilde{\mathbf{h}}_t = \frac{\mathbf{C}_t \mathbf{q}_t}{\max\{|\mathbf{n}_t^\top \mathbf{q}_t|, 1\}}. \quad (6.7)$$

Bentuknya mirip ingatan asosiatif: simpan pasangan key dan value, lalu ambil dengan query. Keterbatasan ketiga adalah pelatihan yang tidak bisa diparalelkan. Pada mLSTM, gerbang, key, value, dan query tidak bergantung pada $\mathbf{h}_{t-1}$, sehingga pelatihannya bisa paralel seperti transformer. Blok sLSTM dan mLSTM kemudian ditumpuk dengan koneksi residual. Hasil yang dilaporkan menunjukkan xLSTM bersaing dengan transformer dan state space model dalam pemodelan bahasa, dengan biaya inferensi yang linear terhadap panjang deret.

6.7 Deret di luar bahasa

Model deret yang lahir dari bahasa dipakai jauh di luar teks. Kuliah LSTM sendiri sudah menunjukkan klasifikasi protein dari urutan asam amino dan analisis detak jantung. Dalam genomik, urutan DNA dibaca oleh model yang sama dengan model bahasa, dan base calling pada pengurutan nanopore memakai jaringan rekuren untuk mengubah sinyal arus listrik menjadi huruf (Bab 18). Dalam kimia, string SMILES dibangkitkan huruf demi huruf oleh LSTM untuk merancang molekul baru (Bab 19). Dalam sensor

yang dikenakan, deret akselerometer dikenali sebagai aktivitas (Bab 16), dan di sistem rekomendasi, urutan lagu yang baru didengar menjadi konteks untuk lagu berikutnya (Bab 30).

Simulasi yang berjalan dalam waktu juga sebuah deret: keadaan medan di langkah t menentukan keadaan di langkah $t + 1$. Model deret dipakai di sini dengan beberapa cara. Yang paling umum adalah memproyeksikan medan aliran ke beberapa mode POD (Bab 27), lalu membiarkan LSTM atau transformer mempelajari evolusi koefisien mode-mode itu, pendekatan yang dibahas dalam ulasan Brunton et al. (2020). Ingatan juga berguna ketika sensor hanya mengukur sebagian keadaan, karena masa lalu membantu menebak bagian yang tersembunyi. Transformer bahkan tidak harus berjalan sepanjang waktu. Ia bisa memperlakukan titik-titik mesh atau partikel sebagai token. Universal Physics Transformers (Alkin et al., 2024), dari kelompok riset di Linz, memakai gagasan ini untuk neural operator yang tidak terikat pada grid (Bab 26). Di semua pendekatan itu, masalah yang selalu muncul adalah galat yang menumpuk ketika model menebak berulang-ulang dari tebakannya sendiri. Masalah ini kita bahas di Bab 26.

CATATAN SUMBER. Bab ini mengikuti slide LSTM and Recurrent Neural Nets dari Sepp Hochreiter (RNN, BPTT, RTRL, vanishing gradient, LSTM, attention, Mamba, xLSTM) dan slide Natural Language Processing (representasi kata, n-gram, embedding, model bahasa besar). Bacaan utama: Hochreiter and Schmidhuber (1997a), Vaswani et al. (2017), Beck et al. (2024), dan Gu and Dao (2023).

7 Melihat: Computer Vision

Computer vision adalah salah satu kuliah pertama saya, di semester musim dingin 2024/25. Kuliahnya bergerak dari fisika lensa sampai NeRF dalam sebelas pertemuan, dan setiap beberapa minggu ada tugas praktikum: pengolahan citra dasar, ekstraksi fitur dan segmentasi, optical flow dan deteksi objek, lalu rekonstruksi tiga dimensi. Di bagian optical flow, untuk pertama kalinya di buku ini, gambar dan aliran fluida bertemu.

Bagi manusia, melihat terasa tanpa usaha. Bagi komputer, gambar hanyalah tabel angka. Bab ini menyusuri jalan dari cahaya yang masuk ke lensa sampai mesin yang bisa merekonstruksi dunia tiga dimensi. Di sepanjang jalan akan terlihat bahwa banyak alat klasik, seperti konvolusi, transformasi Fourier, dan kuadrat terkecil, tetap hidup di dalam jaringan saraf modern.

7.1 Dari cahaya ke piksel

Kamera paling sederhana adalah kotak gelap dengan lubang kecil. Setiap titik di dunia memancarkan cahaya ke segala arah, dan hanya satu berkas yang lolos lewat lubang itu, sehingga terbentuk gambar terbalik yang tajam. Kelemahannya, lubang kecil berarti sedikit cahaya. Lensa mengumpulkan banyak berkas dan memfokuskannya ke satu titik. Model lensa tipis menghubungkan panjang fokus f , jarak benda z_o , dan jarak bayangan z_i :

$$\frac{1}{f} = \frac{1}{z_o} + \frac{1}{z_i}. \quad (7.1)$$

Hanya benda pada satu jarak yang tepat fokus. Benda lain membentuk lingkaran kabur, dan f -number $N = f/D$, dengan D diameter bukaan, menentukan seberapa cepat kekaburan itu tumbuh. Bukaan kecil memberi *depth of field* yang lebar, dengan harga gambar yang lebih gelap.

Untuk komputasi, kita kembali ke model lubang jarum. Titik (X, Y, Z) dalam koordinat kamera jatuh di bidang gambar pada $x = fX/Z$ dan $y = fY/Z$. Pembagian dengan Z membuat proyeksi ini tidak linear, dan trik *homogeneous coordinates* mengembalikannya ke bentuk perkalian matriks dengan menambahkan satu koordinat:

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} [\mathbf{R} \mid \mathbf{t}] \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}. \quad (7.2)$$

Matriks pertama, $\mathbf{K}$, berisi sifat kamera sendiri: panjang fokus dalam piksel, titik pusat, dan kemiringan. Rotasi $\mathbf{R}$ dan translasi $\mathbf{t}$ menyatakan posisi kamera di dunia. Lensa nyata menambahkan distorsi radial, $x_d = x(1 + k_1 r^2 + k_2 r^4)$, yang membuat garis

lurus tampak melengkung di tepi gambar. *Camera calibration* memperkirakan semua parameter ini, biasanya dari beberapa foto papan catur yang posisi sudut-sudutnya diketahui.

Proyeksi perspektif mirip bayangan tangan di dinding saat kita bermain wayang dengan senter. Tangan yang dekat ke senter membuat bayangan besar, yang jauh membuat bayangan kecil. Dari bayangannya saja, kita tidak bisa tahu apakah itu tangan kecil di dekat senter atau tangan besar di dekat dinding. Kedalaman hilang saat proyeksi, dan sebagian besar kuliah ini adalah usaha untuk mendapatkannya kembali.

Sensor digital menambahkan lapisan lain. Ada noise foton dan noise elektronik, ada kuantisasi ke delapan atau dua belas bit, dan ada filter warna Bayer yang membuat setiap piksel hanya mengukur satu warna. Dua warna lainnya ditebak dari piksel tetangga dalam proses yang disebut *demosaiing*.

7.2 Pengolahan citra

Gambar bisa diperlakukan sebagai fungsi $I(x, y)$, dan banyak operasi dasarnya berupa konvolusi dengan kernel kecil (Bab 5). Filter kotak atau Gauss meratakan gambar dan menekan noise. Turunan, misalnya kernel Sobel, menonjolkan tepi. Laplacian, $\nabla^2 I = I_{xx} + I_{yy}$, menonjolkan perubahan tajam ke segala arah. Besar dan arah gradien ∇I menjadi bahan dasar deteksi tepi.

Banyak sifat filter lebih mudah dipahami di domain frekuensi. *Convolution theorem* menyatakan bahwa konvolusi di ruang sama dengan perkalian di frekuensi, $\mathcal{F}\{I * k\} = \mathcal{F}\{I\} \cdot \mathcal{F}\{k\}$. Filter Gauss tetap berbentuk Gauss di domain frekuensi, sehingga ia meredam frekuensi tinggi dengan halus. Filter kotak berubah menjadi fungsi sinc yang bergelombang, sehingga meninggalkan artefak. Spektrum gambar alami umumnya turun kira-kira seperti $1/f$, artinya sebagian besar energinya berada di frekuensi rendah.

Teorema yang sama menjelaskan mengapa *deconvolution* sulit. Kalau gambar kabur $B = I * k + n$ dibagi di domain frekuensi, $\mathcal{F}\{I\} \approx \mathcal{F}\{B\}/\mathcal{F}\{k\}$, noise ikut terbagi oleh bilangan yang hampir nol di frekuensi tempat kernel lemah, dan hasilnya meledak. Solusinya adalah regularisasi, misalnya filter Wiener. Tema yang sama akan muncul lagi pada masalah invers dalam simulasi.

Objek bisa tampak besar atau kecil. *Gaussian pyramid* membuat salinan gambar yang berulang kali dihaluskan lalu diperkecil setengahnya, sehingga algoritme yang bekerja pada satu ukuran bisa dijalankan di setiap tingkat piramida untuk menangkap semua skala. Piramida akan muncul lagi di optical flow, di U-Net, dan di solver multigrid.

7.3 Machine learning untuk visi

Gambar wajah berukuran 100×100 piksel adalah sebuah titik di ruang berdimensi sepuluh ribu, tetapi wajah tidak memenuhi seluruh ruang itu. *Principal component analysis* (Hotelling, 1933; Pearson, 1901) mencari arah dengan varians terbesar. Untuk data terpusat $\mathbf{X} \in \mathbb{R}^{N \times D}$, arah-arahnya adalah vektor eigen matriks kovarians $\Sigma = \frac{1}{N} \mathbf{X}^\top \mathbf{X}$ dengan nilai eigen terbesar. Turk and Pentland (1991) menyebut vektor eigen wajah sebagai *eigenfaces*. Setiap wajah bisa didekati sebagai rata-rata ditambah kombinasi

beberapa puluh eigenface, dan jumlah komponen dipilih dari fraksi varians yang dijelaskan, $\sum_{k \leq K} \lambda_k / \sum_k \lambda_k$.

Bayangkan memotret sekumpulan balon panjang yang melayang miring. Dari satu sudut, balon-balon tampak sebagai titik-titik yang bertumpuk. Dari sudut lain, panjangnya terlihat jelas dan balon-balon tampak tersebar. PCA memilih sudut pandang tempat data paling tersebar, karena di sanalah perbedaan antar objek paling terlihat. PCA akan muncul lagi dengan nama lain di Bab 27, yaitu *proper orthogonal decomposition*, alat utama untuk mereduksi medan aliran.

Selama dua dekade, visi komputer bergantung pada fitur yang dirancang manusia. Contoh paling terkenal adalah SIFT (Lowe, 2004). SIFT mencari titik khas sebagai ekstrem lokal dari *difference of Gaussians* di seluruh skala, sehingga titik yang sama tetap ditemukan meski gambarnya diperbesar. Di sekitar setiap titik, ia menghitung histogram arah gradien relatif terhadap arah dominan, sehingga deskriptornya tahan terhadap rotasi. Hasilnya adalah vektor 128 dimensi yang bisa dicocokkan antar foto.

AlexNet (Krizhevsky et al., 2017) mengubah keadaan. Jaringan konvolusi yang dilatih pada ImageNet mempelajari fiturnya sendiri: kernel lapisan pertama menjadi detektor tepi dan warna, dan lapisan yang lebih dalam menjadi detektor bagian-bagian objek. Fitur dari jaringan terlatih seperti VGG atau ResNet ternyata berguna jauh di luar klasifikasi. Kuliah menunjukkan contohnya untuk fusi gambar dan estimasi kedalaman dari fokus.

7.4 Segmentasi

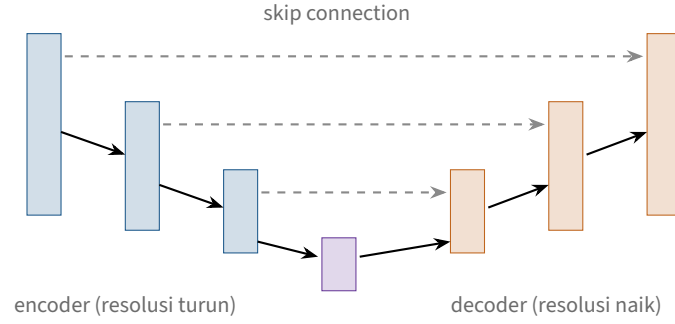
Segmentasi memberi label pada setiap piksel. *Semantic segmentation* memberi kelas seperti jalan, mobil, atau pohon, sedangkan *instance segmentation* juga membedakan mobil yang satu dari mobil yang lain.

Cara klasik memandang segmentasi sebagai pengelompokan. *k*-means (Lloyd, 1982) mengelompokkan piksel berdasarkan warna dan posisi. Pendekatan graf menganggap piksel sebagai simpul dengan bobot kemiripan, lalu memotong graf di tempat ikatannya paling lemah, dan pemotongan ini bisa didekati dengan vektor eigen Laplacian graf, konsep yang muncul lagi di Bab 9. *Hough transform* mendeteksi bentuk parametrik seperti garis: setiap titik tepi “memilih” semua garis $\rho = x \cos \theta + y \sin \theta$ yang melewatinya, dan garis yang nyata mengumpulkan suara terbanyak.

Cara modern memakai jaringan encoder dan decoder. Encoder mengecilkan resolusi sambil menangkap konteks, decoder memperbesarnya kembali dengan transposed convolution. U-Net (Ronneberger et al., 2015) menambahkan *skip connection* dari setiap tingkat encoder ke tingkat decoder yang sesuai, sehingga detail halus tidak hilang (Gambar 7.1). Model fondasi seperti Segment Anything (Kirillov et al., 2023) kini bisa memotong objek apa pun dari petunjuk sederhana seperti satu klik.

7.5 Optical flow

Video adalah deret gambar. *Optical flow* adalah medan vektor (u, v) yang menyatakan ke mana setiap piksel bergerak dari satu bingkai ke bingkai berikutnya. Asumsi



GAMBAR 7.1. Skema U-Net. Encoder mengecilkan resolusi, decoder memperbesarnya lagi, dan skip connection membawa detail dari setiap tingkat encoder ke decoder.

dasarnya adalah *brightness constancy*: titik yang bergerak mempertahankan kecerahannya, $I(x + u, y + v, t + 1) = I(x, y, t)$. Untuk gerak kecil, ruas kiri bisa diuraikan dengan Taylor orde satu, $I(x + u, y + v, t + 1) \approx I(x, y, t) + I_x u + I_y v + I_t$, dengan I_x , I_y , dan I_t turunan parsial kecerahan. Mengurangkan $I(x, y, t)$ dari kedua ruas memberi persamaan optical flow,

$$I_x u + I_y v + I_t = 0. \quad (7.3)$$

Ini satu persamaan dengan dua bilangan yang tidak diketahui. Di setiap piksel, kita hanya bisa mengetahui komponen gerak yang searah gradien kecerahan. Masalah ini disebut *aperture problem*. Contoh angka membuatnya jelas. Misalkan di sebuah piksel $I_x = 2$, $I_y = 0$, dan $I_t = -4$. Persamaannya menjadi $2u - 4 = 0$, sehingga $u = 2$, sedangkan v bisa bernilai berapa saja. Tepi vertikal hanya memberi tahu gerak horizontal. Untuk menentukan v , kita butuh piksel lain yang gradiennya berarah berbeda, misalnya di sebuah sudut.

Tiang pembatas jalan yang bergaris miring dan berputar memberi ilusi yang sama. Kalau dilihat lewat lubang kecil, garis-garisnya tampak bergerak naik, padahal tiangnya berputar ke samping. Melalui celah sempit, hanya gerak yang tegak lurus terhadap garis yang terlihat.

Ada dua cara klasik untuk menutupi kekurangan informasi ini. Lucas-Kanade menganggap semua piksel dalam sebuah jendela kecil bergerak sama, lalu menyelesaikan masalah kuadrat terkecil

$$\begin{pmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix} = - \begin{pmatrix} \sum I_x I_t \\ \sum I_y I_t \end{pmatrix}. \quad (7.4)$$

Asal persamaan ini sederhana. Setiap piksel p di jendela memberi satu persamaan $I_x(p)u + I_y(p)v = -I_t(p)$. Dengan banyak piksel, kita punya lebih banyak persamaan daripada bilangan yang dicari, dan kita meminimalkan jumlah kuadrat sisanya, $E(u, v) = \sum_p (I_x(p)u + I_y(p)v + I_t(p))^2$. Menurunkan terhadap u dan v lalu menyamakannya dengan nol memberi dua persamaan normal, yang ditulis dalam bentuk matriks di atas. Matriks di ruas kiri hanya bisa dibalik kalau jendela berisi gradien ke berbagai arah, seperti di sudut dan tekstur. Di sepanjang tepi lurus, semua gradien sejajar, matriksnya berpangkat satu, dan aperture problem muncul kembali. Nilai eigen matriks yang sama

juga dipakai detektor sudut Harris untuk mencari titik yang baik untuk dilacak. Untuk gerak lebih dari satu piksel, Taylor orde satu tidak berlaku lagi, dan solusinya adalah piramida: hitung flow kasar di resolusi rendah, lalu perhalus tingkat demi tingkat. Horn–Schunck (Horn and Schunck, 1981) mencari medan flow yang padat dengan meminimalkan

$$E(u, v) = \iint (I_x u + I_y v + I_t)^2 + \alpha^2 (\|\nabla u\|^2 + \|\nabla v\|^2) dx dy. \quad (7.5)$$

Suku pertama memaksa kecocokan dengan data, suku kedua memaksa medan yang halus. Ini regularisasi dalam bentuk paling murni: data yang tidak lengkap dilengkapi dengan prior.

Jaringan saraf kemudian belajar optical flow langsung dari data. FlowNet (Fischer et al., 2015) memakai CNN encoder dan decoder, dan varian korelasinya membandingkan fitur dua bingkai secara eksplisit. RAFT (Teed and Deng, 2020) membangun volume korelasi untuk semua pasangan piksel lalu memperbaiki flow secara iteratif dengan sebuah unit rekuren.

Hubungan dengan fluida di sini langsung. Di laboratorium, aliran diberi partikel kecil dan disinari laser, lalu dua foto berurutan dibandingkan untuk mendapatkan medan kecepatan. Teknik ini disebut *particle image velocimetry*, dan secara matematis ia adalah optical flow. Hubungannya bahkan lebih dalam. Untuk zat pewarna yang terbawa aliran tak termampatkan, kekekalan konsentrasi c berbunyi $\partial_t c + \mathbf{u} \cdot \nabla c = 0$, dan itu adalah Persamaan (7.3) dengan I diganti c . Suku kehalusan dalam Persamaan (7.5) bisa diganti syarat fisika seperti $\nabla \cdot \mathbf{u} = 0$. Raissi et al. (2020) membawa gagasan ini sampai ujungnya: dari video konsentrasi zat pewarna saja, sebuah PINN menyimpulkan medan kecepatan dan tekanan yang tidak pernah diukur, dengan memaksakan persamaan Navier–Stokes di dalam loss (Bab 25).

7.6 Deteksi objek

Deteksi menjawab dua pertanyaan sekaligus: apa, dan di mana. Keluarannya berupa kotak pembatas beserta kelasnya. Tumpang tindih antara kotak tebakan A dan kotak benar B diukur dengan *intersection over union*, $|A \cap B|/|A \cup B|$.

Pendekatan dua tahap dimulai dengan R-CNN, yang mengusulkan sekitar dua ribu wilayah dengan *selective search* lalu mengklasifikasi masing-masing dengan CNN. Fast R-CNN (Girshick, 2015) menghitung peta fitur sekali untuk seluruh gambar dan memotong wilayah dari peta itu. Faster R-CNN (Ren et al., 2015) mengganti *selective search* dengan jaringan pengusul wilayah yang dipelajari, dan menambahkan cabang mask ke arsitektur ini menghasilkan instance segmentation. Pendekatan satu tahap seperti YOLO (Redmon et al., 2015) membagi gambar menjadi grid dan membiarkan setiap sel langsung memprediksi kotak dan kelas dalam satu lintasan. Jauh lebih cepat, cocok untuk waktu nyata.

Banyak kotak yang tumpang tindih untuk objek yang sama dibersihkan dengan *non-maximum suppression*: simpan kotak dengan skor tertinggi, buang kotak lain yang tumpang tindihnya dengan kotak itu melewati ambang. Kinerja diukur dengan kurva

presisi terhadap recall, dan luas di bawahnya, *average precision*, dirata-ratakan atas semua kelas.

7.7 Geometri banyak pandang

Satu kamera kehilangan kedalaman, tetapi dua kamera bisa mendapatkannya kembali, persis seperti dua mata kita (Hartley and Zisserman, 2004). Titik x di gambar kiri bisa berasal dari titik mana pun di sepanjang sinar dari pusat kamera kiri. Dilihat dari kamera kanan, sinar itu tampak sebagai garis, yang disebut *epipolar line*, dan pasangan titik di gambar kanan pasti terletak di garis tersebut. Secara aljabar, $x'^T F x = 0$, dengan F matriks fundamental 3×3 berpangkat dua. Untuk kamera terkalibrasi, matriks padanannya adalah matriks esensial $E = [t]_{\times} R$. Pencarian pasangan titik berubah dari pencarian dua dimensi menjadi satu dimensi.

Setelah *rectification*, garis epipolar menjadi horizontal. Pergeseran horizontal titik yang sama antara kedua gambar disebut *disparity* d , dan segitiga sebangun memberi

$$Z = \frac{f B}{d}, \quad (7.6)$$

dengan f panjang fokus dalam piksel dan B jarak antar kamera. Penurunannya hanya memakai proyeksi lubang jarum. Letakkan kamera kiri di titik asal dan kamera kanan bergeser sejauh B di sumbu x . Titik (X, Z) jatuh di gambar kiri pada $x_L = fX/Z$ dan di gambar kanan pada $x_R = f(X - B)/Z$. Selisihnya, $d = x_L - x_R = fB/Z$, tidak lagi bergantung pada X , dan membalik persamaan itu memberi $Z = fB/d$. Kamera stereo dengan $f = 700$ piksel dan $B = 0,12$ meter yang melihat titik dengan disparity dua puluh piksel menempatkan titik itu pada 4,2 meter. Kalau disparity salah satu piksel saja, kedalaman menjadi 4,0 atau 4,4 meter. Semakin jauh objeknya, semakin kecil disparity-nya, dan semakin besar galat relatif kedalamannya.

Structure from motion memperluas gagasan ini ke banyak foto dari kamera yang bergerak: cocokkan fitur antar foto, perkirakan pose kamera, triangulasi titik-titik tiga dimensi, lalu perhalus semuanya bersama-sama.

7.8 Visi tiga dimensi

Dunia tiga dimensi bisa direpresentasikan dengan banyak cara. Peta kedalaman memberi satu angka per piksel. Grid voxel sederhana, tetapi memorinya tumbuh kubik. Awan titik hemat, tetapi tidak punya permukaan. Mesh punya permukaan eksplisit, tetapi topologinya sulit diubah. Permukaan implisit $f(x) = 0$ menyimpan bentuk sebagai himpunan nol sebuah fungsi. Memprediksi kedalaman dari satu gambar mengandung ambiguitas skala, karena miniatur yang dekat dan bangunan nyata yang jauh bisa menghasilkan gambar yang sama. Karena itu galat sering diukur dengan ukuran yang tidak menghukum faktor skala global.

Kuliah ditutup dengan NeRF (Mildenhall et al., 2020). Sebuah MLP menerima posisi x dan arah pandang d , lalu mengeluarkan kerapatan $\sigma(x)$ dan warna $c(x, d)$. Warna

satu piksel dihitung dengan *volume rendering* sepanjang sinar $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$,

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt, \quad T(t) = \exp \left(- \int_{t_n}^t \sigma(\mathbf{r}(s)) ds \right), \quad (7.7)$$

dengan $T(t)$ peluang cahaya lolos sampai titik t tanpa terhalang. Jaringan dilatih dengan membandingkan warna hasil render dengan foto, dan setelah terlatih, ia bisa merender sudut pandang yang belum pernah difoto. NeRF adalah contoh *neural field*, fungsi kontinu dari koordinat ke besaran fisik. Neural field dibahas lebih luas di Bab 11, dan kekerabatannya dengan PINN dekat sekali.

Pertemuan terakhir membahas arah-arrah baru: *vision transformer* yang memotong gambar menjadi petak 16×16 dan memperlakukannya seperti kata (Dosovitskiy et al., 2020), model fondasi yang dilatih pada pasangan gambar dan teks, dan *computational imaging* yang merancang kamera dan algoritme bersama-sama. Kuliah juga menekankan batasnya. Model besar tidak selalu benar, dan stiker kecil yang dirancang khusus bisa membuat detektor gagal mengenali rambu berhenti (Eykholt et al., 2017). Alat seperti Grad-CAM (Selvaraju et al., 2016) membantu melihat bagian gambar mana yang dipakai model untuk memutuskan, topik yang kembali di Bab 13.

CATATAN SUMBER. Bab ini mengikuti sebelas kuliah Computer Vision beserta praktikumnya: pembentukan citra, pengolahan citra, machine learning untuk visi, ekstraksi fitur, segmentasi, optical flow, deteksi objek, geometri banyak pandang, visi tiga dimensi, dan tren. Buku rujukan yang direkomendasikan adalah Szeliski (2022) dan Hartley and Zisserman (2004).

8 Arsitektur Modern dan Model Generatif

Kuliah lanjutan deep learning saya ikuti pada musim panas 2025, dan latihannya setahun kemudian. Tugas-tugas latihannya membentuk sebuah tangga: PyTorch dan training loop, jaringan visi dan transfer learning, pemantauan eksperimen dan CNN lanjutan, autoencoder dan VAE, contoh adversarial dan GAN, lalu normalizing flow. Bab ini menaiki tangga yang sama.

Bab 5 membangun jaringan dari bata-bata dasarnya. Bab ini membahas dua hal yang membuat deep learning modern berbeda: arsitektur yang memungkinkan jaringan amat dalam dilatih dengan stabil, dan model yang tidak hanya mengenali data tetapi juga membangkitkan data baru.

8.1 Dari LeNet ke ResNet

LeNet (Lecun et al., 1998) dan AlexNet (Krizhevsky et al., 2017) masih memakai kernel yang relatif besar. Keluarga VGG hanya memakai kernel 3×3 yang ditumpuk. Dua lapisan 3×3 melihat daerah 5×5 , tiga lapisan melihat 7×7 , dengan parameter yang lebih sedikit dan nonlinearitas yang lebih banyak. Dengan 64 kanal masuk dan keluar, satu lapisan 5×5 membutuhkan $25 \cdot 64^2 = 102\,400$ bobot, sedangkan dua lapisan 3×3 dengan receptive field yang sama cukup dengan $2 \cdot 9 \cdot 64^2 = 73\,728$ bobot, sekitar 28 persen lebih sedikit, dan memberi dua nonlinearitas alih-alih satu.

Menambah lapisan seharusnya tidak merugikan, karena lapisan tambahan bisa saja belajar menjadi fungsi identitas. Kenyataannya, jaringan polos yang dalam justru punya galat *latih* lebih tinggi, jadi masalahnya terletak pada optimasi, bukan overfitting. ResNet (He et al., 2016) mengubah tugas setiap blok dari mempelajari $H(x)$ menjadi mempelajari selisihnya,

$$y = x + F(x). \quad (8.1)$$

Kalau identitas sudah cukup baik, blok cukup mendorong F ke nol. Gradien pun mendapat jalan tol, $\partial y / \partial x = I + \partial F / \partial x$, sehingga sinyal mundur tidak harus melewati semua nonlinearitas. Gagasannya sama dengan constant error carousel pada LSTM (Bab 6), hanya saja kali ini di sepanjang kedalaman, bukan di sepanjang waktu.

Editor yang baik tidak menulis ulang seluruh esai setiap kali merevisi. Ia menulis catatan perubahan di pinggir halaman, dan kalau esainya sudah bagus, catatannya kosong. Blok residual bekerja seperti editor itu: ia hanya menambahkan koreksi pada apa yang sudah ada.

Latihan membahas dua variasi. Pada ResNet *pre-activation*, normalisasi dan aktivasi diletakkan sebelum konvolusi, sehingga bloknya berbentuk $x + F(\phi(x))$ alih-alih $\phi(x + F(x))$. Ketika jalur pintas dan jalur utama berbeda jumlah kanal, konvolusi 1×1 dipakai untuk menyamakannya.

Ada juga blok yang dirancang untuk efisiensi. *Depthwise-separable convolution* memecah konvolusi menjadi dua langkah. Setiap kanal dikonvolusi sendiri-sendiri, lalu kanal-kanal dicampur dengan konvolusi 1×1 . Untuk kernel 3×3 dan 64 kanal, jumlah bobot turun dari $9 \cdot 64 \cdot 64 = 36\,864$ menjadi $9 \cdot 64 + 64 \cdot 64 = 4\,672$. Blok *squeeze-and-excitation* merata-ratakan setiap kanal menjadi satu angka, melewatkannya ke MLP kecil, lalu memakai hasilnya untuk menskalakan kanal, sehingga jaringan belajar kanal mana yang penting untuk gambar tertentu. *Vision transformer* memotong gambar menjadi petak-petak $P \times P$ dan memperlakukan setiap petak sebagai token (Dosovitskiy et al., 2020). Gambar 224×224 dengan $P = 16$ menghasilkan $14 \times 14 = 196$ token. MLP-Mixer bergantian mencampur informasi antar token dan antar kanal dengan MLP biasa, tanpa konvolusi dan tanpa attention.

Melatih jaringan besar dari nol membutuhkan data besar. *Transfer learning* memakai jaringan yang sudah dilatih pada ImageNet, membekukan sebagian besar lapisannya, lalu melatih ulang kepala klasifikasinya saja. Fitur umum seperti tepi dan tekstur sudah dipelajari, sehingga data yang sedikit pun cukup untuk tugas baru.

8.2 Dari mengenali ke membangkitkan

Model diskriminatif mempelajari $p(y | x)$. Model generatif mempelajari $p(x)$ itu sendiri, sehingga bisa membangkitkan contoh baru, mengisi bagian yang hilang, atau menilai seberapa wajar sebuah data. Kuliah membahas lima keluarga.

8.2.1 Autoencoder

Autoencoder memampatkan x menjadi kode $z = e(x)$ yang berdimensi kecil, lalu merekonstruksinya menjadi $\hat{x} = d(z)$. Loss rekonstruksinya mengikuti asumsi noise (Tabel 1.1): MSE untuk data kontinu yang dianggap Gauss, binary cross-entropy dengan keluaran sigmoid untuk data di antara nol dan satu. Autoencoder linear dengan loss kuadrat menemukan subruang yang sama dengan PCA. Kelemahannya, autoencoder biasa tidak menjamin ruang kode yang rapi. Titik acak di ruang kode bisa didekode menjadi sesuatu yang tidak bermakna.

8.2.2 Variational autoencoder

VAE (Kingma and Welling, 2013) memperbaiki hal itu dengan memberi struktur probabilistik. Encoder mengeluarkan distribusi $q_\phi(z | x) = \mathcal{N}(\mu, \text{diag } \sigma^2)$, decoder memodelkan $p_\theta(x | z)$, dan ruang kode diberi prior $p(z) = \mathcal{N}(0, I)$. Pelatihannya memaksimalkan *evidence lower bound*:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)] - \text{KL}(q_\phi(z | x) \| p(z)). \quad (8.2)$$

Suku pertama mengukur rekonstruksi, suku kedua mengukur kedekatan ke prior. Untuk Gauss diagonal, divergensi KL punya bentuk tertutup, $\text{KL} = \frac{1}{2} \sum_j (\sigma_j^2 + \mu_j^2 - 1 - \log \sigma_j^2)$. Untuk satu dimensi, rumus ini didapat dari definisi $\text{KL}(q \| p) = \mathbb{E}_q[\log q - \log p]$ dengan $q = \mathcal{N}(\mu, \sigma^2)$ dan $p = \mathcal{N}(0, 1)$. Log kerapatan keduanya adalah $\log q = -\frac{1}{2} \log(2\pi\sigma^2) - (z - \mu)^2 / (2\sigma^2)$ dan $\log p = -\frac{1}{2} \log 2\pi - z^2 / 2$. Di bawah q , $\mathbb{E}[(z - \mu)^2] = \sigma^2$ dan $\mathbb{E}[z^2] = \sigma^2 + \mu^2$, sehingga $\mathbb{E}_q[\log q - \log p] = -\frac{1}{2} \log \sigma^2 - \frac{1}{2} + \frac{1}{2}(\sigma^2 + \mu^2)$, yang sama dengan rumus

di atas. Untuk dimensi yang independen, KL-nya tinggal dijumlahkan. Satu dimensi laten dengan $\mu = 1$ dan $\sigma = 1$ memberi KL sebesar 0,5. Dengan $\mu = 0$ dan $\sigma = 0,5$, hasilnya $\frac{1}{2}(0,25 - 1 + 1,386) \approx 0,318$. KL bernilai nol hanya kalau $\mu = 0$ dan $\sigma = 1$, yaitu tepat sama dengan prior.

Untuk membawa gradien melewati pengambilan sampel acak, VAE memakai *reparameterization trick*. Sampel ditulis sebagai $z = \mu + \sigma \odot \epsilon$ dengan $\epsilon \sim \mathcal{N}(0, I)$. Keacakan dipindahkan ke ϵ , sehingga z menjadi fungsi yang bisa diturunkan terhadap μ dan σ .

Saya membayangkan encoder sebagai orang yang melihat foto lalu menulis deskripsi singkat yang sengaja agak samar, misalnya “wajah bulat, rambut pendek, kira-kira tersenyum”. Decoder adalah pelukis yang menggambar dari deskripsi itu. Suku KL memaksa semua deskripsi memakai kosakata yang sama dan tidak terlalu spesifik, sehingga deskripsi baru yang dikarang bebas pun tetap menghasilkan wajah yang masuk akal.

Latihan menekankan satu kenop praktis: bobot keseimbangan antara rekonstruksi dan KL. Bobot KL yang terlalu besar membuat gambar kabur dan ruang laten tidak terpakai. Bobot yang terlalu kecil membuat VAE kembali menjadi autoencoder biasa. Penurunan ELBO yang lebih teliti, termasuk versi hierarkis yang mengarah ke model difusi, ada di Bab 11.

8.2.3 Generative adversarial networks

GAN (Goodfellow et al., 2014) mempertandingkan dua jaringan. Generator G mengubah noise z menjadi sampel palsu, dan diskriminator D menebak apakah sebuah sampel asli atau palsu:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_z [\log (1 - D(G(z)))]. \quad (8.3)$$

Bentuk ini tidak lain adalah binary cross-entropy diskriminator yang tandanya dibalik untuk generator. Di awal pelatihan, diskriminator mudah menang dan $\log(1 - D(G(z)))$ jenuh, sehingga gradien generator kecil. Jalan keluarnya adalah *non-saturating loss*, di mana generator memaksimalkan $\log D(G(z))$ saja.

Apa yang sebenarnya dioptimalkan permainan ini? Untuk generator yang tetap, dengan distribusi sampel palsu p_g , tujuan diskriminator bisa ditulis sebagai integral $\int [p_{\text{data}}(x) \log D(x) + p_g(x) \log(1 - D(x))] dx$. Di setiap titik x , fungsi $a \log D + b \log(1 - D)$ maksimal pada $D = a/(a + b)$, sehingga diskriminator terbaik adalah

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}. \quad (8.4)$$

Memasukkan D^* kembali ke tujuan permainan memberi $-\log 4 + 2 \text{JS}(p_{\text{data}} \| p_g)$, dengan JS divergensi Jensen–Shannon (Goodfellow et al., 2014). Jadi generator, ketika melawan diskriminator yang sempurna, sebenarnya meminimalkan jarak antara distribusi data dan distribusi sampelnya, dan minimumnya tercapai tepat ketika keduanya sama, saat $D^* = \frac{1}{2}$ di mana-mana: diskriminator hanya bisa menebak dengan melempar koin.

Pemalsu uang belajar mencetak uang yang lolos pemeriksaan, sementara detektif belajar membedakan uang asli dan palsu. Setiap kali detektif menemukan cacat baru,

pemalsu memperbaikinya. Kalau keduanya seimbang, uang palsu akhirnya tidak bisa dibedakan dari yang asli. Masalah khas GAN adalah *mode collapse*, ketika generator menemukan beberapa sampel yang selalu lolos lalu berhenti menghasilkan variasi lain. Pelatihannya juga tidak stabil, karena tidak ada satu fungsi loss yang turun secara monoton.

8.2.4 Normalizing flows

Flow membangun distribusi yang rumit dengan mengubah distribusi sederhana melalui fungsi yang bisa dibalik (Rezende and Mohamed, 2015). Kalau $x = g(z)$ dengan g bijektif, rumus perubahan variabel memberi kerapatan yang eksak,

$$\log p_X(x) = \log p_Z(g^{-1}(x)) + \log \left| \det \frac{\partial g^{-1}}{\partial x} \right|. \quad (8.5)$$

Dalam satu dimensi, rumus ini berasal dari kekekalan peluang. Peluang yang jatuh di selang kecil $[z, z + dz]$ harus sama dengan peluang di selang padanannya $[x, x + dx]$, sehingga $p_X(x)|dx| = p_Z(z)|dz|$, atau $p_X(x) = p_Z(z)|dz/dx|$. Kalau $x = 2z$ dengan $z \sim \mathcal{N}(0, 1)$, setiap selang diregangkan dua kali, sehingga kerapatannya harus turun separuh: $p_X(x) = \frac{1}{2}p_Z(x/2)$, yang memang kerapatan $\mathcal{N}(0, 4)$. Determinan Jacobian adalah versi banyak dimensi dari faktor regangan ini, dan ia mengukur perubahan volume lokal. Untuk dimensi besar, determinan umum mahal dihitung, jadi lapisan flow dirancang agar Jacobian-nya segitiga. *Coupling layer* membagi vektor menjadi dua bagian, membiarkan bagian pertama apa adanya, dan mengubah bagian kedua dengan fungsi dari bagian pertama. Untuk coupling aditif, $x'_2 = x_2 + t(x_1)$, determinannya bernilai satu dan inversnya analitis. Syarat bijeksi berarti dimensi tidak boleh berubah. Imbalannya, flow memberi likelihood yang eksak, bukan batas bawah seperti ELBO.

8.2.5 Model difusi dan model energi

Model difusi (Ho et al., 2020; Sohl-Dickstein et al., 2015) merusak data perlahan-lahan dengan noise Gauss, lalu melatih jaringan untuk membalik proses itu langkah demi langkah. Mereka kini menjadi model generatif gambar yang paling kuat, dan matematikanya, yang melibatkan persamaan diferensial stokastik dan persamaan Fokker–Planck, dibahas lengkap di Bab 11. Model berbasis energi, termasuk mesin Boltzmann, mendefinisikan $p(x) \propto \exp(-E(x))$. Mudah ditulis, tetapi sulit dilatih, karena konstanta normalisasinya adalah integral atas seluruh ruang.

Tabel 8.1 merangkum kelima keluarga ini.

8.3 Contoh adversarial

Jaringan yang akurat bisa dibohongi dengan perubahan yang tidak terlihat oleh mata. *Fast gradient sign method* menggeser input ke arah yang paling menaikkan loss,

$$x' = x + \epsilon \text{sign}(\nabla_x \mathcal{L}(x, y)). \quad (8.6)$$

Projected gradient ascent mengulang langkah ini beberapa kali, dan setiap kali memproyeksikan hasilnya kembali ke bola $\|x' - x\|_\infty \leq \epsilon$ serta ke rentang piksel yang sah.

TABEL 8.1. Lima keluarga model generatif.

Model	Dilatih dengan	Kekuatan	Kelemahan
VAE	ELBO	ruang laten terstruktur, stabil	sampel cenderung kabur
GAN	permainan minimax	sampel tajam	tidak stabil, mode collapse
Flow	likelihood eksak	kerapatan eksak, bisa dibalik	arsitektur terbatas, dimensi tetap
Difusi	denoising	kualitas dan keragaman	sampling lambat
Energi	beragam, sering MCMC	fleksibel	normalisasi sulit

Serangan seperti ini bekerja juga di dunia fisik (Eykholt et al., 2017). Melatih model dengan contoh adversarial membuatnya lebih tahan, dengan harga akurasi yang sedikit turun pada data bersih.

8.4 Sedikit teori

Naskah kuliah ditutup dengan beberapa hasil teoretis yang menjelaskan mengapa semua ini bekerja. Jaringan dalam paling mudah dilatih kalau nilai singular Jacobian input–output mendekati satu, sehingga sinyal maju dan gradien mundur sama-sama tidak menyusut atau meledak. Sifat ini disebut isometri dinamis, dan ia adalah versi matriks dari argumen varians di Bab 5. Pada lebar tak hingga dan laju belajar kecil, pelatihan jaringan berperilaku seperti regresi kernel dengan kernel yang tetap, yaitu *neural tangent kernel* (Jacot et al., 2018). Teori ini menjelaskan konvergensi jaringan yang lebar sekali dan, seperti akan kita lihat di Bab 25, juga kegagalan pelatihan PINN. Sebelum dilatih, keluaran jaringan acak yang lebar mendekati proses Gauss, sebuah hubungan yang menjembatani deep learning dengan statistik Bayesian (Rasmussen and Williams, 2006).

Jaringan biasa hanya memberi satu tebakan, tanpa keterangan seberapa yakin ia. MC dropout menjalankan dropout saat inferensi dan memperlakukan variasi keluarannya sebagai ketidakpastian (Gal and Ghahramani, 2015). Deep ensembles melatih beberapa jaringan dari inisialisasi yang berbeda dan melihat seberapa jauh mereka berbeda pendapat (Lakshminarayanan et al., 2016).

8.5 Di luar kelas

Arsitektur dan model generatif di bab ini menyebar ke hampir semua bidang. U-Net lahir untuk citra biomedis dan tetap menjadi standar segmentasi organ dan tumor (Bab 21). Transfer learning membuat jaringan visi yang dilatih pada foto sehari-hari berguna untuk citra mikroskop dan citra medis yang datanya sedikit. VAE dan model generatif berbasis teks dipakai untuk merancang molekul baru, dan metrik seperti Fréchet ChemNet Distance meminjam cara mengevaluasi GAN untuk kimia (Bab 19). Contoh adversarial mengingatkan bahwa sensor dan sistem persepsi kendaraan otonom bisa diserang (Bab 17).

Di simulasi, untuk medan pada grid teratur, U-Net adalah pilihan bawaan sebagai model pengganti. Ia menggabungkan konteks yang luas, penting karena tekanan di satu titik dipengaruhi seluruh domain, dengan detail lokal seperti lapisan batas. Blok residual cocok untuk model langkah waktu, karena $u_{t+1} = u_t + F(u_t)$ tidak lain adalah integrator Euler dengan F yang dipelajari.

Encoder sebuah autoencoder bisa memetakan medan aliran ke ruang laten berdimensi kecil. Dinamikanya lalu dipelajari di ruang laten itu, dan decoder mengembalikannya ke medan penuh. Ini salah satu bentuk model tereduksi yang dibahas di Bab 27.

Turbulensi bersifat chaos, sehingga lebih jujur memprediksi distribusi keadaan masa depan daripada satu keadaan tunggal. Model difusi kini dipakai untuk prakiraan cuaca probabilistik (Price et al., 2025) dan untuk simulasi aliran turbulen (Kohl et al., 2023). Model pengganti yang dipakai untuk keputusan teknik juga harus bisa mengatakan bahwa ia tidak yakin, dan ensemble serta model generatif memberi kemampuan itu.

CATATAN SUMBER. Bab ini mengikuti bagian kedua naskah kuliah Deep Learning and Neural Nets II, yang kini bernama Deep Learning: Architectures and Generative Techniques (arsitektur lanjut, model generatif, teori deep learning, optimasi orde tinggi, Bayesian deep learning), beserta enam tugas latihannya. Bacaan pendamping: Goodfellow et al. (2016), Prince (2023), dan Kingma and Welling (2019).

9 Geometri dan Simetri

Kuliah geometric deep learning datang di semester terakhir, pada musim panas 2026. Materinya mengikuti kursus Geometric Deep Learning dari Oxford yang dikembangkan Michael Bronstein dan rekan-rekannya, dengan buku proto karya Bronstein et al. (2021) sebagai acuan. Kuliah-kuliah sebelumnya membahas arsitektur satu per satu. Kuliah ini menunjukkan bahwa konvolusi, DeepSets, jaringan graf, dan transformer adalah anak-anak dari satu resep yang sama.

Resep itu bertumpu pada satu pengamatan. Di Bab 1 kita melihat bahwa belajar di dimensi tinggi hampir mustahil tanpa asumsi. Asumsi yang paling berharga ternyata adalah simetri: kita sering tahu transformasi apa yang tidak mengubah jawaban. Kucing yang digeser tetap kucing. Molekul yang diputar tetap molekul yang sama. Urutan penomoran atom dalam berkas tidak mengubah sifat kimianya. Kalau pengetahuan seperti ini ditanamkan ke dalam arsitektur, ruang yang harus dicari menyusut drastis.

9.1 Sinyal, domain, dan grup

Bahasa geometric deep learning dimulai dengan memisahkan dua hal: domain dan sinyal. Domain Ω adalah tempat data hidup, misalnya grid piksel, himpunan titik, graf, atau permukaan. Sinyal adalah fungsi $x : \Omega \rightarrow \mathbb{R}^c$ yang memberi setiap titik domain sebuah vektor fitur, misalnya warna untuk setiap piksel atau sifat kimia untuk setiap atom. Ruang semua sinyal pada sebuah domain, $X(\Omega)$, adalah ruang vektor: dua sinyal bisa dijumlahkan dan dikalikan skalar titik demi titik. Struktur linear inilah yang memungkinkan kita berbicara tentang lapisan linear.

Simetri dirumuskan dengan *grup*. Grup adalah himpunan G dengan operasi komposisi yang memenuhi empat aksioma: hasil komposisi dua anggota tetap di dalam G , komposisinya asosiatif, ada anggota identitas, dan setiap anggota punya invers. Rotasi bidang membentuk grup, begitu pula pergeseran grid dan permutasi himpunan. Dua grup bisa memiliki struktur yang sama walau anggotanya berbeda, dan kesamaan ini dinyatakan dengan *homomorphism*, pemetaan φ yang menghormati komposisi, $\varphi(gh) = \varphi(g)\varphi(h)$. Determinan adalah contohnya: ia memetakan matriks yang bisa dibalik ke bilangan real tak nol, dan $\det(AB) = \det A \det B$. Fungsi eksponensial memberi contoh yang lebih menarik, karena ia memetakan bilangan real dengan penjumlahan ke bilangan positif dengan perkalian, $e^{a+b} = e^a e^b$, dan bisa dibalik dengan logaritma. Homomorfisme yang bisa dibalik disebut isomorfisme, dan dua grup yang isomorfik secara struktural sama.

Grup bekerja pada domain melalui *group action*: setiap g memindahkan titik u ke $g.u$, dengan identitas tidak memindahkan apa pun dan aksi dua anggota berturut-turut sama dengan aksi komposisinya. Dari aksi pada domain, kita mendapat aksi pada

sinyal,

$$(\rho(g)x)(u) = x(g^{-1}.u). \quad (9.1)$$

Pemetaan $g \mapsto \rho(g)$ ke operator linear pada ruang sinyal disebut *representasi*. Invers pada Persamaan (9.1) bukan hiasan. Untuk menggeser sebuah gambar ke kanan, nilai piksel di posisi baru diambil dari posisi lama di sebelah kirinya. Tanpa invers, komposisi dua pergeseran akan keluar dengan urutan terbalik, dan ρ tidak lagi menjadi homomorfisme.

9.2 Invariansi dan ekuivariansi

Misalkan ρ dan ρ' representasi grup yang sama pada ruang input dan ruang output. Fungsi f disebut *invariant* kalau $f(\rho(g)x) = f(x)$ untuk setiap g , dan *equivariant* kalau

$$f(\rho(g)x) = \rho'(g)f(x) \quad \text{untuk setiap } g \in G. \quad (9.2)$$

Klasifikasi gambar seharusnya invarian terhadap pergeseran: labelnya tidak berubah. Segmentasi seharusnya ekuivarian: kalau gambar digeser, peta segmentasinya ikut bergeser. Secara visual, ekuivariansi berarti diagram berikut komut: menerapkan transformasi lalu f memberi hasil yang sama dengan menerapkan f lalu transformasi padanannya. Sinyal-sinyal yang hanya berbeda karena transformasi grup membentuk sebuah *orbit*. Peta ekuivarian memetakan setiap orbit di ruang input ke sebuah orbit di ruang fitur, sehingga struktur simetrinya terbawa utuh.

Ada cara umum untuk membuat sembarang fungsi menjadi invarian terhadap grup yang terbatas, yaitu merata-ratakannya atas seluruh grup,

$$(S_G f)(x) = \frac{1}{|G|} \sum_{g \in G} f(\rho(g)x). \quad (9.3)$$

Operator ini kadang disebut operator simetrisasi. Ambil himpunan dua elemen dan fungsi $f(x_1, x_2) = x_1$ yang jelas tidak peduli pada simetri. Rata-rata atas dua permutasi memberi $(x_1 + x_2)/2$, yang invarian. Kalau fungsi target memang invarian, membatasi diri pada kelas fungsi invarian tidak menambah galat aproksimasi, karena targetnya masih ada di dalam kelas. Galat estimasi justru bisa turun, karena setiap contoh data kini “mewakili” seluruh orbitnya. Hanya saja, kalau grupnya kecil dibanding dimensi masalah, galat estimasi tetap bisa terkena kutukan dimensi. Simetri membantu, tetapi tidak membebaskan sepenuhnya.

9.3 Skala dan cetak biru

Simetri saja belum cukup. Prior kedua adalah *scale separation*: banyak sinyal bisa dipahami secara bertingkat. Detail lokal memengaruhi detail lokal di sekitarnya, dan struktur besar bisa diringkas dari versi sinyal yang lebih kasar. Dua bahan dasarnya adalah operator lokal yang bekerja di lingkungan kecil, dan operasi *coarse graining* yang meringkas sinyal ke domain yang lebih kasar, seperti pooling pada gambar. Gabungan simetri dan skala lebih kuat dari masing-masing, karena fungsi yang bekerja secara lokal dan bertingkat hanya perlu belajar interaksi jarak pendek di setiap tingkat.

Dari kedua prior ini, Bronstein et al. (2021) merumuskan sebuah cetak biru. Jaringan disusun dari lapisan linear yang ekuivarian terhadap G (sebut B), nonlinearitas yang diterapkan titik demi titik (σ), pooling lokal yang mengasarkan domain (P), dan di ujungnya pooling global yang invarian (A), sehingga

$$f = A \circ \sigma \circ B_J \circ P_{J-1} \circ \cdots \circ P_1 \circ \sigma \circ B_1. \quad (9.4)$$

CNN adalah cetak biru ini pada grid dengan grup translasi. DeepSets adalah cetak biru pada himpunan dengan grup permutasi. Jaringan graf adalah cetak biru pada graf. Kita akan melihat ketiganya satu per satu.

Ada satu kehalusan lagi. Gambar nyata jarang bertransformasi persis menurut grup. Wajah yang sedikit tersenyum bukan rotasi atau pergeseran dari wajah yang datar, melainkan deformasi kecil. Representasi yang baik sebaiknya *hampir* invarian terhadap transformasi yang *hampir* berada di dalam grup: perubahan keluarannya sebanding dengan seberapa jauh deformasi itu dari transformasi grup. Sifat ini disebut stabilitas terhadap deformasi, dan ia menjadi ukuran kualitas representasi yang akan muncul lagi pada wavelet dan pada filter spektral.

9.4 Himpunan

Domain paling sederhana adalah himpunan tanpa struktur. Simetrinya adalah grup permutasi: penomoran elemen tidak boleh berpengaruh. Sinyal pada himpunan n elemen bisa ditulis sebagai matriks fitur $\mathbf{X} \in \mathbb{R}^{n \times c}$, satu baris per elemen. Kalau elemen dipermutasi, baris-baris matriks ikut dipermutasi, $\mathbf{X} \mapsto \mathbf{P}\mathbf{X}$, dengan $\mathbf{P}$ matriks permutasi yang berisi tepat satu angka satu di setiap baris dan kolom.

Peta linear yang ekuivarian terhadap semua permutasi ternyata hanya punya dua derajat kebebasan per pasangan kanal: satu bagian yang bekerja pada setiap elemen sendiri, dan satu bagian yang melihat jumlah seluruh elemen,

$$F(\mathbf{X}) = \mathbf{X}\mathbf{\Lambda} + \frac{1}{n}\mathbf{1}\mathbf{1}^\top \mathbf{X}\mathbf{\Gamma}. \quad (9.5)$$

DeepSets (Zaheer et al., 2017) membangun fungsi invarian dari bagian-bagian ini: $f(\mathbf{X}) = \phi(\sum_i \psi(\mathbf{x}_i))$. Fungsi ψ dipakai bersama untuk setiap elemen dan bersifat ekuivarian, penjumlahan adalah pooling yang invarian, dan ϕ bekerja pada ringkasan global. Kedua fungsi dipelajari, sedangkan penjumlahannya tetap.

9.5 Graf

Graf menambahkan satu bahan ke himpunan: hubungan antar elemen. Graf tak berarah dengan atribut simpul bisa ditulis sebagai matriks fitur $\mathbf{X}$ dan matriks ketetanggaan $\mathbf{A}$, dengan $a_{uv} = 1$ kalau simpul u dan v terhubung. Grup simetrinya tetap permutasi, tetapi sekarang permutasi bekerja pada keduanya: $\mathbf{X} \mapsto \mathbf{P}\mathbf{X}$ dan $\mathbf{A} \mapsto \mathbf{P}\mathbf{A}\mathbf{P}^\top$. Fungsi tingkat graf harus invarian, $f(\mathbf{P}\mathbf{X}, \mathbf{P}\mathbf{A}\mathbf{P}^\top) = f(\mathbf{X}, \mathbf{A})$, dan fungsi tingkat simpul harus ekuivarian, $F(\mathbf{P}\mathbf{X}, \mathbf{P}\mathbf{A}\mathbf{P}^\top) = \mathbf{P}F(\mathbf{X}, \mathbf{A})$.

9.5.1 Message passing

Graph neural network modern hampir semuanya mengikuti pola *message passing* (Battaglia et al., 2018; Gilmer et al., 2017). Setiap simpul u mengumpulkan pesan dari tetangganya, lalu memperbarui keadaannya:

$$h_u = \phi\left(x_u, \bigoplus_{v \in \mathcal{N}_u} \psi(x_u, x_v)\right), \quad (9.6)$$

dengan $\bigoplus$ operator agregasi yang tidak peduli urutan, seperti jumlah, rata-rata, atau maksimum. Bedanya dengan DeepSets ada di sini: setiap simpul tidak lagi meringkas seluruh himpunan, tetapi hanya tetangganya, sehingga sisi graf menentukan siapa berbicara dengan siapa.

Pembagian yang dipakai di kuliah membedakan tiga rasa. Pada varian konvolusional, pesan dari tetangga dikalikan bobot tetap yang bergantung pada struktur graf, seperti pada GCN (Kipf and Welling, 2016). Pada varian attentional, bobotnya dihitung dari fitur kedua simpul, seperti pada GAT (Veličković et al., 2017). Pada varian message passing yang paling umum, pesan ψ adalah fungsi sembarang dari kedua simpul, dan kedua varian sebelumnya adalah kasus khususnya.

Sebagai contoh kecil, ambil graf garis dengan tiga simpul, $1-2-3$, dengan fitur skalar $x = (1, 2, 3)$. Dengan aturan $h_u = x_u + \sum_{v \in \mathcal{N}_u} x_v$, simpul pertama mendapat $1 + 2 = 3$, simpul kedua $2 + 1 + 3 = 6$, dan simpul ketiga $3 + 2 = 5$. Kalau penomoran dibalik menjadi $3-2-1$, fitur menjadi $(3, 2, 1)$ dan hasilnya $(5, 6, 3)$, yaitu hasil semula dalam urutan baru. Itulah ekuivariansi.

Contoh ini bukan kebetulan. Dalam bentuk matriks, aturan tadi adalah $h = (I + A)x$. Kalau simpul dipermutasi dengan P , input menjadi Px dan ketetanggaan menjadi PAP^T , sehingga keluarannya $(I + PAP^T)Px = Px + PAx = P(I + A)x = Ph$, karena $P^T P = I$. Argumen yang sama berlaku untuk message passing umum: fungsi ψ dan ϕ yang dipakai bersama di semua simpul tidak peduli nomor simpulnya, dan agregasi atas tetangga tidak peduli urutan tetangganya. Satu-satunya hal yang bisa membuat jaringan bergantung pada penomoran adalah memberi setiap simpul bobot pribadinya sendiri, dan itulah yang dihindari.

Teman yang mengabarkan berita di sebuah desa adalah gambaran yang cocok. Setiap pagi, setiap orang mendengar kabar dari tetangga sebelah rumahnya dan memperbarui pengetahuannya. Setelah satu pagi, kabar dari rumah sebelah sudah sampai. Setelah tiga pagi, kabar dari rumah yang berjarak tiga sudah sampai. Satu lapisan message passing adalah satu pagi.

9.5.2 Seberapa kuat jaringan graf?

Bisakah jaringan graf membedakan dua graf yang tidak isomorfik? Pertanyaan ini terhubung dengan uji klasik *Weisfeiler–Leman* (WL). Uji ini mewarnai setiap simpul, lalu berulang kali memberi warna baru berdasarkan warna lama simpul itu dan multihimpunan warna tetangganya. Kalau histogram warna dua graf berbeda, graf itu pasti tidak isomorfik. Kalau sama, kita tidak bisa menyimpulkan apa-apa. Xu et al. (2018) dan Morris et al. (2018) menunjukkan bahwa message passing paling banter sekuat

WL, dan mencapai kekuatan itu kalau agregasinya injektif terhadap multihimpunan. Penjumlahan memenuhi syarat ini, sedangkan rata-rata dan maksimum tidak, karena keduanya tidak bisa membedakan, misalnya, satu tetangga berfitur satu dari dua tetangga berfitur satu.

Batas WL mudah diperlihatkan. Bandingkan dua segitiga yang terpisah dengan satu lingkaran enam simpul. Keduanya punya enam simpul, dan setiap simpul punya tepat dua tetangga. Pada setiap iterasi WL, semua simpul menerima informasi yang sama, sehingga semua warna tetap identik dan uji tidak bisa membedakan keduanya, padahal yang satu berisi segitiga dan yang lain tidak. Jaringan message passing biasa pun gagal di sini.

Ada beberapa cara melampaui batas ini. *Positional encoding* memberi setiap simpul “alamat”, misalnya dari vektor eigen Laplacian graf atau dari langkah acak, sehingga simpul yang strukturnya sama tetap bisa dibedakan. Jaringan substruktur menghitung kemunculan pola kecil, seperti segitiga atau siklus, di sekitar setiap simpul. Subgraph GNN menjalankan jaringan pada banyak versi graf yang sedikit diubah, misalnya dengan satu simpul dihapus, lalu menggabungkan hasilnya. Gagasan ini berkerabat dengan *reconstruction conjecture*, dugaan lama dalam teori graf bahwa sebuah graf dengan setidaknya tiga simpul ditentukan sepenuhnya oleh kumpulan subgraf yang masing-masing kehilangan satu simpul.

Transformer pun bisa dilihat sebagai message passing pada graf lengkap, di mana setiap token adalah simpul yang menerima pesan dari semua simpul lain dengan bobot attention. Tanpa positional encoding, attention memang ekuivarian terhadap permutasi. Dengan positional encoding seperti yang dipakai dalam praktik, sifat itu sengaja dilepas, karena urutan kata memang bermakna.

9.5.3 Oversquashing

Untuk menangkap interaksi jarak jauh, kita menambah lapisan. Tetapi jumlah simpul dalam radius r bisa tumbuh secara eksponensial, sementara semua informasinya harus dimampatkan ke vektor berukuran tetap. Gejala ini disebut *oversquashing* (Alon and Yahav, 2020). Secara formal, kepekaan keadaan simpul u setelah r lapisan terhadap fitur awal simpul v , $\|\partial \mathbf{h}_u^{(r)} / \partial x_v\|$, dibatasi oleh entri matriks ketetanggaan ternormalisasi yang dipangkatkan, $(\hat{\mathbf{A}}^r)_{uv}$, dikalikan konstanta yang bergantung pada bobot dan nonlinearitas. Lebar jaringan dan kedalaman ikut berperan, tetapi faktor yang paling menentukan adalah topologi graf: leher botol, yaitu sedikit sisi yang menghubungkan dua kelompok besar, dan daerah dengan kelengkungan negatif (Topping et al., 2021). Kabar dari desa sebelah harus lewat satu jembatan sempit, dan di jembatan itu sebagian besar berita hilang.

9.6 Grid

Grid adalah kasus yang paling kita kenal. Grid satu dimensi bisa dilihat sebagai graf berbentuk lingkaran, kalau tepinya disambung secara periodik. Grup simetrinya adalah pergeseran siklik. Gambar *I* yang digeser satu piksel ke kanan bisa ditulis $I' = \mathbf{AIB}$, dengan $\mathbf{A}$ matriks identitas karena baris tidak berubah, dan $\mathbf{B}$ matriks pergeseran siklik

yang memindahkan setiap kolom satu tempat. Pergeseran ke bawah memakai matriks pergeseran di sisi kiri.

Peta linear yang ekuivarian terhadap pergeseran adalah tepat peta yang komut dengan operator geser S , yaitu $CS = SC$. Matriks seperti ini adalah *circulant matrix*: setiap barisnya adalah baris sebelumnya yang digeser satu tempat. Mengalikan dengan matriks sirkulan sama dengan konvolusi. Inilah alasan mendalam mengapa konvolusi muncul di CNN: ia bukan sekadar pilihan desain, melainkan satu-satunya operasi linear yang menghormati simetri geser. Semua matriks sirkulan punya vektor eigen yang sama, yaitu basis Fourier, sehingga transformasi Fourier diskret mendiagonalkan semuanya sekaligus. Akibatnya, konvolusi di ruang menjadi perkalian titik demi titik di frekuensi, dan dengan FFT (Cooley and Tukey, 1965), konvolusi panjang bisa dihitung dalam $O(n \log n)$ operasi.

Fourier punya keterbatasan. Basisnya tersebar di seluruh ruang, sehingga lokalisasi sempurna di frekuensi berarti tidak ada lokalisasi di ruang. Prinsip ketidakpastian membatasi seberapa sempit sebuah fungsi bisa sekaligus di kedua domain. Transformasi Fourier berjendela memotong sinyal menjadi potongan pendek, tetapi lebar jendelanya sama untuk semua frekuensi. Lebih penting lagi, modulus Fourier tidak stabil terhadap dilatasi kecil. Meregangkan sinyal sedikit menggeser frekuensi rendah sedikit, tetapi menggeser frekuensi tinggi jauh, sehingga dua sinyal yang hampir sama bisa punya spektrum frekuensi tinggi yang berbeda.

Wavelet menyelesaikan ini dengan memakai satu fungsi induk yang digeser dan diskalakan. Di frekuensi tinggi, wavelet sempit di ruang dan lebar di frekuensi. Di frekuensi rendah, sebaliknya. Wavelet Haar, misalnya, adalah tangga naik-turun yang cocok sekali untuk mendeteksi tepi dan struktur lokal. *Scattering transform* (Mallat, 2012) menumpuk tiga bahan: konvolusi wavelet, modulus, dan rata-rata. Modulus adalah nonlinearitas yang krusial. Tanpanya, rata-rata dari koefisien wavelet akan bernilai nol, karena wavelet berosilasi. Modulus memindahkan energi frekuensi tinggi ke frekuensi rendah, sehingga informasinya selamat dari rata-rata. Hasilnya adalah representasi yang invarian terhadap pergeseran dan stabil terhadap deformasi, sebuah CNN yang filternya tidak dipelajari tetapi dirancang, dan dalam banyak hal menjadi model teoretis untuk memahami CNN.

9.7 Konvolusi grup

Konvolusi bisa dipandang sebagai “transformasikan lalu cocokkan”: geser filter ke setiap posisi, lalu ukur kecocokannya dengan sinyal. Kalau grupnya lebih besar dari pergeseran, misalnya pergeseran ditambah rotasi, kita bisa melakukan hal yang sama untuk setiap anggota grup (Cohen and Welling, 2016):

$$(x \star \psi)(g) = \int_{\Omega} x(u) \psi(g^{-1} \cdot u) du. \quad (9.7)$$

Keluarannya kini adalah fungsi pada grup, bukan pada domain asal. Pada grup p_4 , yang terdiri atas pergeseran dan empat rotasi kelipatan sembilan puluh derajat, lapisan pertama mengubah gambar menjadi empat peta fitur, satu untuk setiap orientasi filter.

Lapisan berikutnya bekerja pada fungsi di grup itu sendiri, sehingga filternya juga punya dimensi orientasi. Kalau gambar input diputar sembilan puluh derajat, setiap peta fitur ikut berputar, dan keempat orientasi bergeser secara siklis.

Konsep pendukungnya adalah *homogeneous space*. Grup bekerja secara transitif pada sebuah domain kalau setiap titik bisa dicapai dari titik lain oleh suatu anggota grup. Stabilizer sebuah titik adalah subgrup yang membiarkan titik itu di tempatnya. Bola S^2 di bawah rotasi tiga dimensi adalah contoh *homogeneous space*, dan stabilizer kutub utaranya adalah rotasi di sekitar sumbu yang melewati kutub. Bidang di bawah gerak kaku, yaitu pergeseran dan rotasi, adalah contoh lainnya, dengan stabilizer setiap titik berupa rotasi di sekitar titik itu.

9.8 Manifold dan mesh

Banyak data hidup di permukaan melengkung: tubuh manusia, sayap pesawat, permukaan bumi. Manifold adalah model matematisnya. Secara informal, manifold adalah ruang yang di sekitar setiap titik tampak seperti ruang Euklides. *Chart* adalah peta lokal yang meratakan sepotong manifold, seperti halaman atlas yang meratakan sepotong bumi. Di setiap titik ada *tangent space*, ruang semua arah yang mungkin di titik itu. Untuk mengukur panjang dan sudut vektor singgung, kita butuh *Riemannian metric*, sebuah hasil kali dalam yang bervariasi mulus dari titik ke titik. Dengan metrik, kita bisa mendefinisikan panjang kurva, *geodesic* sebagai jalur terpendek, dan jarak geodesik.

Besaran yang hanya bergantung pada metrik disebut intrinsik. Ia tidak berubah kalau permukaan ditekuk tanpa diregangkan. Selebar kertas yang digulung menjadi silinder punya geometri intrinsik yang sama dengan kertas datar, sehingga jarak geodesik antara dua titik di atasnya tidak berubah. Deformasi yang mempertahankan metrik disebut isometri. Berbeda dari *homogeneous space*, manifold umumnya tidak punya grup simetri global, sehingga stabilitas terhadap deformasi menjadi lebih penting daripada ekuivariansi yang persis.

Kalkulus pada manifold bertumpu pada tiga operator. Gradien ∇f sebuah medan skalar menunjuk ke arah kenaikan tercepat dan didefinisikan melalui metrik, sehingga intrinsik. Divergensi sebuah medan vektor mengukur sumber dan sumur lokal. Keduanya saling adjoint, $\langle \nabla f, X \rangle = -\langle f, \operatorname{div} X \rangle$, persis seperti integrasi parsial. Gabungannya adalah *Laplace–Beltrami operator*, $\Delta f = -\operatorname{div} \nabla f$, yang mengukur seberapa jauh nilai fungsi di sebuah titik berbeda dari rata-rata di sekitarnya. Laplacian ini simetris dan semidefinit positif, sehingga punya basis fungsi eigen yang ortogonal, generalisasi alami dari basis Fourier.

Dengan basis itu, konvolusi bisa didefinisikan secara spektral: transformasikan sinyal ke basis eigen Laplacian, kalikan dengan filter di setiap frekuensi, lalu kembalikan (Bruna et al., 2013). Konvolusi spektral ini intrinsik, tetapi naif. Basis eigen bisa berubah drastis karena deformasi kecil, terutama untuk frekuensi tinggi, sehingga filter yang dipelajari pada satu bentuk tidak berpindah ke bentuk lain. Jalan keluarnya adalah filter yang mulus terhadap frekuensi, misalnya polinomial dalam Laplacian. Filter polinomial berderajat K hanya melihat tetangga dalam radius K , sehingga lokal dan stabil, dan ChebNet (Defferrard et al., 2016) memakai polinomial Chebyshev untuk tujuan ini.

Dalam praktik, manifold didiskretisasi menjadi mesh segitiga atau graf. Laplacian graf yang paling sederhana adalah $L = D - A$, dengan D matriks diagonal derajat simpul, sehingga

$$(Lx)_u = \sum_{v \in \mathcal{N}_u} a_{uv} (x_u - x_v). \quad (9.8)$$

Pada mesh, bobot a_{uv} dipilih dari geometri segitiga, misalnya dengan rumus kotangen, agar Laplacian diskret mendekati Laplace–Beltrami yang kontinu. Bentuk di atas sudah memperlihatkan bahwa Laplacian adalah sebuah langkah message passing: setiap simpul menjumlahkan selisih dirinya dengan tetangganya. Konvolusi spektral, jaringan graf, dan operator diferensial pada permukaan ternyata adalah satu keluarga.

9.9 Simetri di luar kelas

Cetak biru ini muncul di banyak bidang dengan nama yang berbeda. Molekul adalah graf atom, dan sifatnya tidak berubah kalau molekul diputar atau atom-atomnya dinomori ulang, sehingga jaringan graf dan jaringan ekuivarian menjadi standar dalam kimia komputasi (Bab 19). AlphaFold menempatkan residu protein sebagai benda kaku di ruang dengan attention yang tidak peduli rotasi global (Bab 20). Awan titik lidar pada kendaraan otonom adalah himpunan tanpa urutan, dan PointNet adalah DeepSets untuk awan titik (Bab 17). Pengguna dan item dalam sistem rekomendasi membentuk graf bipartit, dan message passing di atasnya menyebarkan selera (Bab 30).

Bagi simulasi, bab ini barangkali yang paling penting di Bagian II. Hukum fisika punya simetri: persamaan Navier–Stokes tidak berubah kalau sistem digeser atau diputar, dan gaya antar partikel tidak bergantung pada urutan penomoran partikel. Model pengganti yang tidak menghormati simetri ini harus mempelajarinya dari data, dan sering gagal di luar data latihnya. Jaringan graf yang ekuivarian terhadap rotasi dan translasi tiga dimensi, seperti EGNN (Satorras et al., 2021), membangunnya langsung ke dalam arsitektur.

Mesh simulasi pada dasarnya adalah graf. Metode volume hingga, yang akan kita bahas di Bab 23, menghitung perubahan di setiap sel dari fluks yang melewati sisi-sisinya ke sel tetangga. Bentuknya persis message passing: setiap sel mengumpulkan “pesan” fluks dari tetangganya, lalu memperbarui keadaannya. Tidak mengherankan bahwa simulator yang dipelajari dengan jaringan graf menjadi salah satu keberhasilan terbesar di bidang ini. Graph Network-based Simulator (Sanchez-Gonzalez et al., 2020) meniru dinamika partikel cairan dan pasir, MeshGraphNets (Pfaff et al., 2020) bekerja pada mesh simulasi aerodinamika dan struktur, dan Message Passing Neural PDE Solvers (Brandstetter et al., 2022) memakai message passing sebagai solver PDE umum. GraphCast (Lam et al., 2023) membawa gagasan yang sama ke prakiraan cuaca global, dengan graf di atas mesh ikosahedral yang membungkus bumi. Semuanya dibahas lebih dalam di Bab 26.

Laplacian graf juga punya makna fisik yang langsung. Persamaan difusi panas pada graf, $\dot{x} = -\kappa Lx$, didiskretisasi dengan Euler eksplisit menjadi $x^{n+1} = x^n - \kappa \Delta t Lx^n$. Satu langkah waktu difusi adalah satu lapisan message passing dengan bobot yang sudah ditentukan. Pandangan ini juga menjelaskan gejala *oversmoothing* pada jaringan

graf yang terlalu dalam: seperti panas yang lama-lama merata, fitur semua simpul menjadi semakin mirip.

CATATAN SUMBER. Bab ini mengikuti kuliah Deep Learning: Geometric Techniques, yang memakai materi kursus Geometric Deep Learning dari Oxford dan daftar topik ujiannya: kutukan dimensi, prior geometris, himpunan, graf, grid, konvolusi grup, manifold, dan konvolusi spektral. Acuan utama adalah Bronstein et al. (2021), dengan latar belakang di Bronstein et al. (2017).

10 Bernalar dalam Ketidakpastian

Kuliah model probabilistik saya ikuti pada musim dingin 2025/26. Slidenya banyak bersandar pada buku Koller and Friedman (2009) dan Russell and Norvig (2021). Bab ini mengikuti urutan kuliahnya: representasi, inferensi, pembelajaran, lalu model yang berjalan dalam waktu.

Bab-bab sebelumnya kebanyakan membahas fungsi yang memetakan input ke output. Bab ini membahas cara lain memandang dunia: sebagai kumpulan variabel acak yang saling bergantung. Kita tidak lagi bertanya “apa jawabannya?”, tetapi “seberapa yakin kita pada setiap kemungkinan jawaban, dan bagaimana keyakinan itu berubah ketika ada bukti baru?”.

10.1 Mengapa butuh struktur

Distribusi gabungan atas n variabel biner butuh $2^n - 1$ angka. Untuk dua puluh variabel, itu sekitar sejuta angka, dan untuk tiga puluh variabel, satu miliar. Tidak ada yang bisa menyimpan, apalagi mempelajari, tabel sebesar itu. Jalan keluarnya adalah *conditional independence*. Variabel X dan Y independen bersyarat terhadap Z kalau, setelah Z diketahui, mengetahui Y tidak lagi menambah informasi tentang X : $p(X | Y, Z) = p(X | Z)$. Petir dan suara guntur sangat berkorelasi, tetapi keduanya disebabkan oleh peristiwa yang sama. Kalau kita sudah tahu ada sambaran listrik di awan, mendengar guntur tidak menambah informasi tentang kilatnya.

Aturan rantai probabilitas selalu berlaku: $p(x_1, \dots, x_n) = \prod_i p(x_i | x_1, \dots, x_{i-1})$. Independensi bersyarat memangkas setiap faktor sehingga hanya bergantung pada beberapa variabel.

10.2 Bayesian network

Bayesian network adalah graf berarah tanpa siklus yang setiap simpulnya adalah variabel acak. Setiap variabel diberi distribusi bersyarat terhadap induk-induknya, dan distribusi gabungannya adalah hasil kali semuanya:

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i | \text{pa}(x_i)). \quad (10.1)$$

Kalau setiap variabel biner punya paling banyak k induk, jaringan butuh paling banyak $n 2^k$ angka. Dengan $n = 20$ dan $k = 3$, itu 160 angka, bukan sejuta.

Contoh yang dipakai banyak buku teks, termasuk Koller and Friedman (2009), adalah seorang mahasiswa. Nilai ujiannya, G , bergantung pada tingkat kesulitan mata kuliah, D , dan kecerdasan mahasiswa, I . Kecerdasan juga memengaruhi skor tes masuk, S , dan nilai memengaruhi kualitas surat rekomendasi, L . Graf ini memungkinkan

beberapa pola penalaran. Penalaran kausal berjalan searah panah: mahasiswa cerdas cenderung mendapat nilai baik. Penalaran evidensial berjalan melawan panah: nilai buruk menurunkan keyakinan bahwa mahasiswanya cerdas. Pola yang paling menarik adalah penalaran antarsebab. Nilai buruk menaikkan keyakinan bahwa mata kuliahnya sulit dan menurunkan keyakinan bahwa mahasiswanya cerdas. Kalau kemudian kita tahu mata kuliahnya memang sulit, nilai buruk itu sudah “terjelaskan”, dan keyakinan tentang kecerdasan mahasiswa naik kembali. Pola ini disebut *explaining away*. Bayesian network menangani semua pola ini dengan satu algoritme inferensi yang sama, tanpa aturan khusus untuk masing-masing.

10.2.1 Membaca independensi dari graf

Kapan dua variabel independen bersyarat bisa dibaca langsung dari bentuk graf. Ada tiga pola dasar untuk jalur $X - Z - Y$. Pada rantai $X \rightarrow Z \rightarrow Y$ dan pada garpu $X \leftarrow Z \rightarrow Y$, jalur informasi terbuka selama Z tidak diamati dan tertutup ketika Z diamati. Pola ketiga, *v-structure* $X \rightarrow Z \leftarrow Y$, bekerja terbalik. Kalau Z tidak diamati, jalurnya tertutup: kesulitan mata kuliah dan kecerdasan mahasiswa tidak saling memengaruhi. Kalau Z , atau salah satu turunannya, diamati, jalur terbuka, dan itulah sebabnya *explaining away* terjadi. Dua variabel disebut *d-separated* oleh sekumpulan pengamatan kalau semua jalur di antara keduanya tertutup, dan dalam hal itu mereka pasti independen bersyarat.

Satu himpunan yang istimewa adalah *Markov blanket* sebuah variabel: induknya, anaknya, dan induk lain dari anak-anaknya. Kalau Markov blanket diketahui, variabel itu independen dari seluruh sisa jaringan. Sifat ini nanti menjadi kunci Gibbs sampling.

10.3 Inferensi eksak

Inferensi berarti menghitung distribusi sebuah variabel kueri setelah sebagian variabel lain diamati, misalnya $p(I \mid G = \text{buruk})$. Cara naif adalah menjumlahkan distribusi gabungan atas semua variabel lain, dengan biaya eksponensial. *Variable elimination* memanfaatkan faktorisasi: penjumlahan didorong sedalam mungkin ke dalam hasil kali, sehingga setiap variabel dijumlahkan hanya dari faktor-faktor yang menyebutnya. Hasil antara disimpan sebagai faktor baru.

Urutan eliminasi menentukan biaya. Urutan yang baik menjaga faktor antara tetap kecil, sedangkan urutan yang buruk bisa membuat faktor sebesar tabel gabungan. Untuk graf berbentuk pohon, biayanya linear. Untuk graf umum, inferensi eksak termasuk masalah NP-hard, dan biayanya bergantung secara eksponensial pada ukuran faktor terbesar yang tidak bisa dihindari.

10.4 Inferensi dengan sampel

Kalau inferensi eksak terlalu mahal, kita bisa memperkirakan peluang dengan menarik sampel. Cara paling sederhana adalah *forward sampling*: tarik setiap variabel mengikuti urutan graf, dari induk ke anak. Untuk menghitung peluang bersyarat, *rejection sampling* membuang sampel yang tidak cocok dengan pengamatan. Cara ini boros sekali kalau

pengamatannya jarang terjadi, karena hampir semua sampel dibuang.

Likelihood weighting tidak membuang apa pun. Variabel yang diamati dipaksa bernilai sesuai pengamatan, dan setiap sampel diberi bobot sebesar peluang pengamatan itu terjadi. Ini adalah contoh *importance sampling*: sampel ditarik dari distribusi yang mudah, lalu dikoreksi dengan bobot. Kelemahannya muncul ketika pengamatan berada di bagian bawah graf. Variabel di atasnya ditarik tanpa memperhatikan pengamatan, sehingga banyak sampel mendapat bobot yang hampir nol.

Markov chain Monte Carlo mengambil jalan yang berbeda. Kita membangun sebuah rantai Markov yang distribusi stasionernya adalah distribusi posterior yang dicari, lalu menjalankan rantai itu cukup lama. *Gibbs sampling* memperbarui satu variabel pada satu waktu, menariknya dari distribusi bersyarat terhadap semua variabel lain. Berkat Markov blanket, distribusi bersyarat itu hanya melibatkan beberapa faktor lokal. Metropolis–Hastings (Hastings, 1970; Metropolis et al., 1953) lebih umum: usulkan langkah acak, lalu terima dengan peluang yang dipilih tepat agar rantai menyeimbangkan distribusi target. Kedua metode ini lahir dari fisika statistik, untuk menghitung sifat gas dan cairan, dan kembali ke fisika di Bab 24.

Bayangkan menaksir bentuk sebuah ruangan gelap dengan berjalan acak di dalamnya, sambil lebih sering melangkah ke tempat yang lebih “terang”. Setelah cukup lama, jejak langkah Anda menggambarkan seberapa terang setiap sudut ruangan. Langkah-langkah awal sebelum rantai mencapai keseimbangan biasanya dibuang, dan sampel yang berurutan saling berkorelasi, sehingga jumlah sampel efektifnya lebih kecil dari jumlah sampel mentah.

10.5 Belajar parameter dan struktur

Kalau struktur graf diketahui dan datanya lengkap, belajar parameter ternyata hanya menghitung. Maximum likelihood untuk $p(x_i \mid \text{pa}(x_i))$ adalah frekuensi relatif di data. Masalahnya muncul untuk kombinasi yang jarang atau tidak pernah terlihat, yang langsung mendapat peluang nol. Pendekatan Bayesian memberi prior Dirichlet pada setiap tabel peluang, dan efeknya setara dengan menambahkan hitungan semu sebelum melihat data. Kalau sebuah kejadian muncul 3 kali dari 10, maximum likelihood memberi 0,3. Dengan satu hitungan semu untuk setiap dari dua kemungkinan, yaitu Laplace smoothing, perkiraannya menjadi $4/12 \approx 0,33$. Pengaruh hitungan semu ini memudar ketika data bertambah.

Kalau sebagian data hilang, atau ada variabel tersembunyi yang tidak pernah diamati, hitungan sederhana tidak bisa dilakukan. Algoritme EM (Dempster et al., 1977) bergantian antara dua langkah. Langkah E menghitung distribusi variabel yang hilang dengan parameter saat ini, sehingga menghasilkan hitungan yang diharapkan. Langkah M memperbarui parameter seolah-olah hitungan yang diharapkan itu adalah data lengkap. Setiap putaran tidak menurunkan likelihood. EM akan muncul lagi sebagai kerangka di balik VAE dan model campuran Gauss di Bab 11.

Belajar struktur graf lebih sulit, karena jumlah graf yang mungkin tumbuh super-eksponensial terhadap jumlah variabel. Kuliah membahas dua keluarga pendekatan. Pendekatan berbasis kendala menjalankan uji independensi bersyarat pada data, lalu

membangun graf yang sesuai dengan hasil uji itu. Pendekatan berbasis skor mencari graf yang memaksimalkan sebuah skor, misalnya BIC,

$$\text{BIC} = \log p(\mathcal{D} \mid \hat{\theta}, \mathcal{G}) - \frac{\log N}{2} \dim(\mathcal{G}), \quad (10.2)$$

yang menyeimbangkan kecocokan dengan data dan jumlah parameter. Tanpa suku penalti, graf yang terhubung penuh selalu menang, dan itulah overfitting dalam bentuk struktur. Pencariannya biasanya heuristik, misalnya hill climbing dengan menambah, menghapus, atau membalik satu sisi pada satu waktu, atau tabu search yang melarang kembali ke langkah yang baru saja diambil. Pendekatan ketiga, Bayesian model averaging, tidak memilih satu graf, tetapi merata-ratakan prediksi atas banyak graf yang diberi bobot posterior.

10.6 Model yang berjalan dalam waktu

Banyak sistem berubah dari waktu ke waktu. *Dynamic Bayesian network* memodelkannya dengan mengulang struktur yang sama di setiap langkah waktu, dengan dua asumsi: keadaan sekarang hanya bergantung pada keadaan sebelumnya (asumsi Markov), dan aturan transisinya tidak berubah dari waktu ke waktu. Seluruh model cukup ditentukan oleh satu irisan waktu beserta sambungannya ke irisan berikutnya, lalu bisa “dibuka” sepanjang yang dibutuhkan.

Ada empat tugas inferensi yang berulang di semua model temporal. *Filtering* menghitung keadaan sekarang dari semua pengamatan sampai sekarang. *Prediction* menghitung keadaan masa depan. *Smoothing* memperbaiki perkiraan keadaan masa lalu dengan pengamatan yang datang belakangan. Dan *most likely explanation* mencari urutan keadaan yang paling mungkin menjelaskan semua pengamatan.

10.6.1 Hidden Markov model

Hidden Markov model (Rabiner, 1989) adalah DBN dengan satu variabel keadaan diskret yang tersembunyi dan satu pengamatan di setiap langkah. Ia ditentukan oleh matriks transisi $a_{ij} = p(s_t = j \mid s_{t-1} = i)$, peluang emisi $b_j(o) = p(o_t = o \mid s_t = j)$, dan distribusi awal. Filtering dihitung dengan *forward algorithm*,

$$\alpha_t(j) = b_j(o_t) \sum_i \alpha_{t-1}(i) a_{ij}, \quad (10.3)$$

yang bergantian antara memprediksi (penjumlahan atas keadaan sebelumnya) dan memperbarui dengan pengamatan baru (perkalian dengan peluang emisi). Algoritme backward menghitung pesan yang sama dari arah sebaliknya, dan gabungan keduanya memberi smoothing. Algoritme Viterbi mengganti penjumlahan dengan maksimum untuk mencari urutan keadaan terbaik, dan Baum–Welch adalah EM untuk mempelajari parameter HMM dari pengamatan saja.

Contoh klasik dari Russell and Norvig (2021) adalah penjaga yang tidak pernah keluar gedung dan hanya bisa menebak cuaca dari apakah direktornya datang membawa payung. Misalkan peluang hujan hari ini sama dengan kemarin adalah 0,7, peluang

membawa payung saat hujan 0,9, dan saat tidak hujan 0,2, dengan keyakinan awal 0,5. Pada hari pertama direktur membawa payung. Prediksinya tetap 0,5, dan pembaruan memberi hujan sebanding dengan $0,9 \times 0,5 = 0,45$ dan tidak hujan sebanding dengan $0,2 \times 0,5 = 0,10$. Setelah dinormalisasi, peluang hujan 0,818. Pada hari kedua prediksinya $0,818 \times 0,7 + 0,182 \times 0,3 \approx 0,627$, dan pengamatan payung kedua menaikannya menjadi sekitar 0,883. Setiap hari, keyakinan mengalir lewat dua langkah yang sama: ramalkan, lalu koreksi dengan bukti.

10.6.2 Kalman filter

Kalau keadaannya kontinu dan semua hubungan linear dengan noise Gauss, filtering bisa dikerjakan secara eksak dengan Kalman filter (Kalman, 1960). Modelnya $\mathbf{x}_t = \mathbf{A}\mathbf{x}_{t-1} + \mathbf{w}_t$ dan $z_t = \mathbf{H}\mathbf{x}_t + v_t$, dengan noise proses $\mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$ dan noise pengukuran $v_t \sim \mathcal{N}(0, \mathbf{R})$. Karena Gauss tetap Gauss di bawah transformasi linear, cukup melacak rata-rata dan kovarians:

$$\begin{aligned}\hat{\mathbf{x}}_t^- &= \mathbf{A}\hat{\mathbf{x}}_{t-1}, & \mathbf{P}_t^- &= \mathbf{A}\mathbf{P}_{t-1}\mathbf{A}^\top + \mathbf{Q}, \\ \mathbf{K}_t &= \mathbf{P}_t^- \mathbf{H}^\top (\mathbf{H}\mathbf{P}_t^- \mathbf{H}^\top + \mathbf{R})^{-1}, & & \\ \hat{\mathbf{x}}_t &= \hat{\mathbf{x}}_t^- + \mathbf{K}_t(z_t - \mathbf{H}\hat{\mathbf{x}}_t^-), & \mathbf{P}_t &= (\mathbf{I} - \mathbf{K}_t\mathbf{H})\mathbf{P}_t^-. \end{aligned} \quad (10.4)$$

Baris pertama adalah prediksi, dan baris terakhir adalah koreksi. Matriks $\mathbf{K}_t$, *Kalman gain*, menentukan seberapa besar kita mempercayai pengukuran dibanding prediksi.

Kalman filter seperti mendengarkan dua saksi. Saksi pertama adalah model, yang meramalkan posisi dari posisi sebelumnya. Saksi kedua adalah sensor. Keduanya bisa keliru, dan kita mempercayai masing-masing sesuai ketelitiannya. Dalam satu dimensi, kalau ketidakpastian prediksi punya varians 4 dan pengukuran punya varians 1, gain-nya $4/(4 + 1) = 0,8$: hasil akhir bergerak 80 persen ke arah pengukuran, dan variansnya turun menjadi 0,8, lebih kecil dari keduanya. Menggabungkan dua sumber yang tidak sempurna menghasilkan perkiraan yang lebih baik dari masing-masing.

Rumus satu dimensi ini bisa diturunkan langsung dengan teorema Bayes. Prediksi memberi prior $\mathcal{N}(\hat{x}^-, P^-)$, dan pengukuran z memberi likelihood $\mathcal{N}(z; x, R)$. Posterior sebanding dengan hasil kali keduanya, dan eksponennya adalah $-\frac{1}{2}[(x - \hat{x}^-)^2/P^- + (z - x)^2/R]$. Jumlah dua kuadrat dalam x tetap kuadrat dalam x , jadi posteriornya Gauss. Mengumpulkan koefisien x^2 memberi presisi posterior $1/P = 1/P^- + 1/R$: presisi, yaitu kebalikan varians, langsung dijumlahkan. Mengumpulkan koefisien x memberi rata-rata posterior $\hat{x} = P(\hat{x}^-/P^- + z/R)$, rata-rata berbobot dengan bobot sebanding presisi masing-masing. Dengan sedikit aljabar, keduanya bisa ditulis ulang sebagai $\hat{x} = \hat{x}^- + K(z - \hat{x}^-)$ dan $P = (1 - K)P^-$ dengan $K = P^-/(P^- + R)$, persis bentuk Kalman. Untuk angka di atas, $1/P = 1/4 + 1/1 = 1,25$, sehingga $P = 0,8$.

Untuk sistem yang tidak linear, Kalman filter diperluas dengan melinearkan model di sekitar perkiraan saat ini (*extended Kalman filter*), atau diganti dengan *particle filter* yang mewakili distribusi dengan sekumpulan sampel berbobot. Keduanya dipakai di kendaraan otonom untuk melacak posisi dari GPS, odometri, dan lidar.

10.7 Di luar kelas

Model-model di bab ini bekerja di belakang banyak sistem yang kita temui di bagian-bagian berikutnya. Kendaraan otonom melacak posisinya dengan filter Bayes dan melacak mobil di sekitarnya dengan Kalman filter (Bab 17). Pelacak gerak inersia menggabungkan giroskop dan akselerometer dengan versi sederhana Kalman filter (Bab 16). Filter Wiener dan estimator MMSE dalam pemrosesan sinyal adalah saudara dekatnya (Bab 14). Dalam bioinformatika, hidden Markov model dipakai untuk mencari gen dan menjajarkan keluarga protein (Durbin et al., 1998), dan algoritme EM menjadi dasar banyak metode analisis ekspresi gen (Bab 18).

Di dunia simulasi, ide Kalman filter dikenal sebagai *data assimilation*. Prakiraan cuaca modern bekerja dengan cara ini: model atmosfer meramalkan keadaan beberapa jam ke depan, lalu ramalan itu dikoreksi dengan pengamatan dari satelit, balon, dan stasiun cuaca, dengan bobot sesuai ketidakpastian masing-masing. Model atmosfer punya jutaan variabel, sehingga matriks kovarians P tidak mungkin disimpan. Ensemble Kalman filter (Evensen, 1994) memperkirakannya dari sekumpulan simulasi yang berjalan paralel dengan kondisi awal sedikit berbeda. Sebaran ensemble menjadi ukuran ketidakpastian ramalan.

Gagasan yang sama berguna di skala laboratorium. Simulasi CFD sebuah tangki dan pengukuran temperatur dari sensor di dalam tangki yang sama jarang cocok sempurna. Model turbulensi, sifat fluida, dan syarat batas semuanya menyimpan galat. Daripada memilih salah satu, kita bisa menarik simulasi secara perlahan ke arah pengukuran di titik-titik sensor, dengan kekuatan yang mencerminkan seberapa dipercaya masing-masing. Teknik paling sederhana seperti ini disebut *nudging*, dan ia adalah kerabat dekat Kalman filter dengan gain yang dipilih tangan.

Cara berpikir probabilistik juga membuka pintu ke masalah invers. Kalau kita ingin menyimpulkan parameter fisika, misalnya konduktivitas atau syarat batas, dari pengukuran yang berisik, jawaban yang jujur bukan satu angka, melainkan sebuah distribusi posterior. MCMC, inferensi variasional, dan model generatif dari Bab 11 adalah alat untuk menghitungnya.

CATATAN SUMBER. Bab ini mengikuti kuliah Probabilistic Models (Bayesian network, inferensi eksak dan aproksimasi, pembelajaran parameter dan struktur, DBN, HMM, Kalman filter) beserta latihannya. Slide kuliah bersandar pada Koller and Friedman (2009) dan Russell and Norvig (2021), yang juga menjadi bacaan pendamping terbaik. Bagian Kalman filter dan particle filter juga muncul di kuliah kendaraan otonom.

11 Dari Estimasi ke Difusi

Kuliah machine learning lanjut saya ikuti pada musim dingin 2025/26, di bawah Johannes Brandstetter, yang risetnya sendiri berada di persimpangan deep learning dan simulasi fisika. Delapan unitnya membentuk satu alur panjang: dari teori estimasi dan statistik Bayesian, lewat VAE, persamaan diferensial biasa, parsial, dan stokastik, sampai model difusi, flow matching, dan neural field. Tugas-tugasnya meliputi probabilistic PCA, score-based diffusion dan neural ODE, serta variational diffusion model. Di antara semua kuliah, inilah yang paling langsung mengarah ke AI untuk simulasi.

Benang merah bab ini sederhana untuk diucapkan, meski butuh beberapa halaman untuk dipahami: model generatif modern adalah persamaan diferensial. Model difusi adalah persamaan difusi yang dijalankan mundur. Flow matching adalah persamaan transport. Neural field adalah fungsi kontinu yang bisa diatur oleh persamaan fisika. Untuk sampai ke sana, kita mulai dari pertanyaan yang paling dasar: seberapa baik kita bisa menaksir sesuatu dari data.

11.1 Teori estimasi

Misalkan data $\mathcal{D}$ dibangkitkan oleh distribusi dengan parameter θ yang tidak diketahui, dan kita membuat sebuah estimator $\hat{\theta}(\mathcal{D})$. Karena data acak, estimator pun acak. Kualitasnya diukur dengan mean squared error yang dirata-ratakan atas semua kemungkinan data, dan MSE itu terurai menjadi varians dan bias kuadrat,

$$\text{MSE}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta)^2] = \text{Var}(\hat{\theta}) + (\mathbb{E}[\hat{\theta}] - \theta)^2. \quad (11.1)$$

Bentuknya sama dengan dekomposisi bias–varians di Bab 1, kali ini untuk parameter, bukan untuk prediksi. Rata-rata sampel adalah estimator tak bias untuk rata-rata distribusi. Varians sampel yang dibagi N bias ke bawah, dan pembagian dengan $N - 1$ menghilangkan bias itu. Estimator tak bias dengan varians terkecil di antara semua estimator tak bias disebut *minimum variance unbiased* (MVU).

Ada batas bawah mutlak untuk varians estimator tak bias mana pun. *Fisher information*

$$I(\theta) = -\mathbb{E}\left[\frac{\partial^2}{\partial \theta^2} \log p(\mathcal{D} \mid \theta)\right] \quad (11.2)$$

mengukur seberapa tajam puncak log-likelihood di sekitar nilai yang benar. Puncak yang tajam berarti data membawa banyak informasi tentang θ . Batas Cramér–Rao menyatakan $\text{Var}(\hat{\theta}) \geq 1/I(\theta)$ untuk setiap estimator tak bias. Untuk rata-rata distribusi normal dengan varians σ^2 yang diketahui dan N sampel, $I = N/\sigma^2$, sehingga varians terkecil yang mungkin adalah σ^2/N . Rata-rata sampel mencapai batas ini, jadi ia estimator yang efisien. Maximum likelihood, secara umum, tidak selalu tak bias untuk

sampel kecil, tetapi untuk sampel besar ia konsisten dan mencapai batas Cramér–Rao secara asimtotik.

Batas Cramér–Rao punya terjemahan langsung di dunia simulasi. Kalau kita ingin menaksir sebuah parameter fisika, misalnya koefisien perpindahan panas, dari pengukuran temperatur, Fisher information mengatakan apakah percobaan kita cukup informatif. Kalau log-likelihood hampir datar terhadap parameter itu, tidak ada metode pintar yang bisa menaksirnya dengan baik. Masalahnya ada pada desain percobaan, bukan pada algoritmenya.

11.2 Cara pandang Bayesian

Pendekatan frekuentis menganggap θ tetap dan datanya acak. Pendekatan Bayesian menganggap θ sendiri tidak pasti dan mewakilinya dengan distribusi. Teorema Bayes memperbarui prior menjadi posterior setelah melihat data. Untuk beberapa pasangan likelihood dan prior, posterior punya bentuk yang sama dengan prior, dan pasangan seperti ini disebut *conjugate*. Contoh yang dibahas kuliah adalah Gamma dan Poisson. Kalau jumlah kejadian per hari mengikuti Poisson dengan laju λ , dan prior $\lambda \sim \text{Gamma}(\alpha, \beta)$, maka setelah mengamati $x_1, \dots, x_N$, posteriornya $\text{Gamma}(\alpha + \sum_i x_i, \beta + N)$. Dengan prior $\text{Gamma}(2, 1)$ dan pengamatan 3, 5, dan 4 kejadian, posteriornya $\text{Gamma}(14, 4)$ dengan rata-rata 3,5. Prior bertindak seperti dua kejadian yang “sudah teramati” dalam satu hari semu, dan bobotnya memudar seiring datangnya data.

Estimasi MAP mengambil puncak posterior, dan dengan prior Gauss pada bobot ia menghasilkan weight decay, seperti di Bab 1. Pendekatan Bayesian penuh tidak berhenti di puncak. Ia memprediksi dengan merata-ratakan atas seluruh posterior, dan normalisasi posterior, $p(\mathcal{D}) = \int p(\mathcal{D} | \theta)p(\theta)d\theta$, disebut *evidence*. Evidence bisa dipakai untuk membandingkan model: model yang terlalu sederhana tidak bisa menjelaskan data, sedangkan model yang terlalu fleksibel menyebar peluangnya ke terlalu banyak kemungkinan data, sehingga keduanya mendapat evidence yang rendah.

11.3 Variabel laten dan probabilistic PCA

Banyak data lebih mudah dijelaskan kalau kita membayangkan ada sebab tersembunyi di baliknya. *Latent variable model* menulis $p(x) = \int p(x | z)p(z)dz$, dengan z yang tidak pernah diamati. Contoh paling sederhana adalah probabilistic PCA (Tipping and Bishop, 1999):

$$x = Wz + \mu + \varepsilon, \quad z \sim \mathcal{N}(0, I_q), \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I_D). \quad (11.3)$$

Karena semuanya Gauss, distribusi marginalnya juga Gauss, $x \sim \mathcal{N}(\mu, WW^\top + \sigma^2 I)$, dan maximum likelihood punya solusi tertutup. Kolom W terbentang oleh q vektor eigen utama dari kovarians sampel, diskalakan oleh $(\lambda_i - \sigma^2)^{1/2}$, dan σ^2 adalah rata-rata nilai eigen yang dibuang. Ketika $\sigma^2 \rightarrow 0$, PCA biasa muncul kembali. Keuntungan versi probabilistik adalah adanya likelihood, sehingga data yang hilang bisa ditangani, model bisa dibandingkan, dan beberapa model bisa digabung.

Kalau $p(x | z)$ tidak lagi linear, misalnya berupa jaringan saraf, integral marginal tidak bisa dihitung lagi, dan posterior $p(z | x)$ pun tidak. Dari sinilah VAE lahir.

11.4 ELBO, sekali lagi dan lebih teliti

Bab 8 memperkenalkan VAE secara intuitif. Kuliah ini menurunkannya. Ambil sembarang distribusi $q(z | x)$. Log-likelihood bisa ditulis sebagai

$$\log p(x) = \underbrace{\mathbb{E}_{q(z|x)} \left[\log \frac{p(x, z)}{q(z | x)} \right]}_{\text{ELBO}} + \text{KL}(q(z | x) \| p(z | x)). \quad (11.4)$$

Identitas ini didapat dengan satu trik. Karena $\log p(x)$ tidak bergantung pada z , ia sama dengan ekspektasinya di bawah q . Lalu tulis $p(x) = p(x, z)/p(z | x)$, kalikan dan bagi dengan $q(z | x)$ di dalam logaritma, dan pisahkan menjadi dua suku:

$$\log p(x) = \mathbb{E}_q \left[\log \frac{p(x, z)}{q(z | x)} \right] + \mathbb{E}_q \left[\log \frac{q(z | x)}{p(z | x)} \right].$$

Suku kedua adalah definisi divergensi KL. Divergensi KL tidak pernah negatif, sehingga suku pertama adalah batas bawah untuk $\log p(x)$. Memaksimalkan ELBO terhadap q sama dengan mendekatkan q ke posterior yang sebenarnya, dan memaksimalkannya terhadap parameter model menaikkan likelihood. Kalau suku pertama diuraikan, kita mendapat dua bagian yang sudah dikenal: rekonstruksi, $\mathbb{E}_q[\log p(x | z)]$, dan pencocokan prior, $-\text{KL}(q(z | x) \| p(z))$.

Divergensi KL sendiri layak dipahami. $\text{KL}(q \| p) = \mathbb{E}_q[\log q - \log p]$ mengukur seberapa banyak informasi yang hilang kalau p dipakai untuk mewakili q . Ia tidak simetris. Meminimalkan $\text{KL}(q \| p)$ terhadap q membuat q menghindari daerah tempat p kecil, sehingga q cenderung memilih satu puncak p dan mengabaikan puncak lain.

Model campuran Gauss memberi contoh yang menjembatani. Data dianggap berasal dari salah satu dari K distribusi Gauss, dan label komponennya adalah variabel laten diskret. Algoritme EM untuk model ini, yang sudah kita temui di Bab 10, ternyata adalah maksimisasi ELBO bergantian: langkah E memilih q sama dengan posterior yang tepat, dan langkah M memperbarui parameter. VAE melakukan hal yang sama, tetapi karena posteriornya tidak bisa dihitung, q diwakili oleh jaringan encoder, dan gradien dibawa melewati sampling dengan reparameterization trick.

Langkah terakhir menuju model difusi adalah VAE hierarkis. Alih-alih satu lapisan laten, kita punya rantai $x \rightarrow z_1 \rightarrow z_2 \rightarrow \dots \rightarrow z_T$, dan pada versi Markov setiap lapisan hanya bergantung pada lapisan di sebelahnya. ELBO-nya terurai menjadi jumlah suku untuk setiap transisi. Model difusi adalah VAE hierarkis Markov dengan dua keputusan istimewa: dimensi setiap laten sama dengan dimensi data, dan encoder-nya tidak dipelajari, melainkan ditetapkan sebagai penambahan noise Gauss sedikit demi sedikit (Luo, 2022).

11.5 Persamaan diferensial biasa, parsial, dan stokastik

Sebelum masuk ke model difusi, kuliah berhenti sejenak untuk mengulang matematika yang akan dipakai.

11.5.1 ODE

Persamaan diferensial biasa, $\dot{x} = f(x, t)$ dengan $x(0) = x_0$, menggambarkan sistem yang perubahannya ditentukan oleh keadaannya sekarang. Untuk sistem linear $\dot{x} = Ax$, solusinya $x(t) = e^{At}x_0$, dan nilai eigen A menentukan nasibnya: bagian real negatif berarti peluruhan, bagian real positif berarti pertumbuhan, bagian imajiner berarti osilasi. Teorema Picard–Lindelöf menjamin solusi ada dan tunggal kalau f Lipschitz.

Kebanyakan ODE tidak punya solusi tertutup, jadi kita menyelesaikannya secara numerik. Metode Euler, $x_{n+1} = x_n + h f(x_n, t_n)$, punya galat global sebanding dengan langkah h . Runge–Kutta orde empat mengevaluasi f empat kali per langkah dan galatnya sebanding dengan h^4 . Untuk sistem kaku (*stiff*), yaitu yang punya skala waktu cepat sekali dan lambat sekaligus, metode eksplisit terpaksa memakai langkah kecil sekali agar stabil, dan metode implisit atau langkah adaptif menjadi pilihan yang lebih baik. Topik ini dibahas lebih dalam di Bab 24.

11.5.2 PDE dan persamaan panas

Persamaan diferensial parsial melibatkan turunan terhadap lebih dari satu variabel. Contoh yang dibahas panjang lebar di kuliah adalah persamaan panas pada selang $(0, 1)$,

$$\partial_t u = \kappa \partial_{xx} u, \quad u(0, t) = u(1, t) = 0. \quad (11.5)$$

Pemisahan variabel memberi solusi

$$u(x, t) = \sum_{n=1}^{\infty} b_n e^{-\kappa(n\pi)^2 t} \sin(n\pi x), \quad (11.6)$$

dengan koefisien b_n diambil dari kondisi awal. Setiap mode Fourier meluruh dengan laju yang sebanding dengan kuadrat frekuensinya. Detail halus hilang paling cepat, dan setelah cukup lama yang tersisa hanya gelombang paling panjang. Pada garis bilangan penuh, transformasi Fourier memberi gambaran yang sama, $\hat{u}(k, t) = \hat{u}(k, 0) e^{-\kappa k^2 t}$, yang di ruang posisi berarti konvolusi dengan kernel Gauss yang semakin lebar. Pengamatan ini akan berguna sebentar lagi: menambahkan noise Gauss pada data sama dengan menjalankan persamaan panas pada distribusinya.

11.5.3 Gerak Brown dan SDE

Gerak Brown W_t adalah batas acak dari jalan acak dengan langkah yang semakin kecil. Kenaikannya independen dan berdistribusi normal, $W_{t+\Delta t} - W_t \sim \mathcal{N}(0, \Delta t)$. Simpangannya tumbuh seperti $\sqrt{\Delta t}$, bukan Δt , sehingga lintasannya kontinu tetapi tidak bisa diturunkan di mana pun. Sebuah jalan acak memberi gambaran: kalau setiap langkah waktu Δt kita melangkah ke kiri atau ke kanan sejauh $\sqrt{\Delta t}$, setelah waktu $t = n\Delta t$ variansnya $n\Delta t = t$, tidak bergantung pada seberapa halus langkahnya. Kalau

langkahnya sebanding Δt , variansnya akan menyusut ke nol ketika langkah diperhalus. *Stochastic differential equation* menambahkan suku acak ini ke ODE,

$$dx = f(x, t) dt + g(t) dW_t, \quad (11.7)$$

dengan f sebagai *drift* dan g sebagai *diffusion*. Kalkulus untuk persamaan ini punya satu kejutan. Karena $(dW)^2$ berukuran dt , bukan nol, aturan rantai mendapat suku tambahan. Lemma Itô untuk fungsi skalar $\varphi(x)$ berbunyi

$$d\varphi = \left(\varphi' f + \frac{1}{2} g^2 \varphi'' \right) dt + \varphi' g dW. \quad (11.8)$$

Asal suku tambahan itu terlihat dari Taylor orde dua: $d\varphi = \varphi' dx + \frac{1}{2} \varphi'' (dx)^2 + \dots$. Dengan $dx = f dt + g dW$, kuadratnya memuat $g^2 (dW)^2$, dan karena $(dW)^2$ berukuran dt , suku itu bertahan, sedangkan $(dt)^2$ dan $dt dW$ lebih kecil dan hilang. Contoh klasiknya gerak Brown geometris, $dS = \mu S dt + \sigma S dW$, model sederhana harga saham. Dengan $\varphi = \log S$, lemma Itô memberi $d \log S = (\mu - \frac{1}{2} \sigma^2) dt + \sigma dW$, sehingga $S_t = S_0 \exp((\mu - \frac{1}{2} \sigma^2)t + \sigma W_t)$. Suku $-\frac{1}{2} \sigma^2$ adalah jejak kalkulus Itô yang tidak akan muncul dalam kalkulus biasa.

Distribusi $p(x, t)$ dari solusi SDE berevolusi menurut persamaan Fokker–Planck,

$$\partial_t p = -\nabla \cdot (f p) + \frac{1}{2} g^2 \Delta p. \quad (11.9)$$

Suku pertama adalah transport oleh drift, suku kedua adalah difusi. Untuk $f = 0$, ini persis persamaan panas. Sebuah SDE yang bekerja pada partikel-partikel menjadi PDE yang bekerja pada kerapatannya.

11.6 Neural ODE

Blok residual, $\mathbf{h}_{l+1} = \mathbf{h}_l + F(\mathbf{h}_l)$, terlihat seperti satu langkah Euler. Kalau jumlah lapisan dibuat tak hingga dan setiap langkahnya kecil, kita mendapat

$$\frac{d\mathbf{h}}{dt} = f_\theta(\mathbf{h}, t), \quad (11.10)$$

sebuah *neural ODE* (Chen et al., 2018). Lintasan maju dihitung dengan solver ODE apa pun, termasuk yang langkahnya adaptif. Untuk gradien, kita bisa saja melakukan backprop melewati setiap langkah solver, tetapi memorinya sebanding dengan jumlah langkah. Metode adjoint, yang dikenal lama dalam teori kendali optimal, menghindarinya. Definisikan $\mathbf{a}(t) = \partial \mathcal{L} / \partial \mathbf{h}(t)$. Vektor ini memenuhi ODE-nya sendiri yang berjalan mundur,

$$\frac{d\mathbf{a}}{dt} = -\mathbf{a}^\top \frac{\partial f_\theta}{\partial \mathbf{h}}, \quad \frac{d\mathcal{L}}{d\theta} = - \int_{t_1}^{t_0} \mathbf{a}^\top \frac{\partial f_\theta}{\partial \theta} dt. \quad (11.11)$$

Keadaan $\mathbf{h}$, adjoint $\mathbf{a}$, dan gradien terhadap parameter diintegrasikan mundur bersama dalam satu sistem yang diperbesar. Memorinya konstan terhadap jumlah langkah. Persamaan adjoint ini tidak lain adalah delta backprop dari Bab 5 dalam waktu kontinu.

Neural ODE cocok sekali untuk sistem fisika yang memang berevolusi secara kontinu. Data yang diambil pada waktu tidak teratur bisa langsung dipakai, dan model dinamika yang dipelajari bisa dijalankan dengan solver standar. Hubungan dengan model difusi juga erat: sampling deterministik dari model difusi adalah menyelesaikan sebuah ODE.

11.7 Model difusi

Model difusi denoising (Ho et al., 2020; Sohl-Dickstein et al., 2015) menetapkan proses maju yang merusak data selangkah demi selangkah,

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}), \quad (11.12)$$

dengan jadwal noise β_t yang kecil. Karena komposisi Gauss tetap Gauss, kita bisa melompat langsung dari data ke langkah mana pun: dengan $\bar{\alpha}_t = \prod_{s \leq t} (1 - \beta_s)$,

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (11.13)$$

Bentuk tertutup ini bisa diperiksa dengan dua langkah. Tulis $\alpha_t = 1 - \beta_t$. Satu langkah memberi $\mathbf{x}_1 = \sqrt{\alpha_1} \mathbf{x}_0 + \sqrt{1 - \alpha_1} \boldsymbol{\epsilon}_1$, dan langkah berikutnya $\mathbf{x}_2 = \sqrt{\alpha_2} \mathbf{x}_1 + \sqrt{1 - \alpha_2} \boldsymbol{\epsilon}_2 = \sqrt{\alpha_1 \alpha_2} \mathbf{x}_0 + \sqrt{\alpha_2(1 - \alpha_1)} \boldsymbol{\epsilon}_1 + \sqrt{1 - \alpha_2} \boldsymbol{\epsilon}_2$. Dua noise Gauss independen yang dijumlahkan tetap Gauss, dengan varians yang dijumlahkan: $\alpha_2(1 - \alpha_1) + (1 - \alpha_2) = 1 - \alpha_1 \alpha_2$. Jadi $\mathbf{x}_2 = \sqrt{\alpha_1 \alpha_2} \mathbf{x}_0 + \sqrt{1 - \alpha_1 \alpha_2} \boldsymbol{\epsilon}$, dan induksi memberi rumus umumnya. Dengan jadwal β_t yang naik linear dari 10^{-4} ke 0,02 dalam seribu langkah, seperti pada Ho et al. (2020), nilai $\bar{\alpha}_T$ sekitar 4×10^{-5} : sinyal aslinya tinggal kurang dari satu persen dalam amplitudo. Setelah cukup banyak langkah, $\mathbf{x}_T$ hampir murni noise. Proses mundur dipelajari oleh sebuah jaringan. Dari ELBO VAE hierarkis, setelah beberapa langkah aljabar, loss pelatihannya menjadi sederhana: tebak noise yang ditambahkan,

$$\mathcal{L} = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}} \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2. \quad (11.14)$$

Pada saat sampling, kita mulai dari noise murni dan berulang kali mengurangi noise yang ditebak jaringan, ditambah sedikit noise baru, sampai kembali ke data.

Bayangkan setetes tinta jatuh ke segelas air. Tinta menyebar perlahan sampai airnya berwarna rata, dan bentuk awal tetes itu tampak hilang. Model difusi belajar memutar video itu mundur: dari air yang keruh merata, langkah demi langkah, sampai tetes tinta terbentuk kembali. Setiap langkah mundur hanya perlu sedikit menebak, karena setiap langkah maju hanya sedikit merusak.

Kelemahan DDPM adalah lambat, karena sampling membutuhkan ratusan sampai ribuan langkah jaringan. DDIM (Song et al., 2020a) menunjukkan bahwa jaringan yang sama bisa dipakai dengan proses sampling deterministik yang melompati banyak langkah, tanpa pelatihan ulang. Variational diffusion models (Kingma et al., 2021) merumuskan jadwal noise sebagai rasio sinyal terhadap noise yang bisa dipelajari dan menunjukkan bahwa ELBO model difusi hanya bergantung pada jadwal itu melalui nilai ujung-ujungnya.

11.8 Score dan persamaan diferensial stokastik

Song et al. (2020b) menyatukan model difusi dalam bahasa SDE. Proses maju adalah SDE $d\mathbf{x} = f(\mathbf{x}, t) dt + g(t) d\mathbf{W}$ yang mengubah distribusi data menjadi Gauss. Hasil lama dari Anderson (1982) menyatakan bahwa proses ini punya kebalikan dalam waktu, yang juga sebuah SDE,

$$d\mathbf{x} = [f(\mathbf{x}, t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x})] dt + g(t) d\bar{\mathbf{W}}, \quad (11.15)$$

yang berjalan dari $t = T$ ke $t = 0$. Satu-satunya besaran yang tidak diketahui adalah *score* $\nabla_x \log p_t(x)$, yaitu arah tempat kerapatan data naik paling cepat. Jaringan $s_\theta(x, t)$ dilatih untuk menebak *score* ini. Melatihnya ternyata mudah berkat *denoising score matching* (Vincent, 2011): kalau $\tilde{x} = x + \sigma\epsilon$, *score* dari distribusi bersyarat $p(\tilde{x} | x)$ adalah $-(\tilde{x} - x)/\sigma^2 = -\epsilon/\sigma$, sehingga menebak *score* sama dengan menebak *noise*, persis seperti DDPM.

Ada juga ODE yang menghasilkan distribusi marginal yang sama di setiap waktu, yaitu *probability flow ODE*,

$$dx = \left[f(x, t) - \frac{1}{2}g(t)^2 \nabla_x \log p_t(x) \right] dt. \quad (11.16)$$

Mengapa SDE dan ODE bisa menghasilkan marginal yang sama? Persamaan Fokker-Planck menjawabnya. Suku difusi $\frac{1}{2}g^2\Delta p$ bisa ditulis sebagai $\nabla \cdot (\frac{1}{2}g^2 p \nabla \log p)$, sehingga difusi pada kerapatan bisa diganti dengan transport deterministik dengan kecepatan $-\frac{1}{2}g^2 \nabla \log p$. Partikel-partikel individu bergerak berbeda, tetapi kerumunannya berevolusi dengan cara yang sama. Kuliah merangkum bahwa ketiga cara melatih model difusi, yaitu menebak data bersih, menebak *noise*, dan menebak *score*, setara sampai faktor skala. Untuk membangkitkan sampel bersyarat, misalnya gambar dari teks, *score* ditambah dengan gradien sebuah penilai syarat, atau syarat langsung diberikan sebagai input ke jaringan.

11.9 Flow matching

Kalau ujung-ujungnya kita menyelesaikan sebuah ODE, mengapa tidak langsung belajar ODE-nya? *Flow matching* (Lipman et al., 2022) melakukan hal itu. Kita mencari medan kecepatan $v_\theta(x, t)$ sehingga ODE $\dot{x} = v_\theta(x, t)$ membawa sampel dari distribusi sederhana p_0 , misalnya Gauss, ke distribusi data p_1 . Pemetaan yang dihasilkan disebut *flow* dalam arti matematis, dan barisan distribusi antara p_t disebut *probability path*.

Kesulitannya, medan kecepatan yang benar untuk seluruh distribusi tidak diketahui. Kunci *flow matching* adalah bekerja secara bersyarat. Untuk satu pasangan titik, sampel *noise* x_0 dan sampel data x_1 , kita bisa memilih jalur yang paling sederhana, garis lurus $x_t = (1 - t)x_0 + tx_1$, yang kecepatannya konstan, $x_1 - x_0$. *Loss conditional flow matching* memaksa jaringan meniru kecepatan bersyarat itu,

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t, x_0, x_1} \left\| v_\theta(x_t, t) - (x_1 - x_0) \right\|^2, \quad (11.17)$$

dan hasil utamanya adalah bahwa gradien loss ini sama dengan gradien loss untuk medan kecepatan marginal yang sebenarnya. Kita tidak pernah perlu mengetahui medan marginal itu. Gagasan jalur lurus juga dikembangkan secara independen sebagai *rectified flow* (Liu et al., 2022).

Analogi yang saya pakai adalah rombongan penumpang yang harus dipindahkan dari terminal ke berbagai alamat di kota. Kita tidak tahu jalur terbaik untuk seluruh rombongan, tetapi kita tahu jalur lurus untuk setiap pasangan terminal dan alamat. Kalau sopir belajar rata-rata dari jutaan perjalanan lurus seperti itu, ia akhirnya tahu arah yang tepat di setiap persimpangan untuk seluruh arus penumpang. Persamaan

kontinuitas yang mengatur probability path, $\partial_t p_t + \nabla \cdot (p_t v_t) = 0$, adalah persamaan kekekalan massa yang sama dengan di mekanika fluida.

11.10 Neural field

Unit terakhir kuliah meninggalkan model generatif dan membahas cara lain merepresentasikan data. Dalam fisika, medan adalah besaran yang punya nilai di setiap titik ruang dan waktu: temperatur, tekanan, kecepatan. *Neural field* adalah jaringan saraf yang mewakili medan seperti itu secara langsung, $f_\theta : (x, t) \mapsto \text{nilai}$ (Xie et al., 2021). Alih-alih menyimpan nilai di grid, kita menyimpan bobot jaringan. Representasinya kontinu, tidak terikat resolusi, dan bisa diturunkan di mana saja dengan autodiff.

Bentuk tiga dimensi bisa diwakili sebagai occupancy field, yaitu peluang sebuah titik berada di dalam benda, atau sebagai *signed distance field* $f(x)$, jarak bertanda dari titik ke permukaan, positif di luar dan negatif di dalam (Park et al., 2019). Permukaannya adalah himpunan nol $f(x) = 0$, dan bisa diekstrak dengan algoritme seperti marching cubes. Fungsi jarak yang sejati memenuhi persamaan Eikonal, $\|\nabla f(x)\| = 1$. Menambahkan $(\|\nabla f_\theta\| - 1)^2$ ke loss di titik-titik acak memaksa jaringan menjadi fungsi jarak yang sah (Gropp et al., 2020). Ini adalah loss yang diinformasikan fisika: sebuah persamaan diferensial parsial ditegakkan di dalam loss.

NeRF, yang sudah kita lihat di Bab 7, adalah neural field untuk kerapatan dan warna yang dilatih melalui persamaan volume rendering. Kuliah menutup dengan pernyataan yang menjembatani ke Bagian III: dari sudut pandang ini, PINN hanyalah neural field yang diatur oleh persamaan fisika. Bedanya dengan NeRF hanya pada persamaan apa yang ditegakkan.

Satu masalah praktis menghubungkan semuanya. MLP biasa dengan aktivasi ReLU cenderung belajar komponen frekuensi rendah lebih dulu dan kesulitan mewakili detail halus, gejala yang disebut *spectral bias* (Rahaman et al., 2018). Dua obat yang populer adalah memetakan input dengan fungsi sinus dan kosinus berbagai frekuensi sebelum masuk jaringan (Tancik et al., 2020), atau memakai fungsi sinus sebagai aktivasi seperti pada SIREN (Sitzmann et al., 2020). Spectral bias juga menjadi salah satu alasan PINN sulit mempelajari solusi yang berosilasi cepat (Bab 25).

11.11 Di luar kelas

Alat-alat dari bab ini menyebar jauh. Model difusi membangkitkan gambar dan video (Bab 32), dan modul struktur AlphaFold3 adalah model difusi yang membangkitkan koordinat atom (Bab 20). Teori estimasi dan batas Cramér–Rao adalah bahasa sehari-hari pemrosesan sinyal (Bab 14). Probabilistic PCA dan analisis faktor dipakai untuk meringkas data ekspresi gen yang berisik (Bab 18). Neural ODE cocok untuk data medis yang diukur pada waktu tidak teratur, seperti kunjungan pasien yang jaraknya berbeda-beda.

Bagi Bagian V, bab ini memberi hampir semua alatnya. Neural ODE adalah cara alami untuk mempelajari dinamika sistem fisika dari data. Neural field adalah kerangka PINN. Model difusi dan flow matching dipakai untuk membangkitkan medan turbulen

yang realistis, untuk memperbaiki prediksi jangka panjang model pengganti (Lippe et al., 2023), dan untuk prakiraan cuaca probabilistik (Price et al., 2025). Bahkan cara berpikirnya pun berasal dari fisika: model difusi adalah persamaan panas pada ruang distribusi, dan flow matching adalah persamaan kontinuitas. Batas Cramér–Rao mengingatkan bahwa ketika kita mencoba menyimpulkan parameter fisika dari data, informasi yang terkandung di dalam data membatasi apa yang bisa dicapai oleh metode mana pun.

CATATAN SUMBER. Bab ini mengikuti delapan unit kuliah Machine Learning: Advanced Techniques: teori estimasi, statistik Bayesian, probabilistic PCA, VAE dan ELBO, ODE, PDE, dan SDE, neural ODE, model difusi (DDPM, DDIM), score-based model dan flow matching, Fokker–Planck, dan neural field. Bacaan utamanya adalah makalah-makalah aslinya: Tipping and Bishop (1999), Kingma and Welling (2013), Chen et al. (2018), Ho et al. (2020), Song et al. (2020b), Lipman et al. (2022), dan ulasan Luo (2022).

12 Belajar dari Ganjaran: Reinforcement Learning

Reinforcement learning datang kepada saya dalam dua tahap. Pada musim panas 2025, kuliah deep reinforcement learning langsung membawa kami ke imitation learning, policy gradient, dan credit assignment, dengan proyek latihan berupa agen DQN, agen PPO, dan sebuah tantangan akhir. Pada musim dingin berikutnya, kuliah reinforcement learning mengisi fondasinya secara rapi: proses keputusan Markov, dynamic programming, metode Monte Carlo, dan temporal difference. Bab ini menyusunnya dalam urutan yang lebih logis, fondasi dulu, lalu deep RL.

Dua tipe belajar di bab-bab sebelumnya selalu punya data yang sudah tersedia. Reinforcement learning berbeda. Agen harus mengumpulkan datanya sendiri dengan bertindak, dan umpan baliknya bukan jawaban yang benar, melainkan angka ganjaran yang sering datang terlambat. Seekor anjing yang belajar duduk tidak diberi tahu gerakan otot apa yang harus dilakukan. Ia mencoba sesuatu, dan kadang mendapat camilan. Tugasnya adalah mencari tahu tindakan mana yang membuat camilan itu datang.

12.1 Proses keputusan Markov

Kerangka formalnya adalah *Markov decision process* (MDP): himpunan keadaan $\mathcal{S}$, himpunan aksi $\mathcal{A}$, dinamika $p(s', r | s, a)$ yang memberi peluang keadaan berikutnya dan ganjarannya, serta faktor diskon $\gamma \in [0, 1)$ (Sutton and Barto, 2018). Di setiap langkah, agen melihat keadaan s_t , memilih aksi a_t menurut *policy* $\pi(a | s)$, lalu menerima ganjaran r_{t+1} dan keadaan baru. Tujuannya memaksimalkan *return*, jumlah ganjaran yang didiskon,

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \quad (12.1)$$

Faktor diskon membuat ganjaran yang jauh di masa depan bernilai lebih kecil, dan secara matematis menjaga jumlahnya tetap terhingga.

Seberapa baik berada di sebuah keadaan diukur oleh *value function* $V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$, dan seberapa baik mengambil sebuah aksi di sebuah keadaan oleh *action-value function* $Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$. Kedua fungsi memenuhi persamaan rekursif yang menjadi jantung seluruh bidang ini, yaitu persamaan Bellman:

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma V^\pi(s')]. \quad (12.2)$$

Nilai sebuah keadaan sama dengan ganjaran langsung ditambah nilai keadaan berikutnya yang didiskon, dirata-ratakan atas semua yang mungkin terjadi. Untuk policy

terbaik, rata-rata atas aksi diganti dengan maksimum, dan kita mendapat persamaan Bellman optimal, $V^*(s) = \max_a \sum_{s',r} p(s', r | s, a)[r + \gamma V^*(s')]$.

Persamaan ini turun langsung dari definisi return. Karena $G_t = r_{t+1} + \gamma G_{t+1}$, ekspektasinya dari keadaan s adalah $\mathbb{E}[r_{t+1} + \gamma G_{t+1} | s_t = s]$. Ekspektasi ini dihitung bertahap: pilih aksi menurut π , dapatkan ganjaran dan keadaan berikutnya menurut p , lalu ganti $\mathbb{E}[G_{t+1} | s_{t+1} = s']$ dengan $V^\pi(s')$ berkat sifat Markov, karena masa depan setelah s' tidak bergantung pada bagaimana kita sampai di sana.

Contoh paling kecil sudah memperlihatkan maknanya. Bayangkan satu keadaan dengan satu aksi yang selalu memberi ganjaran satu dan kembali ke keadaan yang sama. Persamaan Bellman menjadi $V = 1 + \gamma V$, sehingga $V = 1/(1 - \gamma)$. Dengan $\gamma = 0,9$, nilainya 10. Faktor diskon bisa dibaca sebagai cakrawala efektif: dengan $\gamma = 0,9$, agen kira-kira peduli pada sepuluh langkah ke depan.

12.2 Dynamic programming

Kalau dinamika MDP diketahui, persamaan Bellman bisa diselesaikan dengan dynamic programming (Bellman, 1957). *Policy evaluation* menghitung V^π dengan menerapkan persamaan Bellman berulang-ulang sebagai aturan update sampai nilainya tidak berubah lagi. Karena faktor diskon membuat update ini kontraktif, iterasinya pasti konvergen. *Policy improvement* lalu membuat policy baru yang serakah terhadap V^π , memilih di setiap keadaan aksi dengan nilai harapan tertinggi. Teorema perbaikan policy menjamin policy baru ini tidak lebih buruk dari yang lama. Bergantian antara keduanya disebut *policy iteration*, dan ia berakhir di policy optimal dalam jumlah langkah terhingga untuk MDP terhingga. *Value iteration* memadatkan keduanya dengan langsung menerapkan persamaan Bellman optimal sebagai update.

Kuliah memakai dunia grid kecil sebagai contoh. Agen bergerak di petak-petak, setiap langkah memberi ganjaran -1 , dan permainan berakhir di petak sudut. Setelah policy evaluation untuk policy acak, nilai setiap petak mencerminkan rata-rata jumlah langkah yang dibutuhkan untuk mencapai sudut dengan berjalan acak. Policy yang serakah terhadap nilai-nilai itu sudah langsung menunjuk jalur terpendek.

12.3 Belajar dari pengalaman

Di dunia nyata, dinamika jarang diketahui. Agen harus belajar dari sampel.

Metode *Monte Carlo* menunggu sampai satu episode selesai, lalu memakai return yang benar-benar diperoleh sebagai perkiraan nilai setiap keadaan yang dikunjungi. Perkiraan ini tak bias, tetapi variansnya besar, karena satu return bergantung pada semua keputusan acak sampai akhir episode. Metode *temporal difference* (TD) tidak menunggu. Setelah satu langkah, ia memperbarui perkiraan dengan memakai perkiraannya sendiri tentang keadaan berikutnya,

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]. \quad (12.3)$$

Selisih di dalam kurung disebut *TD error*. Pendekatan ini, belajar tebakan dari tebakan, disebut *bootstrapping*. Variansnya kecil karena hanya satu langkah acak yang terlibat,

tetapi ada bias selama perkiraan $V(s_{t+1})$ belum benar. Di antara keduanya terbentang metode n -langkah dan eligibility traces yang mencampur keduanya.

Bayangkan memperkirakan waktu perjalanan pulang dari kantor. Metode Monte Carlo menunggu sampai Anda tiba di rumah, lalu mencatat total waktunya. Metode TD memperbarui perkiraan di setiap persimpangan: “jalanan ternyata macet, perkiraan saya untuk sisa perjalanan harus naik”, tanpa menunggu tiba.

Untuk mengendalikan, bukan hanya menilai, kita mempelajari Q . SARSA memperbarui $Q(s, a)$ dengan aksi berikutnya yang benar-benar diambil, sehingga ia belajar nilai policy yang sedang dijalankan. Q-learning (Watkins and Dayan, 1992) memakai aksi terbaik di keadaan berikutnya,

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)], \quad (12.4)$$

sehingga ia belajar nilai policy optimal sambil menjalankan policy lain yang lebih eksploratif. Perbedaan ini disebut on-policy dan off-policy.

Di sini muncul dilema yang tidak dikenal di supervised learning: *exploration* melawan *exploitation*. Kalau agen selalu memilih aksi yang sejauh ini terbaik, ia mungkin tidak pernah menemukan aksi yang sebenarnya lebih baik. Kalau ia terus mencoba hal baru, ia membuang ganjaran. Strategi paling sederhana adalah ϵ -greedy: pilih aksi terbaik hampir selalu, tetapi dengan peluang kecil ϵ pilih aksi acak. Ini mirip memilih tempat makan. Kalau selalu ke warung langganan, kita tidak pernah tahu ada warung yang lebih enak di gang sebelah.

12.4 Deep Q-network

Untuk keadaan yang amat banyak, seperti piksel layar permainan, tabel Q diganti jaringan saraf $Q_\theta(s, a)$. Gabungan ini ternyata tidak stabil, karena data yang dikumpulkan berurutan sangat berkorelasi dan targetnya terus berubah seiring jaringannya sendiri berubah. DQN (Mnih et al., 2015) memperbaikinya dengan dua trik. *Experience replay* menyimpan pengalaman di sebuah buffer dan melatih jaringan dengan mini-batch acak dari buffer itu, sehingga korelasi terputus. *Target network* adalah salinan jaringan yang diperbarui jarang-jarang dan dipakai untuk menghitung target, sehingga target tidak bergeser di setiap langkah. Dengan dua trik ini, satu arsitektur yang sama belajar memainkan puluhan permainan Atari dari piksel. Operator maksimum dalam target Q-learning cenderung melebih-lebihkan nilai, dan varian-varian DQN selanjutnya memisahkan pemilihan aksi dari penilaian aksinya untuk mengurangi bias itu.

12.5 Policy gradient

Pendekatan lain melewati fungsi nilai dan langsung mengoptimalkan policy berparameter $\pi_\theta(a | s)$. Teorema policy gradient memberi gradien return harapan $J(\theta)$ dalam bentuk yang bisa diperkirakan dari sampel,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) G_t \right]. \quad (12.5)$$

Kuncinya adalah *log-derivative trick*. Untuk satu lintasan τ dengan peluang $p_\theta(\tau)$ dan return $G(\tau)$, gradien return harapan adalah

$$\nabla_\theta \int p_\theta(\tau) G(\tau) d\tau = \int p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) G(\tau) d\tau,$$

karena $\nabla p = p \nabla \log p$. Bentuk kedua adalah sebuah ekspektasi, sehingga bisa diperkirakan dengan merata-ratakan atas lintasan yang dijalankan. Peluang lintasan adalah hasil kali peluang aksi dari policy dan peluang transisi dari lingkungan. Setelah logaritma diambil, hasil kali itu menjadi jumlah, dan suku-suku transisi lingkungan tidak bergantung pada θ , sehingga hilang saat diturunkan. Yang tersisa hanya $\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t)$. Inilah alasan policy gradient bisa dipakai tanpa model lingkungan sama sekali. Mengganti $G(\tau)$ dengan return sejak langkah t saja diperbolehkan, karena aksi di langkah t tidak bisa memengaruhi ganjaran yang sudah diterima sebelumnya.

Algoritme REINFORCE (Williams, 1992) memakai rumus ini langsung: naikan peluang aksi yang diikuti return tinggi, turunkan peluang aksi yang diikuti return rendah. Estimasi ini tak bias, tetapi variansnya besar. Mengurangkan sebuah *baseline* yang tidak bergantung pada aksi, biasanya perkiraan $V(s_t)$, tidak mengubah ekspektasinya tetapi mengurangi variansnya. Selisih $A_t = G_t - V(s_t)$ disebut *advantage*: seberapa jauh aksi ini lebih baik daripada rata-rata. Metode *actor-critic* melatih dua jaringan sekaligus, aktor yang memilih aksi dan kritikus yang menaksir nilai (Mnih et al., 2016), dan *generalized advantage estimation* (Schulman et al., 2015b) mencampur perkiraan *advantage* dari berbagai panjang langkah untuk mengatur keseimbangan bias dan varians.

Policy gradient punya kelemahan praktis: satu langkah yang terlalu besar bisa merusak policy, dan data baru yang dikumpulkan dengan policy rusak itu ikut buruk, sehingga pelatihan bisa runtuh. TRPO (Schulman et al., 2015a) membatasi setiap update agar policy baru tidak terlalu jauh dari yang lama dalam arti divergensi KL. PPO (Schulman et al., 2017) mencapai efek serupa dengan cara yang jauh lebih sederhana, yaitu memotong rasio peluang $\rho_t = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{lama}}}(a_t | s_t)$:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t) \right]. \quad (12.6)$$

Kalau sebuah aksi bagus ($A_t > 0$), peluangnya boleh naik, tetapi tidak lebih dari faktor $1 + \epsilon$ dalam satu update. Kalau aksi buruk, peluangnya boleh turun, tetapi tidak lebih dari faktor $1 - \epsilon$. PPO menjadi algoritme bawaan di banyak aplikasi karena kuat dan mudah disetel. Untuk aksi kontinu, soft actor-critic (Haarnoja et al., 2018) menambahkan entropi policy ke dalam tujuan, sehingga agen dihargai karena tetap acak selama hal itu tidak terlalu merugikan.

12.6 Meniru dan memberi kredit

12.6.1 Imitation learning

Kadang lebih mudah menunjukkan cara melakukan sesuatu daripada menulis fungsi ganjarannya. *Behavioral cloning* memperlakukan demonstrasi ahli sebagai data supervised: input keadaan, target aksi. Masalahnya adalah *covariate shift*. Begitu agen

sedikit menyimpang dari jalur ahli, ia sampai di keadaan yang tidak pernah dilihat ahli, membuat kesalahan lebih besar, dan menyimpang semakin jauh. DAgger (Ross et al., 2010) mengatasinya dengan menjalankan policy agen, meminta ahli memberi label pada keadaan yang dikunjungi agen, lalu melatih ulang. Pendekatan lain mempelajari fungsi ganjaran dari demonstrasi (*inverse reinforcement learning*), atau mempertandingkan agen dengan diskriminator yang membedakan perilakunya dari perilaku ahli, seperti pada GAIL (Ho and Ermon, 2016).

12.6.2 Credit assignment

Masalah tersulit RL adalah ganjaran yang tertunda. Dalam sebuah pertandingan sepak bola, gol terjadi di menit ke-80, tetapi umpan yang menentukan mungkin terjadi lima menit sebelumnya, dan kesalahan lawan yang membuka ruang mungkin terjadi lebih awal lagi. Metode TD menyebarkan kredit mundur satu langkah demi satu langkah, sehingga untuk penundaan panjang, belajarnya lambat sekali. RUDDER (Arjona-Medina et al., 2018), yang dikembangkan di Linz, mengambil pendekatan berbeda. Sebuah LSTM dilatih untuk memprediksi return total dari seluruh urutan keadaan dan aksi. Lalu, dengan menganalisis seberapa besar setiap langkah mengubah prediksi itu, return dibagi ulang ke langkah-langkah yang benar-benar berkontribusi. Ganjaran yang tertunda diubah menjadi ganjaran langsung, dan nilai harapan yang tersisa untuk dipelajari menjadi hampir nol. Kuliah deep RL di Linz membahas topik ini dengan nama direct credit assignment.

12.6.3 Kendali sebagai inferensi

Kuliah juga membahas cara pandang yang menyatukan RL dengan inferensi probabilistik (Bab 10). Kalau kita memperkenalkan variabel “optimalitas” yang peluangnya sebanding dengan eksponensial ganjaran, mencari policy optimal menjadi mirip menghitung posterior atas lintasan, dengan syarat bahwa lintasan itu optimal. Hasil dari cara pandang ini adalah RL dengan entropi maksimum, yang menjelaskan mengapa soft actor-critic menambahkan entropi ke dalam tujuannya.

12.7 Model, permainan, dan banyak agen

Buku Plaet (2022), yang menjadi bacaan kuliah, menyusun deep RL menjadi beberapa keluarga. Metode berbasis model mempelajari dinamika lingkungan lalu merencanakan di dalam model itu, sehingga butuh jauh lebih sedikit interaksi. *Self-play* melatih agen melawan salinan dirinya sendiri, seperti AlphaZero (Silver et al., 2018). RL multi-agen menghadapi lingkungan yang berubah karena agen lain juga belajar. RL hierarkis memecah tugas panjang menjadi subtugas, dan meta-learning melatih agen untuk cepat beradaptasi ke tugas baru.

12.8 Reinforcement learning di dunia nyata

Di luar permainan, RL dipakai untuk masalah kendali yang sulit dimodelkan secara analitis. Rabault et al. (2019) melatih agen untuk mengendalikan jet kecil di permukaan

silinder dalam simulasi CFD, dan agen itu menemukan strategi yang mengurangi gaya hambat. Ulasan Garnier et al. (2021) merangkum puluhan studi serupa dalam mekanika fluida, dari kendali aliran sampai optimasi bentuk. Dalam robotika, agen dilatih di simulator lalu dipindahkan ke robot nyata. Dalam sistem rekomendasi, memilih item yang ditampilkan kepada pengguna bisa dirumuskan sebagai masalah keputusan berurutan.

Penerapan yang paling berpengaruh beberapa tahun terakhir justru datang dari bahasa. Model bahasa yang sudah dilatih kemudian disetel dengan *reinforcement learning from human feedback*. Manusia membandingkan pasangan jawaban model, sebuah model ganjaran dilatih dari preferensi itu (Christiano et al., 2017), lalu model bahasa dioptimasi dengan PPO terhadap model ganjaran tersebut (Ouyang et al., 2022). Perkembangan ini kita bahas lagi di Bab 32.

CATATAN SUMBER. Bab ini menggabungkan kuliah Reinforcement Learning (MDP, dynamic programming, Monte Carlo, temporal difference, policy gradient, PPO) dan kuliah serta latihan Deep Reinforcement Learning (imitation learning, policy gradient, direct credit assignment, kendali dan inferensi, proyek DQN dan PPO). Bacaan utama: Sutton and Barto (2018) dan Plaatt (2022).

13 Membuka Kotak Hitam: AI dan Visualisasi

Kuliah AI and visualization saya ikuti pada musim dingin 2025/26, diberikan oleh kelompok Visual Data Science di Linz. Kuliahnya bertumpu pada satu rumus yang muncul sejak pertemuan pertama: AI ditambah visualisasi ditambah manusia sama dengan explainable AI. Penjelasan bukan sesuatu yang dikeluarkan mesin begitu saja. Ia harus dirancang untuk seseorang, dan visualisasi adalah salah satu alat terbaik untuk menyampaikannya.

Model yang paling akurat sering yang paling sulit dipahami. Untuk menyarankan film, itu tidak masalah. Untuk menolak kredit, mendiagnosis penyakit, atau menyetujui desain sebuah reaktor, kita perlu tahu alasan di balik keputusan. Bab ini membahas dua arah hubungan antara AI dan visualisasi: visualisasi untuk memahami AI, dan AI untuk membuat visualisasi.

13.1 Siapa yang butuh penjelasan, dan untuk apa

Penjelasan yang dibutuhkan pengembang model berbeda dari yang dibutuhkan dokter, regulator, atau orang yang permohonan kreditnya ditolak. Pengembang ingin menemukan bug dan bias. Pengguna ahli ingin tahu kapan model bisa dipercaya. Orang yang terdampak ingin tahu apa yang harus diubah agar keputusannya berbeda. Kuliah menyusun teknik-teknik XAI menurut pertanyaan pengguna: mengapa prediksi ini, mengapa bukan yang lain, bagaimana kalau input diubah, dan contoh mana yang mirip.

Ada juga pertukaran yang sering digambarkan antara kinerja dan keterjelasan. Regresi linear dan pohon keputusan kecil mudah dibaca, jaringan dalam tidak. Rudin (2019) berpendapat bahwa untuk keputusan berisiko tinggi, kita sebaiknya memakai model yang bisa ditafsirkan sejak awal daripada menjelaskan kotak hitam setelah jadi, karena penjelasan sesudahnya bisa saja tidak setia pada apa yang sebenarnya dilakukan model.

Kuliah membedakan dua arah. *Vis for ML* memakai visualisasi untuk memahami, men-debug, dan membandingkan model. *ML for Vis* memakai machine learning untuk membantu membuat visualisasi, misalnya merekomendasikan jenis grafik atau menghasilkan dashboard dari permintaan dalam bahasa biasa.

13.2 Dasar-dasar visualisasi

Visualisasi bekerja karena sistem visual manusia memproses pola dengan cepat. Tetapi tidak semua saluran visual sama kuatnya. Munzner (2014) menyusun peringkat untuk data kuantitatif: posisi pada sumbu yang sama paling akurat dibaca, disusul

panjang, sudut, luas, lalu kecerahan dan saturasi warna. Itulah sebabnya grafik batang biasanya lebih jujur daripada diagram lingkaran, dan mengapa grafik tiga dimensi yang memiringkan sumbu sering menyesatkan. Prinsip Gestalt menjelaskan bagaimana mata mengelompokkan objek: berdekatan, mirip warna atau bentuk, terhubung garis, atau berada di dalam wadah yang sama.

Visualisasi juga bisa bercerita. Cerita yang digerakkan penulis membawa pembaca melalui urutan yang sudah ditentukan, seperti artikel berita. Cerita yang digerakkan pembaca membiarkan orang menjelajah sendiri. Banyak visualisasi terbaik memakai bentuk campuran: dimulai dengan narasi, lalu membuka ruang untuk dijelajahi.

Kuliah memakai visualisasi untuk menjelaskan algoritme. Algoritme pengurutan bisa digambar sebagai garis-garis yang bertukar tempat dari langkah ke langkah. *k*-means bisa dianimasikan sebagai pusat yang bergeser. Teorema Bayes bisa dijelaskan dengan kotak berisi seratus orang. Contoh yang dibahas adalah tes penyakit dengan sensitivitas 90 persen, tingkat positif palsu 9 persen, dan prevalensi 1 persen. Dari seribu orang, sepuluh sakit dan sembilan di antaranya terdeteksi, sedangkan dari 990 orang sehat, sekitar 89 mendapat hasil positif palsu. Peluang benar-benar sakit setelah hasil positif hanya $9/(9 + 89) \approx 9$ persen. Banyak orang, termasuk dokter, menebak jauh lebih tinggi. Gambar seratus kotak kecil membuat hasil ini langsung terasa masuk akal.

13.3 Menjelaskan lewat proyeksi

Data berdimensi tinggi, termasuk aktivasi di dalam jaringan saraf, bisa diproyeksikan ke dua dimensi untuk dilihat. PCA (Bab 4) mempertahankan varians global, t-SNE mempertahankan tetangga lokal, dan UMAP (McInnes et al., 2018) berusaha menjaga keduanya. Kuliah meluangkan waktu khusus untuk salah baca yang umum pada t-SNE: hyperparameter seperti perplexity amat menentukan, ukuran kelompok di peta tidak berarti apa-apa, jarak antar kelompok bisa menipu, noise acak kadang tampak berpola, dan untuk melihat topologi kita mungkin butuh beberapa peta dengan setelan berbeda.

Proyeksi juga bisa menunjukkan proses yang berjalan dalam waktu. Kalau setiap keadaan sebuah proses, misalnya setiap langkah algoritme, setiap posisi permainan catur, atau setiap epoch pelatihan jaringan, diproyeksikan sebagai satu titik, lalu titik-titik itu dihubungkan, kita mendapat lintasan. Lintasan pelatihan dua jaringan bisa dibandingkan: apakah keduanya menempuh jalan yang mirip, atau satu berputar-putar sebelum menemukan solusi. Kelompok visualisasi di Linz juga mengembangkan alat seperti ini untuk kimia, sehingga ahli kimia bisa menjelajahi ruang molekul yang dilihat oleh model prediksi mereka.

13.4 Menjelaskan dalam ruang input

Sekelompok teknik menjelaskan model dengan melihat bagaimana keluarannya berubah terhadap input. *Partial dependence plot* menampilkan efek rata-rata satu fitur terhadap prediksi, dengan merata-ratakan atas nilai fitur-fitur lain. *Individual conditional expectation* menampilkan kurva yang sama untuk setiap contoh secara terpisah, sehingga interaksi antar fitur yang tersembunyi di balik rata-rata menjadi terlihat.

LIME (Ribeiro et al., 2016) menjelaskan satu prediksi dengan membangun model sederhana yang hanya berlaku di sekitar contoh itu. Contoh diganggu sedikit demi sedikit, misalnya dengan menyembunyikan potongan-potongan gambar (*superpixel*), model asli ditanya untuk setiap versi, lalu model linear dilatih untuk meniru jawabannya secara lokal. Bobot model linear itu menjadi penjelasan. Model surrogate global melakukan hal serupa untuk seluruh data, dengan risiko bahwa model sederhana itu tidak cukup setia.

Nilai Shapley berasal dari teori permainan kooperatif. Pertanyaannya: kalau sekelompok pemain bersama menghasilkan keuntungan, berapa bagian yang adil untuk setiap pemain? Jawaban Shapley adalah rata-rata kontribusi marginal pemain itu atas semua urutan kedatangan pemain. Dalam XAI, pemainnya adalah fitur dan keuntungannya adalah prediksi. Ambil contoh dua fitur, A dan B. Tanpa fitur apa pun, prediksi model 0. Dengan A saja 10, dengan B saja 20, dan dengan keduanya 40. Kalau A datang lebih dulu, kontribusinya 10, dan kalau datang setelah B, kontribusinya 20. Rata-ratanya 15. Dengan cara yang sama, B mendapat $(20 + 30)/2 = 25$. Jumlah keduanya tepat 40, prediksi penuh. SHAP (Lundberg and Lee, 2017) membuat perhitungan ini praktis untuk model dengan banyak fitur, karena jumlah urutan yang mungkin tumbuh secara faktorial.

13.5 Menjelaskan lewat gradien

Untuk jaringan saraf, gradien keluaran terhadap input memberi peta kepekaan langsung: piksel mana yang paling mengubah skor kelas kalau digeser sedikit. *Saliency map* vanilla (Simonyan et al., 2013) menampilkan besar gradien ini. Hasilnya sering berbintik dan berisik. Integrated gradients (Sundararajan et al., 2017) menjumlahkan gradien sepanjang jalur dari gambar acuan, misalnya gambar hitam, ke gambar asli, dan memenuhi sifat bahwa jumlah atribusinya sama dengan selisih keluaran. Grad-CAM (Selvaraju et al., 2016) memakai gradien terhadap peta fitur lapisan konvolusi terakhir untuk menimbang peta-peta itu, sehingga menghasilkan peta panas kasar yang menunjukkan daerah gambar yang dipakai model. Layer-wise relevance propagation (Bach et al., 2015) membagi skor keluaran mundur ke input lapisan demi lapisan dengan aturan kekekalan relevansi.

Kuliah memakai contoh dari sebuah studi klasifikasi jamur. Peta Grad-CAM kadang menunjukkan bahwa model melihat bagian yang memang penting secara biologis, seperti tudung dan bilah, tetapi kadang juga latar belakang, misalnya rumput atau tangan yang memegang jamur. Peta seperti ini tidak membuktikan bahwa model benar, tetapi amat berguna untuk menemukan jalan pintas yang tidak diinginkan.

13.6 Menjelaskan lewat contoh dan konsep

Manusia sering menjelaskan dengan contoh: “ini mirip kasus yang kemarin”. Model pun bisa menjelaskan prediksinya dengan menunjukkan contoh data latih yang paling mirip di ruang fiturnya. *Prototype* adalah contoh yang mewakili sebuah kelompok, dan *criticism* adalah contoh yang tidak terwakili dengan baik oleh prototype mana pun.

Keduanya bersama memberi gambaran lebih jujur tentang data.

Penjelasan berbasis konsep bertanya apakah sebuah konsep yang bermakna bagi manusia, misalnya “bergaris” atau “berlatar salju”, tersimpan di dalam jaringan. *Network dissection* mencocokkan unit-unit di dalam jaringan dengan konsep yang dianotasi. TCAV (Kim et al., 2017) mempelajari arah di ruang aktivasi yang mewakili sebuah konsep dari contoh-contohnya, lalu mengukur seberapa peka prediksi sebuah kelas terhadap arah itu. Jenis temuan yang dicari mirip contoh terkenal dari Ribeiro et al. (2016), di mana penjelasan LIME mengungkapkan bahwa sebuah pengklasifikasi serigala dan anjing husky sebenarnya terutama melihat salju di latar belakang.

13.7 Visualisasi di era AI generatif

Pertemuan terakhir membahas arah sebaliknya: AI yang membuat visualisasi. Model bahasa bisa menulis kode grafik dari permintaan dalam bahasa biasa, menyusun dashboard, atau membuat penjelasan bagi pengguna baru. Kuliah menyoroti risikonya. Gambar yang dihasilkan bisa tampak meyakinkan tetapi salah secara data. Model membuat asumsi tentang maksud pengguna yang tidak pernah diucapkan. Dan jawabannya bergantung pada gambaran yang dibentuk model tentang siapa penggunanya. Kuliah mengusulkan antarmuka yang membuat asumsi-asumsi itu terlihat dan bisa dikoreksi pengguna.

13.8 Di luar kelas

Keterjelasan penting di setiap bidang yang dibahas buku ini, dengan alasan yang berbeda-beda. Dalam kedokteran, peta Grad-CAM pada citra rontgen bisa mengungkapkan model yang melihat penanda rumah sakit di sudut gambar alih-alih paru-paru. Dalam penemuan obat, atribusi pada atom-atom molekul membantu ahli kimia memutuskan bagian mana yang perlu diubah. Dalam pemrosesan sinyal dan sistem tertanam, kita perlu tahu sensor mana yang paling menentukan sebuah keputusan. Dalam simulasi, keterjelasan bisa diambil sejauh-jauhnya: metode seperti SINDy (Bab 27) tidak menjelaskan model kotak hitam, melainkan menghasilkan persamaan yang bisa dibaca langsung. Itu barangkali bentuk penjelasan yang paling disukai Rudin (2019).

CATATAN SUMBER. Bab ini mengikuti enam kuliah AI and Visualization beserta labnya: dasar visualisasi, menjelaskan algoritme, proyeksi, teknik penjelasan visual (PDP, ICE, LIME, Shapley, saliency, Grad-CAM, contoh, konsep), dan AI generatif untuk visualisasi. Bacaan pendamping: Molnar (2022) untuk interpretable machine learning dan Munzner (2014) untuk visualisasi.

Bagian III

Sinyal, Sensor, dan Sistem

14 Menangkap Sinyal: Estimasi, Filter, dan Radar

Pemrosesan sinyal masuk ke perjalanan saya lewat dua mata kuliah. Kuliah optimum and adaptive signal processing systems, dengan latihan kelompok berisi enam tugas, membahas teori estimasi, filter optimal, dan filter adaptif. Pada musim dingin 2025/26, latihan radar system engineering mengajak kami membangun radar di dalam notebook: persamaan radar, korelasi, FFT, deteksi, CFAR, radar FMCW, dan beamforming digital, sampai sebuah simulator radar sederhana.

Pemrosesan sinyal sudah “belajar dari data” jauh sebelum istilah machine learning populer. Filter adaptif yang meredam gema di telepon memperbarui bobotnya dengan gradien stokastik, persis seperti jaringan saraf. Banyak konsep di bab ini, dari batas Cramér–Rao sampai Kalman filter, sudah kita temui dari sisi machine learning. Di sini kita melihatnya dari sisi insinyur sinyal.

14.1 Sinyal acak

Sinyal yang menarik hampir selalu bercampur noise, sehingga kita memperlakukannya sebagai proses acak. Sebuah proses disebut stasioner dalam arti luas kalau rata-ratanya tetap dan fungsi autokorelasinya, $r_x[k] = \mathbb{E}[x[n]x[n+k]]$, hanya bergantung pada jarak waktu k , bukan pada waktu absolut. Transformasi Fourier dari autokorelasi adalah *power spectral density*, yang menyatakan bagaimana daya sinyal tersebar di seluruh frekuensi. Noise putih punya autokorelasi yang hanya tidak nol di $k = 0$, sehingga spektrumnya datar: semua frekuensi mendapat daya yang sama, seperti cahaya putih yang memuat semua warna.

14.2 Estimasi klasik

Kuliah teori estimasi dimulai dengan contoh paling sederhana: menaksir level tegangan tetap A dari pengukuran yang berisik, $x[n] = A + w[n]$ untuk $n = 0, \dots, N-1$, dengan $w[n]$ noise putih Gauss bervarians σ^2 . Rata-rata sampel tak bias, variansnya σ^2/N , dan batas Cramér–Rao (Bab 11) untuk masalah ini juga σ^2/N . Jadi rata-rata sampel adalah estimator *minimum variance unbiased*, dan tidak ada estimator tak bias lain yang lebih baik (Kay, 1993).

Untuk model yang lebih umum, beberapa jalan tersedia. Kalau model linear dalam parameter, $\mathbf{x} = \mathbf{H}\boldsymbol{\theta} + \mathbf{w}$, estimator *best linear unbiased* memberi varians terkecil di antara semua estimator linear tak bias, dan untuk noise Gauss ia sama dengan MVU. Estimator kuadrat terkecil, $\hat{\boldsymbol{\theta}} = (\mathbf{H}^\top \mathbf{H})^{-1} \mathbf{H}^\top \mathbf{x}$, tidak memerlukan asumsi statistik apa pun, hanya meminimalkan selisih kuadrat antara data dan model. Maximum likelihood

menghubungkan semuanya: untuk noise Gauss, ia menghasilkan kuadrat terkecil, dan untuk sampel besar ia mendekati batas Cramér–Rao.

14.3 Estimasi Bayesian

Kalau kita punya pengetahuan awal tentang parameter, misalnya bahwa sebuah tegangan hampir pasti di sekitar lima volt, kita memperlakukan parameter sebagai variabel acak dengan prior. Estimator yang meminimalkan galat kuadrat rata-rata, *minimum mean square error*, ternyata adalah rata-rata posterior $\mathbb{E}[\theta | \mathbf{x}]$. Estimator MAP mengambil puncak posterior. Untuk model linear Gauss, keduanya sama, dan rata-rata posterior adalah kompromi berbobot antara prior dan data: kalau data sedikit atau berisik, hasilnya dekat ke prior, kalau data banyak, hasilnya mengikuti data. Ini adalah logika yang sama dengan Kalman gain di Bab 10, dan memang Kalman filter adalah estimator MMSE yang berjalan berurutan dalam waktu.

14.4 Filter optimal

Misalkan kita ingin mendapatkan sinyal $d[n]$ dari pengamatan $x[n]$ yang terganggu, dengan sebuah filter linear berbobot $\mathbf{w}$ yang bekerja pada M sampel terakhir, $y[n] = \mathbf{w}^\top \mathbf{x}[n]$. Filter Wiener memilih bobot yang meminimalkan galat kuadrat rata-rata $\mathbb{E}[(d[n] - y[n])^2]$. Menurunkan terhadap $\mathbf{w}$ dan menyamakannya dengan nol memberi persamaan Wiener–Hopf,

$$\mathbf{R}\mathbf{w} = \mathbf{p}, \quad \mathbf{w}_{\text{opt}} = \mathbf{R}^{-1}\mathbf{p}, \quad (14.1)$$

dengan $\mathbf{R} = \mathbb{E}[\mathbf{x}\mathbf{x}^\top]$ matriks autokorelasi input dan $\mathbf{p} = \mathbb{E}[\mathbf{x}d]$ korelasi silang antara input dan sinyal yang diinginkan. Hasil ini punya tafsiran geometris yang indah, yaitu *orthogonality principle*: pada bobot optimal, galat tegak lurus terhadap semua data yang dipakai. Tidak ada lagi informasi dalam data yang bisa dipakai untuk mengurangi galat.

Filter Wiener membutuhkan statistik $\mathbf{R}$ dan $\mathbf{p}$, yang dalam praktik harus diperkirakan dari data. Menggantinya dengan rata-rata sampel memberi filter kuadrat terkecil, yang optimal untuk data yang ada, bukan untuk ekspektasinya (Manolakis et al., 2005).

14.5 Filter adaptif

Ketika statistik sinyal berubah dari waktu ke waktu, misalnya gema di ruang yang orangnya berpindah tempat, filter harus terus menyesuaikan diri. Algoritme LMS mengambil satu langkah gradient descent pada galat kuadrat sesaat di setiap sampel,

$$\mathbf{w}[n+1] = \mathbf{w}[n] + \mu e[n] \mathbf{x}[n], \quad e[n] = d[n] - \mathbf{w}[n]^\top \mathbf{x}[n]. \quad (14.2)$$

Ini SGD dengan mini-batch berukuran satu, lima puluh tahun sebelum deep learning. Langkah μ harus cukup kecil agar stabil, kira-kira $0 < \mu < 2/\lambda_{\max}$ dengan $\lambda_{\max}$ nilai eigen terbesar $\mathbf{R}$, dan kecepatan konvergensinya ditentukan oleh sebaran nilai eigen $\mathbf{R}$. Inilah masalah condition number dari Bab 2, dalam pakaian yang berbeda. Algoritme

RLS memperbarui perkiraan $\mathbf{R}^{-1}$ secara rekursif sehingga setiap langkah mirip langkah Newton. Ia konvergen jauh lebih cepat dan tidak terpengaruh sebaran nilai eigen, dengan biaya sekitar M^2 operasi per sampel, dibanding M untuk LMS.

Bayangkan headphone peredam bising. Mikrofon luar menangkap suara mesin pesawat, dan filter adaptif belajar memprediksi seberapa banyak suara itu bocor ke telinga. Speaker lalu memutar prediksi dengan tanda terbalik. Setiap kali pesawat berbelok atau penumpang menggeser kepala, galat sisa berubah, dan bobot filter ikut bergeser beberapa ribu kali per detik.

14.6 Radar

Radar memancarkan gelombang elektromagnetik dan mendengarkan pantulannya. Daya yang kembali mengikuti persamaan radar,

$$P_r = \frac{P_t G^2 \lambda^2 \sigma}{(4\pi)^3 R^4}, \quad (14.3)$$

dengan P_t daya pancar, G penguatan antena, λ panjang gelombang, σ *radar cross section* target, dan R jaraknya. Pangkat empat pada jarak datang dari dua perjalanan: gelombang melemah sebanding R^2 dalam perjalanan pergi dan sekali lagi dalam perjalanan pulang. Akibatnya, untuk melipatgandakan jangkauan, daya pancar harus dinaikkan enam belas kali.

Untuk menemukan pantulan lemah di tengah noise, sinyal yang diterima dikorelasikan dengan sinyal yang dipancarkan. Filter ini, *matched filter*, memaksimalkan rasio sinyal terhadap noise untuk noise putih (Richards, 2014). Puncak korelasi menunjukkan waktu tunda, dan waktu tunda menunjukkan jarak, $R = c\tau/2$.

Setelah itu muncul pertanyaan deteksi: apakah sebuah puncak benar-benar target, atau hanya noise yang kebetulan tinggi? Kriteria Neyman–Pearson menetapkan ambang yang memaksimalkan peluang deteksi untuk peluang alarm palsu tertentu. Masalahnya, tingkat noise dan clutter berubah dari satu tempat ke tempat lain. *Constant false alarm rate* (CFAR) menghitung ambang secara lokal. Untuk setiap sel yang diuji, tingkat noise diperkirakan dari sel-sel latih di sekitarnya, dengan beberapa sel penjaga di antaranya agar target sendiri tidak ikut terhitung. Varian cell-averaging merata-ratakan sel latih, sedangkan varian order-statistic memakai nilai berperingkat tertentu dan lebih tahan ketika ada beberapa target berdekatan (Rohling, 1983).

14.6.1 Radar FMCW

Radar mobil dan radar di dalam ponsel umumnya memakai gelombang kontinu yang frekuensinya naik secara linear, sebuah *chirp*. Pantulan dari target pada jarak R tiba dengan tunda $2R/c$, sehingga pada setiap saat frekuensinya sedikit lebih rendah daripada sinyal yang sedang dipancarkan. Mencampur keduanya menghasilkan *beat frequency* yang sebanding dengan jarak,

$$f_b = \frac{2RS}{c}, \quad S = \frac{B}{T_c}, \quad (14.4)$$

dengan B lebar pita chirp dan T_c durasinya. Dengan $B = 1$ GHz dan $T_c = 50 \mu\text{s}$, kemiringannya 2×10^{13} Hz per detik, dan target pada sepuluh meter menghasilkan beat sekitar 1,33 MHz. FFT dari sinyal beat langsung memberi profil jarak. Resolusi jaraknya $c/(2B)$, lima belas sentimeter untuk lebar pita satu gigahertz.

Kecepatan diperoleh dari deretan chirp. Target yang bergerak menggeser fase pantulan sedikit dari satu chirp ke chirp berikutnya, sebanding dengan kecepatannya. FFT kedua di sepanjang chirp mengubah geseran fase itu menjadi kecepatan, dan hasil kedua FFT bersama-sama membentuk peta jarak dan kecepatan. Dengan beberapa antenna penerima, perbedaan fase antar antenna menunjukkan arah datangnya pantulan, $\Delta\phi = 2\pi d \sin \theta / \lambda$. FFT ketiga, atau *digital beamforming*, mengubahnya menjadi sudut. Latihan kami berakhir dengan simulator yang menggabungkan semua langkah ini: memancarkan chirp ke beberapa target, menambahkan noise, lalu menjalankan seluruh rantai pemrosesan untuk menemukannya kembali.

14.7 Di mana machine learning masuk

Batas antara pemrosesan sinyal dan machine learning semakin kabur. Peta jarak dan kecepatan radar diperlakukan sebagai gambar dan dimasukkan ke jaringan konvolusi untuk mengenali gerakan tangan atau membedakan pejalan kaki dari sepeda. Detektor CFAR klasik bisa diganti detektor yang dipelajari dari data, dengan risiko bahwa jaminan alarm palsunya hilang. Algoritme iteratif, misalnya untuk rekonstruksi sinyal, bisa “dibuka” (*unrolled*): setiap iterasi menjadi satu lapisan jaringan yang parameternya dipelajari, sehingga struktur algoritme klasik tetap terjaga. Dan di arah sebaliknya, gagasan pemrosesan sinyal, seperti filter, frekuensi, dan wavelet, memberi bahasa untuk memahami apa yang dilakukan jaringan saraf (Bab 9). Fondasinya tetap sama: model, noise, dan batas atas apa yang bisa diestimasi dari data.

CATATAN SUMBER. Bab ini mengikuti kuliah dan latihan Optimum and Adaptive Signal Processing Systems (variabel dan proses acak, estimasi klasik dan Bayesian, filter Wiener dan kuadrat terkecil, filter adaptif) serta latihan Radar System Engineering (RCS dan persamaan radar, korelasi, FFT, deteksi, CFAR, FMCW, beamforming digital, simulator). Bacaan yang disebut kuliah: Kay (1993) dan Manolakis et al. (2005). Untuk radar, Richards (2014).

15 Mengendalikan Sistem

Di semester pertama saya mengambil dua mata kuliah yang sekilas tidak ada hubungannya dengan machine learning. Control systems membahas sistem dinamis linear dari model ruang keadaan sampai pengendali diskret, dengan latihan dan ujian berbasis MATLAB. Production automation systems membahas dunia pabrik: perangkat otomasi, sistem waktu nyata, pemrograman PLC, fieldbus, dan standar IEC 61499 untuk sistem otomasi terdistribusi. Belakangan terlihat bahwa teori kendali adalah saudara kandung reinforcement learning, dan bahwa model ruang keadaan adalah bahasa yang sama yang dipakai Kalman filter, state space model seperti Mamba, dan penemuan persamaan dari data.

15.1 Sistem dan model

Sebuah sistem dinamis dijelaskan oleh keadaannya, yaitu sekumpulan besaran yang cukup untuk menentukan masa depannya kalau input diketahui. Untuk rangkaian listrik RC, keadaannya tegangan kapasitor. Untuk kaskade tangki air yang saling mengalirkan, keadaannya ketinggian air di setiap tangki. Untuk populasi pemangsa dan mangsa, keadaannya jumlah masing-masing hewan, dan persamaan Lotka–Volterra menggambarkan siklus naik turun yang khas: mangsa berlimpah membuat pemangsa bertambah, pemangsa yang banyak membuat mangsa berkurang, dan seterusnya. Bentuk umumnya adalah

$$\dot{x} = f(x, u), \quad y = h(x, u), \quad (15.1)$$

dengan u input yang bisa kita atur dan y keluaran yang bisa kita ukur.

Sebagian besar teori dibangun untuk sistem linear yang tidak berubah terhadap waktu,

$$\dot{x} = Ax + Bu, \quad y = Cx + Du. \quad (15.2)$$

Sistem nonlinear dilinearkan di sekitar titik kesetimbangan dengan mengganti f dengan matriks Jacobiannya. Solusi sistem linear adalah $x(t) = e^{At}x_0 + \int_0^t e^{A(t-\tau)}Bu(\tau) d\tau$, dan perilakunya ditentukan oleh nilai eigen A . Sistem stabil secara asimtotik, artinya kembali ke kesetimbangan dari gangguan apa pun, tepat ketika semua nilai eigen punya bagian real negatif. Bentuk Jordan menguraikan sistem menjadi mode-mode independen, sehingga setiap nilai eigen bisa dibaca sebagai satu cara sistem meluruh, tumbuh, atau berosilasi.

15.2 Bisa dicapai, bisa diamati

Dua pertanyaan menentukan apakah sebuah sistem bisa dikendalikan dengan baik. Pertama, apakah input bisa membawa keadaan ke mana saja? Untuk sistem linear,

jawabannya ya tepat ketika matriks

$$\mathcal{R} = [B \ AB \ A^2B \ \dots \ A^{n-1}B] \quad (15.3)$$

berpangkat penuh. Kedua, apakah keadaan bisa direkonstruksi dari keluaran? Jawabannya ya tepat ketika matriks yang disusun dari $C, CA, \dots, CA^{n-1}$ berpangkat penuh. Dekomposisi Kalman memisahkan keadaan menjadi bagian yang bisa dicapai dan diamati, bagian yang hanya salah satunya, dan bagian yang tidak keduanya. Bagian yang tidak terjangkau dan tidak stabil adalah kabar buruk, karena tidak ada pengendali yang bisa menyelamatkannya.

15.3 Pengendali dan pengamat

Dengan umpan balik keadaan $u = -Kx$, sistem tertutup menjadi $\dot{x} = (A - BK)x$, dan kita bisa memilih K agar nilai eigen sistem tertutup berada di tempat yang kita inginkan, selama sistemnya bisa dicapai. Ambil contoh paling sederhana, sebuah massa tanpa gesekan, $\ddot{x} = u$. Dengan keadaan posisi dan kecepatan serta $u = -k_1x - k_2\dot{x}$, persamaan karakteristiknya $s^2 + k_2s + k_1 = 0$. Kalau kita menginginkan kedua kutub di -2 , polinomialnya harus $(s + 2)^2 = s^2 + 4s + 4$, sehingga $k_1 = k_2 = 4$. Rumus Ackermann melakukan perhitungan yang sama untuk sistem dengan satu input berukuran berapa pun.

Kalau keadaan tidak diukur seluruhnya, kita membangun pengamat yang menirukan sistem dan mengoreksi dirinya dengan galat keluaran,

$$\dot{\hat{x}} = A\hat{x} + Bu + L(y - C\hat{x}). \quad (15.4)$$

Prinsip pemisahan menyatakan bahwa pengendali dan pengamat bisa dirancang terpisah lalu digabung, dan kutub-kutub sistem gabungan adalah gabungan kutub keduanya.

Alih-alih memilih kutub, kita juga bisa meminta pengendali terbaik menurut sebuah biaya. Linear quadratic regulator meminimalkan

$$J = \int_0^\infty (x^\top Qx + u^\top Ru) dt, \quad (15.5)$$

yang menimbang seberapa jauh keadaan menyimpang terhadap seberapa besar tenaga kendali yang dipakai. Jawabannya tetap berupa umpan balik linear, dengan K dari persamaan Riccati. Ketika pengukurannya berisik, pengamat optimalnya adalah Kalman filter (Kalman, 1960), dan gabungan LQR dengan Kalman filter disebut LQG. Dari sudut pandang Bab 12, LQR adalah masalah reinforcement learning yang modelnya diketahui sempurna, dinamikanya linear, dan biayanya kuadratik, sehingga persamaan Bellman bisa diselesaikan secara tertutup.

15.4 Domain frekuensi

Transformasi Laplace mengubah persamaan diferensial linear menjadi persamaan aljabar. Hubungan antara input dan keluaran menjadi fungsi alih, $G(s) = C(sI - A)^{-1}B + D$. Sistem stabil dalam arti BIBO, yaitu input terbatas selalu menghasilkan keluaran

terbatas, kalau semua kutub $G(s)$ berada di setengah bidang kiri. Respons frekuensi $G(i\omega)$ menyatakan bagaimana sinyal sinus diperkuat dan digeser fasenya, dan diagram Bode menggambarinya dalam skala logaritmik.

Dalam lingkaran umpan balik, pengendali $R(s)$ dan proses $G(s)$ membentuk fungsi lingkaran terbuka $L(s) = R(s)G(s)$, dan banyak sifat sistem tertutup bisa dibaca dari bentuk L . Pengendali proporsional-integral menambahkan suku integral yang menghapus galat keadaan tunak, karena selama galat masih ada, integralnya terus tumbuh dan mendorong input. Kuliah merancang pengendali seperti ini dengan membentuk respons langkah yang diinginkan, lalu memeriksa kestabilannya dengan kriteria grafis. Bagian terakhir kuliah membawa semuanya ke dunia digital. Pengendali di komputer bekerja pada sampel, dan pemetaan $z = e^{sT}$ menghubungkan domain kontinu dengan domain diskret. Transformasi Tustin memberi cara praktis untuk mengubah pengendali kontinu menjadi algoritme diskret.

15.5 Otomasi di pabrik

Teori kendali bertemu kenyataan di lantai pabrik. Kuliah production automation memperlihatkan lapisan-lapisan yang menjalankan kendali itu. *Programmable logic controller* (PLC) membaca semua input, menjalankan programnya, lalu menulis semua output, dalam siklus yang berulang setiap beberapa milidetik. Programnya ditulis dalam bahasa standar IEC 61131-3, seperti diagram tangga yang meniru rangkaian relai, diagram blok fungsi, atau teks terstruktur. Sistem waktu nyata membedakan tenggat keras, yang tidak boleh terlewat sama sekali karena akibatnya berbahaya, dari tenggat lunak, yang sesekali boleh terlewat. Fieldbus menghubungkan sensor, aktuator, dan pengendali dengan komunikasi yang waktunya bisa dipastikan. Standar IEC 61499 memecah aplikasi otomasi menjadi blok fungsi yang digerakkan oleh kejadian dan bisa disebar ke banyak perangkat, sebuah langkah menuju pabrik yang bisa dikonfigurasi ulang dengan cepat, tema utama gerakan Industri 4.0.

15.6 Kendali dan machine learning

Hubungan antara kendali dan machine learning berjalan ke dua arah. Dari kendali ke machine learning: model ruang keadaan menjadi state space model di deep learning (Bab 6), metode adjoint dari kendali optimal menjadi cara melatih neural ODE (Bab 11), dan Kalman filter menjadi kerangka data assimilation (Bab 10). Dari machine learning ke kendali: model dinamika bisa dipelajari dari data alih-alih diturunkan dari hukum fisika, lalu dipakai dalam pengendali prediktif yang mengoptimalkan beberapa langkah ke depan di setiap saat. Teori operator Koopman (Koopman, 1931) menawarkan cara melihat sistem nonlinear sebagai sistem linear di ruang fungsi yang lebih besar, sehingga alat-alat kendali linear bisa dipakai kembali, sebuah gagasan yang kita temui lagi di Bab 27. Reinforcement learning sendiri bisa dipandang sebagai kendali optimal tanpa model, dan pada sistem nyata keduanya sering digabung: model fisika memberi dasar, pembelajaran mengoreksi apa yang tidak dimodelkan.

CATATAN SUMBER. Bab ini mengikuti naskah kuliah Control Systems (sistem dan model, sifat sistem, sistem LTI, keterjangkauan dan keteramatan, pengendali dan pengamat keadaan, LQR dan LQG, transformasi Laplace, desain pengendali, sistem diskret) dan kuliah Production Automation Systems (perangkat otomatisasi, sistem waktu nyata, PLC, fieldbus, IEC 61499). Bacaan lanjut tentang hubungan kendali dan data: Brunton and Kutz (2022).

16 Komputasi yang Menghilang

Pada musim dingin 2025/26 saya mengikuti dua kuliah pervasive computing dari Alois Ferscha. Yang satu membahas sistem dan lingkungan: visi komputasi pervasif, sensor dan modalitas, jaringan sensor, cara mengenali manusia dan benda, arsitektur pengenalan konteks, dan komputer yang dikenakan di kepala. Yang lain membahas perancangan dari sisi manusia: persepsi visual, gerak mata, pendengaran, perabaan, atensi, emosi, antarmuka yang bisa disentuh, dan gerakan motorik. Latihannya meminta kami mengumpulkan data sensor sendiri, lalu membangun pengenalan aktivitas dan sistem pelacak gerak.

16.1 Visi Weiser

Weiser (1991) membuka artikelnya dengan kalimat yang sering dikutip: teknologi yang paling mendalam adalah teknologi yang menghilang, yang menenun dirinya ke dalam kehidupan sehari-hari sampai tidak bisa dibedakan darinya. Menulis adalah contohnya. Kita tidak lagi memikirkan teknologi tulisan ketika membaca papan nama jalan. Weiser membayangkan komputer yang sama: tersebar di dinding, di meja, di pakaian, bekerja tanpa diminta, dan tidak menuntut perhatian. Hari ini ponsel, jam pintar, dan rumah pintar mendekati visi itu, dengan semua manfaat dan masalahnya.

Komputasi pervasif membutuhkan tiga kemampuan: merasakan dunia lewat sensor, memahami konteksnya, dan bertindak dengan cara yang tidak mengganggu. Bagian tengah, memahami konteks, adalah tempat machine learning bekerja.

16.2 Sensor dan modalitas

Ponsel biasa membawa akselerometer, giroskop, magnetometer, mikrofon, kamera, GPS, dan sensor cahaya serta jarak. Di luar ponsel, ada label RFID yang bisa dibaca tanpa kontak, pelacak mata, sensor tekanan di lantai, dan sensor fisiologis seperti detak jantung dan konduktansi kulit. Setiap modalitas punya kelebihan dan kekurangannya. Kamera kaya informasi tetapi mengganggu privasi dan boros energi. Akselerometer murah dan hemat, tetapi sinyalnya sulit ditafsirkan. Kuliah menekankan satu hal: tidak ada sensor yang langsung mengukur konteks. Tidak ada “sensor rapat” atau “sensor sedang memasak”. Konteks harus disimpulkan dari kombinasi sinyal mentah.

Jaringan sensor menambahkan masalah komunikasi dan energi. Simpul-simpul kecil bertenaga baterai harus memutuskan kapan mengukur, kapan mengirim, dan kapan tidur. Memproses data di simpul itu sendiri, alih-alih mengirim semuanya ke server, sering menghemat energi karena mengirim satu bit lewat radio lebih mahal daripada banyak operasi hitung.

16.3 Rantai pengenalan konteks

Arsitektur pengenalan konteks yang dibahas kuliah mengikuti rantai yang juga dijelaskan dalam tutorial Bulling et al. (2014). Data mentah dari sensor dipotong menjadi jendela-jendela waktu yang tumpang tindih, misalnya dua detik dengan geseran satu detik. Pada frekuensi sampling 50 Hz, setiap jendela berisi seratus sampel per sumbu. Dari setiap jendela dihitung fitur: rata-rata, simpangan baku, korelasi antar sumbu, energi di pita frekuensi tertentu dari FFT, atau jumlah puncak. Fitur-fitur itu dimasukkan ke pengklasifikasi, misalnya k -nearest neighbour, pohon keputusan, SVM, atau jaringan saraf, yang menghasilkan label aktivitas untuk setiap jendela. Langkah terakhir menghaluskan urutan label, karena seseorang tidak berganti dari berjalan ke duduk lalu ke berjalan lagi dalam tiga detik.

Rantai ini terlihat sederhana, tetapi kuliah dan latihan memperlihatkan jebakannya. Yang paling penting adalah cara evaluasi. Kalau jendela-jendela dari orang yang sama masuk ke data latih dan data uji sekaligus, model belajar mengenali gaya gerak orang itu, bukan aktivitasnya, dan skornya tampak terlalu bagus. Evaluasi yang jujur meninggalkan satu orang di luar pelatihan, lalu mengujinya pada orang itu, sebuah variasi dari pembagian data yang hati-hati di Bab 1. Dalam latihan kami, yang dikenali adalah cara menggenggam dan gerakan memasang sekrup, direkam dengan akselerometer ponsel. Setiap jendela diringkas menjadi sekitar empat puluh fitur, dari rata-rata, simpangan baku, kemiringan distribusi, dan laju perpotongan nol di setiap sumbu, sampai korelasi antar sumbu, frekuensi dominan, dan entropi spektral.

16.4 Melacak gerak

Sensor inersia memberi kecepatan sudut dari giroskop dan percepatan dari akselerometer. Mengintegrasikan kecepatan sudut memberi orientasi, tetapi setiap bias kecil ikut terintegrasi dan orientasi perlahan melenceng. Akselerometer, saat sensor diam atau bergerak pelan, mengukur arah gravitasi dan memberi orientasi kemiringan yang tidak melenceng, tetapi berisik. Kedua sumber ini saling melengkapi. Filter komplementer menggabungkannya dengan cara yang sederhana:

$$\theta_k = \alpha(\theta_{k-1} + \omega_k \Delta t) + (1 - \alpha) \theta_k^{\text{acc}}, \quad (16.1)$$

dengan α dekat satu, misalnya 0,98. Giroskop dipercaya untuk perubahan cepat, akselerometer untuk kebenaran jangka panjang. Ini versi tangan dari Kalman filter di Bab 10, dengan gain yang dipilih sendiri alih-alih dihitung dari kovarians. Untuk melacak posisi, percepatan harus diintegrasikan dua kali, dan galatnya tumbuh kuadratik terhadap waktu, sehingga pelacak posisi inersia murni hanya berguna untuk waktu singkat kecuali ada sumber koreksi lain seperti kamera atau GPS.

16.5 Manusia di dalam lingkaran

Sepuluh kedua materi membahas sisi manusia, karena sistem pervasif berinteraksi langsung dengan persepsi dan perhatian kita. Mata tidak bergerak mulus. Ia melompat

dalam gerakan cepat yang disebut sakade, lalu berhenti dalam fiksasi, dan hanya di daerah kecil di tengah penglihatan kita melihat dengan tajam. Pelacak mata merekam pola ini, dan pola itu mengungkap apa yang menarik perhatian, seberapa berat beban pikiran, atau apakah seseorang mengantuk. Memakai pandangan mata sebagai input punya masalah klasik yang disebut masalah sentuhan Midas: mata kita melihat banyak hal tanpa bermaksud memilihnya, sehingga sistem harus bisa membedakan melihat dari memilih.

Komputasi afektif mencoba mengenali emosi. Model emosi yang dipakai beragam, dari kategori emosi dasar seperti senang, sedih, marah, dan takut, sampai ruang dua dimensi valensi dan gairah, di mana setiap emosi adalah sebuah titik. Sinyalnya bisa berupa wajah, suara, postur, atau sinyal fisiologis. Kuliah mengingatkan bahwa emosi yang diungkapkan tidak selalu sama dengan yang dirasakan, dan bahwa pengenalan emosi menyentuh wilayah privasi yang pribadi.

Antarmuka yang bisa disentuh, atau *tangible user interface*, memberi bentuk fisik pada informasi digital, sehingga kita bisa memegang, memindah, dan menyusun benda nyata untuk mengendalikan komputer. Kuliah juga melihat ke depan, ke materi yang bisa diprogram dan mengubah bentuknya sendiri.

16.6 Di mana machine learning masuk

Hampir seluruh rantai pengenalan konteks kini bisa diganti jaringan saraf yang belajar fiturnya sendiri dari sinyal mentah, seperti jaringan konvolusi satu dimensi atau LSTM pada data akselerometer (Bab 6). Analisis adegan akustik mengenali tempat dan kejadian dari suara, dan pada prinsipnya memakai jaringan visi pada spektrogram. Tantangan khas bidang ini tetap ada: data berlabel mahal karena harus direkam dengan orang sungguhan, setiap orang bergerak berbeda, sensor bergeser posisinya, dan perangkatnya punya baterai dan memori kecil. Model kecil yang bisa berjalan di mikrokontroler, sering disebut TinyML, menjadi cabang tersendiri. Dan karena sensornya melekat di tubuh dan di rumah, pertanyaan privasi dan persetujuan tidak bisa dipisahkan dari pertanyaan teknis (Bab 31).

CATATAN SUMBER. Bab ini mengikuti kuliah Pervasive Computing: Systems and Environments (visi, sensor, jaringan sensor, identifikasi manusia dan benda, arsitektur pengenalan konteks, komputasi yang dikenakan di kepala) dan Pervasive Computing: Design and Development (persepsi, pelacakan mata, pendengaran, perabaan, atensi, komputasi afektif, antarmuka tangible, gerak motorik), beserta latihan pengenalan aktivitas dan pelacakan gerak.

17 Robot dan Kendaraan Otonom

Musim panas 2025 membawa saya ke dua kuliah tentang mesin yang bergerak. Introduction to robotic systems membahas mekanika lengan robot: kerangka koordinat, kinematika maju dan balik, Jacobian, persamaan gerak dari prinsip Lagrange, visual servoing, dan perencanaan jalur. Introduction to autonomous vehicles membahas mobil yang mengemudi sendiri: lokalisasi dengan pendekatan Markov, ICP, dan NDT, sensor lidar, deteksi objek tiga dimensi, Kalman filter, dan pelacakan banyak objek, dengan proyek lokalisasi dan proyek persepsi.

Robot adalah tempat semua yang sudah kita bahas bertemu. Ia harus melihat (Bab 7), menaksir keadaannya di tengah ketidakpastian (Bab 10), mengendalikan gerakannya (Bab 15), dan memutuskan apa yang dilakukan (Bab 3 and 12). Bab ini melihat dua jenis robot: lengan industri yang bekerja di tempat, dan kendaraan yang bergerak di dunia terbuka.

17.1 Koordinat dan kinematika

Robot hidup di antara banyak kerangka koordinat: dunia, dasar robot, setiap sendi, ujung lengan, kamera. Posisi dan orientasi sebuah kerangka relatif terhadap kerangka lain dinyatakan dengan matriks rotasi dan vektor translasi, yang digabung menjadi matriks transformasi homogen 4×4 , sehingga transformasi berurutan cukup dikalikan. Konvensi Denavit–Hartenberg (Denavit and Hartenberg, 1955) memberi resep standar untuk menempatkan kerangka di setiap sendi, sehingga setiap sambungan hanya butuh empat parameter.

Forward kinematics menghitung posisi ujung lengan dari sudut-sudut sendi. Untuk lengan planar dua ruas dengan panjang l_1 dan l_2 serta sudut q_1 dan q_2 ,

$$x = l_1 \cos q_1 + l_2 \cos(q_1 + q_2), \quad y = l_1 \sin q_1 + l_2 \sin(q_1 + q_2). \quad (17.1)$$

Dengan $l_1 = l_2 = 1$, $q_1 = 0$, dan $q_2 = 90^\circ$, ujung lengan berada di $(1, 1)$. *Inverse kinematics* berjalan ke arah sebaliknya: dari posisi yang diinginkan, cari sudut sendinya. Untuk lengan dua ruas ada dua jawaban, siku di atas atau siku di bawah, dan untuk posisi di luar jangkauan tidak ada jawaban sama sekali. Lengan dengan lebih banyak sendi dari yang dibutuhkan, yang disebut redundan, punya tak hingga jawaban, dan kebebasan itu bisa dipakai untuk menghindari rintangan.

Jacobian menghubungkan kecepatan sendi dengan kecepatan ujung lengan, $\dot{p} = J(q)\dot{q}$. Untuk lengan dua ruas tadi, determinannya $l_1 l_2 \sin q_2$. Ketika lengan terbentang lurus atau terlipat penuh, $\sin q_2 = 0$ dan Jacobian kehilangan pangkat. Di konfigurasi *singular* seperti ini, ada arah gerak yang tidak bisa dicapai sama sekali, dan kecepatan sendi yang dibutuhkan untuk bergerak di dekatnya membengkak.

17.2 Dinamika

Kinematika tidak peduli pada massa dan gaya. Untuk mengendalikan lengan yang sebenarnya, kita perlu tahu torsi apa yang menghasilkan gerak tertentu. Kuliah menurunkannya dengan prinsip Hamilton dan persamaan Lagrange. Tuliskan energi kinetik T dan energi potensial V sebagai fungsi koordinat sendi, bentuk fungsi Lagrange $L = T - V$, lalu terapkan

$$\frac{d}{dt} \frac{\partial L}{\partial \dot{q}_i} - \frac{\partial L}{\partial q_i} = \tau_i. \quad (17.2)$$

Hasilnya selalu berbentuk $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau$, dengan matriks massa yang bergantung pada konfigurasi, suku Coriolis dan sentrifugal, dan gravitasi. Struktur ini dipakai langsung untuk kendali: kalau persamaannya diketahui, pengendali bisa menghitung torsi yang membatalkan nonlinearitas dan menyisakan sistem linear yang mudah dikendalikan. Deskripsi robot untuk perangkat lunak, seperti format URDF, menyimpan semua massa, inersia, dan geometri yang dibutuhkan untuk perhitungan ini (Lynch and Park, 2017).

17.3 Melihat dan merencanakan

Visual servoing mengendalikan robot dengan umpan balik dari kamera. Model kamera dan kalibrasinya sama dengan di Bab 7. Pada visual servoing berbasis posisi, pose objek diperkirakan dari gambar, lalu galat pose diumpankan ke pengendali.

Untuk bergerak tanpa menabrak, kuliah membahas medan potensial buatan. Tujuan memberi potensial yang menarik, rintangan memberi potensial yang menolak, dan robot mengikuti gradien negatif jumlahnya, seperti bola yang menggelinding di lanskap. Cara ini elegan dan murah, tetapi punya masalah yang sudah kita kenal dari optimasi: minimum lokal. Robot bisa terjebak di lekukan antara dua rintangan, di mana tarikan dan tolakan seimbang. Perencana berbasis graf seperti A* (Bab 3) atau perencana berbasis sampel menghindarinya dengan mencari secara global. Gerak sepanjang jalur kemudian diberi profil kecepatan yang halus, misalnya trapesium dengan fase percepatan, kecepatan tetap, dan perlambatan, supaya motor tidak dihentak.

17.4 Di mana aku berada: lokalisasi

Mobil otonom harus tahu posisinya dengan ketelitian beberapa sentimeter, jauh lebih teliti dari GPS biasa. Kuliah membahas tiga pendekatan. *Markov localization* adalah filter Bayes dari Bab 10 di atas grid posisi (Thrun et al., 2005). Keyakinan tentang posisi disebar menurut model gerak, karena roda selip dan odometri tidak sempurna, lalu dipertajam dengan membandingkan pengukuran sensor dengan peta. Bayangkan berjalan di koridor gelap dengan tangan meraba dinding: setiap langkah menambah keraguan, setiap pintu yang teraba mengurangi keraguan.

Dua pendekatan lain bekerja dengan mencocokkan awan titik lidar dengan peta. *Iterative closest point* (Besl and McKay, 1992) bergantian antara memasang setiap titik pindaian dengan titik peta terdekat, lalu mencari rotasi dan translasi kaku yang meminimalkan jarak kuadrat pasangan-pasangan itu. Langkah kedua punya solusi

tertutup melalui dekomposisi nilai singular. ICP cepat konvergen kalau tebakan awalnya baik, dan bisa tersesat kalau tidak. *Normal distributions transform* (Biber and Strasser, 2004) membagi peta menjadi sel-sel dan mewakili titik-titik di setiap sel dengan sebuah distribusi Gauss. Pindaian baru dicocokkan dengan memaksimalkan likelihood titik-titiknya di bawah campuran Gauss itu, yang lebih mulus dan lebih tahan terhadap tebakan awal yang kurang tepat.

17.5 Melihat jalan: persepsi dan pelacakan

Lidar memancarkan pulsa laser dan mengukur waktu tempuhnya, sehingga menghasilkan awan titik tiga dimensi dengan jarak yang akurat. Kamera memberi warna dan tekstur, dan radar (Bab 14) memberi kecepatan langsung dan tetap bekerja dalam hujan dan kabut. Fusi sensor menggabungkan kelebihan masing-masing.

Mendeteksi objek dalam awan titik punya kesulitan khas: titik-titiknya tidak tersusun dalam grid dan urutannya tidak bermakna. PointNet (Qi et al., 2016) menanganinya dengan menerapkan MLP yang sama pada setiap titik lalu menggabungkan hasilnya dengan pooling maksimum, sebuah fungsi yang invarian terhadap permutasi persis seperti DeepSets di Bab 9. PointPillars (Lang et al., 2018) mengelompokkan titik ke dalam pilar-pilar vertikal di atas grid tanah, mengubahnya menjadi gambar pandangan atas, lalu memakai jaringan konvolusi dua dimensi yang cepat.

Deteksi di setiap bingkai belum cukup. Kita perlu tahu bahwa mobil di bingkai ini adalah mobil yang sama dengan di bingkai sebelumnya, dan ke mana ia bergerak. Pelacakan banyak objek bekerja dalam siklus. Untuk setiap jejak yang sudah ada, Kalman filter atau versi diperluasnya meramalkan posisi di bingkai berikutnya. Deteksi baru lalu dipasangkan dengan jejak-jejak itu. Pemasangan ini adalah masalah penugasan: diberikan matriks biaya berisi jarak antara setiap ramalan dan setiap deteksi, cari pasangan satu-satu dengan biaya total terkecil. Algoritme Hungaria (Kuhn, 1955) menyelesaikannya secara eksak. Deteksi yang tidak punya pasangan bisa menjadi jejak baru, dan jejak yang lama tidak mendapat deteksi dihapus. Setiap jejak yang terpasang diperbarui dengan langkah koreksi Kalman.

17.6 Di mana machine learning masuk

Robotika klasik dibangun dari model yang diturunkan: kinematika, dinamika, model sensor. Machine learning masuk di tempat model sulit ditulis. Persepsi hampir seluruhnya dikuasai deep learning. Dinamika yang sulit dimodelkan, seperti gesekan, kabel, atau benda lunak, bisa dipelajari dari data dan dipakai sebagai koreksi atas model fisika. Policy untuk manipulasi yang rumit dilatih dengan reinforcement learning di simulator lalu dipindahkan ke robot nyata, atau dipelajari dari demonstrasi manusia (Bab 12). Model fondasi yang memadukan penglihatan, bahasa, dan aksi mulai dipakai untuk memberi perintah robot dengan kalimat biasa (Bab 32). Tetapi untuk bagian yang menyangkut keselamatan, seperti lokalisasi, pelacakan, dan kendali tingkat rendah, metode klasik dengan jaminan yang bisa dianalisis tetap menjadi tulang punggung.

CATATAN SUMBER. Bab ini mengikuti kuliah Introduction to Robotic Systems (transformasi kerangka, kecepatan benda kaku, Denavit–Hartenberg, kinematika maju dan balik, Jacobian dan singularitas, persamaan Lagrange, visual servoing, medan potensial, profil trayektori) dan Introduction to Autonomous Vehicles (Markov localization, ICP, NDT, lidar, deteksi objek tiga dimensi, Kalman dan extended Kalman filter, pelacakan banyak objek). Buku rujukan: Lynch and Park (2017) dan Thrun et al. (2005).

Bagian IV

AI untuk Ilmu Hayati

18 Membaca Kode Kehidupan

Pada musim panas 2025 saya membuka pintu ke ilmu hayati lewat beberapa kuliah sekaligus: pengantar genetika, analisis genom dan transkriptomik, bioinformatika struktural, dan AI dalam ilmu hayati. Kuliah genom dimulai dari dasar sekali, dari apa itu kehidupan dan bagaimana DNA disusun, lalu bergerak ke teknologi microarray, pengurutan DNA generasi demi generasi, perakitan genom, pemetaan bacaan, pemanggilan varian, dan analisis RNA-seq. Bagi pembaca yang datang dari teknik atau ilmu komputer, seperti saya, bab ini adalah pengenalan dengan bahasa yang dipakai bab-bab berikutnya.

Biologi molekuler modern adalah ilmu data. Satu genom manusia berisi sekitar tiga miliar huruf, satu percobaan pengurutan menghasilkan ratusan juta potongan bacaan, dan satu pengukuran ekspresi gen mencatat aktivitas puluhan ribu gen sekaligus. Hampir setiap langkah dari sampel ke kesimpulan melibatkan algoritme, statistik, dan semakin sering machine learning.

18.1 Dogma sentral

Informasi genetik tersimpan dalam DNA, rantai ganda yang tersusun dari empat basa: adenin, sitosin, guanin, dan timin. Basa pada satu untai berpasangan dengan basa pada untai lain, A dengan T dan C dengan G, sehingga setiap untai menyimpan salinan lengkap informasi untai pasangannya. Struktur heliks ganda ini diusulkan Watson dan Crick pada 1953, dengan bantuan penting dari foto difraksi sinar-X yang dibuat Rosalind Franklin, sebuah kisah yang juga disinggung di kuliah.

Dogma sentral biologi molekuler menggambarkan aliran informasi. DNA disalin menjadi RNA dalam proses transkripsi, lalu RNA pembawa pesan diterjemahkan menjadi protein di ribosom. Setiap tiga basa, satu kodon, menyandikan satu asam amino. Ada $4^3 = 64$ kodon untuk dua puluh asam amino dan sinyal berhenti, sehingga kodenya berlebih: beberapa kodon berbeda menyandikan asam amino yang sama. Pada sel eukariot, bagian-bagian RNA dipotong dan disambung ulang sebelum diterjemahkan, dan penyambungan alternatif memungkinkan satu gen menghasilkan beberapa protein berbeda. Protein lalu melipat menjadi bentuk tiga dimensi yang menentukan fungsinya, topik Bab 20.

Bayangkan perpustakaan besar yang buku-bukunya tidak boleh dibawa keluar. Untuk membangun sesuatu, seseorang memfotokopi satu halaman resep (transkripsi), membawa fotokopinya ke dapur (ribosom), lalu memasak sesuai resep itu (translasi). Sel yang berbeda di tubuh kita membawa perpustakaan yang sama, tetapi memfotokopi halaman yang berbeda. Itulah sebabnya sel otot dan sel saraf begitu berbeda meski DNA-nya identik, dan itulah sebabnya mengukur RNA, yaitu fotokopi yang sedang

beredar, memberi tahu apa yang sedang dikerjakan sebuah sel.

18.2 Variasi genetik

Dua manusia berbeda di sekitar satu dari setiap seribu basa. Variasi yang paling umum adalah *single nucleotide polymorphism* (SNP), satu posisi tempat sebagian orang punya satu basa dan sebagian lain punya basa berbeda. Genetika populasi mempelajari bagaimana frekuensi varian berubah. Hukum Hardy–Weinberg menyatakan bahwa dalam populasi besar yang kawin acak tanpa seleksi, kalau frekuensi dua alel adalah p dan $q = 1 - p$, maka frekuensi genotipenya p^2 , $2pq$, dan q^2 . Dengan $p = 0,7$, sekitar 49 persen individu homozigot untuk alel pertama, 42 persen heterozigot, dan 9 persen homozigot untuk alel kedua. Penyimpangan dari proporsi ini menjadi petunjuk adanya seleksi, migrasi, atau kesalahan pengukuran.

Varian yang letaknya berdekatan di kromosom cenderung diwariskan bersama, sebuah keadaan yang disebut *linkage disequilibrium*. Studi asosiasi seluruh genom (GWAS) memanfaatkan hal ini: mereka mengukur ratusan ribu SNP pada ribuan orang dan mencari varian yang frekuensinya berbeda antara penderita dan bukan penderita sebuah penyakit. Karena jumlah uji statistiknya banyak sekali, ambang signifikansinya harus ketat sekali.

18.3 Mengukur ekspresi gen

Microarray DNA adalah kaca kecil yang ditemplei jutaan potongan DNA pendek yang dikenal, yang disebut probe. RNA dari sampel diubah menjadi DNA berlabel fluoresen, lalu dibiarkan menempel pada probe yang cocok. Semakin terang sebuah titik, semakin banyak RNA dari gen itu. Jalan dari gambar ke angka ternyata panjang: koreksi latar belakang, normalisasi antar chip agar distribusinya sebanding, misalnya dengan normalisasi kuantil, lalu meringkas banyak probe dari satu gen menjadi satu nilai ekspresi. Kuliah membahas metode ringkasan seperti RMA, dan FARMs, metode berbasis analisis faktor (Bab 4) dari kelompok Sepp Hochreiter, yang memperlakukan sinyal bersama antar probe sebagai faktor laten dan noise pribadi setiap probe sebagai noise analisis faktor.

18.4 Pengurutan DNA

Pengurutan Sanger, generasi pertama, membaca satu potongan DNA sepanjang beberapa ratus basa dengan akurasi tinggi, dan dipakai dalam Proyek Genom Manusia. Generasi kedua, yang dipelopori platform Illumina, membaca ratusan juta potongan pendek secara paralel, sering berpasangan dari kedua ujung sebuah fragmen. Generasi ketiga dan keempat membaca satu molekul DNA secara langsung, dengan bacaan yang jauh lebih panjang, dari puluhan ribu sampai jutaan basa, meski dulu dengan galat yang lebih tinggi. Pada pengurutan nanopore, DNA ditarik melewati pori kecil dan setiap kombinasi basa mengubah arus listrik dengan cara yang khas. Mengubah sinyal arus menjadi urutan basa, *base calling*, kini dikerjakan jaringan saraf rekuren dan konvolusi

yang dilatih dengan loss connectionist temporal classification, karena panjang keluaran tidak diketahui sebelumnya.

Setiap basa yang dibaca diberi skor kualitas PHRED, $Q = -10 \log_{10} P_{\text{galat}}$. Skor 30 berarti peluang salah satu per seribu. Kedalaman cakupan, $C = NL/G$, menyatakan berapa kali rata-rata setiap posisi genom dibaca, dengan N jumlah bacaan, L panjang bacaan, dan G panjang genom. Untuk genom manusia sepanjang tiga miliar basa, cakupan tiga puluh kali dengan bacaan 150 basa membutuhkan sekitar 600 juta bacaan.

18.5 Dari bacaan ke genom

Kalau genom acuan sudah ada, setiap bacaan dipetakan ke posisinya di acuan. Mencocokkan ratusan juta bacaan pendek ke tiga miliar huruf dengan toleransi terhadap galat membutuhkan indeks yang cerdas. Transformasi Burrows–Wheeler, yang juga dipakai dalam kompresi data, memungkinkan pencarian pola yang amat cepat dengan memori kecil (Li and Durbin, 2009). Strategi umumnya adalah *seed and extend*: cari potongan pendek yang cocok persis, lalu perluas dengan penjajaran yang toleran terhadap galat. Untuk bacaan panjang, potongan-potongan yang cocok dirangkai dulu menjadi rantai yang konsisten sebelum penjajaran rinci.

Setelah bacaan dipetakan, *variant calling* memutuskan di posisi mana sampel berbeda dari acuan. Tantangannya membedakan varian sungguhan dari galat pengurutan dan galat pemetaan. DeepVariant (Poplin et al., 2018) mengubah tumpukan bacaan di sekitar sebuah posisi menjadi gambar dan memakai jaringan konvolusi untuk mengklasifikasikan genotipenya, sebuah contoh menarik dari visi komputer yang dipinjam oleh genomik.

Kalau tidak ada acuan, genom harus dirakit dari nol, seperti menyusun teka-teki jigsaw raksasa dari potongan-potongan yang saling tumpang tindih. Pendekatan *overlap-layout-consensus* membangun graf yang simpulnya bacaan dan sisinya tumpang tindih, lalu mencari jalur yang melewati setiap simpul tepat sekali, sebuah jalur Hamilton yang sulit dicari secara umum. Graf de Bruijn membalik sudut pandangnya: bacaan dipotong menjadi semua kata sepanjang k , dan rakitan menjadi jalur yang melewati setiap sisi tepat sekali, sebuah jalur Euler yang bisa dicari dengan efisien. Bagian yang berulang di genom adalah musuh utama keduanya, karena potongan dari dua tempat berbeda tampak identik. Bacaan panjang membantu justru karena mereka bisa melompati pengulangan. Kuliah juga menunjukkan bahwa jaringan graf (Bab 9) mulai dipakai untuk menyederhanakan graf rakitan.

18.6 RNA-seq

Pengurutan juga bisa mengukur ekspresi gen. RNA diubah menjadi DNA, diurutkan, lalu jumlah bacaan yang jatuh pada setiap gen dihitung. Hitungan ini harus dinormalisasi karena sampel yang berbeda diurutkan dengan kedalaman berbeda. Untuk mencari gen yang ekspresinya berubah antara dua kondisi, DESeq2 (Love et al., 2014) memodelkan hitungan dengan distribusi binomial negatif, yang mengizinkan varians lebih besar dari rata-rata seperti yang selalu terlihat pada data biologis. Karena setiap

gen hanya punya sedikit ulangan, perkiraan dispersi dan perubahan lipat dari banyak gen “dipinjamkan” satu sama lain dan ditarik ke arah tren bersama, sebuah bentuk prior Bayesian empiris. Dengan puluhan ribu gen yang diuji sekaligus, nilai p dikoreksi untuk mengendalikan proporsi temuan palsu.

18.7 Di mana machine learning masuk

Genomik adalah salah satu medan tempat machine learning paling banyak dipakai di ilmu hayati. Jaringan saraf memanggil basa dari sinyal nanopore, memanggil varian dari tumpukan bacaan, dan memprediksi dari urutan DNA saja di mana protein akan menempel atau seberapa kuat sebuah gen diekspresikan. Model bahasa yang dilatih pada urutan DNA dan protein meniru resep model bahasa manusia di Bab 6. Metode tanpa label seperti biclustering dan analisis faktor (Bab 4) menemukan modul gen yang bekerja bersama dalam data ekspresi. Karakter datanya menuntut kehati-hatian khusus: jumlah fitur puluhan ribu dengan jumlah sampel puluhan, batch percobaan yang meninggalkan jejak sistematis, dan korelasi antar sampel dari pasien atau keluarga yang sama, semua sumber kebocoran yang dibahas di Bab 1.

CATATAN SUMBER. Bab ini mengikuti kuliah Genome Analysis and Transcriptomics (dasar biologi molekuler, genetika populasi, microarray dan pemrosesannya, pengurutan DNA, pemetaan bacaan, pemanggilan varian, perakitan genom, RNA-seq) dan Introduction to Genetics. Untuk analisis urutan secara probabilistik, Durbin et al. (1998) tetap menjadi rujukan klasik.

19 Molekul, Obat, dan Data Medis

Kuliah AI in life sciences saya ikuti pada musim panas 2025. Materinya luas: penemuan obat dan representasi molekul, QSAR dan virtual screening, geometric deep learning untuk molekul, model multimodal, model generatif, data medis, sampai perencanaan sintesis kimia. Banyak metode yang dibahas lahir di Linz sendiri, dari DeepTox sampai jaringan Hopfield modern untuk kimia, sehingga kuliahnya banyak memperlihatkan hasil riset dari kelompok-kelompok di kampus yang sama.

Menemukan obat baru memakan waktu lebih dari satu dekade dan biaya yang amat besar, dan sebagian besar kandidat gagal di tengah jalan. Pipeline-nya panjang: memilih *target* biologis, biasanya sebuah protein yang terlibat dalam penyakit, mencari *hit* yang memengaruhi target itu, mengembangkannya menjadi *lead*, mengoptimalkan sifat-sifatnya, lalu menguji pada hewan dan manusia. Machine learning tidak bisa menggantikan uji klinis, tetapi bisa membuat langkah-langkah awal jauh lebih murah, misalnya dengan menyaring jutaan molekul di komputer sebelum satu pun disintesis.

19.1 Bagaimana mesin melihat molekul

Molekul organik kecil adalah atom-atom yang dihubungkan ikatan kimia. Representasi paling alami adalah graf: atom sebagai simpul, ikatan sebagai sisi, masing-masing dengan atribut seperti jenis unsur, muatan, dan orde ikatan. Untuk disimpan dan dicari di basis data, graf ini ditulis sebagai string. SMILES (Weininger, 1988) berjalan menyusuri graf dan mencatat atom yang dilewati: etanol ditulis CCO, benzena c1ccccc1, dengan angka yang menandai titik pembukaan dan penutupan cincin. Satu molekul bisa punya banyak SMILES yang sah, tergantung dari atom mana penyusuran dimulai, sehingga dibutuhkan bentuk kanonik untuk membandingkan. SELFIES dirancang agar setiap string yang mungkin selalu mewakili molekul yang sah, sifat yang berguna untuk model generatif. InChI dan InChIKey memberi pengenalan unik yang bisa dipakai sebagai kunci basis data.

Untuk machine learning klasik, molekul diubah menjadi vektor fitur. *Molecular descriptor* adalah besaran yang bisa dihitung, seperti berat molekul, jumlah donor ikatan hidrogen, atau koefisien partisi oktanol-air ($\log P$) yang mengukur seberapa “berminyak” sebuah molekul. *Molecular fingerprint* adalah vektor biner panjang yang menandai keberadaan pola-pola substruktur. Extended connectivity fingerprint (Rogers and Hahn, 2010) dibangun dengan cara yang mirip uji Weisfeiler–Leman di Bab 9. Setiap atom mulai dengan pengenalan dari sifat-sifatnya sendiri. Di setiap iterasi, pengenalan atom diperbarui dengan meng-*hash* pengenalnya bersama pengenalan tetangga-tetangganya, sehingga setelah dua atau tiga iterasi, setiap pengenalan mewakili lingkungan berjari-jari dua atau tiga ikatan. Semua pengenalan kemudian dilipat ke dalam vektor berukuran

tetap, misalnya 2048 bit. ECFP adalah message passing tanpa parameter yang dipelajari.

Kemiripan dua molekul diukur dengan koefisien Tanimoto pada fingerprint-nya. Koefisien ini adalah rasio irisan terhadap gabungan,

$$T(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{c}{a + b - c}, \quad (19.1)$$

dengan a dan b jumlah bit yang menyala pada masing-masing fingerprint dan c jumlah bit yang menyala pada keduanya. Bentuk kedua didapat karena $|A \cup B| = |A| + |B| - |A \cap B|$: bit yang ada di keduanya terhitung dua kali dalam $a + b$, sehingga harus dikurangi sekali. Kalau molekul A punya 20 bit menyala, B punya 18, dan 12 di antaranya sama, kemiripannya $12/(20 + 18 - 12) = 12/26 \approx 0,46$. Dalam praktik, nilai di atas sekitar 0,7 untuk ECFP sering dianggap tanda dua molekul cukup mirip untuk kemungkinan punya aktivitas serupa, tetapi ambang ini bergantung pada jenis fingerprint.

19.2 Aktivitas biologis

Seberapa kuat sebuah senyawa memengaruhi targetnya diukur dalam percobaan yang disebut *assay*. Pada konsentrasi yang semakin tinggi, efeknya biasanya naik mengikuti kurva berbentuk S. Bentuk ini bisa diturunkan dari kesetimbangan pengikatan yang paling sederhana. Misalkan reseptor R dan ligan L membentuk kompleks RL dengan konstanta disosiasi

$$K_d = \frac{[R][L]}{[RL]}. \quad (19.2)$$

Jumlah reseptor total tetap, $[R]_{\text{tot}} = [R] + [RL]$. Fraksi reseptor yang terikat adalah $\theta = [RL]/[R]_{\text{tot}}$. Dari definisi K_d , $[R] = K_d[RL]/[L]$, sehingga

$$\theta = \frac{[RL]}{K_d[RL]/[L] + [RL]} = \frac{[L]}{K_d + [L]}. \quad (19.3)$$

Ketika konsentrasi ligan sama dengan K_d , tepat separuh reseptor terikat. Pada skala logaritmik konsentrasi, fungsi ini menjadi kurva S yang terkenal. Untuk penghambatan, konsentrasi yang memberi separuh efek maksimum disebut IC_{50} , dan untuk pemodelan biasanya dipakai $pIC_{50} = -\log_{10}(IC_{50} \text{ dalam molar})$. Senyawa dengan IC_{50} 100 nano-molar punya pIC_{50} sebesar 7, dan setiap kenaikan satu satuan berarti senyawa sepuluh kali lebih kuat. Skala logaritmik ini membuat target regresi lebih mendekati distribusi normal, sesuai asumsi noise Gauss dari Bab 1.

Obat tidak cukup kuat, ia juga harus sampai ke tempatnya. Sifat *ADME*, yaitu penyerapan, distribusi, metabolisme, dan ekskresi, menentukan hal ini. Aturan lima Lipinski (Lipinski et al., 1997) adalah heuristik terkenal untuk obat yang diminum: berat molekul tidak lebih dari 500 dalton, $\log P$ tidak lebih dari 5, paling banyak 5 donor dan 10 akseptor ikatan hidrogen. Aturan ini dirumuskan dari pengamatan pada obat-obat yang sudah berhasil, bukan dari hukum fisika, dan banyak pengecualiannya.

Data aktivitas dikumpulkan dalam basis data publik seperti ChEMBL (Gaulton et al., 2012). Kalau disusun sebagai matriks senyawa terhadap target, matriks ini jarang: sebagian besar pasangan tidak pernah diukur. Datanya juga tidak seimbang, karena senyawa aktif jauh lebih sedikit dari yang tidak aktif, dan berisik, karena laboratorium berbeda memberi angka berbeda untuk pengukuran yang sama.

19.3 QSAR dan virtual screening

Quantitative structure–activity relationship (QSAR) adalah nama lama untuk apa yang kini kita sebut supervised learning pada molekul: memprediksi aktivitas dari struktur. Pipeline-nya mengikuti Bab 1: pilih representasi, pilih model, pilih hyperparameter dengan validasi, lalu evaluasi. *Virtual screening* memakai model seperti ini, atau sekadar pencarian kemiripan, untuk menyaring pustaka berisi jutaan sampai miliaran senyawa dan memilih segelintir yang layak diuji di laboratorium. Pendekatan berbasis ligan hanya memakai senyawa aktif yang sudah diketahui. Pendekatan berbasis struktur memakai bentuk tiga dimensi protein target untuk menilai seberapa baik sebuah senyawa bisa menempel.

Kuliah memberi penekanan besar pada satu jebakan: cara membagi data. Ahli kimia bekerja dalam seri senyawa, yaitu banyak varian kecil dari satu kerangka yang sama. Kalau data dibagi acak, varian dari seri yang sama masuk ke data latih dan data uji sekaligus, dan model tampak hebat karena pada dasarnya hanya mengenali saudara dekat. Pembagian berdasarkan kerangka molekul atau berdasarkan kluster memaksa model diuji pada kimia yang benar-benar baru, dan skornya biasanya turun drastis. Bias ini disebut *compound series bias*. Pada model multitask, kebocoran bisa terjadi lewat tugas lain yang mengukur senyawa yang sama, dan pada prediksi sinergi dua obat, lewat pasangan obat yang sama dalam urutan terbalik. Validasi silang bersarang dipakai agar pemilihan hyperparameter tidak ikut membocorkan data uji.

DeepTox (Mayr et al., 2016), dari kelompok di Linz, menunjukkan kekuatan jaringan saraf multitask untuk memprediksi toksisitas. Banyak tugas toksisitas berbagi mekanisme, sehingga satu jaringan yang belajar semuanya bersama-sama membangun representasi yang lebih baik daripada model terpisah untuk setiap tugas. ChemProp (Yang et al., 2019) membawa message passing langsung ke graf molekul, sehingga fiturnya dipelajari alih-alih dirancang. Untuk sifat yang bergantung pada bentuk tiga dimensi, jaringan ekuivarian seperti EGNN (Satorras et al., 2021) menjamin bahwa prediksi tidak berubah kalau molekul diputar.

19.4 Belajar dari banyak modalitas

Data ilmu hayati datang dalam banyak bentuk: struktur molekul, deskripsi teks sebuah assay, gambar mikroskop sel, urutan protein. Kuliah membahas cara menghubungkannya dengan *contrastive learning*. Gagasan dasarnya berasal dari CLIP (Radford et al., 2021), yang melatih encoder gambar dan encoder teks agar pasangan yang benar berdekatan dan pasangan yang salah berjauhan di ruang embedding bersama.

Loss-nya bisa diturunkan sebagai masalah klasifikasi. Dalam satu batch ada N pasangan. Untuk molekul ke- i , kita hitung kemiripan kosinus s_{ij} dengan setiap deskripsi j dalam batch, lalu memperlakukan pertanyaan “deskripsi mana yang cocok dengan molekul ini?” sebagai klasifikasi N kelas dengan softmax. Cross-entropy untuk jawaban yang benar memberi

$$\mathcal{L}_i = -\log \frac{\exp(s_{ii}/\tau)}{\sum_{j=1}^N \exp(s_{ij}/\tau)}, \quad (19.4)$$

dengan τ suhu yang mengatur ketajaman softmax. Loss ini dihitung ke dua arah, dari molekul ke teks dan dari teks ke molekul, lalu dirata-ratakan. Sebagai contoh, misalkan batch berisi tiga pasangan, $\tau = 1$, dan untuk molekul pertama kemiripannya (0,9; 0,2; 0,1). Bobot softmax untuk pasangan yang benar adalah $e^{0,9}/(e^{0,9} + e^{0,2} + e^{0,1}) \approx 2,46/(2,46 + 1,22 + 1,11) \approx 0,51$, dan loss-nya $-\ln 0,51 \approx 0,67$. Menurunkan suhu menjadi 0,1 membuat bobot itu mendekati satu.

CLAMP (Seidl et al., 2023) memakai gagasan ini untuk menghubungkan molekul dengan deskripsi teks assay, sehingga model bisa memprediksi aktivitas untuk assay baru hanya dari deskripsinya, tanpa satu pun contoh berlabel. Ini adalah *zero-shot learning*. Ketika hanya ada segelintir contoh berlabel, pendekatan *few-shot* memakai jaringan Hopfield modern (Bab 6) untuk membandingkan molekul kueri dengan contoh-contoh yang ada dan dengan konteks dari molekul lain. *Active learning* memilih molekul berikutnya yang paling informatif untuk diukur, biasanya yang prediksinya paling tidak pasti.

19.5 Merancang molekul baru

Model generatif membalik arah QSAR: bukan dari struktur ke sifat, melainkan dari sifat yang diinginkan ke struktur. Pendekatan paling sederhana memperlakukan SMILES sebagai teks dan melatih LSTM untuk menebak karakter berikutnya (Segler et al., 2018a). Setelah dilatih pada jutaan molekul, model ini menghasilkan string baru yang sebagian besar adalah molekul sah. Untuk mengarahkannya ke sifat tertentu, model disetel dengan reinforcement learning (Bab 12), dengan model QSAR sebagai fungsi ganjaran, atau dilatih ulang pada molekul-molekul terbaik. VAE (Gómez-Bombarelli et al., 2018) memetakan molekul ke ruang laten kontinu, sehingga optimasi bisa dilakukan dengan gradien di ruang itu.

Ada beberapa jebakan. Model yang dioptimasi terhadap satu skor cenderung menemukan celah dalam skor itu, menghasilkan molekul yang secara angka sempurna tetapi aneh secara kimia. Mode collapse membuat model terus menghasilkan variasi kecil dari satu molekul. Untuk mengevaluasi model generatif, kita memeriksa validitas, keunikan, kebaruan, dan keragaman molekul yang dihasilkan, serta seberapa mirip distribusinya dengan molekul nyata. Fréchet ChemNet Distance (Preuer et al., 2018), dari Linz juga, mengukur hal terakhir dengan membandingkan distribusi aktivasi sebuah jaringan prediksi bioaktivitas untuk kedua himpunan molekul. Kalau kedua distribusi dianggap Gauss, jarak Fréchet di antaranya punya bentuk tertutup,

$$d^2 = \|\mu_1 - \mu_2\|^2 + \text{Tr}(\Sigma_1 + \Sigma_2 - 2(\Sigma_1 \Sigma_2)^{1/2}). \quad (19.5)$$

Dalam satu dimensi, rumus ini menyederhana menjadi $(\mu_1 - \mu_2)^2 + (\sigma_1 - \sigma_2)^2$, karena $\sigma_1^2 + \sigma_2^2 - 2\sigma_1\sigma_2 = (\sigma_1 - \sigma_2)^2$: selisih rata-rata ditambah selisih lebar sebaran. Kuliah juga mengingatkan bahwa metrik ini menangkap banyak bias dari jaringan yang dipakainya, sehingga angka FCD yang baik belum menjamin molekul yang baik.

19.6 Data medis

Satu bagian kuliah membahas data klinis. Catatan kesehatan elektronik berisi data tabular dengan campuran variabel kontinu, kategorikal, dan waktu, dengan banyak nilai yang hilang. Nilai yang hilang jarang hilang secara acak: tes yang tidak dilakukan sering berarti dokter tidak menganggapnya perlu, dan fakta itu sendiri membawa informasi. Kuliah membahas contoh model yang belajar sinyal yang salah, seperti penanda rumah sakit atau jenis alat pemindai, alih-alih sinyal penyakit.

Untuk data tabular, model berbasis pohon masih sulit dikalahkan, meski arsitektur khusus terus dikembangkan. TabPFN (Hollmann et al., 2025) mengambil jalan yang tidak biasa: sebuah transformer dilatih sekali pada jutaan masalah tabular sintetis, lalu untuk data baru ia memprediksi langsung dalam satu lintasan maju, dengan data latih sebagai konteks, tanpa pelatihan ulang. Data medis juga tersebar di banyak rumah sakit yang tidak boleh saling berbagi data pasien. *Federated learning* melatih model bersama dengan hanya bertukar pembaruan parameter, bukan data mentahnya.

19.7 Merencanakan sintesis

Molekul yang dirancang komputer tidak berguna kalau tidak bisa dibuat. *Retrosynthesis* bekerja mundur: dari molekul target, cari reaksi yang bisa menghasilkannya dari bahan yang lebih sederhana, lalu ulangi sampai tiba di bahan yang bisa dibeli. Satu langkah mundur bisa dirumuskan sebagai klasifikasi atas aturan reaksi yang diekstrak dari basis data, sebagai terjemahan dari SMILES produk ke SMILES reaktan, atau sebagai pencarian aturan yang paling cocok. MHNreact (Seidl et al., 2022) memakai jaringan Hopfield modern untuk mencocokkan molekul dengan templat reaksi, sehingga templat yang jarang muncul pun bisa dipilih. Untuk rencana banyak langkah, masalahnya mirip permainan: setiap langkah membuka banyak kemungkinan, dan kita mencari jalur yang berakhir di bahan yang tersedia. Segler et al. (2018b) memakai Monte Carlo tree search yang dipandu jaringan saraf, gagasan yang sama dengan AlphaGo (Bab 3).

19.8 Benang merah

Bab ini memperlihatkan bahwa hampir semua alat dari bagian-bagian sebelumnya menemukan tempatnya di kimia dan kedokteran. Message passing dan ekuivariansi untuk molekul, LSTM dan transformer untuk SMILES dan urutan, contrastive learning untuk menghubungkan modalitas, reinforcement learning dan pencarian pohon untuk desain dan sintesis, jaringan Hopfield untuk belajar dari sedikit contoh. Yang membedakan bidang ini adalah datanya: sedikit, berisik, tidak seimbang, dan penuh struktur tersembunyi yang bisa membocorkan jawaban. Di sini, cara mengevaluasi sering lebih menentukan daripada arsitektur model.

CATATAN SUMBER. Bab ini mengikuti kuliah AI in Life Sciences: penemuan obat, representasi molekul, aktivitas biologis dan ADME, QSAR dan virtual screening, evaluasi dan pembagian data, geometric deep learning untuk molekul, model multimodal dan few-shot, model generatif, data medis, dan perencanaan sintesis.

20 Bentuk Protein

Kuliah bioinformatika struktural saya ikuti pada musim panas 2025. Naskah kuliahnya, yang ditulis Sepp Hochreiter dan Noura Chelbat, bergerak dari ikatan kimia dan sudut torsi ke perbandingan struktur, prediksi struktur sekunder, pemodelan homologi, dan prediksi *ab initio*. Bagian praktisnya berupa tantangan di sebuah server penilaian: untuk setiap protein, tebak residu mana yang membentuk *binding pocket*, kantong tempat molekul kecil seperti obat bisa menempel. Prediksi dikirim ke server dan langsung muncul di papan peringkat.

Di Bab 18 kita melihat bahwa gen menyandikan urutan asam amino. Urutan itu baru setengah cerita. Rantai asam amino melipat menjadi bentuk tiga dimensi yang khas, dan bentuk itulah yang menentukan apa yang bisa dilakukan sebuah protein: mengikat oksigen, memotong molekul lain, atau mengirim sinyal melintasi membran sel. Memprediksi bentuk dari urutan adalah salah satu masalah besar biologi selama setengah abad, sampai deep learning mengubahnya.

20.1 Dari rantai ke bentuk

Dua puluh jenis asam amino berbeda pada rantai sampingnya: ada yang suka air, ada yang menghindarinya, ada yang bermuatan. Asam amino tersambung lewat ikatan peptida, yang hampir datar dan kaku. Kebebasan rantai utama terletak pada dua sudut putar di setiap residu, yang disebut sudut torsi ϕ dan ψ . Tidak semua pasangan sudut mungkin, karena atom-atom akan bertabrakan, dan peta pasangan yang diizinkan, diagram Ramachandran, menunjukkan beberapa daerah yang padat.

Struktur protein dijelaskan dalam beberapa tingkat. Struktur primer adalah urutan asam aminonya. Struktur sekunder adalah pola lokal yang berulang, terutama heliks alfa dan lembaran beta, yang distabilkan ikatan hidrogen antar rantai utama. Struktur tersier adalah lipatan tiga dimensi seluruh rantai, dan struktur kuarterner adalah susunan beberapa rantai menjadi satu kompleks. Gaya pendorong utama pelipatan adalah efek hidrofobik: residu yang menghindari air bersembunyi di dalam inti protein, sementara residu yang suka air berada di permukaan.

Seberapa sulit pelipatan itu? Kalau setiap residu dari protein seratus residu hanya punya tiga konformasi, jumlah kemungkinannya $3^{100} \approx 5 \times 10^{47}$. Mencoba satu konformasi setiap 10^{-13} detik pun membutuhkan sekitar 5×10^{34} detik, jauh lebih lama dari umur alam semesta. Padahal protein nyata melipat dalam mikrodetik sampai detik. Paradoks Levinthal ini menunjukkan bahwa pelipatan bukan pencarian acak, melainkan perjalanan menuruni lanskap energi berbentuk corong yang mengarahkan rantai ke lipatan aslinya.

20.2 Membandingkan struktur

Sebelum memprediksi, kita perlu cara mengukur seberapa mirip dua struktur. Ukuran paling umum adalah *root mean square deviation* setelah kedua struktur disuperposisikan sebaik mungkin,

$$\text{RMSD} = \sqrt{\frac{1}{N} \sum_{i=1}^N \|\mathbf{R}x_i + \mathbf{t} - y_i\|^2}, \quad (20.1)$$

dengan x_i dan y_i posisi atom padanan, biasanya atom karbon alfa, dan $\mathbf{R}, \mathbf{t}$ rotasi dan translasi yang meminimalkan jumlah itu. Translasi terbaik ternyata sederhana: geser kedua struktur sehingga pusat massanya berimpit. Untuk rotasinya, setelah keduanya dipusatkan, jumlah kuadrat bisa diuraikan menjadi $\sum \|x_i\|^2 + \sum \|y_i\|^2 - 2 \sum y_i^\top \mathbf{R}x_i$. Dua suku pertama tidak bergantung pada $\mathbf{R}$, jadi meminimalkan RMSD sama dengan memaksimalkan $\sum_i y_i^\top \mathbf{R}x_i = \text{Tr}(\mathbf{R}\mathbf{H})$ dengan $\mathbf{H} = \sum_i x_i y_i^\top$. Kalau $\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ adalah dekomposisi nilai singularnya, maksimum tercapai pada $\mathbf{R} = \mathbf{V}\mathbf{U}^\top$, dengan koreksi tanda kalau determinannya negatif agar hasilnya rotasi sejati dan bukan pencerminan. Prosedur ini, yang dikenal sebagai algoritme Kabsch, sama dengan langkah inti ICP di Bab 17.

RMSD punya kelemahan: satu bagian yang fleksibel, seperti ekor yang bergoyang, bisa membuat dua struktur yang intinya identik tampak jauh berbeda. Karena itu dipakai juga ukuran yang lebih tahan, misalnya fraksi residu yang posisinya berada dalam batas jarak tertentu dari posisi yang benar.

20.3 Struktur sekunder

Prediksi struktur sekunder memberi setiap residu satu dari tiga label: heliks, lembaran, atau lainnya. Metode awal memakai kecenderungan statistik setiap asam amino untuk berada di heliks atau lembaran. Lompatan besar datang dari dua gagasan: memakai jendela residu di sekitar posisi yang diprediksi, dan memakai profil evolusi dari banyak urutan homolog alih-alih satu urutan saja. Naskah kuliah membahas arsitektur jaringan saraf dua tingkat, di mana tingkat pertama memprediksi dari jendela urutan dan tingkat kedua menghaluskan prediksi itu dengan melihat prediksi tetangganya, karena heliks dan lembaran selalu muncul dalam potongan yang berurutan. Kinerjanya diukur dengan Q3, fraksi residu yang labelnya benar.

20.4 Struktur tiga dimensi

Kalau sebuah protein punya kerabat evolusi yang strukturnya sudah diketahui, kita bisa meminjam strukturnya. Pemodelan homologi menjajarkan urutan baru dengan urutan kerabatnya, menyalin koordinat rantai utama untuk bagian yang terjajar, lalu membangun bagian yang berbeda, seperti lengkungan yang lebih panjang. Prinsip dasarnya adalah bahwa struktur dilestarikan evolusi jauh lebih kuat daripada urutan. Kalau tidak ada kerabat yang jelas, *threading* mencoba memasang urutan baru ke setiap lipatan yang dikenal dan menilai kecocokannya dengan potensial kontak statistik. Pendekatan *ab initio* seperti Rosetta merakit struktur dari potongan-potongan pendek

yang diambil dari struktur yang dikenal, lalu mencari konformasi berenergi rendah dengan Monte Carlo. Simulasi dinamika molekul (Bab 24) bisa memperhalus struktur, tetapi terlalu mahal untuk melipat protein dari nol.

Perkembangan bidang ini diukur dengan kompetisi CASP, di mana kelompok-kelompok memprediksi struktur yang sudah dipecahkan secara eksperimen tetapi belum dipublikasikan. Selama bertahun-tahun, kemajuannya lambat.

20.5 AlphaFold

Pada CASP14 tahun 2020, AlphaFold2 (Jumper et al., 2021) memprediksi banyak struktur dengan akurasi yang mendekati percobaan. Gagasan kuncinya sudah lama dikenal dalam bioinformatika, tetapi dimanfaatkan dengan cara baru. Kalau dua residu bersentuhan di dalam struktur, mutasi pada salah satunya sering diikuti mutasi pengimbang pada yang lain, supaya kontakannya tetap terjaga. Jadi pola mutasi yang berkorelasi dalam penjabaran banyak urutan homolog adalah petunjuk tentang residu mana yang berdekatan di ruang tiga dimensi.

AlphaFold2 memproses dua representasi sekaligus: penjabaran banyak urutan, dan representasi pasangan yang menyimpan hubungan antara setiap pasangan residu. Blok Evoformer bertukar informasi di antara keduanya dengan attention, termasuk operasi yang menegakkan konsistensi geometris, misalnya bahwa jarak-jarak di antara tiga residu harus memenuhi ketaksamaan segitiga. Modul struktur kemudian menempatkan setiap residu sebagai benda kaku di ruang dengan attention yang invarian terhadap rotasi dan translasi global, gagasan dari Bab 9. Model ini juga mengeluarkan perkiraan keyakinannya sendiri untuk setiap residu, sehingga pengguna tahu bagian mana yang bisa dipercaya. AlphaFold3 (Abramson et al., 2024) memperluasnya ke kompleks protein dengan DNA, RNA, dan molekul kecil, dengan modul struktur berbasis difusi (Bab 11) yang membangkitkan koordinat atom secara langsung.

Hadiah Nobel Kimia 2024 sebagian diberikan untuk prediksi struktur protein ini, pengakuan bahwa sebuah metode machine learning telah menyelesaikan masalah ilmiah yang mendasar.

20.6 Binding pocket dan obat

Untuk penemuan obat (Bab 19), yang paling penting sering bukan seluruh struktur, melainkan kantong tempat obat menempel. Tantangan di kuliah merumuskan prediksi binding pocket sebagai klasifikasi setiap residu: masuk kantong atau tidak. Fiturnya bisa berasal dari urutan, dari struktur tiga dimensi seperti seberapa terkubur sebuah residu, atau dari embedding model bahasa protein. Karena residu kantong hanya sebagian kecil dari seluruh protein, akurasi biasa menyesatkan, dan ukuran seperti F1 atau tumpang tindih antara himpunan residu tebakan dan himpunan yang benar lebih jujur, persis seperti pelajaran tentang kelas tidak seimbang di Bab 1.

CATATAN SUMBER. Bab ini mengikuti kuliah Structural Bioinformatics dan naskah Bioinformatics III (latar kimia dan fisika, perbandingan dan penjabaran struktur, prediksi struktur sekunder, pemodelan homologi dan threading, prediksi ab initio dan dinamika molekul), beserta tantangan

prediksi binding pocket. Perkembangan terbaru diambil dari makalah AlphaFold (Abramson et al., 2024; Jumper et al., 2021).

21 Melihat ke Dalam Tubuh

Kuliah deep learning for medical imaging dimulai bukan dengan jaringan saraf, melainkan dengan fisika. Naskah dasarnya membahas osilasi dan gelombang, analisis Fourier, gelombang elektromagnetik, fenomena gelombang seperti pembiasan, interferensi, hamburan, dan penyerapan, lalu cara menghasilkan gambar dengan sinar-X, tomografi komputer, dan kedokteran nuklir. Pilihannya masuk akal: kita tidak bisa menilai apa yang dilakukan sebuah jaringan pada citra CT tanpa memahami dari mana angka-angka di citra itu datang.

Citra medis berbeda dari foto biasa. Setiap piksel adalah hasil pengukuran fisik dengan satuan dan noise yang khas, gambar sering berbentuk volume tiga dimensi, data berlabel mahal karena butuh dokter ahli, dan kesalahan bisa berakibat fatal. Bab ini mengikuti jalan dari foton ke diagnosis.

21.1 Sinar-X dan penyerapan

Sinar-X dihasilkan dengan menembakkan elektron yang dipercepat tegangan tinggi ke sebuah anoda. Energi foton terbesar yang bisa muncul sama dengan energi kinetik elektron, $E = h\nu = eU$, sehingga tabung dengan tegangan 100 kilovolt menghasilkan foton sampai 100 kiloelektronvolt. Ketika berkas sinar-X melewati tubuh, sebagian foton diserap atau dihamburkan. Penurunan intensitas di setiap irisan tipis sebanding dengan intensitas yang datang dan dengan ketebalan irisan, $dI = -\mu I ds$, dengan μ koefisien atenuasi jaringan. Memisahkan variabel dan mengintegrasikan memberi hukum Beer–Lambert,

$$I = I_0 \exp \left(- \int \mu(s) ds \right). \quad (21.1)$$

Untuk jaringan seragam dengan $\mu = 0,2 \text{ cm}^{-1}$ setebal 10 cm, hanya $e^{-2} \approx 13,5$ persen foton yang tembus. Tulang menyerap jauh lebih kuat dari jaringan lunak, itulah sebabnya tulang tampak terang pada foto rontgen. Kuliah juga membahas sisi praktisnya: grid anti hamburan yang menyaring foton yang berbelok arah, detektor digital, dan dosis radiasi, yang di Eropa Tengah secara alami sekitar dua milisievert per tahun.

21.2 Tomografi komputer

Foto rontgen biasa menumpuk seluruh tubuh menjadi satu bayangan. Tomografi komputer mengukur banyak proyeksi dari berbagai sudut lalu merekonstruksi peta μ di setiap titik irisan. Dari hukum Beer–Lambert, setiap pengukuran memberi integral garis koefisien atenuasi,

$$p(\theta, r) = -\ln \frac{I}{I_0} = \int_{\text{garis}(\theta, r)} \mu(x, y) ds, \quad (21.2)$$

dan kumpulan semua integral garis disebut transformasi Radon. Masalah utama CT adalah membalik transformasi ini.

Cara paling naif adalah *backprojection*: setiap proyeksi “dioleskan” kembali sepanjang garisnya, lalu semua sudut dijumlahkan. Hasilnya kabur, karena setiap titik terang ikut menyebar ke seluruh garis yang melewatinya, dengan kekaburan yang meluruh seperti $1/r$. Teorema irisan pusat menjelaskan obatnya: transformasi Fourier satu dimensi dari sebuah proyeksi sama dengan satu irisan melalui pusat dari transformasi Fourier dua dimensi gambar. Karena irisan-irisan itu berkumpul rapat di dekat pusat dan jarang di frekuensi tinggi, frekuensi rendah terwakili berlebihan. Mengalikan setiap proyeksi dengan filter yang naik linear terhadap frekuensi, $|k|$, sebelum *backprojection* mengoreksi ketidakseimbangan itu. Inilah *filtered backprojection* (Kak and Slaney, 1988), algoritme yang bekerja di dalam hampir semua pemindai CT sejak pemindai pertama yang dibangun Hounsfield (Hounsfield, 1973).

Nilai yang ditampilkan bukan μ langsung, melainkan satuan Hounsfield, $HU = 1000(\mu - \mu_{\text{air}})/\mu_{\text{air}}$, dengan μ_{air} koefisien atenuasi air. Air bernilai 0 dan udara sekitar -1000 , sedangkan tulang padat bisa beberapa ratus sampai lebih dari seribu. Skala yang dibakukan ini membuat citra dari pemindai berbeda bisa dibandingkan, dan menjadi alasan mengapa normalisasi intensitas untuk jaringan saraf pada CT berbeda dari normalisasi pada foto biasa. Benda logam seperti tambalan gigi menyerap hampir semua foton dan merusak asumsi rekonstruksi, sehingga muncul artefak bergaris yang khas.

21.3 Kedokteran nuklir

Pada skintigrafi dan PET, sumber radiasinya ada di dalam tubuh. Pasien diberi zat penanda radioaktif yang terkumpul di jaringan dengan aktivitas tertentu, misalnya sel tumor yang memakan banyak glukosa. Pada PET, positron yang dipancarkan bertemu elektron dan musnah menjadi dua foton yang terbang ke arah berlawanan. Detektor yang menangkap kedua foton pada saat yang hampir bersamaan tahu bahwa peristiwanya terjadi di sepanjang garis penghubung keduanya, sehingga rekonstruksinya kembali menjadi masalah tomografi. Gambarnya menunjukkan fungsi, bukan anatomi, dan sering digabung dengan CT yang memberi anatomi.

21.4 Deep learning untuk citra medis

Tugas-tugas utama di citra medis sama dengan di Bab 7: klasifikasi, deteksi, dan segmentasi, dengan rekonstruksi sebagai tugas tambahan yang khas. Klasifikasi lesi kulit dengan jaringan konvolusi yang dilatih pada lebih dari seratus ribu gambar mencapai kinerja setara dokter kulit dalam percobaan Esteva et al. (2017). Untuk segmentasi organ dan tumor, U-Net (Ronneberger et al., 2015) yang awalnya dirancang untuk citra biomedis menjadi standar. nnU-Net (Isensee et al., 2021) menunjukkan bahwa sebagian besar keberhasilan datang dari menyetel resep dengan benar untuk setiap himpunan data, yaitu resolusi, normalisasi, augmentasi, dan arsitektur, dan bahwa resep itu bisa dibuat otomatis dari sifat-sifat datanya.

Segmentasi dinilai dengan koefisien Dice, $2|A \cap B|/(|A| + |B|)$, ukuran tumpang tindih antara daerah tebakan A dan daerah yang benar B . Dice berkerabat dekat dengan Tanimoto di Bab 19: kalau Tanimoto $T = |A \cap B|/|A \cup B|$, maka $\text{Dice} = 2T/(1 + T)$, karena $|A| + |B| = |A \cup B| + |A \cap B|$. Tumor yang kecil membuat Dice peka sekali: meleset beberapa voxel saja sudah menurunkan skor dengan tajam.

Rekonstruksi juga mulai dibantu jaringan saraf. CT dosis rendah memakai lebih sedikit foton sehingga lebih aman, tetapi gambarnya lebih berisik. Jaringan bisa membersihkan noise setelah rekonstruksi, atau algoritme rekonstruksi iteratif bisa dibuka menjadi lapisan-lapisan yang dipelajari, seperti yang disinggung di Bab 14. Di semua penerapan ini, pengetahuan fisika tentang bagaimana citra terbentuk tetap menjadi pegangan untuk memeriksa apakah sebuah hasil masuk akal.

21.5 Batas dan tanggung jawab

Kuliah AI dalam ilmu hayati dan kuliah tentang AI dan masyarakat sama-sama menyoroti batas sistem seperti ini. Model bisa belajar jalan pintas, misalnya penanda di sudut gambar yang hanya muncul pada pasien dari rumah sakit tertentu. Model yang dilatih di satu rumah sakit bisa jatuh kinerjanya di rumah sakit lain dengan pemindai berbeda. Dan model bisa bekerja lebih buruk untuk kelompok pasien yang kurang terwakili dalam data latih. Alat-alat XAI di Bab 13 membantu menemukan masalah seperti ini, dan regulasi di Bab 31 menetapkan kewajiban bagi sistem AI berisiko tinggi, termasuk alat diagnosis.

CATATAN SUMBER. Bab ini mengikuti naskah dasar *Deep Learning for Medical Imaging* (gelombang, analisis Fourier, gelombang elektromagnetik, pencitraan sinar-X, tomografi komputer, kedokteran nuklir). Untuk rekonstruksi tomografi, Kak and Slaney (1988) adalah rujukan klasik.

Bagian V

AI untuk Simulasi

22 Fluida yang Ingin Kita Tiru

Kuliah metode numerik dalam mekanika fluida saya ikuti pada musim dingin 2025/26, diberikan oleh Stefan Pirker, yang kemudian juga membimbing tugas akhir saya. Gaya kuliahnya khas. Setiap topik dibuka dengan pertanyaan seperti “aliran apa yang terlintas di kepala Anda?”, dan beberapa slide meminta ChatGPT menulis puisi tentang neraca massa atau pidato perpisahan untuk persamaan Navier–Stokes, sebelum kembali ke persamaan yang sesungguhnya. Bab ini dan bab berikutnya merangkum kuliah itu. Bagian III secara keseluruhan memakai fluida sebagai contoh utama, karena di sanalah saya paling banyak bekerja, tetapi gagasan-gagasannya berlaku untuk banyak sistem fisika lain.

Sebelum membicarakan bagaimana machine learning bisa membantu simulasi, kita perlu tahu apa yang disimulasikan. Bab ini membahas persamaan yang mengatur aliran fluida dan panas, serta bilangan tak berdimensi yang menentukan wataknya.

22.1 Wajah-wajah aliran

Aliran fluida punya banyak wajah. Madu yang dituang mengalir pelan dan teratur. Asap rokok naik lurus sebentar, lalu tiba-tiba pecah menjadi pusaran yang kacau. Awan di belakang pulau kecil di tengah samudra membentuk deretan pusaran yang bergantian ke kiri dan ke kanan, dan foto satelit pulau Alexander Selkirk di Pasifik selatan menjadi contoh yang sering dipakai. Jembatan Tacoma Narrows runtuh pada 1940 karena angin yang berinteraksi dengan getaran strukturnya. Semua ini diatur oleh persamaan yang sama. Yang membedakannya adalah perbandingan antara gaya-gaya yang bekerja.

Bilangan Reynolds menyatakan perbandingan antara inersia dan gesekan kental,

$$\text{Re} = \frac{UL}{\nu}, \quad (22.1)$$

dengan U kecepatan khas, L panjang khas, dan ν viskositas kinematik. Untuk Re di bawah sekitar sepuluh, aliran merayap dan gesekan mendominasi. Pada Re menengah, aliran laminar tetapi bisa melepaskan pusaran secara periodik. Pada Re tinggi, aliran menjadi turbulen. Air dengan viskositas kinematik sekitar $10^{-6} \text{ m}^2/\text{s}$ yang mengalir satu meter per detik di pipa berdiameter lima sentimeter punya Re sekitar lima puluh ribu, jauh di daerah turbulen.

Untuk aliran yang digerakkan perbedaan temperatur, bilangan lain menjadi penting. Bilangan Prandtl membandingkan difusi momentum dengan difusi panas, bilangan Grashof atau Rayleigh mengukur kekuatan gaya apung dibanding gesekan, dan bilangan Péclet membandingkan perpindahan oleh aliran dengan perpindahan oleh difusi. Dua aliran dengan bilangan-bilangan tak berdimensi yang sama berperilaku sama, sekecil atau sebesar apa pun ukurannya. Prinsip kesamaan ini memungkinkan percobaan

dengan model kecil di laboratorium, dan juga menentukan seberapa jauh sebuah model pengganti bisa diharapkan berlaku.

22.2 Hukum kekekalan

Semua persamaan fluida adalah neraca. Untuk sebuah volume kecil di ruang, perubahan jumlah suatu besaran di dalamnya sama dengan apa yang mengalir masuk dikurangi apa yang mengalir keluar, ditambah apa yang diproduksi di dalamnya. Ungkapan matematisnya adalah persamaan transport umum untuk besaran ϕ per satuan massa,

$$\frac{\partial(\rho\phi)}{\partial t} + \nabla \cdot (\rho \mathbf{u}\phi) = \nabla \cdot (\Gamma \nabla \phi) + S_\phi, \quad (22.2)$$

dengan suku perubahan terhadap waktu, suku konveksi oleh aliran, suku difusi, dan suku sumber. Hampir semua persamaan di bab ini adalah kasus khusus bentuk ini.

Kekekalan massa mengambil $\phi = 1$, tanpa difusi dan sumber:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0. \quad (22.3)$$

Untuk fluida tak termampatkan dengan massa jenis tetap, persamaan ini menjadi syarat $\nabla \cdot \mathbf{u} = 0$: apa yang masuk ke sebuah volume harus keluar lagi.

Kekekalan momentum adalah hukum Newton untuk sebuah paket fluida. Percepatan paket itu dikalikan massanya sama dengan gaya tekanan, gaya gesekan kental, dan gaya badan seperti gravitasi. Untuk fluida Newtonian tak termampatkan dengan viskositas tetap, kita mendapat persamaan Navier–Stokes,

$$\rho \left(\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} \right) = -\nabla p + \mu \nabla^2 \mathbf{u} + \rho \mathbf{g}. \quad (22.4)$$

Suku $(\mathbf{u} \cdot \nabla) \mathbf{u}$ adalah sumber semua kesulitan. Ia nonlinear, karena kecepatan mengangkut dirinya sendiri, dan dialah yang melahirkan turbulensi. Tekanan tidak punya persamaan evolusinya sendiri dalam aliran tak termampatkan. Ia muncul sebagai pengali yang menjaga agar $\nabla \cdot \mathbf{u} = 0$ terpenuhi di setiap saat, dan kopling inilah yang membuat solver numerik rumit (Bab 23).

Kekekalan energi untuk aliran dengan perpindahan panas, dalam bentuk temperatur, berbunyi

$$\rho c_p \left(\frac{\partial T}{\partial t} + \mathbf{u} \cdot \nabla T \right) = \nabla \cdot (\lambda \nabla T) + S_T, \quad (22.5)$$

dengan c_p kapasitas panas, λ konduktivitas termal, dan S_T sumber panas seperti reaksi kimia atau radiasi. Kekekalan spesies untuk fraksi massa c_i sebuah zat terlarut mengikuti bentuk yang sama, dengan difusivitas massa sebagai pengganti konduktivitas dan laju reaksi sebagai sumber.

Ketika perbedaan temperatur kecil, perubahan massa jenis bisa diabaikan di semua tempat kecuali di suku gravitasi. Pendekatan Boussinesq menulis $\rho = \rho_0(1 - \beta(T - T_0))$ hanya di suku gaya badan, dengan β koefisien muai. Fluida yang lebih panas menjadi sedikit lebih ringan dan naik. Pendekatan ini cukup untuk menjelaskan sel konveksi di panci yang dipanaskan, angin laut, dan pancaran air hangat yang naik di dalam tangki.

22.3 Batas dan awal

Persamaan saja tidak menentukan sebuah aliran. Kita butuh syarat batas dan syarat awal. Di dinding padat, fluida menempel dan kecepatannya nol, syarat yang disebut *no-slip*. Di saluran masuk, kecepatan atau laju aliran ditentukan. Di saluran keluar, tekanan biasanya ditetapkan. Untuk temperatur, dinding bisa diberi temperatur tetap, fluks panas tetap, atau dibuat adiabatik, yaitu tanpa perpindahan panas sama sekali. Pilihan syarat batas sering lebih menentukan hasil simulasi daripada skema numeriknya, dan seperti akan kita lihat, sebagian besar galat dalam membandingkan simulasi dengan percobaan berasal dari sini.

22.4 Mengapa simulasi aliran mahal

Untuk aliran turbulen, pusaran terkecil yang membuang energi menjadi panas berukuran amat kecil dibanding ukuran domain. Menyelesaikan semua skala itu secara langsung, yang disebut *direct numerical simulation*, membutuhkan jumlah titik grid yang tumbuh kira-kira seperti $Re^{9/4}$ dalam tiga dimensi, dan langkah waktu yang ikut mengecil (Pope, 2000). Untuk aliran industri dengan Re jutaan, itu jauh melampaui komputer mana pun. Simulasi praktis karena itu memodelkan efek skala kecil alih-alih menyelesaikannya, dan pemodelan itulah yang membuka pintu bagi machine learning: sebagai model untuk skala kecil, sebagai pengganti seluruh solver, atau sebagai cara mempercepat solver yang ada. Bab-bab berikutnya membahas ketiganya.

CATATAN SUMBER. Bab ini mengikuti kuliah Numerical Methods in Fluid Mechanics bagian pengantar dan persamaan atur (neraca massa, momentum, energi, spesies). Buku rujukan yang baik adalah Versteeg and Malalasekera (2007), Ferziger et al. (2020), dan untuk turbulensi Pope (2000).

23 Dari Persamaan ke Solver

Setelah persamaan-persamaan diatur, kuliah metode numerik fluida bergerak ke cara menyelesaikannya: metode volume hingga, kopling tekanan dan kecepatan, turbulensi dalam tiga pertemuan, perpindahan panas dan spesies, aliran multifase, dan satu pertemuan khusus tentang recurrence CFD. Bab ini merangkum semuanya kecuali yang terakhir, yang mendapat babnya sendiri di Bab 29.

23.1 Metode volume hingga

Metode volume hingga membagi domain menjadi sel-sel kecil dan menegakkan neraca Persamaan (22.2) untuk setiap sel. Mengintegrasikan persamaan atas volume sebuah sel, lalu memakai teorema divergensi Gauss, mengubah integral divergensi menjadi jumlah fluks yang melewati sisi-sisi sel:

$$\frac{d}{dt} \int_V \rho \phi \, dV + \sum_{\text{sisi } f} (\rho \mathbf{u} \phi - \Gamma \nabla \phi)_f \cdot \mathbf{n}_f A_f = \int_V S_\phi \, dV. \quad (23.1)$$

Keindahan bentuk ini adalah sifat konservatifnya. Fluks yang keluar dari satu sel lewat sebuah sisi adalah persis fluks yang masuk ke sel tetangganya lewat sisi yang sama, sehingga tidak ada massa, momentum, atau energi yang hilang atau muncul di dalam domain kecuali lewat batas dan sumber. Di Bab 9 kita melihat bahwa bentuk ini adalah message passing: setiap sel mengumpulkan “pesan” fluks dari tetangganya.

Nilai di sisi sel harus diperkirakan dari nilai di pusat sel. Untuk suku difusi, beda pusat bekerja baik. Untuk suku konveksi, pilihannya lebih rumit. Beda pusat akurat orde dua, tetapi menghasilkan osilasi yang tidak fisis kalau konveksi jauh lebih kuat daripada difusi di skala sel, yaitu kalau bilangan Péclet sel lebih besar dari dua. Skema *upwind* mengambil nilai dari sel di hulu aliran. Ia stabil dan tidak berosilasi, tetapi hanya orde satu, dan galatnya berperilaku seperti difusi tambahan yang meratakan gradien tajam, gejala yang disebut *numerical diffusion*. Skema orde lebih tinggi seperti second-order *upwind* dan skema dengan limiter mencari jalan tengah. Kuliah memperlihatkan contohnya pada perambatan gelombang permukaan, di mana antarmuka air dan udara mengabur dengan skema orde satu dan tetap tajam dengan skema yang lebih baik.

Untuk waktu, metode eksplisit menghitung keadaan baru langsung dari keadaan lama. Murah per langkah, tetapi langkahnya dibatasi oleh syarat Courant–Friedrichs–Lewy (Courant et al., 1928),

$$\text{CFL} = \frac{u \, \Delta t}{\Delta x} \leq C_{\max}, \quad (23.2)$$

yang secara intuitif berarti informasi tidak boleh melompati lebih dari sekitar satu sel dalam satu langkah. Metode implisit menyelesaikan sistem persamaan untuk keadaan baru. Mahal per langkah, tetapi stabil untuk langkah yang jauh lebih besar.

23.2 Tekanan dan kecepatan

Untuk aliran tak termampatkan, tekanan tidak punya persamaannya sendiri. Ia harus diatur sedemikian rupa sehingga medan kecepatan yang dihasilkan persamaan momentum memenuhi kekekalan massa. Algoritme SIMPLE (Patankar, 1980; Patankar and Spalding, 1972), yang dibahas langkah demi langkah di kuliah, menyelesaikan masalah ini secara iteratif. Pertama, tebak medan tekanan dan selesaikan persamaan momentum untuk mendapat kecepatan sementara. Kecepatan ini umumnya belum memenuhi kekekalan massa. Kedua, tulis hubungan antara koreksi kecepatan dan koreksi tekanan. Ketiga, masukkan hubungan itu ke persamaan kekekalan massa, sehingga diperoleh persamaan untuk koreksi tekanan. Selesaikan, perbarui tekanan dan kecepatan, lalu ulangi sampai konvergen. Pendekatan yang menyelesaikan persamaan-persamaan ini bergantian disebut *segregated solver*, sedangkan solver *coupled* menyelesaikannya bersama dalam satu sistem besar.

Kuliah menekankan perbedaan antara konvergensi numerik dan konvergensi fisis. Residu yang turun beberapa orde menunjukkan bahwa sistem persamaan diskret sudah terselesaikan. Itu belum berarti besaran yang kita pedulikan, misalnya gaya hambat atau temperatur di sebuah titik, sudah tidak berubah lagi, apalagi bahwa hasilnya benar secara fisis. Memantau besaran fisis dan menguji kebebasan hasil terhadap ukuran grid adalah bagian wajib setiap simulasi.

23.3 Turbulensi

23.3.1 Rata-rata Reynolds

Aliran turbulen penuh dengan fluktuasi acak di semua skala. Untuk teknik, biasanya cukup mengetahui rata-ratanya. Dekomposisi Reynolds menulis setiap besaran sebagai rata-rata ditambah fluktuasi, $u = \bar{u} + u'$. Ketika dekomposisi ini dimasukkan ke persamaan Navier–Stokes dan dirata-ratakan, hampir semuanya rapi kecuali satu suku baru dari nonlinearitas, yaitu *Reynolds stress* $-\rho \overline{u'_i u'_j}$. Suku ini tidak diketahui dari besaran rata-rata, sehingga persamaannya tidak tertutup. Inilah *closure problem* turbulensi.

Boussinesq mengusulkan agar Reynolds stress diperlakukan seperti tegangan kental, dengan sebuah viskositas turbulen μ_t yang jauh lebih besar dari viskositas molekuler. Masalahnya bergeser menjadi menentukan μ_t . Model k - ε (Launder and Spalding, 1974) menyelesaikan dua persamaan transport tambahan, untuk energi kinetik turbulen k dan laju disipasinya ε , lalu menulis $\mu_t = \rho C_\mu k^2 / \varepsilon$. Konstanta-konstantanya ditetapkan dari aliran sederhana yang dipahami dengan baik, seperti peluruhan turbulensi di belakang grid dan penyebaran jet. Model k - ω SST (Menter, 1994) memakai formulasi k - ω di dekat dinding dan beralih ke k - ε di aliran bebas, dan menjadi salah satu model paling banyak dipakai di industri. Ketika anisotropi turbulensi penting, seperti di dalam siklon gas atau pusaran ujung sayap, model Reynolds stress menyelesaikan persamaan transport untuk setiap komponen tegangan.

Turbulensi sering digambarkan sebagai kaskade energi. Pusaran besar menerima energi dari aliran rata-rata, pecah menjadi pusaran yang lebih kecil, dan seterusnya,

sampai pusaran terkecil cukup kecil untuk diredam viskositas menjadi panas. Seperti air terjun bertingkat, energi mengalir dari kolam besar ke kolam-kolam yang semakin kecil, dan model turbulensi hanya perlu tahu berapa banyak air yang jatuh ke dasar, bukan bentuk setiap riaknya.

23.3.2 *Large eddy simulation*

Large eddy simulation (LES) menyelesaikan pusaran besar secara langsung dan hanya memodelkan pusaran yang lebih kecil dari ukuran grid. Model Smagorinsky (SMAGORINSKY, 1963) memberi viskositas sub-grid $\mu_{SGS} = \rho(C_s \Delta)^2 |\bar{S}|$, sebanding dengan kuadrat ukuran grid dan laju deformasi yang terselesaikan. LES jauh lebih mahal dari RANS karena harus berjalan dalam waktu dengan grid halus, tetapi jauh lebih murah dari DNS, dan ia menangkap struktur aliran tak tunak yang hilang dalam rata-rata.

23.4 Panas, spesies, dan reaksi

Dalam aliran turbulen, panas dan zat terlarut juga diaduk oleh pusaran. Konduktivitas efektif menjadi jumlah konduktivitas molekuler dan sumbangan turbulen, $\lambda_{\text{eff}} = \lambda + \mu_t c_p / \text{Pr}_t$, dengan bilangan Prandtl turbulen biasanya sekitar 0,85. Radiasi menambah suku sumber dalam persamaan energi, dan reaksi kimia menambah suku sumber dalam persamaan spesies dan energi sekaligus. Kuliah menyinggung contoh-contoh industri, dari nosel Laval sampai pemanasan logam cair, untuk menunjukkan bahwa sebagian besar kerja pemodelan ada pada suku-suku sumber ini, bukan pada solvernya.

23.5 Aliran multifase

Banyak aliran industri melibatkan lebih dari satu fase: gelembung gas di dalam cairan, butiran di dalam gas, tetesan di dalam udara. Kuliah membahas beberapa keluarga pendekatan.

Metode *volume of fluid* (Hirt and Nichols, 1981) melacak antarmuka antara dua fluida yang tidak bercampur dengan sebuah fraksi volume yang bernilai satu di satu fase dan nol di fase lain. Massa jenis dan viskositas dirata-ratakan menurut fraksi itu, dan tegangan permukaan masuk lewat lompatan tekanan Young–Laplace di antarmuka, $\Delta p = \sigma \kappa$, dengan κ kelengkungan antarmuka. Contohnya di kuliah termasuk gelombang di tangki yang bergoyang dan aliran kapiler di saluran mikro.

Untuk partikel atau tetesan yang tersebar, pendekatan Euler–Lagrange melacak setiap partikel, atau sekelompok partikel yang mewakilinya, dalam kerangka Lagrange, sementara fase kontinu diselesaikan di grid. Kopling bisa satu arah, ketika partikel hanya dibawa aliran, atau dua arah, ketika partikel juga memberi gaya balik ke fluida. Untuk aliran butiran padat, metode elemen diskret (Cundall and Strack, 1979) menyelesaikan gerak setiap butir dan tumbukannya, dan gabungannya dengan CFD dipakai untuk unggun terfluidisasi dan pengangkutan pneumatik. Ketika partikel terlalu banyak untuk dilacak satu per satu, model dua fluida memperlakukan kedua fase sebagai kontinum yang saling menembus, digandeng oleh fraksi volume dan pertukaran momentum.

23.6 Di mana machine learning masuk

Bab ini memperlihatkan tiga titik lemah simulasi aliran. Model penutup, seperti viskositas turbulen dan model sub-grid, dibangun dari asumsi yang sering tidak berlaku. Simulasi yang cukup teliti mahal sekali. Dan setiap kasus baru, dengan geometri atau syarat batas berbeda, harus disimulasikan dari awal. Machine learning masuk lewat ketiga pintu itu: model penutup yang dipelajari dari data simulasi teliti (Bab 28), model pengganti yang menggantikan solver (Bab 25 and 26), dan metode yang memanfaatkan pola berulang agar simulasi panjang bisa diekstrapolasi dengan murah (Bab 29).

CATATAN SUMBER. Bab ini mengikuti kuliah Numerical Methods in Fluid Mechanics bagian volume hingga, kopling tekanan dan kecepatan, turbulensi, panas dan spesies, serta multifase. Bacaan pendamping: Versteeg and Malalasekera (2007), Ferziger et al. (2020), Patankar (1980), dan Pope (2000).

24 Fisika Komputasi

Kuliah fisika komputasi saya ikuti pada musim dingin 2025/26. Catatan kuliahnya berupa tulisan tangan, tiga belas pertemuan yang masing-masing membahas satu alat numerik dasar: interpolasi, transformasi Fourier dan FFT, mencari akar, persamaan diferensial biasa, metode simplektik, masalah nilai batas, masalah nilai eigen, persamaan diferensial parsial yang bergantung waktu, metode split-operator, lalu Monte Carlo. Alat-alat ini tersembunyi di dalam hampir setiap simulasi, dan juga di dalam banyak algoritme machine learning.

24.1 Interpolasi dan Fourier

Interpolasi mencari fungsi mulus yang melewati titik-titik data yang diketahui. Polinomial interpolasi Lagrange berderajat $n - 1$ melewati n titik dengan tepat, tetapi untuk derajat tinggi pada titik berjarak sama, ia berosilasi liar di dekat tepi selang. Spline kubik memakai polinomial derajat tiga per potongan yang disambung mulus, dan biasanya jauh lebih jinak. Interpolasi dengan polinomial trigonometri, yaitu jumlah sinus dan kosinus, cocok untuk data periodik, dan koefisiennya tidak lain adalah transformasi Fourier diskret.

Menghitung transformasi Fourier diskret dari N titik secara langsung membutuhkan sekitar N^2 operasi. FFT (Cooley and Tukey, 1965) memecah jumlah itu menjadi dua jumlah setengah ukuran, satu untuk indeks genap dan satu untuk indeks ganjil, lalu mengulang pemecahan itu secara rekursif. Biayanya turun menjadi sekitar $N \log_2 N$. Untuk satu juta titik, selisihnya kira-kira dari 10^{12} ke 2×10^7 operasi. FFT menjadi tulang punggung pemrosesan sinyal, pengolahan citra, solver spektral untuk PDE, dan neural operator berbasis Fourier di Bab 26.

24.2 Mencari akar

Banyak masalah berujung pada mencari x yang memenuhi $f(x) = 0$. Metode bagi dua mulai dari selang tempat f berganti tanda, lalu terus membagi selang menjadi dua dan mempertahankan separuh yang masih mengandung pergantian tanda. Lambat tetapi pasti: setiap langkah memberi tambahan satu bit ketelitian. Metode Newton–Raphson memakai garis singgung, $x_{n+1} = x_n - f(x_n)/f'(x_n)$, dan konvergen secara kuadratik di dekat akar, tetapi bisa melompat jauh kalau tebakan awalnya buruk atau turunannya hampir nol. Dalam banyak dimensi, Newton menjadi penyelesaian sistem linear dengan Jacobian di setiap langkah, dan itulah yang terjadi di dalam setiap solver implisit.

24.3 Persamaan diferensial biasa

Masalah nilai awal $\dot{y} = f(y, t)$ dengan $y(0)$ diketahui diselesaikan selangkah demi selangkah. Metode Euler eksplisit, $y_{n+1} = y_n + h f(y_n, t_n)$, hanya akurat orde satu. Runge–Kutta orde empat mengevaluasi f empat kali per langkah dan galat globalnya sebanding dengan h^4 , sehingga memperkecil langkah setengah kali mengurangi galat enam belas kali.

Osilator harmonik memperlihatkan bahwa akurasi saja tidak cukup. Persamaan Newton $\ddot{x} = -\omega^2 x$ punya energi yang kekal, dan di ruang fase posisi dan kecepatan, lintasannya adalah elips tertutup. Euler eksplisit membuat lintasan berputar keluar, karena energinya bertambah sedikit di setiap langkah. Runge–Kutta orde empat jauh lebih akurat, tetapi untuk simulasi yang panjang sekali energinya tetap perlahan bergeser. Metode *simplektik* seperti velocity Verlet (Verlet, 1967) tidak menjaga energi persis, tetapi galat energinya berosilasi dalam batas yang tetap dan tidak menumpuk, karena metode ini menjaga struktur geometris ruang fase, yaitu volume ruang fase, sesuai teorema Liouville. Untuk dinamika molekul dan mekanika benda langit yang berjalan jutaan langkah, sifat ini jauh lebih penting daripada orde akurasi.

Masalah nilai batas berbeda. Syaratnya diberikan di dua ujung, misalnya $u(a) = u_a$ dan $u(b) = u_b$. Metode *shooting* menebak kemiringan awal, menyelesaikan masalah nilai awal, melihat seberapa jauh hasilnya meleset di ujung lain, lalu memperbaiki tebakan dengan pencarian akar. Alternatifnya, turunan diganti beda hingga di grid, sehingga masalahnya menjadi sistem persamaan linear tridiagonal.

24.4 Nilai eigen

Persamaan Schrödinger stasioner dalam satu dimensi adalah masalah nilai eigen: cari energi E dan fungsi gelombang ψ yang memenuhi $-\frac{\hbar^2}{2m}\psi'' + V\psi = E\psi$. Dengan beda hingga, operator turunan kedua menjadi matriks tridiagonal, dan masalahnya menjadi mencari nilai eigen sebuah matriks. Algoritme QR melakukan ini secara iteratif. Matriks diuraikan menjadi hasil kali matriks ortogonal dan matriks segitiga atas, dikalikan dalam urutan terbalik, lalu diulang. Untuk matriks simetris, iterasinya konvergen ke matriks diagonal yang berisi nilai-nilai eigen. Penguraian QR sendiri dibangun dengan refleksi Householder yang mengonvolkan kolom demi kolom.

Nilai eigen muncul di mana-mana di buku ini: arah utama PCA, kestabilan sistem linear, frekuensi getaran struktur, dan mode-mode aliran pada dekomposisi POD.

24.5 PDE yang bergantung waktu

Persamaan difusi $\partial_t u = D \partial_{xx} u$ yang didiskretisasi dengan beda maju dalam waktu dan beda pusat dalam ruang memberi skema eksplisit

$$u_j^{n+1} = u_j^n + \frac{D\Delta t}{\Delta x^2} (u_{j+1}^n - 2u_j^n + u_{j-1}^n). \quad (24.1)$$

Analisis kestabilan von Neumann memasukkan satu gelombang Fourier $u_j^n = g^n e^{ikj\Delta x}$ ke dalam skema dan menghitung faktor penguat $g = 1 - 4r \sin^2(k\Delta x/2)$ dengan

$r = D\Delta t/\Delta x^2$. Supaya tidak ada gelombang yang tumbuh, $|g| \leq 1$ untuk semua k , yang berarti $r \leq 1/2$. Dengan $D = 1$ dan $\Delta x = 0,01$, langkah waktu tidak boleh melebihi 5×10^{-5} . Memperhalus grid dua kali memaksa langkah waktu empat kali lebih kecil. Skema implisit menghapus batas ini dengan harga menyelesaikan sistem linear di setiap langkah.

Untuk persamaan Schrödinger yang bergantung waktu, kuliah membahas metode *split-operator*. Operator Hamilton terdiri atas energi kinetik, yang sederhana di ruang momentum, dan energi potensial, yang sederhana di ruang posisi. Satu langkah waktu dipecah menjadi setengah langkah potensial, satu langkah kinetik setelah FFT ke ruang momentum, lalu setengah langkah potensial lagi setelah FFT balik. Gagasan memecah operator menjadi bagian-bagian yang masing-masing mudah juga dipakai di banyak solver fluida, misalnya untuk memisahkan konveksi dari difusi.

24.6 Monte Carlo

Integral berdimensi tinggi tidak bisa dihitung dengan grid, karena jumlah titiknya tumbuh eksponensial. Integrasi Monte Carlo merata-ratakan fungsi pada titik-titik acak. Galatnya turun seperti $1/\sqrt{N}$, lambat, tetapi tidak bergantung pada dimensi sama sekali. Itulah sebabnya Monte Carlo menang untuk masalah berdimensi tinggi, dan itulah juga hubungannya dengan kutukan dimensi di Bab 1.

Dalam fisika statistik, kita ingin rata-rata besaran atas distribusi Boltzmann $p(\mathbf{X}) \propto \exp(-H(\mathbf{X})/k_B T)$, dengan H energi total sistem banyak partikel. Konstanta normalisasinya, fungsi partisi, mustahil dihitung. Algoritme Metropolis (Metropolis et al., 1953) tidak membutuhkannya. Usulkan gerakan kecil acak sebuah partikel. Kalau energinya turun, terima. Kalau naik sebesar ΔH , terima dengan peluang $\exp(-\Delta H/k_B T)$. Rantai Markov yang dihasilkan mengunjungi konfigurasi dengan frekuensi sesuai distribusi Boltzmann. Ini adalah MCMC yang sama dengan di Bab 10, di tempat kelahirannya.

24.7 Simulasi partikel

Fisika komputasi juga membawa kita ke simulasi partikel. Dinamika molekul (Alder and Wainwright, 1959) mengintegrasikan persamaan Newton untuk setiap atom dengan gaya dari potensial antaratom, biasanya dengan velocity Verlet, dan menghitung sifat makroskopis seperti tekanan dan temperatur sebagai rata-rata (Frenkel and Smit, 2002). Metode elemen diskret melakukan hal serupa untuk butiran yang bertumbukan (Bab 23). *Smoothed particle hydrodynamics* (Gingold and Monaghan, 1977; Monaghan, 1992) memakai partikel untuk mewakili fluida itu sendiri, dengan besaran di setiap titik diperoleh dengan merata-ratakan partikel tetangga menggunakan kernel penghalus. Tanpa grid, metode ini nyaman untuk permukaan bebas dan percikan.

Semua simulasi partikel punya struktur yang sama: setiap partikel berinteraksi dengan tetangga di dekatnya. Struktur ini persis struktur graf, dan simulator berbasis jaringan graf di Bab 26 belajar meniru simulasi partikel seperti ini dengan memperlakukan partikel sebagai simpul dan interaksi sebagai sisi.

CATATAN SUMBER. Bab ini mengikuti tiga belas catatan kuliah Computational Physics I yang ditulis tangan. Salah satu buku yang disebut di awal kuliah adalah Numerical Recipes (Press et al., 2007). Untuk simulasi partikel, Frenkel and Smit (2002) adalah pengantar yang baik.

25 Physics-Informed Neural Networks

PINN tidak punya mata kuliahnya sendiri dalam perjalanan saya. Ia muncul sedikit-sedikit: sebagai neural field yang diatur persamaan di akhir kuliah machine learning lanjut (Bab 11), sebagai kemampuan autodiff menurunkan keluaran terhadap input di kuliah deep learning dasar (Bab 5), dan sebagai salah satu pendekatan yang terus muncul dalam bacaan tentang machine learning untuk mekanika fluida selama mengerjakan tugas akhir. Bab ini merangkai potongan-potongan itu.

Gagasan PINN sederhana. Alih-alih menyelesaikan persamaan diferensial di grid, kita mewakili solusinya dengan jaringan saraf dan melatih jaringan itu agar memenuhi persamaannya. Gagasan ini sudah dicoba pada akhir 1990-an (Lagaris et al., 1998), dan dipopulerkan kembali dengan nama physics-informed neural networks oleh Raissi et al. (2019), setelah autodiff dan GPU membuatnya praktis.

25.1 Resep dasar

Ambil persamaan panas satu dimensi, $\partial_t u = \kappa \partial_{xx} u$ pada $x \in [0, 1]$ dan $t \in [0, T]$, dengan syarat awal $u(x, 0) = u_0(x)$ dan syarat batas $u(0, t) = u(1, t) = 0$. Sebuah MLP $u_\theta(x, t)$ menerima koordinat dan mengeluarkan temperatur. Untuk titik mana pun, autodiff memberi $\partial_t u_\theta$ dan $\partial_{xx} u_\theta$ secara eksak, sehingga kita bisa menghitung *residual* persamaan,

$$r_\theta(x, t) = \partial_t u_\theta(x, t) - \kappa \partial_{xx} u_\theta(x, t). \quad (25.1)$$

Solusi sejati membuat residual nol di mana-mana. Maka kita meminimalkan jumlah kuadrat residual di sekumpulan titik acak di dalam domain, yang disebut *collocation points*, ditambah suku untuk syarat awal dan syarat batas:

$$\mathcal{L}(\theta) = \frac{\lambda_r}{N_r} \sum_{i=1}^{N_r} r_\theta(x_i, t_i)^2 + \frac{\lambda_0}{N_0} \sum_{j=1}^{N_0} (u_\theta(x_j, 0) - u_0(x_j))^2 + \frac{\lambda_b}{N_b} \sum_{k=1}^{N_b} u_\theta(x_k^b, t_k)^2. \quad (25.2)$$

Kalau ada pengukuran, suku keempat menambahkan selisih kuadrat antara u_θ dan data di titik-titik pengukuran. Bobot λ menentukan seberapa penting masing-masing suku, dan seperti akan kita lihat, memilihnya ternyata salah satu bagian tersulit.

Mengapa ini masuk akal? Dari sudut pandang Bab 1, suku residual adalah prior: di antara semua fungsi yang cocok dengan data dan syarat batas, pilih yang memenuhi hukum fisika. Dari sudut pandang Bab 11, u_θ adalah neural field, dan persamaan diferensial adalah aturan yang ditegakkan pada field itu, persis seperti persamaan Eikonal pada signed distance field.

25.2 Contoh kecil yang bisa dihitung tangan

Supaya terasa konkret, ambil persamaan peluruhan $\dot{u} = -ku$ dengan $u(0) = 1$ dan $k = 1$, pada $t \in [0, 1]$. Solusi sejatinya e^{-t} , sehingga $u(1) = e^{-1} \approx 0,368$. Lagaris et al. (1998) menyarankan bentuk solusi yang memenuhi syarat awal secara otomatis, $u(t) = 1 + t N(t)$, dengan N sebuah jaringan. Untuk contoh tangan, kita ganti jaringan itu dengan konstanta a saja, sehingga $u(t) = 1 + at$. Residualnya

$$r(t) = \dot{u} + u = a + 1 + at = a(1 + t) + 1. \quad (25.3)$$

Kita cari a yang meminimalkan $J(a) = \int_0^1 r(t)^2 dt$, versi kontinu dari suku residual dalam Persamaan (25.2). Turunan terhadap a adalah $2 \int_0^1 (a(1+t)+1)(1+t) dt$. Menyamakannya dengan nol memberi

$$a \int_0^1 (1+t)^2 dt = - \int_0^1 (1+t) dt \implies a \cdot \frac{7}{3} = -\frac{3}{2} \implies a = -\frac{9}{14} \approx -0,643. \quad (25.4)$$

Maka $u(1) \approx 1 - 0,643 = 0,357$, hanya sekitar tiga persen di bawah nilai sebenarnya, padahal “jaringan”-nya hanya satu bilangan. Jaringan sungguhan dengan ribuan parameter bisa mendekati lengkungan eksponensial dengan jauh lebih baik. Contoh ini memperlihatkan inti PINN: tidak ada data solusi sama sekali, hanya persamaan dan syarat awal, dan meminimalkan residual sudah cukup untuk menemukan solusi yang masuk akal.

25.3 Masalah invers

Kekuatan PINN yang paling meyakinkan terletak pada masalah invers. Misalkan konduktivitas κ tidak diketahui, tetapi kita punya beberapa pengukuran temperatur. Cukup jadikan κ parameter yang ikut dilatih bersama bobot jaringan. Suku data memaksa u_θ cocok dengan pengukuran, suku residual memaksa u_θ memenuhi persamaan panas dengan κ yang sedang ditaksir, dan optimasi menemukan κ yang membuat keduanya konsisten. Solver klasik harus dijalankan berulang kali di dalam lingkaran optimasi untuk masalah yang sama, sedangkan PINN menyelesaikan masalah maju dan invers sekaligus.

Contoh paling terkenal adalah Raissi et al. (2020), yang disinggung di Bab 7. Dari video konsentrasi zat pewarna di sekitar sebuah benda, PINN memulihkan medan kecepatan dan tekanan yang tidak pernah diukur, karena persamaan transport zat pewarna dan persamaan Navier–Stokes menghubungkan ketiganya.

25.4 Mengapa PINN sulit dilatih

Kalau PINN begitu sederhana, mengapa tidak menggantikan semua solver? Pengalaman komunitas selama beberapa tahun memperlihatkan sejumlah kesulitan yang serius (Cuomo et al., 2022).

Kesulitan pertama adalah keseimbangan suku-suku loss. Wang et al. (2021) memperlihatkan bahwa gradien suku residual bisa jauh lebih besar dari gradien suku batas,

sehingga jaringan memenuhi persamaan di dalam domain tetapi mengabaikan syarat batas. Ini masalah condition number dari Bab 2 dalam bentuk yang khas. Mereka mengusulkan bobot λ yang disesuaikan otomatis berdasarkan besar gradien masing-masing suku. Pendekatan lain memberi setiap titik collocation bobotnya sendiri yang ikut dilatih dengan arah berlawanan, sehingga titik yang sulit mendapat perhatian lebih (McClenny and Braga-Neto, 2023).

Kesulitan kedua berasal dari spectral bias (Bab 11). Analisis neural tangent kernel untuk PINN (Wang et al., 2022) menunjukkan bahwa komponen solusi dengan frekuensi tinggi dipelajari jauh lebih lambat. Solusi dengan gradien tajam atau osilasi cepat, yang justru menarik dalam fluida, menjadi sulit. Fourier features dan aktivasi sinus membantu.

Kesulitan ketiga diperlihatkan Krishnapriyan et al. (2021). Untuk persamaan konveksi dengan koefisien konveksi yang besar, PINN standar gagal total meski arsitekturnya cukup ekspresif. Masalahnya terletak pada lanskap loss yang sulit sekali dioptimasi, bukan pada kapasitas jaringan. Melatih bertahap, mulai dari koefisien kecil lalu dinaikkan, atau memecah domain waktu menjadi potongan yang dilatih berurutan, memperbaiki keadaan. Gagasan terakhir ini juga menyentuh masalah kausalitas: PINN melatih seluruh rentang waktu sekaligus, sehingga bisa saja “memenuhi” persamaan di waktu akhir sebelum benar di waktu awal, padahal informasi fisik mengalir dari masa lalu ke masa depan.

Ada beberapa trik lain yang sering dipakai: syarat batas yang dibangun langsung ke dalam bentuk solusi, seperti $1 + tN(t)$ pada contoh di atas, sehingga suku batas tidak lagi dibutuhkan; pemilihan titik collocation secara adaptif, dengan menambah titik di tempat residualnya besar; dan resep optimasi Adam lalu L-BFGS dari Bab 2.

25.5 Penilaian yang jujur

Untuk masalah maju yang tipikal, misalnya aliran di sekitar benda pada bilangan Reynolds menengah, PINN umumnya lebih lambat dan kurang akurat daripada solver klasik yang sudah matang. Setiap kasus baru juga butuh pelatihan baru, karena PINN mempelajari satu solusi, bukan cara menyelesaikan. PINN paling bernilai di tempat lain: masalah invers dan asimilasi data, masalah dengan data sebaran yang tidak lengkap, geometri yang sulit di-mesh, dan masalah berdimensi tinggi di mana grid tidak mungkin dibangun (Karniadakis et al., 2021). Untuk mempelajari cara menyelesaikan sebuah keluarga persamaan sekaligus, kita butuh pendekatan lain: neural operator dan simulator yang dipelajari, tema bab berikutnya.

CATATAN SUMBER. Bab ini dirangkai dari kuliah Machine Learning: Advanced Techniques (neural field dan loss yang diinformasikan fisika), kuliah deep learning dasar (autodiff), dan bacaan selama tugas akhir. Bacaan utama: Raissi et al. (2019), ulasan Karniadakis et al. (2021) dan Cuomo et al. (2022), serta studi tentang kegagalan pelatihan oleh Wang et al. (2021), Wang et al. (2022), dan Krishnapriyan et al. (2021).

26 Belajar Menyelesaikan: Neural Operator dan Simulator

Bab ini juga tidak berasal dari satu mata kuliah. Kuliah geometric deep learning memberi bahasa graf dan ekuivariansi, kuliah machine learning lanjut memberi transformasi Fourier dan neural ODE, dan dosennya sendiri, Johannes Brandstetter, termasuk peneliti yang membentuk bidang ini, dari message passing PDE solver sampai transformer untuk fisika. Sisanya datang dari bacaan.

PINN mempelajari satu solusi. Kalau syarat awalnya berubah sedikit, pelatihan harus diulang. Neural operator dan simulator yang dipelajari mengambil tujuan yang lebih ambisius: belajar *cara* menyelesaikan sebuah keluarga persamaan, sehingga solusi untuk syarat awal, parameter, atau geometri baru bisa diperoleh dalam satu lintasan maju, sering ribuan kali lebih cepat dari solver klasik.

26.1 Dari fungsi ke fungsi

Solver numerik pada dasarnya adalah sebuah *operator*: ia memetakan satu fungsi, misalnya medan temperatur awal atau distribusi konduktivitas, ke fungsi lain, yaitu solusinya. Jaringan saraf biasa memetakan vektor berukuran tetap ke vektor berukuran tetap, sehingga terikat pada satu resolusi grid. *Operator learning* ingin belajar pemetaan antar ruang fungsi itu sendiri, sehingga model yang dilatih pada grid kasar bisa dievaluasi pada grid yang lebih halus (Kovachki et al., 2021).

26.1.1 DeepONet

DeepONet (Lu et al., 2021) bertumpu pada teorema aproksimasi universal untuk operator. Operator G yang memetakan fungsi input v ke solusi $G(v)$, dievaluasi di titik y , didekati sebagai

$$G(v)(y) \approx \sum_{k=1}^p b_k(v) t_k(y). \quad (26.1)$$

Jaringan *branch* menerima nilai v di sejumlah titik sensor tetap dan mengeluarkan koefisien b_k . Jaringan *trunk* menerima koordinat y dan mengeluarkan fungsi basis $t_k(y)$. Bentuk ini mengingatkan pada ekspansi deret seperti Fourier atau POD (Bab 27), dengan perbedaan bahwa basis dan koefisiennya sama-sama dipelajari. Karena trunk menerima koordinat kontinu, keluarannya bisa dievaluasi di titik mana pun.

26.1.2 Fourier neural operator

Fourier neural operator (Li et al., 2020) berangkat dari bentuk solusi banyak PDE linear: konvolusi syarat awal dengan sebuah kernel, seperti kernel Gauss pada persamaan

panas di Bab 11. Satu lapisan FNO memakai operator integral dengan kernel yang dipelajari,

$$(\mathcal{K}v)(x) = \int \kappa(x - y) v(y) dy. \quad (26.2)$$

Karena kernel hanya bergantung pada selisih $x - y$, operator ini adalah konvolusi, dan teorema konvolusi (Bab 7) mengubahnya menjadi perkalian di domain frekuensi:

$$\mathcal{K}v = \mathcal{F}^{-1}(\mathbf{R} \cdot \mathcal{F}v), \quad (26.3)$$

dengan $\mathbf{R}$ bobot yang dipelajari untuk setiap frekuensi. FNO hanya menyimpan bobot untuk sejumlah kecil frekuensi terendah dan membuang sisanya. Satu lapisan lengkap menjumlahkan operator spektral ini dengan transformasi linear lokal, lalu menerapkan nonlinearitas. Dengan FFT, biayanya sebanding dengan $N \log N$ untuk N titik grid, dan karena bobotnya didefinisikan per frekuensi, bukan per titik grid, model yang sama bisa dijalankan pada resolusi yang berbeda. Pemotongan frekuensi juga punya sisi buruk: struktur yang tajam, seperti gelombang kejut, sulit diwakili oleh sedikit mode Fourier.

26.2 Simulator berbasis graf

Grid teratur jarang ditemui dalam simulasi teknik. Mesh menyesuaikan diri dengan geometri, dan partikel bergerak bebas. Di sini jaringan graf (Bab 9) menjadi alat alami. Graph Network-based Simulator (Sanchez-Gonzalez et al., 2020) memperlakukan setiap partikel sebagai simpul dan menghubungkan partikel yang berdekatan dengan sisi. Arsitekturnya mengikuti pola *encode-process-decode*: keadaan setiap partikel dikodekan menjadi vektor laten, beberapa langkah message passing menyebarkan informasi interaksi, lalu decoder mengeluarkan percepatan setiap partikel. Posisi baru dihitung dengan integrator sederhana, dan langkah itu diulang. Model yang dilatih pada air, pasir, dan cairan kental ini menghasilkan rollout yang stabil untuk ribuan langkah.

MeshGraphNets (Pfaff et al., 2020) membawa gagasan yang sama ke mesh simulasi, dengan dua jenis sisi: sisi mesh yang mengikuti konektivitas elemen, dan sisi “dunia” antara simpul yang berdekatan di ruang walau tidak tersambung di mesh, misalnya untuk kain yang menyentuh dirinya sendiri. Model ini bahkan belajar menyesuaikan resolusi mesh. Message Passing Neural PDE Solvers (Brandstetter et al., 2022) merumuskan message passing sebagai solver PDE umum dan menunjukkan bahwa ia mencakup skema numerik klasik seperti beda hingga dan volume hingga sebagai kasus khusus. Hubungan ini sudah kita lihat di Bab 23: volume hingga adalah message passing fluks.

26.3 Masalah rollout

Hampir semua simulator yang dipelajari bekerja secara autoregresif: tebak langkah berikutnya, lalu pakai tebakan itu sebagai input untuk langkah selanjutnya. Galat kecil di setiap langkah tidak hanya menumpuk, tetapi juga bisa diperbesar. Misalkan galat di langkah t adalah e_t , model menambahkan galat baru δ di setiap langkah, dan kepekaan

langkah model terhadap inputnya diwakili faktor J . Maka $e_{t+1} \approx J e_t + \delta$, dan setelah T langkah

$$e_T \approx \delta \sum_{k=0}^{T-1} J^k. \quad (26.4)$$

Kalau $|J| < 1$, galat jenuh di sekitar $\delta/(1 - |J|)$. Kalau $|J|$ sedikit lebih besar dari satu, galat tumbuh eksponensial. Dengan $J = 1,01$, setelah 500 langkah faktor $J^{500} \approx 145$. Model yang tampak sempurna dalam uji satu langkah bisa meledak dalam rollout panjang.

Ada masalah yang lebih halus. Selama pelatihan, model hanya melihat input dari simulasi sejati. Selama rollout, ia melihat input dari tebakannya sendiri, yang sedikit berbeda distribusinya. Ini covariate shift yang sama dengan behavioral cloning di Bab 12. Beberapa obat sudah dicoba: menambahkan noise pada input selama pelatihan agar model terbiasa dengan input yang tidak sempurna (Sanchez-Gonzalez et al., 2020), melatih dengan rollout beberapa langkah, atau, seperti PDE-Refiner (Lippe et al., 2023), memperhalus setiap tebakan dengan beberapa langkah denoising mirip model difusi, supaya komponen frekuensi tinggi dengan amplitudo kecil tidak diabaikan. Analisis spektrum sering menunjukkan bahwa model kehilangan detail halus lebih dulu, dan detail itulah yang kemudian merusak stabilitas.

26.4 Model fondasi untuk fisika dan cuaca

Keberhasilan paling mencolok terjadi di prakiraan cuaca, di mana data reanalisis global berpuluh tahun tersedia. FourCastNet (Pathak et al., 2022) memakai operator Fourier yang adaptif. GraphCast (Lam et al., 2023) memakai message passing di atas mesh ikosahedral berjenjang yang membungkus bumi, dan dalam banyak ukuran mengungguli sistem prakiraan numerik operasional untuk sepuluh hari ke depan, dengan waktu hitung kurang dari satu menit. GenCast (Price et al., 2025) memakai model difusi untuk menghasilkan ensemble prakiraan yang probabilistik. Aurora (Bodnar et al., 2024) dilatih pada beragam data atmosfer sebagai model fondasi yang bisa disetel ke tugas-tugas baru, seperti kualitas udara.

Gagasan model fondasi juga dibawa ke PDE secara umum. Pretraining pada banyak sistem fisika sekaligus (McCabe et al., 2023) ternyata membantu ketika model disetel ke sistem baru dengan data sedikit. Kumpulan data besar seperti The Well (Ohana et al., 2024) menyediakan simulasi dari banyak bidang fisika untuk melatih dan menguji model seperti ini. Universal Physics Transformers (Alkin et al., 2024) memisahkan model dari diskretisasi: titik mesh atau partikel dikodekan ke ruang laten berukuran tetap, dinamika berjalan di ruang laten, dan decoder bisa ditanya di titik mana pun. Gupta and Brandstetter (2022) membandingkan arsitektur secara sistematis pada berbagai skala ruang dan waktu.

26.5 Pertanyaan yang tetap harus diajukan

Kembali ke tiga pertanyaan di Bab 0. Fisika masuk ke model-model ini terutama lewat data dan arsitektur: simulasi yang benar sebagai data latih, dan ekuivariansi, lokalitas,

serta konservasi sebagai struktur jaringan. Di luar jangkauan data latih, misalnya pada bilangan Reynolds lain atau geometri yang jauh berbeda, jaminannya tipis, dan pengujian harus dilakukan secara eksplisit. Biaya pelatihannya besar, sering setara ribuan simulasi klasik untuk menghasilkan data, sehingga keuntungan baru muncul kalau model dipakai berkali-kali, misalnya untuk optimasi desain, kendali waktu nyata, atau ensemble besar. Ulasan Lino et al. (2023) dan Vinuesa and Brunton (2022) memberi gambaran yang seimbang tentang kekuatan dan batas pendekatan ini dalam mekanika fluida.

CATATAN SUMBER. Bab ini dirangkai dari kuliah Deep Learning: Geometric Techniques, Machine Learning: Advanced Techniques, dan bacaan selama tugas akhir. Makalah utamanya: Lu et al. (2021), Li et al. (2020), Kovachki et al. (2021), Sanchez-Gonzalez et al. (2020), Pfaff et al. (2020), Brandstetter et al. (2022), Lippe et al. (2023), dan Lam et al. (2023).

27 Menemukan dan Memampatkan Dinamika

Makalah SINDy (Brunton et al., 2016) sudah saya simpan sejak musim panas 2024, sebelum kuliah pertama dimulai, dan buku Brunton and Kutz (2022) menyusul ke folder bacaan tidak lama kemudian. Bab ini membahas gagasan-gagasan dari buku itu yang menyambung dengan banyak kuliah: PCA dari kuliah unsupervised dan computer vision, nilai eigen dari fisika komputasi, lasso dari dasar-dasar machine learning, dan sistem dinamis dari kuliah kendali.

Model pengganti di Bab 26 umumnya kotak hitam yang besar sekali. Bab ini menempuh arah sebaliknya. Pertanyaannya bukan “bisakah jaringan meniru simulasi?”, melainkan “berapa sedikit yang sebenarnya kita butuhkan untuk menjelaskan dinamika ini?”. Jawabannya sering mengejutkan: medan aliran dengan jutaan derajat kebebasan kadang bisa dijelaskan oleh beberapa mode saja, dan dinamikanya oleh persamaan dengan beberapa suku.

27.1 Proper orthogonal decomposition

Simpan m potret medan aliran, masing-masing dengan n nilai, sebagai kolom-kolom matriks $\mathbf{X} \in \mathbb{R}^{n \times m}$ setelah rata-ratanya dikurangkan. Dekomposisi nilai singular memberi

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top, \quad (27.1)$$

dengan kolom-kolom $\mathbf{U}$ sebagai mode ruang, nilai singular σ_k di diagonal $\mathbf{\Sigma}$, dan baris-baris $\mathbf{V}^\top$ sebagai evolusi waktu setiap mode. Dalam mekanika fluida, prosedur ini disebut *proper orthogonal decomposition* (Berkooz et al., 1993; Taira et al., 2017). Ia tidak lain adalah PCA dari Bab 4, diterapkan pada potret-potret medan.

Mengapa mode-mode ini istimewa? Teorema Eckart–Young menyatakan bahwa di antara semua matriks berpangkat r , potongan SVD yang hanya menyimpan r mode pertama adalah yang paling dekat ke $\mathbf{X}$, dengan galat kuadrat sama dengan jumlah kuadrat nilai singular yang dibuang, $\sum_{k>r} \sigma_k^2$. Kalau medan berupa kecepatan, σ_k^2 sebanding dengan energi kinetik yang ditangkap mode ke- k , sehingga fraksi $\sum_{k \leq r} \sigma_k^2 / \sum_k \sigma_k^2$ dibaca sebagai fraksi energi yang ditangkap. Untuk aliran melewati silinder pada bilangan Reynolds rendah, beberapa mode pertama datang berpasangan dan bersama-sama menangkap hampir seluruh energi fluktuasi, karena pelepasan pusaran adalah gelombang yang berjalan, dan gelombang berjalan butuh dua mode yang bergeser seperempat periode.

Mode POD bisa dipakai untuk membangun *reduced-order model*. Tuliskan $\mathbf{u}(\mathbf{x}, t) \approx \bar{\mathbf{u}}(\mathbf{x}) + \sum_{k=1}^r a_k(t) \boldsymbol{\varphi}_k(\mathbf{x})$, masukkan ke persamaan Navier–Stokes, lalu proyeksikan ke

setiap mode. Proyeksi Galerkin ini menghasilkan sistem ODE kecil untuk koefisien $a_k(t)$, dengan suku linear dari viskositas dan suku kuadratik dari konveksi. Model seperti ini bisa ribuan kali lebih cepat, tetapi sering tidak stabil untuk waktu panjang, karena mode-mode kecil yang dibuang adalah tempat energi seharusnya dibuang, sama seperti pusaran kecil dalam kaskade turbulensi (Bab 23). Model penutup untuk mode yang hilang, atau model deret seperti LSTM pada koefisien a_k (Bab 6), dipakai untuk memperbaikinya.

27.2 Dynamic mode decomposition

POD memilih mode menurut energi, tanpa peduli dinamikanya. *Dynamic mode decomposition* (SCHMID, 2010) memilih mode menurut dinamika. Misalkan potret-potret berurutan dengan jarak waktu Δt , dan susun dua matriks yang bergeser satu langkah, $\mathbf{X} = [x_1, \dots, x_{m-1}]$ dan $\mathbf{X}' = [x_2, \dots, x_m]$. Kita mencari operator linear $\mathbf{A}$ sehingga $\mathbf{X}' \approx \mathbf{A}\mathbf{X}$, yaitu $\mathbf{A} = \mathbf{X}'\mathbf{X}^+$ dengan pseudo-invers $\mathbf{X}^+$. Untuk medan besar, $\mathbf{A}$ berukuran $n \times n$ dan terlalu besar untuk dibentuk, sehingga kita memproyeksikannya ke r mode POD pertama. Dengan $\mathbf{X} \approx \mathbf{U}_r \boldsymbol{\Sigma}_r \mathbf{V}_r^\top$, diperoleh

$$\tilde{\mathbf{A}} = \mathbf{U}_r^\top \mathbf{X}' \mathbf{V}_r \boldsymbol{\Sigma}_r^{-1}, \quad (27.2)$$

matriks kecil $r \times r$ yang nilai eigennya sama dengan nilai eigen utama $\mathbf{A}$. Setiap nilai eigen λ_k membawa satu mode yang berkembang seperti λ_k^t per langkah. Dalam waktu kontinu, $\omega_k = \ln \lambda_k / \Delta t$, dengan bagian real sebagai laju tumbuh atau luruh dan bagian imajiner sebagai frekuensi. Sebagai contoh, $\lambda = 0,99 e^{0,1i}$ dengan $\Delta t = 0,01$ detik memberi laju luruh $\ln 0,99 / 0,01 \approx -1,0$ per detik dan frekuensi sudut 10 radian per detik.

DMD seperti mendengarkan sebuah orkestra lalu menuliskan partitur: nada mana yang dimainkan, seberapa keras, dan apakah ia menguat atau memudar. POD, sebaliknya, hanya mencatat alat musik mana yang paling banyak bersuara.

27.3 Koopman: linear di ruang yang lebih besar

DMD mendekati dinamika dengan operator linear, padahal kebanyakan sistem nonlinear. Teori operator Koopman (Koopman, 1931) memberi pembenarannya. Alih-alih melacak keadaan x , kita melacak fungsi-fungsi pengamatan $g(x)$. Operator Koopman memajukan pengamatan dalam waktu, dan operator itu selalu linear, walau dinamikanya nonlinear, dengan harga bahwa ruang fungsi pengamatan umumnya berdimensi tak hingga.

Contoh dari Brunton and Kutz (2022) memperlihatkan kapan ruang itu ternyata berhingga. Ambil sistem $\dot{x} = \mu x$ dan $\dot{y} = \lambda(y - x^2)$, yang nonlinear karena suku x^2 . Tambahkan pengamatan baru $z = x^2$. Turunannya $\dot{z} = 2x\dot{x} = 2\mu x^2 = 2\mu z$. Dalam koordinat (x, y, z) , sistemnya menjadi

$$\dot{x} = \mu x, \quad \dot{y} = \lambda y - \lambda z, \quad \dot{z} = 2\mu z, \quad (27.3)$$

sepenuhnya linear. Dalam praktik, pengamatan yang membuat dinamika linear jarang bisa ditebak, sehingga Lusch et al. (2018) melatih autoencoder untuk menemukannya:

encoder memetakan keadaan ke koordinat di mana dinamikanya adalah perkalian matriks, dan decoder mengembalikannya. Kalau berhasil, semua alat kendali linear dari Bab 15 bisa dipakai pada sistem nonlinear.

27.4 Menemukan persamaan: SINDy

Kadang kita ingin lebih dari model yang meniru: kita ingin persamaannya. *Sparse identification of nonlinear dynamics* (Brunton et al., 2016) berangkat dari pengamatan bahwa kebanyakan persamaan fisika hanya punya sedikit suku. Susun pustaka fungsi kandidat dari data, misalnya $\Theta(\mathbf{X}) = [1, x, x^2, x^3, \sin x, \dots]$, dan cari koefisien jarang Ξ sehingga

$$\dot{\mathbf{X}} \approx \Theta(\mathbf{X}) \Xi. \quad (27.4)$$

Ini masalah regresi linear dengan prior jarang, persis lasso dari Bab 1. Algoritme yang dipakai dalam makalah aslinya lebih sederhana: selesaikan kuadrat terkecil, nolkan koefisien yang kecil di bawah ambang, lalu ulangi kuadrat terkecil hanya dengan suku yang tersisa.

Contoh kecil memperlihatkan logikanya. Misalkan data berasal dari $\dot{x} = -2x$, dan pustakanya $[1, x, x^2]$. Pada data tanpa noise, $\dot{x}$ di setiap titik tepat -2 kali x , sehingga kuadrat terkecil memberi koefisien $(0, -2, 0)$ dan pengambangan tidak mengubah apa pun. Dengan noise, kuadrat terkecil biasa memberi koefisien kecil pada 1 dan x^2 yang mencoba menjelaskan noise. Pengambangan membuangnya, dan regresi ulang hanya dengan x memulihkan -2 dengan baik. Hasilnya bukan jaringan dengan ribuan bobot, melainkan persamaan yang bisa dibaca, diperiksa, dan diekstrapolasi. Pada data sistem Lorenz yang chaos, SINDy memulihkan ketiga persamaannya hanya dari lintasan yang diukur.

SINDy punya tiga syarat yang sering menjadi titik lemah. Turunan waktu harus diperkirakan dari data, dan diferensiasi numerik memperbesar noise. Pustakanya harus memuat suku yang benar. Dan koordinatnya harus tepat, karena dalam koordinat yang salah, dinamika sederhana bisa tampak rumit. Untuk masalah terakhir, Champion et al. (2019) melatih autoencoder bersama SINDy, sehingga koordinat dan persamaan ditemukan bersamaan. PDE-FIND (Rudy et al., 2017) memperluas gagasan ini ke persamaan diferensial parsial dengan memasukkan turunan ruang ke dalam pustaka.

27.5 Memilih alat

Ketiga keluarga ini mengisi tempat yang berbeda. POD dan model tereduksi cocok ketika kita punya persamaan dan ingin mempercepatnya. DMD dan Koopman cocok untuk menganalisis dan meramalkan dinamika yang hampir periodik atau berosilasi, dan untuk kendali. SINDy cocok ketika persamaannya belum diketahui dan datanya cukup bersih. Semuanya menghasilkan model yang jauh lebih kecil dan lebih mudah ditafsirkan daripada jaringan dalam, dan karena itulah mereka masih dipakai berdampingan dengan neural operator di Bab 26, sering sebagai lapisan di dalamnya.

CATATAN SUMBER. Bab ini dirangkai dari kuliah Machine Learning: Unsupervised Techniques

(PCA), Control Systems (sistem linear dan nilai eigen), Computational Physics I (SVD dan masalah nilai eigen), dan bacaan selama tugas akhir. Rujukan utama: Brunton and Kutz (2022), Taira et al. (2017), SCHMID (2010), dan Brunton et al. (2016).

28 Model Hibrida: Fisika dan Data Bersama

Di antara semua pendekatan di Bagian V, model hibrida adalah yang paling dekat dengan cara kerja insinyur sehari-hari. Kita tidak membuang solver yang sudah teruji puluhan tahun. Kita membiarkannya mengerjakan apa yang ia kuasai, lalu memakai data untuk memperbaiki bagian yang ia kerjakan dengan buruk. Bab ini dirangkai dari kuliah metode numerik fluida, yang membahas model penutup turbulensi beserta kelemahannya, dan dari bacaan selama tugas akhir.

28.1 Di mana solver lemah

Bab 23 memperlihatkan bahwa simulasi praktis selalu memodelkan sesuatu yang tidak bisa diselesaikan: tegangan Reynolds di RANS, pusaran sub-grid di LES, gaya antara fluida dan partikel di aliran multifase, atau perpindahan panas di dekat dinding yang tidak cukup halus grid-nya. Model-model ini dirumuskan dengan asumsi yang disederhanakan dan dikalibrasi pada aliran sederhana. Ketika dipakai pada aliran yang rumit, misalnya dengan pemisahan aliran, kelengkungan kuat, atau putaran, asumsinya runtuh. Data dari simulasi yang lebih teliti, seperti DNS dan LES, atau dari percobaan, bisa dipakai untuk mempelajari model yang lebih baik (Duraismy et al., 2019).

28.2 Model penutup yang dipelajari

Pendekatan paling langsung adalah melatih jaringan untuk memprediksi besaran yang hilang dari besaran yang tersedia. Untuk RANS, targetnya tegangan Reynolds, dan inputnya gradien kecepatan rata-rata. Tetapi jaringan sembarang tidak menjamin sifat fisika yang paling dasar: hasilnya harus sama tidak peduli bagaimana sistem koordinat diputar. Ling et al. (2016) membangun sifat itu ke dalam arsitektur. Mereka memakai hasil klasik bahwa tensor anisotropi Reynolds $\mathbf{b}$, dalam model viskositas turbulen umum, bisa ditulis sebagai kombinasi dari sepuluh tensor basis $\mathbf{T}^{(n)}$ yang disusun dari tensor laju regangan $\mathbf{S}$ dan tensor rotasi $\mathbf{\Omega}$ yang dinormalisasi (Pope, 2000):

$$\mathbf{b} = \sum_{n=1}^{10} g_n(\lambda_1, \dots, \lambda_5) \mathbf{T}^{(n)}. \quad (28.1)$$

Koefisien g_n hanya bergantung pada lima besaran invarian λ_i , seperti $\text{tr}(\mathbf{S}^2)$ dan $\text{tr}(\mathbf{\Omega}^2)$, yang nilainya tidak berubah ketika koordinat diputar. Tensor basis sendiri berputar bersama koordinat. Jadi kalau jaringan hanya mempelajari g_n dari invarian, hasil $\mathbf{b}$ dijamin berputar dengan benar, apa pun bobot jaringannya. Model viskositas turbulen linear dari Bab 23 adalah kasus khusus dengan hanya suku pertama, $\mathbf{T}^{(1)} = \mathbf{S}$, dan koefisien tetap. Inilah contoh nyata inductive bias dari Bab 0: simetri tidak dipelajari dari data, tetapi dibangun ke dalam bentuk modelnya.

28.3 A priori dan a posteriori

Model penutup yang dipelajari sering tampak baik dalam uji *a priori*: ambil medan DNS, hitung input dan target, lalu ukur galat prediksi. Begitu model dimasukkan ke dalam solver dan simulasi dijalankan, yaitu uji *a posteriori*, hasilnya bisa mengecewakan atau bahkan meledak. Penyebabnya sama dengan masalah rollout di Bab 26. Selama pelatihan, model melihat input dari DNS. Selama simulasi, ia melihat input dari medan yang dihasilkan solver bersama model itu sendiri, yang distribusinya bergeser. Galat kecil mengubah medan, medan yang berubah mengubah input model, dan lingkarannya bisa memperbesar galat.

Obatnya adalah melatih model di dalam lingkaran yang sama dengan tempat ia akan dipakai. Pendekatan *solver-in-the-loop* (Um et al., 2020) menulis satu langkah simulasi hibrida sebagai

$$\mathbf{u}_{n+1} = \mathcal{S}(\mathbf{u}_n) + C_\theta(\mathcal{S}(\mathbf{u}_n)), \quad (28.2)$$

dengan $\mathcal{S}$ solver kasar dan C_θ koreksi yang dipelajari. Kalau solver ditulis dalam kerangka yang bisa diturunkan, gradien loss bisa dibawa mundur melewati beberapa langkah solver sekaligus, sehingga koreksi belajar memperbaiki galat yang muncul dari interaksinya sendiri dengan solver. Gagasan solver yang bisa diturunkan, *differentiable physics*, dibahas lengkap dalam buku Thuerey et al. (2021). Kochkov et al. (2021) memakai pendekatan serupa untuk turbulensi dua dimensi: jaringan mempelajari cara menginterpolasi nilai di sisi sel pada grid yang jauh lebih kasar, dan simulasi yang dihasilkan setara akurasi dengan solver klasik pada grid berkali-kali lebih halus, dengan percepatan puluhan kali.

28.4 Ketidakpastian dan banyak tingkat ketelitian

Model hibrida juga cocok untuk memanfaatkan data dari sumber yang berbeda-beda ketelitian dan harganya. Simulasi kasar murah dan banyak, simulasi teliti mahal dan sedikit, percobaan paling dipercaya tetapi paling jarang. Model *multi-fidelity* mempelajari koreksi dari tingkat murah ke tingkat mahal, yang biasanya jauh lebih sederhana untuk dipelajari daripada solusi lengkapnya. Ketidakpastian prediksi bisa diperkirakan dengan ensemble atau model probabilistik (Bab 8), dan ini penting karena model hibrida sering dipakai justru di luar daerah data latihnya.

Asimilasi data dari Bab 10 adalah bentuk hibrida yang paling tua. Solver memberi ramalan, data memberi koreksi, dan keduanya digabung dengan bobot sesuai ketidakpastian masing-masing. Ketika model penutupnya sendiri tidak pasti, parameter model bisa ikut ditaksir dalam proses asimilasi, misalnya dengan menambahkannya ke vektor keadaan Kalman filter ensemble.

28.5 Apa yang dipelajari, dan apa yang tidak

Pola umum model hibrida bisa diringkas dalam satu kalimat: biarkan fisika menangani apa yang sudah diketahui pasti, terutama hukum kekekalan, dan biarkan data menangani apa yang dimodelkan dengan buruk. Pilihan ini punya keuntungan praktis

yang besar. Karena sebagian besar dinamika dihitung oleh solver yang konservatif, model hibrida jauh lebih jarang melanggar kekekalan massa atau energi daripada model pengganti murni. Karena koreksinya kecil, ia butuh lebih sedikit data. Dan karena strukturnya jelas, kegagalannya lebih mudah didiagnosis. Ulasan Vinuesa and Brunton (2022) dan Brunton et al. (2020) menempatkan pendekatan ini sebagai salah satu jalur yang paling menjanjikan untuk mekanika fluida komputasi.

CATATAN SUMBER. Bab ini dirangkai dari kuliah Numerical Methods in Fluid Mechanics (model penutup turbulensi) dan bacaan selama tugas akhir. Rujukan utama: Duraisamy et al. (2019), Ling et al. (2016), Um et al. (2020), Kochkov et al. (2021), dan Thuerey et al. (2021).

29 Recurrence CFD: Simulasi dari Ingatan

Recurrence CFD adalah topik tugas akhir saya, dan kuliah metode numerik fluida memberinya satu pertemuan penuh. Stefan Pirker membukanya dengan Max dan Moritz, dua anak nakal dari cerita bergambar Wilhelm Busch, dan guru mereka, Lehrer Lämpel. Max dan Moritz membuat tebakan yang berani, sang guru mengoreksinya. Menurut Pirker, begitulah cara algoritme SIMPLE menyelesaikan persamaan Navier-Stokes: tebakan yang tidak sempurna, lalu koreksi berbasis fisika. Dan begitulah juga rCFD dikembangkan. Bab ini mengikuti alur kuliah itu, lalu menceritakan sedikit tentang apa yang saya kerjakan dalam tugas akhir.

29.1 Aliran yang mengulang dirinya

Banyak aliran yang menarik bagi industri berjalan lama sekali. Peleburan baja di dalam ladle, pencampuran di tangki berpengaduk, dan gelembung yang naik di kolom gelembung bisa berlangsung menit sampai jam, sedangkan simulasi CFD yang teliti hanya sanggup menjangkau beberapa puluh detik waktu fisis dalam waktu hitung yang wajar. Untungnya, banyak aliran seperti ini, setelah melewati fase awal, tidak menghasilkan pola yang benar-benar baru. Pusaran datang dan pergi, gelembung naik dalam pola yang mirip, dan keadaan aliran berulang kali kembali ke dekat keadaan yang pernah dikunjungi sebelumnya.

Pengulangan ini bisa diukur. Simpan medan aliran dari simulasi penuh sebagai deretan bingkai $\phi_1, \dots, \phi_M$. Untuk setiap pasangan bingkai, hitung jaraknya, lalu normalisasikan:

$$R_{ij} = \frac{\|\phi_i - \phi_j\|}{\max_{k,l} \|\phi_k - \phi_l\|}. \quad (29.1)$$

Matriks R ini disebut *recurrence matrix*. Diagonalnya nol, karena setiap bingkai identik dengan dirinya sendiri. Kalau aliran periodik sempurna, garis-garis gelap sejajar diagonal muncul pada jarak satu periode. Pada aliran turbulen, polanya lebih kabur, tetapi tetap ada daerah-daerah di luar diagonal dengan nilai kecil: pasangan bingkai yang berjauhan dalam waktu tetapi mirip satu sama lain. Dalam bahasa machine learning, R adalah matriks jarak antar sampel, dan pasangan dengan jarak kecil adalah tetangga terdekat di ruang keadaan aliran.

29.2 rCFD berbasis aliran

Gagasan pertama, yang diterbitkan Lichtenegger and Pirker (2016) untuk kolom gelembung, berbunyi seperti kalimat yang berulang di slide: “kita putuskan dengan berani untuk menyusun aliran buatan dengan menjahit potongan-potongan rangkaian aliran”. Mulailah dari sebuah bingkai, mainkan beberapa bingkai berikutnya dari basis data,

lalu lompat ke bingkai lain yang mirip dengan bingkai terakhir menurut recurrence matrix, dan teruskan dari sana. Hasilnya adalah deret medan aliran sepanjang yang kita mau, yang secara statistik meniru aliran asli, tanpa pernah menyelesaikan persamaan Navier–Stokes lagi.

Di atas aliran yang dijahit ini, besaran yang terbawa aliran, misalnya konsentrasi zat terlarut, dilacak dengan persamaan transport implisit (Persamaan (22.2)). Persamaan transport untuk satu besaran skalar jauh lebih murah daripada persamaan Navier–Stokes yang terkopel, dan karena implisit, langkah waktunya bisa jauh lebih besar. Pada kasus kolom gelembung, rCFD berbasis aliran menghasilkan prediksi yang cocok dengan CFD penuh dengan percepatan sekitar seratus kali.

Bayangkan mencoba meramalkan arus lalu lintas sebuah kota selama setahun. Daripada mensimulasikan setiap mobil, kita merekam lalu lintas selama beberapa minggu, lalu menyusun tahun buatan dengan menyambung potongan-potongan hari yang mirip: setelah Senin pagi yang macet biasanya datang siang yang agak sepi. Kalau yang kita pedulikan adalah ke mana sebuah paket akan sampai, rekaman lalu lintas yang dijahit sudah cukup.

29.3 rCFD berbasis transport

Langkah kedua, dari Pirker and Lichtenegger (2018), bahkan melewati persamaan transport. Kuliah menjelaskannya dengan sungai Danube di Linz. Sebuah zat yang dijatuhkan ke sungai dibawa arus dan menyebar. Arus membawa zat dari satu tempat ke tempat lain, yaitu konveksi, dan pusaran kecil membuatnya menyebar, yaitu difusi. Dalam bahasa sel-sel volume hingga, konveksi berarti sebagian isi sebuah sel berpindah ke sel lain, dan difusi berarti dua sel bertetangga saling bertukar isi. Mesh simulasi adalah jaringan komunikasi antar sel. rCFD berbasis transport memutuskan, sekali lagi dengan berani, untuk mengganti konveksi dengan *cell-to-cell shifts* dan difusi dengan *face swaps* yang direkam dari simulasi penuh.

Secara sederhana, satu langkah transport menjadi perkalian dengan sebuah matriks pemindahan. Kalau c_i adalah jumlah zat di sel i dan P_{ji} fraksi isi sel i yang dipindahkan ke sel j dalam satu langkah, maka

$$c_j^{\text{baru}} = \sum_i P_{ji} c_i. \quad (29.2)$$

Massa total kekal tepat ketika setiap kolom P berjumlah satu, $\sum_j P_{ji} = 1$: semua isi sel i pergi ke suatu tempat, tidak ada yang hilang dan tidak ada yang muncul. Pembaca yang ingat Bab 27 akan mengenali bahwa matriks seperti ini adalah versi diskret dari operator transfer, saudara operator Koopman, yang memajukan distribusi alih-alih memajukan pengamatan.

Hasilnya mencengangkan. Untuk simulasi pencampuran di sebuah ladle baja, slide kuliah membandingkan CFD penuh yang membutuhkan 26 jam dengan rCFD berbasis transport yang selesai dalam 1,6 detik pada perangkat keras yang sama, dengan histogram konsentrasi yang cocok di beberapa titik waktu.

29.4 Tiga kegagalan berani dan koreksinya

Bagian paling jujur dari kuliah adalah tiga “kegagalan berani” rCFD dan cara memperbaikinya.

Kegagalan pertama adalah ketidakseimbangan massa. Untuk simulasi yang panjang, hasil rCFD menyimpang dari CFD penuh. Penyebabnya, pemindahan antar sel yang direkam dari bingkai berbeda dan dijahit bersama tidak menjamin bahwa setiap kolom P berjumlah tepat satu. Secara lokal, sebagian massa hilang dan sebagian lain tercipta. Secara global, neraca massa spesies bisa dihitung dengan mudah, sehingga galat rCFD bisa dipantau. Koreksinya memakai pemantau global itu untuk menyesuaikan pola konsentrasi lokal lewat face swaps yang bekerja pada massa, sehingga neraca kembali seimbang.

Kegagalan kedua adalah difusi yang tidak fisis. Face swaps yang sama kuatnya di semua tempat membuat zat menyebar terlalu cepat di daerah tenang dan terlalu lambat di daerah turbulen. Du et al. (2020), yang menerapkan rCFD pada penyebaran polutan di sekitar bangunan, memperbaikinya dengan membuat kekuatan face swaps bergantung pada turbulensi lokal, yang dalam LES dinyatakan oleh viskositas sub-grid.

Kegagalan ketiga adalah pengulangan yang kurang menonjol. Di domain yang besar, tidak ada dua bingkai yang mirip di *semua* tempat sekaligus. Penjahitan global memasang bingkai yang secara rata-rata mirip tetapi secara lokal, misalnya di ekor pusaran di belakang sebuah gedung, tidak mirip sama sekali. Usulan koreksinya adalah memecah domain menjadi “pulau-pulau” dengan pengulangan yang kuat dan “laut” latar dengan pengulangan yang lemah, masing-masing dengan recurrence matrix sendiri.

29.5 Menuju kembaran digital

Kuliah ditutup dengan visi yang lebih besar. Kalau rCFD begitu cepat, ia bisa berjalan bersamaan dengan proses industri yang sebenarnya, sebagai kembaran digital. Masalahnya, proses nyata punya lebih dari satu keadaan operasi, dan rCFD yang dijalankan maju tanpa koreksi pasti melenceng dari kenyataan. Jalan keluarnya mirip asimilasi data di Bab 10: jalankan rCFD sambil mengoreksinya dengan pengukuran proses yang tersedia secara langsung. Kuliah memperlihatkan percobaan sederhana untuk ide ini, dua aliran air panas dan dingin yang dicampur di sebuah tangki dengan deretan sensor temperatur vertikal. Model tangki ideal yang tercampur sempurna terlalu sederhana karena mengabaikan perbedaan di dalam tangki, sedangkan CFD tanpa validasi rawan galat. rCFD yang diturunkan dari CFD dan dikoreksi dengan data sensor menempati tempat di antara keduanya.

29.6 Tugas akhir saya

Tugas akhir saya menerapkan rCFD berbasis transport pada pancaran apung (*buoyant jet*) di dalam sebuah tangki: air hangat yang masuk dan naik karena lebih ringan, lalu bercampur dengan air di sekitarnya. Kasus ini menyentuh dua kegagalan di atas

sekaligus. Pancaran yang naik membentuk lapisan temperatur yang tajam, dan setiap difusi tambahan dari pemindahan antar sel mengaburkan lapisan itu. Ketika dinding tangki kehilangan panas ke lingkungan, alirannya juga tidak lagi berulang sempurna, karena temperatur rata-rata terus turun.

Benang merah pekerjaannya adalah mengambil lebih banyak informasi dari simulasi penuh dan menyimpannya di dalam basis data rCFD, alih-alih memakai satu konstanta global. Salah satu contohnya adalah difusivitas lokal yang direkam dari CFD dan dipakai kembali saat replay, gagasan yang sejalan dengan koreksi Du et al. (2020). Contoh lain adalah difusivitas negatif yang terkendali untuk mempertajam kembali gradien yang terlalu kabur, dengan pembatas seperti pada skema flux-corrected transport agar tidak muncul osilasi. Hasil-hasil kuantitatifnya ada di laporan tugas akhir. Yang ingin saya sampaikan di sini lebih sederhana: setiap perbaikan mengikuti pola yang sama dengan yang diajarkan di kuliah, yaitu tebakan yang berani diikuti koreksi berbasis fisika.

29.7 rCFD dari sudut pandang machine learning

Setelah membaca bab-bab sebelumnya, rCFD bisa dilihat dengan mata machine learning. Basis data bingkai adalah ingatan, recurrence matrix adalah ukuran kemiripan antar keadaan, dan penjahitan adalah mencari tetangga terdekat di ruang keadaan lalu melompat ke sana. Tidak ada jaringan saraf dan tidak ada pelatihan dengan gradien. rCFD adalah model pengganti nonparametrik, mirip k -nearest neighbour (Bab 4) yang diterapkan pada lintasan dinamika. Matriks pemindahan antar sel adalah operator transfer yang dipelajari dari data, saudara dekat DMD dan Koopman (Bab 27). Dan koreksi neraca massa adalah contoh paling jelas dari prinsip model hibrida (Bab 28): biarkan data memberi dinamika, dan biarkan hukum kekekalan menjaga agar hasilnya tetap fisis.

Kekuatannya juga kelemahannya. Karena hanya memutar ulang dan menyusun kembali apa yang pernah dilihat, rCFD tidak bisa menghasilkan keadaan aliran yang benar-benar baru. Untuk aliran yang memang mengulang dirinya, itu justru jaminan: hasilnya tidak akan berhalusinasi. Untuk aliran yang berubah keadaan operasinya, dibutuhkan basis data dengan banyak keadaan dan cara memilih di antaranya, persis pertanyaan yang diangkat kuliah tentang proses nyata yang punya lebih dari satu keadaan operasi. Di sinilah rCFD dan model pengganti berbasis jaringan saraf, seperti yang disinggung di slide terakhir kuliah, mungkin akan saling melengkapi.

CATATAN SUMBER. Bab ini mengikuti pertemuan tentang recurrence CFD dalam kuliah Numerical Methods in Fluid Mechanics dan pengalaman selama tugas akhir. Makalah utamanya: Lichtenegger and Pirker (2016) untuk rCFD berbasis aliran, Pirker and Lichtenegger (2018) untuk rCFD berbasis transport, dan Du et al. (2020) untuk penerapan pada lingkungan bangunan.

Bagian VI

AI, Manusia, dan Masa Depan

30 Belajar dari Jejak Pengguna

Kuliah learning from user-generated data saya ikuti pada musim panas 2025, diberikan oleh kelompok Markus Schedl, yang banyak meneliti rekomendasi musik. Sebelas kuliahnya bergerak dari collaborative filtering klasik ke evaluasi, data teks dan data implisit, sistem rekomendasi berbasis graf, hibrida, dan konteks, lalu ditutup dengan dua topik yang jarang dibahas di kuliah teknis: bias dan keadilan, serta sistem rekomendasi yang diilhami psikologi. Latihannya membangun rekomendasi item-kNN dari umpan balik implisit, model faktor laten, dan evaluasi dengan nDCG.

Sistem rekomendasi barangkali adalah machine learning yang paling sering kita temui tanpa disadari: video berikutnya, lagu berikutnya, produk yang “mungkin juga Anda suka”. Datanya adalah jejak yang ditinggalkan jutaan pengguna, berupa rating, klik, putaran lagu, dan waktu menonton, dan tugasnya adalah menebak apa yang akan disukai seseorang dari apa yang disukai orang-orang lain.

30.1 Collaborative filtering

Data dasarnya adalah matriks pengguna terhadap item, dengan rating di sel yang diketahui dan kosong di sebagian besar sel lainnya. *Collaborative filtering* berbasis pengguna mengikuti logika sederhana: cari pengguna lain yang selera masa lalunya mirip denganmu, lalu rekomendasikan apa yang mereka sukai. Kemiripan biasanya diukur dengan kosinus atau korelasi Pearson antara vektor rating. Untuk dua pengguna dengan rating (5, 3, 4) dan (4, 2, 5) pada tiga item yang sama, kemiripan kosinusnya

$$\frac{5 \cdot 4 + 3 \cdot 2 + 4 \cdot 5}{\sqrt{5^2 + 3^2 + 4^2} \sqrt{4^2 + 2^2 + 5^2}} = \frac{46}{\sqrt{50} \sqrt{45}} \approx 0,97. \quad (30.1)$$

Korelasi Pearson mengurangkan rata-rata rating setiap pengguna lebih dulu, sehingga pengguna yang selalu memberi rating tinggi dan pengguna yang pelit rating tetap bisa dikenali sebagai bersepakat. Prediksi rating lalu dihitung sebagai rata-rata berbobot dari rating tetangga-tetangga terdekat.

Versi berbasis item (Sarwar et al., 2001) membalik sudut pandangnya: item yang sering disukai oleh orang yang sama dianggap mirip, dan kita merekomendasikan item yang mirip dengan yang sudah kamu sukai. Karena jumlah item biasanya lebih stabil daripada jumlah pengguna, kemiripan antar item bisa dihitung lebih jarang dan dipakai ulang, sehingga pendekatan ini lebih mudah diskalakan.

30.2 Faktor laten

Pendekatan yang menjadi inti solusi-solusi terbaik dalam kompetisi Netflix Prize adalah faktorisasi matriks (Koren et al., 2009). Setiap pengguna u dan setiap item i diwakili

oleh vektor berdimensi kecil, $\mathbf{p}_u$ dan $\mathbf{q}_i$, dan rating diprediksi sebagai

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i, \quad (30.2)$$

dengan μ rata-rata global, b_u kecenderungan pengguna memberi rating tinggi atau rendah, dan b_i kecenderungan item dinilai tinggi atau rendah. Vektor-vektor ini dilatih dengan meminimalkan galat kuadrat pada rating yang diketahui saja, ditambah regularisasi L2:

$$\mathcal{L} = \sum_{(u,i) \text{ diketahui}} (r_{ui} - \hat{r}_{ui})^2 + \lambda (b_u^2 + b_i^2 + \|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2). \quad (30.3)$$

Turunan terhadap $\mathbf{p}_u$ untuk satu rating, dengan galat $e_{ui} = r_{ui} - \hat{r}_{ui}$, adalah $-2e_{ui}\mathbf{q}_i + 2\lambda\mathbf{p}_u$. Langkah SGD (Bab 2) dengan laju η , faktor dua diserap ke dalam η , menjadi

$$\mathbf{p}_u \leftarrow \mathbf{p}_u + \eta(e_{ui}\mathbf{q}_i - \lambda\mathbf{p}_u), \quad \mathbf{q}_i \leftarrow \mathbf{q}_i + \eta(e_{ui}\mathbf{p}_u - \lambda\mathbf{q}_i). \quad (30.4)$$

Dimensi-dimensi laten tidak diberi nama, tetapi sering bisa ditafsirkan: untuk film, satu dimensi mungkin memisahkan film serius dari komedi, dimensi lain film untuk penonton dewasa dari film keluarga. Ini analisis faktor dari Bab 4, diterapkan pada matriks yang sebagian besar kosong.

Dalam praktik, kebanyakan data bukan rating, melainkan umpan balik implisit: klik, putaran, pembelian. Data ini tidak punya sinyal negatif yang jelas, karena tidak mengklik bisa berarti tidak suka atau sekadar tidak melihat. Bayesian personalized ranking (Rendle et al., 2012) mengubah tujuannya menjadi peringkat: untuk setiap pengguna, item yang pernah ia pakai seharusnya diberi skor lebih tinggi dari item yang belum. Loss-nya $-\sum \ln \sigma(\hat{x}_{ui} - \hat{x}_{uj})$ untuk pasangan item i yang dipakai dan j yang tidak, sebuah binary cross-entropy atas pasangan. Neural collaborative filtering (He et al., 2017) mengganti hasil kali titik dengan jaringan saraf, dan pendekatan berbasis graf seperti LightGCN (He et al., 2020) memandang pengguna dan item sebagai simpul graf bipartit dan menyebarkan embedding lewat message passing (Bab 9).

30.3 Mengukur rekomendasi yang baik

Galat rating seperti RMSE kurang relevan kalau yang ditampilkan adalah daftar sepuluh teratas. Ukuran yang peka peringkat lebih cocok. Precision dan recall pada k teratas menghitung berapa item relevan yang masuk daftar. *Discounted cumulative gain* (Järvelin and Kekäläinen, 2002) menambahkan bahwa item relevan di urutan atas lebih berharga daripada di urutan bawah,

$$\text{DCG}@k = \sum_{i=1}^k \frac{\text{rel}_i}{\log_2(i+1)}, \quad (30.5)$$

dan versi ternormalisasinya, nDCG, membaginya dengan DCG dari urutan terbaik yang mungkin. Untuk daftar dengan relevansi (3, 2, 0, 1), DCG-nya $3/1 + 2/1,585 + 0/2 + 1/2,322 \approx 3 + 1,262 + 0 + 0,431 = 4,693$. Urutan idealnya (3, 2, 1, 0) dengan DCG

$3 + 1,262 + 0,5 + 0 = 4,762$. Maka $nDCG \approx 0,985$: hampir sempurna, hanya item relevan terakhir yang sedikit terlambat.

Kuliah menekankan bahwa akurasi bukan segalanya. Daftar yang akurat tetapi hanya berisi item yang sudah populer tidak membantu siapa pun menemukan sesuatu yang baru. Ukuran di luar akurasi menilai keragaman daftar, kebaruan, cakupan katalog, dan *serendipity*, yaitu kejutan yang menyenangkan. Evaluasi offline pada data historis juga berbeda dari evaluasi online dengan pengguna sungguhan, dan studi dengan pengguna sering memperlihatkan bahwa yang dirasakan pengguna tidak selalu sejalan dengan angka offline.

30.4 Konten, konteks, dan hibrida

Collaborative filtering gagal untuk item baru yang belum pernah dipakai siapa pun dan pengguna baru yang belum punya riwayat, masalah yang disebut *cold start*. Rekomendasi berbasis konten mengisi celah ini dengan memakai sifat item itu sendiri: teks deskripsi, tag, atau sinyal audio sebuah lagu. Untuk teks, bobot TF-IDF memberi nilai tinggi pada kata yang sering muncul di sebuah dokumen tetapi jarang di dokumen lain, dan model yang lebih maju memakai embedding kata dan model topik. Sistem hibrida menggabungkan kedua sumber, misalnya dengan merata-ratakan skor, beralih di antara keduanya tergantung situasi, atau memasukkan fitur konten ke dalam model faktor.

Konteks juga mengubah apa yang pantas direkomendasikan. Lagu yang cocok untuk berolahraga pagi berbeda dari lagu untuk menjelang tidur. Kuliah memberi contoh dari riset kelompoknya sendiri tentang rekomendasi musik yang memperhitungkan lokasi dan aktivitas, dengan memetakan tempat dan lagu ke tag yang sama. Rekomendasi berbasis graf pengetahuan menyebarkan preferensi pengguna lewat relasi antar item, seperti penyanyi, genre, dan label rekaman.

30.5 Bias, keadilan, dan psikologi

Sistem rekomendasi tidak hanya mencerminkan selera, tetapi juga membentuknya. *Popularity bias* membuat item yang sudah populer semakin sering direkomendasikan, sehingga semakin populer lagi. Kinerja sistem juga bisa berbeda antar kelompok pengguna, misalnya menurut usia, jenis kelamin, atau negara. Kuliah membahas ukuran untuk mendeteksinya, seperti selisih kinerja rekomendasi antar kelompok, dan contoh dari data musik tentang bagaimana algoritme rekomendasi bisa menggeser porsi musik lokal yang diterima pendengar di berbagai negara. Strategi perbaikannya bisa diterapkan sebelum pelatihan, misalnya menyeimbangkan ulang data, selama pelatihan dengan kendala keadilan, atau sesudahnya dengan menyusun ulang daftar.

Kuliah terakhir menghubungkan rekomendasi dengan psikologi kognitif. Ingatan manusia memudar menurut kurva lupa Ebbinghaus, dan lagu yang terakhir didengar beberapa waktu lalu lebih mudah diingat kembali kalau sering diulang. Model memori dari arsitektur kognitif seperti ACT-R dipakai untuk meramalkan lagu mana yang ingin didengar lagi seseorang, dengan menimbang seberapa sering dan seberapa baru ia mendengarkannya. Sistem rekomendasi yang diilhami psikologi ini juga lebih mudah

dijelaskan, karena alasannya bisa diungkapkan dengan bahasa yang dipahami manusia.

30.6 Di luar layar

Rekomendasi adalah masalah keputusan berurutan, sehingga reinforcement learning (Bab 12) mulai dipakai untuk mengoptimalkan kepuasan jangka panjang, bukan klik berikutnya. Gagasan faktorisasi matriks juga muncul di bidang lain: matriks senyawa terhadap target di penemuan obat (Bab 19) sama jaranganya dengan matriks pengguna terhadap film, dan model yang belajar dari banyak target sekaligus memakai logika yang sama dengan collaborative filtering. Dan pertanyaan tentang bias serta dampak sosial rekomendasi membawa kita ke bab berikutnya.

CATATAN SUMBER. Bab ini mengikuti kuliah dan latihan Learning from User-generated Data: collaborative filtering, faktor laten, evaluasi, data konten dan implisit, rekomendasi berbasis graf, hibrida, konteks, bias dan keadilan, serta sistem rekomendasi yang diilhami psikologi.

31 AI, Hukum, dan Manusia

Di sela kuliah teknis, saya mengikuti serangkaian mata kuliah yang melihat AI dari luar: AI and law, yang diberikan oleh pengacara dan ahli hukum teknologi; AI in society dan technology and society dari kacamata sosiologi; robopsychology dan communicating AI dari Martina Mara, yang laboratoriumnya meneliti bagaimana manusia memandang dan memperlakukan mesin. Bab ini merangkum apa yang saya bawa pulang dari kuliah-kuliah itu. Ia lebih sedikit rumusnya, tetapi pertanyaannya tidak kalah sulit.

31.1 Apa itu sistem AI menurut hukum

Hukum butuh definisi, dan definisi AI ternyata sulit. AI Act Uni Eropa (European Parliament and Council of the European Union, 2024) memakai definisi yang berakar pada pekerjaan kelompok ahli tingkat tinggi Komisi Eropa (High-Level Expert Group on Artificial Intelligence, 2019): sistem berbasis mesin yang dirancang untuk beroperasi dengan berbagai tingkat otonomi, mungkin beradaptasi setelah dipasang, dan untuk tujuan tertentu menyimpulkan dari input yang diterimanya cara menghasilkan keluaran seperti prediksi, konten, rekomendasi, atau keputusan yang bisa memengaruhi lingkungan fisik atau virtual. Kata kuncinya adalah *menyimpulkan*. Program yang hanya menjalankan aturan yang ditulis manusia secara eksplisit umumnya tidak termasuk, sedangkan model yang dilatih dari data termasuk.

31.2 AI Act: regulasi berbasis risiko

Slide kuliah menyusun linimasa AI Act: diusulkan Komisi Eropa pada April 2021, diterbitkan di Lembaran Resmi Uni Eropa pada 12 Juli 2024, dan mulai berlaku pada 1 Agustus 2024, dengan ketentuan-ketentuannya berlaku bertahap. Larangan untuk sistem dengan risiko yang tidak dapat diterima berlaku sejak Februari 2025, kewajiban bagi penyedia model AI serbaguna sejak Agustus 2025, dan menurut jadwal awal sebagian besar ketentuan lainnya mulai Agustus 2026. Jadwal ini kemudian diubah. Paket penyederhanaan yang dikenal sebagai Digital Omnibus on AI, yang disepakati pada Mei 2026 dan diterbitkan di Lembaran Resmi pada akhir Juli 2026, menunda kewajiban untuk sistem berisiko tinggi yang berdiri sendiri ke 2 Desember 2027, dan untuk AI yang tertanam di produk yang sudah diatur regulasi lain ke 2 Agustus 2028. Kewajiban transparansi, misalnya untuk konten sintetis, tetap berjalan sejak Agustus 2026, dengan masa transisi sampai Desember 2026 untuk sistem yang sudah beredar. Ketika buku ini ditulis, peraturannya masih bergerak, sehingga pembaca sebaiknya selalu memeriksa versi terbarunya.

Inti AI Act adalah piramida risiko. Di puncak ada praktik yang dilarang sama sekali: penilaian sosial warga negara (*social scoring*), manipulasi yang mengeksploitasi

kerentanan orang, pengenalan emosi di tempat kerja dan sekolah, kategorisasi biometrik untuk menyimpulkan ras, keyakinan politik, atau orientasi seksual, serta pengumpulan gambar wajah secara tidak tertarget. Di bawahnya ada sistem berisiko tinggi, misalnya AI dalam rekrutmen, penilaian kredit, pendidikan, infrastruktur kritis, dan alat kesehatan. Sistem ini boleh dipakai, tetapi penyediaanya wajib menjalankan manajemen risiko, menjaga kualitas data, membuat dokumentasi teknis, mencatat log, memastikan pengawasan manusia, serta memenuhi syarat akurasi dan ketahanan. Lebih ke bawah, sistem dengan risiko terbatas seperti chatbot terutama dikenai kewajiban transparansi: pengguna harus tahu bahwa mereka berbicara dengan mesin. Sistem lainnya praktis bebas.

Model AI serbaguna, seperti model bahasa besar, ditambahkan menjelang akhir perundingan. Penyediaanya wajib menyiapkan dokumentasi teknis dan informasi bagi pihak yang membangun aplikasi di atasnya. Model yang dilatih dengan total komputasi lebih dari 10^{25} operasi floating point dianggap berpotensi membawa risiko sistemik dan dikenai kewajiban tambahan: evaluasi model, mitigasi risiko, pelaporan insiden serius, dan keamanan siber. Untuk pelanggaran, denda bisa mencapai 35 juta euro atau tujuh persen omzet global tahunan, mana yang lebih tinggi.

31.3 Data pribadi dan GDPR

Sebelum AI Act, perangkat hukum utama untuk AI di Eropa adalah GDPR (European Parliament and Council of the European Union, 2016), yang berlaku sejak 25 Mei 2018. GDPR mengatur pemrosesan data pribadi, yaitu informasi apa pun yang berkaitan dengan orang yang bisa diidentifikasi. Enam prinsipnya menjadi kerangka berpikir yang berguna bagi siapa pun yang melatih model: pemrosesan harus sah, adil, dan transparan; data hanya dipakai untuk tujuan yang dinyatakan; data yang dikumpulkan sesedikit mungkin; data harus akurat; data tidak disimpan lebih lama dari perlu; dan integritas serta kerahasiaannya dijaga. Pengendali data bertanggung jawab membuktikan kepatuhan pada semua prinsip itu.

Satu ketentuan berbicara langsung tentang AI. Seseorang berhak untuk tidak tunduk pada keputusan yang semata-mata didasarkan pada pemrosesan otomatis, termasuk profiling, kalau keputusan itu berakibat hukum atau berdampak signifikan baginya. Ada pengecualian, misalnya kalau diperlukan untuk kontrak, diizinkan undang-undang, atau disetujui secara eksplisit, tetapi pesannya jelas: manusia harus tetap punya tempat dalam keputusan penting. Bagi pengembang, ini berarti penilaian dampak perlindungan data sebelum sistem berisiko dijalankan, dan perlindungan privasi yang dirancang sejak awal (*privacy by design*) alih-alih ditambal belakangan.

Kuliah juga membahas pertanggungjawaban. Arahan tanggung jawab produk Uni Eropa yang direvisi memperluas definisi produk sehingga mencakup perangkat lunak, termasuk sistem AI, dan meringankan beban pembuktian bagi korban dalam kasus yang secara teknis rumit. Usulan arahan tanggung jawab AI membahas pengungkapan bukti dan anggapan hubungan sebab-akibat yang bisa dibantah ketika sistem AI terlibat dalam kerugian.

31.4 Teknologi dan masyarakat

Kuliah sosiologi dimulai dengan konsep *sociological imagination* dari Mills (1959): kemampuan melihat bagaimana masalah pribadi terhubung dengan isu sosial yang lebih besar. Orang yang ditolak kreditnya oleh algoritme mengalami masalah pribadi. Kalau ribuan orang dari lingkungan yang sama ditolak dengan cara serupa, itu isu sosial.

Dari sana kuliah menolak *technological determinism*, pandangan bahwa teknologi berkembang menurut logikanya sendiri lalu mengubah masyarakat. Sebaliknya, teknologi dibentuk oleh pilihan sosial: siapa yang mendanai riset, data apa yang dikumpulkan, masalah siapa yang dianggap penting, dan bagaimana sebuah sistem akhirnya dipakai. Strategi AI nasional berbagai negara menunjukkan bahwa satu teknologi yang sama dibayangkan dengan berbeda di tempat yang berbeda. Benjamin (2019) memperlihatkan bagaimana sistem yang tampak netral bisa mereproduksi ketidakadilan rasial, karena ia belajar dari data dan keputusan masa lalu. Zuboff (2019) menyebut ekonomi yang mengubah perilaku manusia menjadi data untuk memprediksi dan memengaruhinya sebagai kapitalisme pengawasan. Kita sudah melihat contoh kecilnya: embedding kata yang menyerap stereotip gender (Bolukbasi et al., 2016), sistem rekomendasi yang memperkuat popularitas (Bab 30), dan model medis yang belajar penanda rumah sakit alih-alih penyakit (Bab 21).

31.5 Bagaimana manusia memandang mesin

Robopsychology bukan psikologi untuk robot, melainkan psikologi manusia ketika berhadapan dengan mesin yang tampak cerdas. Kuliahnya dimulai dengan metode: perbedaan antara psikologi sehari-hari dan psikologi ilmiah, antara korelasi dan sebab-akibat, dan mengapa eksperimen dengan kelompok yang diacak dibutuhkan untuk menyimpulkan efek. Laboratorium Mara, misalnya, memakai realitas virtual untuk menguji bagaimana sinyal-sinyal sebuah robot kolaboratif memengaruhi perasaan aman pekerja di dekatnya.

Temuan klasik bidang ini adalah bahwa manusia memperlakukan komputer secara sosial hampir secara otomatis. Orang bersikap sopan kepada komputer, menerapkan stereotip gender pada suara mesin, dan merasa tersanjung oleh pujian dari program, meski tahu itu hanya program (Nass and Moon, 2000; Reeves and Nass, 1996). *Anthropomorphism*, kecenderungan memberi sifat manusia pada benda, diperkuat oleh tiga faktor menurut Epley et al. (2007): ketersediaan pengetahuan tentang manusia sebagai kerangka penjelasan, kebutuhan untuk memahami dan meramalkan perilaku sesuatu, dan kebutuhan akan hubungan sosial. Semakin mirip manusia sebuah robot, semakin ia disukai, sampai titik di mana kemiripan yang hampir tetapi tidak sempurna justru terasa menyeramkan. Lembah ganjil ini, *uncanny valley*, diusulkan Mori et al. (2012) pada 1970. Kuliah juga membahas kepercayaan, kreativitas mesin, dan kebutuhan psikologis dasar manusia seperti otonomi dan kompetensi ketika bekerja bersama teknologi.

31.6 Berbicara tentang AI

Kuliah communicating AI membahas literasi AI: kompetensi yang dibutuhkan orang awam untuk memahami, menilai, dan memakai AI secara kritis (Long and Magerko, 2020). Kuliah memperlihatkan bahwa liputan media tentang AI meningkat tajam sejak sekitar 2015, dan bahwa gambar yang dipakai media untuk mewakili AI didominasi robot humanoid, otak bercahaya, dan warna biru. Sebuah survei yang dibahas di kuliah meminta peserta menilai gambar-gambar seperti ini menurut seberapa mirip manusia, seberapa menyeramkan, dan seberapa tepat mereka mewakili AI, lalu melihat bagaimana penilaian-penilaian itu saling berkaitan. Narasi fiksi dan berita tentang AI cenderung berayun antara harapan berlebihan dan ketakutan akan kepunahan, dan teori kultivasi menjelaskan bagaimana paparan berulang membentuk keyakinan publik.

Salah satu tugas kuliah meminta kami menganalisis artikel berita tentang AI, dan tugas kelompoknya merancang kampanye literasi AI untuk kelompok sasaran tertentu. Pelajarannya relevan bagi siapa pun yang bekerja dengan AI: cara kita menjelaskan sebuah sistem memengaruhi apakah orang terlalu percaya, terlalu takut, atau memakainya dengan bijak.

31.7 Apa artinya bagi pengembang

Kuliah-kuliah ini meninggalkan beberapa kebiasaan yang patut dibawa ke pekerjaan teknis. Tanyakan sejak awal siapa yang terdampak sistem yang dibangun dan bagaimana mereka bisa menggugat keputusannya. Periksa kinerja model untuk setiap kelompok, bukan hanya rata-ratanya. Kumpulkan data seperlunya. Dokumentasikan apa yang dilakukan model dan di mana batasnya, karena regulasi semakin mewajibkannya dan karena itu adalah praktik teknik yang baik. Dan jelaskan sistem dengan jujur, tanpa melebih-lebihkan kecerdasannya dan tanpa meremehkan risikonya.

CATATAN SUMBER. Bab ini mengikuti kuliah AI and Law I dan II (definisi AI, AI Act, GDPR, kontrak, tanggung jawab produk), AI in Society dan Technology and Society (imajinasi sosiologis, pembentukan sosial teknologi, AI dan ketidakadilan, kapitalisme pengawasan), Robopsychology (metode psikologi, antropomorfisme, kepercayaan, kreativitas), dan Communicating AI (literasi AI, citra dan narasi AI di media).

32 Ke Mana Arah AI

Selama dua tahun kuliah, bidang ini bergerak lebih cepat daripada slide yang mengajarkannya. Ketika kuliah LSTM membahas xLSTM sebagai pesaing transformer, model bahasa besar sudah dipakai jutaan orang setiap hari. Ketika kuliah AI and law membahas usulan aturan untuk model serbaguna, aturan itu sudah berlaku. Bab penutup ini mencoba merangkum perkembangan besar beberapa tahun terakhir dan menghubungkannya kembali dengan fondasi di bab-bab sebelumnya. Isinya pasti cepat usang, dan memang itulah intinya.

32.1 Model fondasi dan hukum penskalaan

Istilah *foundation model* (Bommasani et al., 2021) menamai pergeseran cara membangun AI. Alih-alih melatih model khusus untuk setiap tugas, kita melatih satu model besar pada data yang luas, biasanya dengan self-supervised learning (Bab 1), lalu menyesuaikan untuk banyak tugas. GPT-4 (OpenAI et al., 2023), model-model terbuka seperti LLaMA (Touvron et al., 2023), CLIP untuk gambar dan teks (Radford et al., 2021), dan AlphaFold untuk protein adalah contoh-contohnya.

Pendorong utamanya adalah pengamatan empiris yang disebut *scaling laws*. Kaplan et al. (2020) menemukan bahwa loss model bahasa turun mengikuti hukum pangkat terhadap jumlah parameter N , jumlah data D , dan komputasi C , misalnya $L(N) \approx (N_c/N)^\alpha$ dengan α kecil. Hukum pangkat berarti garis lurus pada skala log-log: setiap kali model diperbesar sepuluh kali, loss turun dengan faktor yang kira-kira tetap. Komputasi pelatihan transformer bisa diperkirakan dengan aturan praktis $C \approx 6ND$ operasi floating point: sekitar dua operasi per parameter per token untuk lintasan maju, dan sekitar empat untuk lintasan mundur. Hoffmann et al. (2022) menunjukkan bahwa untuk anggaran komputasi tertentu, banyak model sebelumnya terlalu besar dan dilatih dengan terlalu sedikit data, dan bahwa jumlah token yang optimal kira-kira dua puluh kali jumlah parameter. Model dengan 70 miliar parameter dengan demikian butuh sekitar $1,4$ triliun token, dan komputasinya $6 \times 7 \times 10^{10} \times 1,4 \times 10^{12} \approx 5,9 \times 10^{23}$ operasi. Angka ini menarik dibandingkan dengan ambang 10^{25} operasi yang dipakai AI Act untuk menandai model dengan risiko sistemik (Bab 31): ambang itu kira-kira tujuh belas kali lebih besar.

Supaya model raksasa lebih murah dijalankan, arsitektur *mixture of experts* (Fedus et al., 2021) hanya mengaktifkan sebagian kecil subjaringan untuk setiap token, sehingga jumlah parameter total bisa besar sekali sementara komputasi per token tetap terkendali. Untuk menyesuaikan model ke tugas baru tanpa melatih ulang semua parameternya, LoRA (Hu et al., 2021) membekukan matriks bobot $\mathbf{W} \in \mathbb{R}^{d \times d}$ dan hanya mempelajari koreksi berpangkat rendah $\mathbf{BA}$ dengan $\mathbf{B} \in \mathbb{R}^{d \times r}$ dan $\mathbf{A} \in \mathbb{R}^{r \times d}$. Untuk $d = 4096$ dan

$r = 8$, satu matriks penuh punya sekitar 16,8 juta parameter, sedangkan koreksinya hanya $2 \times 4096 \times 8 = 65\,536$, kurang dari setengah persen.

32.2 Menyelaraskan model dengan manusia

Model bahasa yang hanya dilatih menebak kata berikutnya belum tentu membantu, jujur, atau aman. InstructGPT (Ouyang et al., 2022) memperkenalkan resep tiga langkah: penyetelan dengan contoh jawaban yang ditulis manusia, pelatihan model ganjaran dari perbandingan pasangan jawaban (Christiano et al., 2017), lalu optimasi model bahasa terhadap model ganjaran itu dengan PPO (Bab 12), sambil menjaga agar ia tidak terlalu jauh dari model awal.

Direct preference optimization (Rafailov et al., 2023) memperlihatkan bahwa langkah reinforcement learning bisa dilewati. Penurunannya elegan. Tujuan RLHF adalah memaksimalkan ganjaran harapan dikurangi penalti KL terhadap model acuan π_{ref} :

$$\max_{\pi} \mathbb{E}_{y \sim \pi(\cdot|x)} [r(x, y)] - \beta \text{KL}(\pi(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x)). \quad (32.1)$$

Masalah ini punya solusi tertutup, $\pi^*(y | x) \propto \pi_{\text{ref}}(y | x) \exp(r(x, y)/\beta)$. Membalik persamaan ini memberi ganjaran dalam bentuk policy, $r(x, y) = \beta \log(\pi^*(y | x)/\pi_{\text{ref}}(y | x))$ ditambah suku yang hanya bergantung pada x . Dalam model preferensi Bradley-Terry, peluang jawaban y_w lebih disukai dari y_l adalah $\sigma(r(x, y_w) - r(x, y_l))$, dan suku yang hanya bergantung pada x saling menghapus dalam selisih itu. Hasilnya, preferensi bisa dipelajari langsung pada policy dengan loss

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)}\right), \quad (32.2)$$

sebuah binary cross-entropy biasa, tanpa model ganjaran terpisah dan tanpa sampling selama pelatihan.

32.3 Model yang bernalar

Wei et al. (2022) menunjukkan bahwa meminta model menuliskan langkah-langkah penalarannya sebelum menjawab, *chain-of-thought*, memperbaiki kinerja pada soal matematika dan logika secara nyata. Generasi berikutnya melatih kemampuan ini secara langsung. DeepSeek-R1 (DeepSeek-AI et al., 2025) memperlihatkan bahwa reinforcement learning dengan ganjaran yang bisa diperiksa secara otomatis, misalnya apakah jawaban akhir sebuah soal matematika benar atau apakah sebuah program lolos uji, membuat model belajar menulis rantai penalaran panjang, memeriksa ulang, dan mencoba jalan lain ketika buntu. Pola ini mengingatkan pada pelajaran dari AlphaZero di Bab 3: memberi model lebih banyak waktu untuk “berpikir” saat menjawab, dengan mencari dan memeriksa, menjadi sumbu penskalaan baru di samping ukuran model.

Hasilnya terlihat jelas dalam matematika. AlphaGeometry (Trinh et al., 2024) menggabungkan model bahasa dengan mesin deduksi simbolik untuk menyelesaikan soal geometri olimpiade. Pada Olimpiade Matematika Internasional Juli 2025, sebuah

versi Gemini Deep Think dari Google DeepMind menyelesaikan lima dari enam soal dalam bahasa alami dan diakui panitia mencapai standar medali emas¹.

32.4 Agen

Model bahasa yang bisa memakai alat berubah dari penjawab pertanyaan menjadi agen. ReAct (Yao et al., 2022) merangkai penalaran dan tindakan secara bergantian: model menulis pikiran, memanggil alat seperti pencarian web atau kalkulator, membaca hasilnya, lalu berpikir lagi. Retrieval-augmented generation (Lewis et al., 2020) memberi model akses ke basis dokumen sehingga jawabannya bisa bersandar pada sumber yang dikutip, bukan hanya pada ingatan parameternya. Agen pemrograman yang bisa membaca kode, menjalankan uji, dan memperbaiki kesalahannya sendiri menjadi salah satu penerapan yang paling cepat berkembang. Sebagian besar masalah lama muncul kembali dalam bentuk baru: credit assignment untuk tugas panjang (Bab 12), galat yang menumpuk sepanjang banyak langkah seperti pada rollout simulator (Bab 26), dan pertanyaan siapa yang bertanggung jawab ketika agen bertindak (Bab 31).

32.5 Gambar, video, dan model dunia

Model difusi dari Bab 11 menjadi mesin di balik generator gambar modern. Latent diffusion (Rombach et al., 2021) menjalankan difusi di ruang laten sebuah autoencoder, bukan di ruang piksel, sehingga jauh lebih murah. Diffusion transformer (Peebles and Xie, 2022) mengganti U-Net dengan transformer dan memperlihatkan bahwa model difusi juga mengikuti hukum penskalaan. Generator video adalah perpanjangan alami dari gagasan yang sama ke dimensi waktu.

Model yang membangkitkan video yang konsisten secara fisis mulai disebut *world models*, sebuah istilah yang dipakai Ha and Schmidhuber (2018) untuk agen yang belajar model internal lingkungannya lalu berlatih di dalam “mimpi”-nya sendiri. Apakah model video benar-benar memahami fisika atau hanya meniru penampilannya adalah pertanyaan terbuka, dan alat dari Bagian V, seperti uji kekekalan dan ekstrapolasi, adalah cara yang baik untuk mengujinya.

32.6 AI untuk sains

Tahun 2024 memberi penanda simbolis. Hadiah Nobel Fisika diberikan kepada John Hopfield dan Geoffrey Hinton untuk penemuan dasar yang memungkinkan machine learning dengan jaringan saraf tiruan, dan setengah Hadiah Nobel Kimia diberikan kepada Demis Hassabis dan John Jumper untuk prediksi struktur protein (Bab 20). Jaringan Hopfield yang dihargai itu adalah leluhur langsung jaringan Hopfield modern di Bab 6.

¹Pengumuman Google DeepMind, Juli 2025: <https://deepmind.google/blog/advanced-version-of-gemini-with-deep-think-officially-achieves-gold-medal-standard-at-the-international-mathematical-olympiad/>. OpenAI melaporkan hasil setara dengan model eksperimentalnya, tetapi hasil itu tidak disertifikasi oleh koordinator IMO.

Di bidang cuaca, model berbasis machine learning seperti GraphCast (Lam et al., 2023), GenCast (Price et al., 2025), dan Aurora (Bodnar et al., 2024) kini bersaing dengan sistem numerik operasional, dan beberapa pusat prakiraan cuaca mulai menjalankannya berdampingan. Model fondasi untuk fisika umum (Alkin et al., 2024; McCabe et al., 2023) dan kumpulan data simulasi besar (Ohana et al., 2024) mencoba mengulang kesuksesan itu untuk PDE secara umum. Di kimia dan biologi, AlphaFold3 (Abramson et al., 2024) memperluas prediksi struktur ke interaksi antar molekul. Pola yang sama muncul di semua bidang ini: data simulasi dan pengukuran yang terkumpul selama puluhan tahun menjadi bahan bakar, dan struktur fisika, seperti simetri dan kekekalan, menjadi kemudi.

32.7 Yang tidak berubah

Di tengah semua perubahan ini, pelajaran dari kuliah-kuliah dasar tetap berlaku, bahkan semakin penting. Generalisasi di luar data latih tetap sulit (Bab 1). Model besar tetap bisa belajar jalan pintas yang salah, dan evaluasi yang jujur tetap memerlukan pembagian data yang hati-hati. Optimasi tetap bergantung pada condition number dan noise (Bab 2). Model generatif tetap bisa berhalusinasi, sehingga penalaran yang bisa diperiksa dan sumber yang bisa dikutip menjadi semakin berharga. Dan sistem yang kuat tetap harus dijelaskan dengan jujur kepada orang-orang yang hidupnya terdampak (Bab 13 and 31).

Kalau ada satu benang merah yang saya bawa dari empat semester ini, mungkin begini: kemajuan terbesar jarang datang dari membuang apa yang sudah diketahui. LSTM bertahan karena constant error carousel adalah gagasan yang benar. Konvolusi bertahan karena simetri geser adalah sifat dunia. Persamaan Navier–Stokes tidak berhenti berlaku karena ada jaringan saraf. Model terbaik adalah yang belajar dari data sambil menghormati apa yang sudah kita ketahui tentang dunia.

CATATAN SUMBER. Bab ini disusun dari bacaan, bukan dari satu mata kuliah. Sebagian topiknya disinggung di kuliah LSTM (xLSTM dan Mamba), NLP (model bahasa besar), AI and Law (model AI serbaguna), dan AI in Life Sciences (model bahasa untuk kimia). Perkembangan yang baru diperiksa dengan pengumuman resminya pada saat penulisan, Oktober 2026.

A Glosarium

Buku ini membiarkan banyak istilah dalam bahasa Inggris, karena istilah itulah yang dipakai di kuliah dan literatur. Daftar berikut memberi penjelasan singkat dan bab tempat istilah itu dibahas.

activation function

Fungsi nonlinear yang diterapkan pada keluaran setiap neuron, seperti ReLU atau sigmoid (Bab 5).

active learning

Memilih data berikutnya yang paling informatif untuk diberi label (Bab 19).

actor-critic

Metode RL dengan satu jaringan yang memilih aksi dan satu yang menilai (Bab 12).

adjoint method

Cara menghitung gradien sistem dinamis dengan menyelesaikan persamaan mundur dalam waktu (Bab 11).

advantage

Selisih nilai sebuah aksi dengan nilai rata-rata keadaan (Bab 12).

attention

Mekanisme membaca ingatan dengan query, key, dan value (Bab 6).

autodiff (automatic differentiation)

Menghitung turunan program secara eksak dengan aturan rantai (Bab 5).

autoencoder

Jaringan yang memampatkan input menjadi kode kecil lalu merekonstruksinya (Bab 8).

backpropagation

Aturan rantai yang dihitung dari keluaran ke input untuk mendapat gradien (Bab 5).

batch normalization

Normalisasi pre-activation dengan statistik mini-batch (Bab 5).

Bayes risk

Risiko terkecil yang mungkin dicapai prediktor apa pun (Bab 1).

Bayesian network

Graf berarah yang menyandikan independensi bersyarat antar variabel acak (Bab 10).

bias-variance trade-off

Pertukaran antara galat karena model terlalu sederhana dan galat karena model terlalu peka pada data (Bab 1).

biclustering

Pengelompokan baris dan kolom matriks sekaligus (Bab 4).

binding pocket

Kantong di permukaan protein tempat molekul kecil menempel (Bab 20).

bootstrapping (RL)

Memperbarui perkiraan dengan perkiraan lain, seperti pada TD (Bab 12).

CFAR (constant false alarm rate)

Detektor radar dengan ambang yang menyesuaikan diri dengan noise lokal (Bab 14).

closure problem

Persamaan rata-rata turbulensi mengandung suku yang tidak diketahui (Bab 23).

collaborative filtering

Rekomendasi dari kemiripan selera antar pengguna atau antar item (Bab 30).

collocation points

Titik-titik tempat residual PDE dievaluasi pada PINN (Bab 25).

condition number

Rasio kelengkungan terbesar dan terkecil, penentu kecepatan gradient descent (Bab 2).

constant error carousel

Jalur aditif pada sel LSTM yang membuat gradien tidak hilang (Bab 6).

contrastive learning

Belajar representasi dengan mendekatkan pasangan yang cocok dan menjauhkan yang tidak (Bab 19).

convolution

Operasi linear dengan kernel lokal yang dipakai di semua posisi, ekuivarian terhadap pergeseran (Bab 5 and 9).

covariate shift

Distribusi input saat dipakai berbeda dari saat dilatih (Bab 12 and 26).

credit assignment

Menentukan tindakan mana yang bertanggung jawab atas ganjaran yang datang terlambat (Bab 12).

cross-validation

Memperkirakan generalisasi dengan melatih dan menguji bergantian pada bagian-bagian data (Bab 1).

curse of dimensionality

Kesulitan belajar yang tumbuh eksponensial dengan dimensi (Bab 1).

data assimilation

Menggabungkan model dan pengukuran dengan bobot sesuai ketidakpastian (Bab 10).

DeepONet

Neural operator dengan jaringan branch dan trunk (Bab 26).

diffusion model

Model generatif yang belajar membalik proses penambahan noise (Bab 11).

DMD (dynamic mode decomposition)

Mode dinamika dari operator linear yang dipasang pada potret berurutan (Bab 27).

double descent

Galat uji turun lagi setelah model mampu menghafal data latih (Bab 1).

dropout

Mematikan neuron acak selama pelatihan sebagai regularisasi (Bab 5).

ELBO (evidence lower bound)

Batas bawah log-likelihood yang dimaksimalkan VAE dan model difusi (Bab 8 and 11).

embedding

Representasi vektor padat untuk kata, molekul, pengguna, atau item (Bab 6).

equivariance

Kalau input ditransformasi, keluaran ikut tertransformasi dengan cara yang sama (Bab 9).

exploration–exploitation

Dilema antara mencoba hal baru dan memakai yang sudah terbukti baik (Bab 12).

few-shot / zero-shot learning

Belajar tugas baru dari sedikit atau tanpa contoh berlabel (Bab 19).

filtered backprojection

Algoritme rekonstruksi CT dengan filter ramp sebelum backprojection (Bab 21).

fingerprint (molecular)

Vektor biner penanda substruktur molekul (Bab 19).

Fisher information

Ukuran informasi yang dibawa data tentang parameter, dasar batas Cramér–Rao (Bab 11).

flow matching

Melatih medan kecepatan ODE yang membawa noise ke data (Bab 11).

FNO (Fourier neural operator)

Neural operator yang memakai konvolusi di domain Fourier (Bab 26).

foundation model

Model besar yang dilatih luas lalu disesuaikan ke banyak tugas (Bab 32).

GAN

Generator dan diskriminator yang dipertandingkan (Bab 8).

generalization error

Loss harapan pada distribusi data sebenarnya (Bab 1).

Gibbs sampling

MCMC yang memperbarui satu variabel pada satu waktu (Bab 10).

gradient descent

Melangkah berlawanan arah gradien untuk meminimalkan fungsi (Bab 2).

graph neural network

Jaringan yang bekerja pada graf lewat message passing (Bab 9).

hidden Markov model

Model deret dengan keadaan diskret tersembunyi (Bab 10).

hyperparameter

Pengaturan model yang dipilih di luar pelatihan, misalnya laju belajar (Bab 1).

ICA (independent component analysis)

Memisahkan sumber independen dari campurannya (Bab 4).

imitation learning

Belajar policy dari demonstrasi ahli (Bab 12).

inductive bias

Asumsi yang dibangun ke dalam arsitektur model (Bab 0 and 9).

invariance

Keluaran tidak berubah kalau input ditransformasi (Bab 9).

Kalman filter

Estimator optimal berurutan untuk sistem linear Gauss (Bab 10 and 15).

kernel (CNN)

Matriks bobot kecil yang digeser pada konvolusi (Bab 5).

Koopman operator

Operator linear yang memajukan fungsi pengamatan sistem dinamis (Bab 27).

latent variable

Variabel tersembunyi yang tidak diamati tetapi menjelaskan data (Bab 11).

learning rate

Panjang langkah pada gradient descent (Bab 2).

LMS

Filter adaptif dengan update gradien stokastik (Bab 14).

LSTM

Jaringan rekuren dengan sel ingatan dan gerbang (Bab 6).

MAP (maximum a posteriori)

Estimasi yang memaksimalkan posterior (Bab 1).

maximum likelihood

Estimasi yang memaksimalkan peluang data di bawah model (Bab 1).

MCMC

Menarik sampel dengan rantai Markov yang distribusi stasionernya adalah target (Bab 10 and 24).

MDP (Markov decision process)

Kerangka formal reinforcement learning (Bab 12).

message passing

Simpul mengumpulkan pesan dari tetangga lalu memperbarui keadaannya (Bab 9).

mini-batch

Sebagian kecil data yang dipakai untuk satu langkah SGD (Bab 2).

mode collapse

Model generatif hanya menghasilkan sebagian kecil variasi data (Bab 8).

momentum

Menambahkan ingatan arah sebelumnya pada gradient descent (Bab 2).

neural field

Jaringan yang mewakili medan kontinu sebagai fungsi koordinat (Bab 11).

neural ODE

Persamaan diferensial biasa dengan ruas kanan berupa jaringan saraf (Bab 11).

neural operator

Model yang belajar pemetaan antar ruang fungsi (Bab 26).

normalizing flow

Model generatif dari transformasi bijektif dengan likelihood eksak (Bab 8).

numerical diffusion

Difusi palsu akibat galat diskretisasi (Bab 23).

optical flow

Medan gerak piksel antar bingkai video (Bab 7).

overfitting

Model menghafal data latih dan gagal pada data baru (Bab 1).

oversquashing

Informasi dari banyak simpul jauh terjepit di vektor berukuran tetap (Bab 9).

PCA / POD

Arah-arrah dengan varians atau energi terbesar dari data (Bab 4 and 27).

PINN

Jaringan yang dilatih agar memenuhi persamaan diferensial lewat loss (Bab 25).

policy

Aturan pemilihan aksi agen (Bab 12).

policy gradient

Mengoptimalkan policy langsung dengan gradien return harapan (Bab 12).

positional encoding

Informasi posisi yang ditambahkan ke token transformer (Bab 6).

posterior / prior

Keyakinan setelah dan sebelum melihat data (Bab 1).

PPO

Policy gradient dengan rasio peluang yang dipotong (Bab 12).

precision / recall

Proporsi tebakan positif yang benar, dan proporsi positif sebenarnya yang tertangkap (Bab 1).

QSAR

Model aktivitas biologis dari struktur molekul (Bab 19).

RANS / LES / DNS

Tiga tingkat pemodelan turbulensi, dari rata-rata sampai penyelesaian penuh (Bab 23).

receptive field

Daerah input yang memengaruhi sebuah neuron (Bab 5).

recurrence matrix

Matriks jarak antar bingkai aliran dalam rCFD (Bab 29).

regularization

Pengekangan model agar tidak menghafal (Bab 5).

reparameterization trick

Menulis sampel acak sebagai fungsi diferensiabel parameter (Bab 8).

residual (PDE)

Seberapa jauh sebuah fungsi gagal memenuhi persamaan diferensial (Bab 25).

residual connection

Jalur pintas $y = x + F(x)$ pada ResNet (Bab 8).

RLHF

Menyetel model bahasa dengan ganjaran dari preferensi manusia (Bab 12 and 32).

rollout

Menjalankan model berulang kali dari tebakannya sendiri (Bab 26).

saliency map

Peta kepekaan keluaran terhadap setiap piksel input (Bab 13).

scaling laws

Hubungan hukum pangkat antara ukuran model, data, komputasi, dan loss (Bab 32).

score

Gradien log kerapatan, $\nabla_x \log p(x)$ (Bab 11).

segmentation

Memberi label pada setiap piksel (Bab 7).

self-supervised learning

Membuat target dari data itu sendiri (Bab 1).

SGD

Gradient descent dengan gradien dari mini-batch acak (Bab 2).

Shapley value

Pembagian kontribusi yang adil antar fitur dari teori permainan (Bab 13).

SINDy

Menemukan persamaan dinamika dengan regresi jarang (Bab 27).

spectral bias

Kecenderungan jaringan mempelajari frekuensi rendah lebih dulu (Bab 11).

state space model

Model dengan keadaan internal yang berevolusi linear, dasar Mamba dan teori kendali (Bab 6 and 15).

surrogate model

Model murah yang meniru simulasi mahal (Bab 1 and 26).

TD (temporal difference)

Belajar nilai dari selisih ramalan berurutan (Bab 12).

transfer learning

Memakai model yang dilatih pada satu tugas untuk tugas lain (Bab 8).

transformer

Arsitektur berbasis attention tanpa rekurensi (Bab 6).

U-Net

Encoder–decoder dengan skip connection antar tingkat (Bab 7).

uncanny valley

Rasa ganjil terhadap mesin yang hampir mirip manusia (Bab 31).

VAE

Autoencoder probabilistik yang dilatih dengan ELBO (Bab 8).

value function

Return harapan dari sebuah keadaan atau pasangan keadaan–aksi (Bab 12).

vanishing gradient

Gradien menyusut eksponensial melewati banyak lapisan atau langkah waktu (Bab 6).

weight decay

Regularisasi L2 pada bobot, setara prior Gauss (Bab 1).

Weisfeiler–Leman test

Uji pewarnaan graf yang membatasi kekuatan message passing (Bab 9).

Bibliografi

- Josh Abramson, Jonas Adler, Jack Dunger, Richard Evans, Tim Green, Alexander Pritzel, Olaf Ronneberger, Lindsay Willmore, Andrew J. Ballard, Joshua Bambrick, Sebastian W. Bodenstein, David A. Evans, Chia-Chun Hung, Michael O'Neill, David Reiman, Kathryn Tunyasuvunakool, Zachary Wu, Akvilė Žemgulytė, Eirini Arvaniti, Charles Beattie, Ottavia Bertolli, Alex Bridgland, Alexey Cherepanov, Miles Congreve, Alexander I. Cowen-Rivers, Andrew Cowie, Michael Figurnov, Fabian B. Fuchs, Hannah Gladman, Rishub Jain, Yousuf A. Khan, Caroline M. R. Low, Kuba Perlin, Anna Potapenko, Pascal Savy, Sukhdeep Singh, Adrian Stecula, Ashok Thillaisundaram, Catherine Tong, Sergei Yakneen, Ellen D. Zhong, Michal Zielinski, Augustin Žídek, Victor Bapst, Pushmeet Kohli, Max Jaderberg, Demis Hassabis, and John M. Jumper. Accurate structure prediction of biomolecular interactions with alphafold 3. *Nature*, 630(8016):493–500, 2024. doi: 10.1038/s41586-024-07487-w.
- B. J. Alder and T. E. Wainwright. Studies in molecular dynamics. i. general method. *The Journal of Chemical Physics*, 31(2):459–466, 1959. doi: 10.1063/1.1730376.
- Benedikt Alkin, Andreas Fürst, Simon Schmid, Lukas Gruber, Markus Holzleitner, and Johannes Brandstetter. Universal physics transformers: A framework for efficiently scaling neural operators. arXiv:2402.12365, 2024.
- Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications. arXiv:2006.05205, 2020.
- Brian D.O. Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982. doi: 10.1016/0304-4149(82)90051-5.
- Jose A. Arjona-Medina, Michael Gillhofer, Michael Widrich, Thomas Unterthiner, Johannes Brandstetter, and Sepp Hochreiter. Rudder: Return decomposition for delayed rewards. arXiv:1806.07857, 2018.
- Larry Armijo. Minimization of functions having lipschitz continuous first partial derivatives. *Pacific Journal of Mathematics*, 16(1):1–3, 1966. doi: 10.2140/pjm.1966.16.1.
- Francis Bach. *Learning Theory from First Principles*. MIT Press, 2024.
- Sebastian Bach, Alexander Binder, Grégoire Montavon, Frederick Klauschen, Klaus-Robert Müller, and Wojciech Samek. On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. *PLOS ONE*, 10(7):e0130140, 2015. doi: 10.1371/journal.pone.0130140.
- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. arXiv:1409.0473, 2014.
- Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro,

- Ryan Faulkner, Caglar Gulcehre, Francis Song, Andrew Ballard, Justin Gilmer, George Dahl, Ashish Vaswani, Kelsey Allen, Charles Nash, Victoria Langston, Chris Dyer, Nicolas Heess, Daan Wierstra, Pushmeet Kohli, Matt Botvinick, Oriol Vinyals, Yujia Li, and Razvan Pascanu. Relational inductive biases, deep learning, and graph networks. arXiv:1806.01261, 2018.
- Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. arXiv:1502.05767, 2015. Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *The Journal of Machine Learning Research*, 18(153):1–43, 2018.
- Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xlstm: Extended long short-term memory. arXiv:2405.04517, 2024.
- Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854, 2019. doi: 10.1073/pnas.1903070116.
- Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- Y. Bengio, P. Simard, and P. Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994. doi: 10.1109/72.279181.
- Ruha Benjamin. *Race After Technology: Abolitionist Tools for the New Jim Code*. Polity, 2019.
- G Berkooz, P Holmes, and J L Lumley. The proper orthogonal decomposition in the analysis of turbulent flows. *Annual Review of Fluid Mechanics*, 25(1):539–575, 1993. doi: 10.1146/annurev.fl.25.010193.002543.
- P.J. Besl and Neil D. McKay. A method for registration of 3-d shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14(2):239–256, 1992. doi: 10.1109/34.121791.
- P. Biber and W. Strasser. The normal distributions transform: a new approach to laser scan matching. In *Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003) (Cat. No.03CH37453)*, volume 3, pages 2743–2748, 2004. doi: 10.1109/IROS.2003.1249285.
- Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- Cristian Bodnar, Wessel P. Bruinsma, Ana Lucic, Megan Stanley, Anna Vaughan, Johannes Brandstetter, Patrick Garvan, Maik Riechert, Jonathan A. Weyn, Haiyu Dong, Jayesh K. Gupta, Kit Thambiratnam, Alexander T. Archibald, Chun-Chieh Wu, Elizabeth Heider, Max Welling, Richard E. Turner, and Paris Perdikaris. A foundation model for the earth system. arXiv:2405.13063, 2024.
- Tolga Bolukbasi, Kai-Wei Chang, James Zou, Venkatesh Saligrama, and Adam Kalai. Man is to computer programmer as woman is to homemaker? debiasing word embeddings. arXiv:1607.06520, 2016.
- Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill,

- Erik Brynjolfsson, Shyamal Buch, Dallas Card, Rodrigo Castellon, Niladri Chatterji, Annie Chen, Kathleen Creel, Jared Quincy Davis, Dora Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, Omar Khattab, Pang Wei Koh, Mark Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avaniika Narayan, Deepak Narayanan, Ben Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, Julian Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Rob Reich, Hongyu Ren, Frieda Rong, Yusuf Roohani, Camilo Ruiz, Jack Ryan, Christopher Ré, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishnan Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. On the opportunities and risks of foundation models. arXiv:2108.07258, 2021.
- Léon Bottou, Frank E. Curtis, and Jorge Nocedal. Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311, 2018. doi: 10.1137/16M1080173.
- Johannes Brandstetter, Daniel Worrall, and Max Welling. Message passing neural pde solvers. arXiv:2202.03376, 2022.
- Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.
- Michael M. Bronstein, Joan Bruna, Yann LeCun, Arthur Szlam, and Pierre Vandergheynst. Geometric deep learning: Going beyond euclidean data. *IEEE Signal Processing Magazine*, 34(4):18–42, 2017. doi: 10.1109/MSP.2017.2693418.
- Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. arXiv:2104.13478, 2021.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. arXiv:2005.14165, 2020.
- Joan Bruna, Wojciech Zaremba, Arthur Szlam, and Yann LeCun. Spectral networks and locally connected networks on graphs. arXiv:1312.6203, 2013.
- Steven L. Brunton and J. Nathan Kutz. *Data-Driven Science and Engineering: Machine*

- Learning, Dynamical Systems, and Control*. Cambridge University Press, 2 edition, 2022.
- Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15):3932–3937, 2016. doi: 10.1073/pnas.1517384113.
- Steven L. Brunton, Bernd R. Noack, and Petros Koumoutsakos. Machine learning for fluid mechanics. *Annual Review of Fluid Mechanics*, 52(1):477–508, 2020. doi: 10.1146/annurev-fluid-010719-060214.
- Andreas Bulling, Ulf Blanke, and Bernt Schiele. A tutorial on human activity recognition using body-worn inertial sensors. *ACM Computing Surveys*, 46(3):1–33, 2014. doi: 10.1145/2499621.
- Kathleen Champion, Bethany Lusch, J. Nathan Kutz, and Steven L. Brunton. Data-driven discovery of coordinates and governing equations. *Proceedings of the National Academy of Sciences*, 116(45):22445–22451, 2019. doi: 10.1073/pnas.1906995116.
- Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. arXiv:1806.07366, 2018.
- Paul Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. arXiv:1706.03741, 2017.
- Taco S. Cohen and Max Welling. Group equivariant convolutional networks. arXiv:1602.07576, 2016. Proceedings of the International Conference on Machine Learning (ICML), 2016.
- James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of Computation*, 19(90):297–301, 1965. doi: 10.1090/S0025-5718-1965-0178586-1.
- Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3): 273–297, 1995. doi: 10.1007/BF00994018.
- R. Courant, K. Friedrichs, and H. Lewy. Über die partiellen differenzengleichungen der mathematischen physik. *Mathematische Annalen*, 100(1):32–74, 1928. doi: 10.1007/BF 01448839.
- P. A. Cundall and O. D. L. Strack. A discrete numerical model for granular assemblies. *Géotechnique*, 29(1):47–65, 1979. doi: 10.1680/geot.1979.29.1.47.
- Salvatore Cuomo, Vincenzo Schiano Di Cola, Fabio Giampaolo, Gianluigi Rozza, Maziar Raissi, and Francesco Piccialli. Scientific machine learning through physics-informed neural networks: Where we are and what’s next. *Journal of Scientific Computing*, 92(3), 2022. doi: 10.1007/s10915-022-01939-z.
- G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314, 1989. doi: 10.1007/BF02551274.
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi

- Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv:2501.12948, 2025. Nature volume 645, pages 633-638 (2025).
- Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. arXiv:1606.09375, 2016. Advances in Neural Information Processing Systems 29 (2016).
- A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the em algorithm. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 39(1):1–22, 1977. doi: 10.1111/j.2517-6161.1977.tb01600.x.
- J. Denavit and R. S. Hartenberg. A kinematic notation for lower-pair mechanisms based on matrices. *Journal of Applied Mechanics*, 22(2):215–221, 1955. doi: 10.1115/1.4011045.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805, 2018.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold,

- Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. arXiv:2010.11929, 2020.
- Yaxing Du, Bert Blocken, and Stefan Pirker. A novel approach to simulate pollutant dispersion in the built environment: Transport-based recurrence cfd. *Building and Environment*, 170:106604, 2020. doi: 10.1016/j.buildenv.2019.106604.
- Richard O. Duda, Peter E. Hart, and David G. Stork. *Pattern Classification*. Wiley, 2 edition, 2001.
- Karthik Duraisamy, Gianluca Iaccarino, and Heng Xiao. Turbulence modeling in the age of data. *Annual Review of Fluid Mechanics*, 51(1):357–377, 2019. doi: 10.1146/annurev-fluid-010518-040547.
- Richard Durbin, Sean R. Eddy, Anders Krogh, and Graeme Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press, 1998.
- Nicholas Epley, Adam Waytz, and John T. Cacioppo. On seeing human: A three-factor theory of anthropomorphism. *Psychological Review*, 114(4):864–886, 2007. doi: 10.1037/0033-295X.114.4.864.
- Andre Esteva, Brett Kuprel, Roberto A. Novoa, Justin Ko, Susan M. Swetter, Helen M. Blau, and Sebastian Thrun. Dermatologist-level classification of skin cancer with deep neural networks. *Nature*, 542(7639):115–118, 2017. doi: 10.1038/nature21056.
- European Parliament and Council of the European Union. Regulation (eu) 2016/679 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (general data protection regulation). Official Journal of the European Union, L 119, 2016.
- European Parliament and Council of the European Union. Regulation (eu) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act). Official Journal of the European Union, 12 July 2024, 2024.
- Geir Evensen. Sequential data assimilation with a nonlinear quasi-geostrophic model using monte carlo methods to forecast error statistics. *Journal of Geophysical Research: Oceans*, 99(C5):10143–10162, 1994. doi: 10.1029/94JC00572.
- Kevin Eykholt, Ivan Evtimov, Earlene Fernandes, Bo Li, Amir Rahmati, Chaowei Xiao, Atul Prakash, Tadayoshi Kohno, and Dawn Song. Robust physical-world attacks on deep learning models. arXiv:1707.08945, 2017.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. arXiv:2101.03961, 2021.
- Joel H. Ferziger, Milovan Perić, and Robert L. Street. *Computational Methods for Fluid Dynamics*. Springer, 4 edition, 2020. doi: 10.1007/978-3-319-99693-6.
- Richard E. Fikes and Nils J. Nilsson. Strips: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4):189–208, 1971. doi: 10.1016/0004-3702(71)90010-5.
- Philipp Fischer, Alexey Dosovitskiy, Eddy Ilg, Philip Häusser, Caner Hazırbaş, Vladimir Golkov, Patrick van der Smagt, Daniel Cremers, and Thomas Brox. Flownet: Learning

- optical flow with convolutional networks. arXiv:1504.06852, 2015.
- R. A. FISHER. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936. doi: 10.1111/j.1469-1809.1936.tb02137.x.
- Daan Frenkel and Berend Smit. *Understanding Molecular Simulation: From Algorithms to Applications*. Academic Press, 2 edition, 2002.
- Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. arXiv:1506.02142, 2015.
- Paul Garnier, Jonathan Viquerat, Jean Rabault, Aurélien Larcher, Alexander Kuhnle, and Elie Hachem. A review on deep reinforcement learning for fluid mechanics. *Computers & Fluids*, 225:104973, 2021. doi: 10.1016/j.compfluid.2021.104973.
- A. Gaulton, L. J. Bellis, A. P. Bento, J. Chambers, M. Davies, A. Hersey, Y. Light, S. McGlinchey, D. Michalovich, B. Al-Lazikani, and J. P. Overington. ChEMBL: a large-scale bioactivity database for drug discovery. *Nucleic Acids Research*, 40(D1): D1100–D1107, 2012. doi: 10.1093/nar/gkr777.
- Stuart Geman, Elie Bienenstock, and René Doursat. Neural networks and the bias/variance dilemma. *Neural Computation*, 4(1):1–58, 1992. doi: 10.1162/neco.1992.4.1.1.
- Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins. Learning to forget: Continual prediction with lstm. *Neural Computation*, 12(10):2451–2471, 2000. doi: 10.1162/089976600300015015.
- Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. arXiv:1704.01212, 2017.
- R. A. Gingold and J. J. Monaghan. Smoothed particle hydrodynamics: theory and application to non-spherical stars. *Monthly Notices of the Royal Astronomical Society*, 181(3):375–389, 1977. doi: 10.1093/mnras/181.3.375.
- Ross Girshick. Fast r-cnn. arXiv:1504.08083, 2015.
- Fred Glover. Tabu search—part i. *ORSA Journal on Computing*, 1(3):190–206, 1989. doi: 10.1287/ijoc.1.3.190.
- Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. arXiv:1406.2661, 2014.
- Amos Gropp, Lior Yariv, Niv Haim, Matan Atzmon, and Yaron Lipman. Implicit geometric regularization for learning shapes. arXiv:2002.10099, 2020.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. arXiv:2312.00752, 2023.
- Jayesh K. Gupta and Johannes Brandstetter. Towards multi-spatiotemporal-scale generalized pde modeling. arXiv:2209.15616, 2022.
- Rafael Gómez-Bombarelli, Jennifer N. Wei, David Duvenaud, José Miguel Hernández-Lobato, Benjamín Sánchez-Lengeling, Dennis Sheberla, Jorge Aguilera-Iparraguirre,

- Timothy D. Hirzel, Ryan P. Adams, and Alán Aspuru-Guzik. Automatic chemical design using a data-driven continuous representation of molecules. *ACS Central Science*, 4(2):268–276, 2018. doi: 10.1021/acscentsci.7b00572.
- David Ha and Jürgen Schmidhuber. World models. arXiv:1803.10122, 2018.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. arXiv:1801.01290, 2018.
- Peter Hart, Nils Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi: 10.1109/TSSC.1968.300136.
- Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2 edition, 2004.
- Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2 edition, 2009.
- W. K. Hastings. Monte carlo sampling methods using markov chains and their applications. *Biometrika*, 57(1):97–109, 1970. doi: 10.1093/biomet/57.1.97.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. arXiv:1502.01852, 2015.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90.
- Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. arXiv:1708.05031, 2017.
- Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. Lightgcn: Simplifying and powering graph convolution network for recommendation. arXiv:2002.02126, 2020.
- High-Level Expert Group on Artificial Intelligence. A definition of AI: Main capabilities and disciplines. European Commission, 2019.
- Frederick S. Hillier and Gerald J. Lieberman. *Introduction to Operations Research*. McGraw-Hill, 10 edition, 2015.
- Geoffrey E. Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. arXiv:1207.0580, 2012.
- C.W Hirt and B.D Nichols. Volume of fluid (vof) method for the dynamics of free boundaries. *Journal of Computational Physics*, 39(1):201–225, 1981. doi: 10.1016/0021-991(81)90145-5.
- Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. arXiv:1606.03476, 2016.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models.

- arXiv:2006.11239, 2020.
- Sepp Hochreiter. Untersuchungen zu dynamischen neuronalen netzen. Diploma thesis, Institut für Informatik, Technische Universität München, 1991.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997a. doi: 10.1162/neco.1997.9.8.1735.
- Sepp Hochreiter and Jürgen Schmidhuber. Flat minima. *Neural Computation*, 9(1):1–42, 1997b. doi: 10.1162/neco.1997.9.1.1.
- Sepp Hochreiter, Ulrich Bodenhofer, Martin Heusel, Andreas Mayr, Andreas Mitterecker, Adetayo Kasim, Tatsiana Khamiakova, Suzy Van Sanden, Dan Lin, Willem Talloen, Luc Bijnens, Hinrich W. H. Göhlmann, Ziv Shkedy, and Djork-Arné Clevert. Fabia: factor analysis for bicluster acquisition. *Bioinformatics*, 26(12):1520–1527, 2010. doi: 10.1093/bioinformatics/btq227.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. arXiv:2203.15556, 2022.
- Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeyer, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, 2025. doi: 10.1038/s41586-024-08328-6.
- Berthold K.P. Horn and Brian G. Schunck. Determining optical flow. *Artificial Intelligence*, 17(1–3):185–203, 1981. doi: 10.1016/0004-3702(81)90024-2.
- Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989. doi: 10.1016/0893-6080(89)90020-8.
- H. Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441, 1933. doi: 10.1037/h0071325.
- G. N. Hounsfield. Computerized transverse axial scanning (tomography): Part 1. description of system. *The British Journal of Radiology*, 46(552):1016–1022, 1973. doi: 10.1259/0007-1285-46-552-1016.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. arXiv:2106.09685, 2021.
- A. Hyvärinen and E. Oja. Independent component analysis: algorithms and applications. *Neural Networks*, 13(4–5):411–430, 2000. doi: 10.1016/S0893-6080(00)00026-5.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. arXiv:1502.03167, 2015.
- Fabian Isensee, Paul F. Jaeger, Simon A. A. Kohl, Jens Petersen, and Klaus H. Maier-Hein. nnu-net: a self-configuring method for deep learning-based biomedical image

- segmentation. *Nature Methods*, 18(2):203–211, 2021. doi: 10.1038/s41592-020-01008-z.
- Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. arXiv:1806.07572, 2018. In *Advances in neural information processing systems* (pp. 8571–8580) 2018.
- John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with alphafold. *Nature*, 596(7873):583–589, 2021. doi: 10.1038/s41586-021-03819-2.
- Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of ir techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002. doi: 10.1145/582415.582418.
- Avinash C. Kak and Malcolm Slaney. *Principles of Computerized Tomographic Imaging*. IEEE Press, 1988.
- R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960. doi: 10.1115/1.3662552.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. arXiv:2001.08361, 2020.
- George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021. doi: 10.1038/s42254-021-00314-5.
- Steven M. Kay. *Fundamentals of Statistical Signal Processing: Estimation Theory*. Prentice Hall, 1993.
- Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. arXiv:1609.04836, 2016.
- Been Kim, Martin Wattenberg, Justin Gilmer, Carrie Cai, James Wexler, Fernanda Viegas, and Rory Sayres. Interpretability beyond feature attribution: Quantitative testing with concept activation vectors (tcav). arXiv:1711.11279, 2017. ICML 2018.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. arXiv:1412.6980, 2014.
- Diederik P Kingma and Max Welling. Auto-encoding variational bayes. arXiv:1312.6114, 2013.
- Diederik P. Kingma and Max Welling. An introduction to variational autoencoders. arXiv:1906.02691, 2019. *Foundations and Trends in Machine Learning*: Vol. 12 (2019): No. 4, pp 307–392.

- Diederik P. Kingma, Tim Salimans, Ben Poole, and Jonathan Ho. Variational diffusion models. *arXiv:2107.00630*, 2021.
- Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv:1609.02907*, 2016.
- Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. *arXiv:2304.02643*, 2023.
- S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi: 10.1126/science.220.4598.671.
- Günter Klambauer, Thomas Unterthiner, Andreas Mayr, and Sepp Hochreiter. Self-normalizing neural networks. *arXiv:1706.02515*, 2017. *Advances in Neural Information Processing Systems 30 (NIPS 2017)*.
- Donald E. Knuth and Ronald W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293–326, 1975. doi: 10.1016/0004-3702(75)90019-3.
- Dmitrii Kochkov, Jamie A. Smith, Ayya Alieva, Qing Wang, Michael P. Brenner, and Stephan Hoyer. Machine learning–accelerated computational fluid dynamics. *Proceedings of the National Academy of Sciences*, 118(21), 2021. doi: 10.1073/pnas.2101784118.
- Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. *Lecture Notes in Computer Science*, pages 282–293, 2006. doi: 10.1007/11871842_29.
- Georg Kohl, Li-Wei Chen, and Nils Thuerey. Benchmarking autoregressive conditional diffusion models for turbulent flow simulation. *arXiv:2309.01745*, 2023.
- Teuvo Kohonen. Self-organized formation of topologically correct feature maps. *Biological Cybernetics*, 43(1):59–69, 1982. doi: 10.1007/BF00337288.
- Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009.
- B. O. Koopman. Hamiltonian systems and transformation in hilbert space. *Proceedings of the National Academy of Sciences*, 17(5):315–318, 1931. doi: 10.1073/pnas.17.5.315.
- Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009. doi: 10.1109/MC.2009.263.
- Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces. *arXiv:2108.08481*, 2021. *The Journal of Machine Learning Research* (2023), Volume 24, Issue 1, Article No 89, pp 4061-4157.
- Aditi S. Krishnapriyan, Amir Gholami, Shandian Zhe, Robert M. Kirby, and Michael W. Mahoney. Characterizing possible failure modes in physics-informed neural networks. *arXiv:2109.01050*, 2021. *NeurIPS 2021*.
- Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6):84–90, 2017. doi: 10.1145/3065386.
- H. W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics*

- Quarterly*, 2(1–2):83–97, 1955. doi: 10.1002/nav.3800020109.
- I.E. Lagaris, A. Likas, and D.I. Fotiadis. Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks*, 9(5):987–1000, 1998. doi: 10.1109/72.712178.
- Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. arXiv:1612.01474, 2016.
- Remi Lam, Alvaro Sanchez-Gonzalez, Matthew Willson, Peter Wirsberger, Meire Fortunato, Ferran Alet, Suman Ravuri, Timo Ewalds, Zach Eaton-Rosen, Weihua Hu, Alexander Merose, Stephan Hoyer, George Holland, Oriol Vinyals, Jacklynn Stott, Alexander Pritzel, Shakir Mohamed, and Peter Battaglia. Learning skillful medium-range global weather forecasting. *Science*, 382(6677):1416–1421, 2023. doi: 10.1126/science.adi2336.
- Alex H. Lang, Sourabh Vora, Holger Caesar, Lubing Zhou, Jiong Yang, and Oscar Beijbom. Pointpillars: Fast encoders for object detection from point clouds. arXiv:1812.05784, 2018.
- B.E. Launder and D.B. Spalding. The numerical computation of turbulent flows. *Computer Methods in Applied Mechanics and Engineering*, 3(2):269–289, 1974. doi: 10.1016/0045-7825(74)90029-2.
- Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi: 10.1038/nature14539.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. arXiv:2005.11401, 2020.
- Heng Li and Richard Durbin. Fast and accurate short read alignment with burrows–wheeler transform. *Bioinformatics*, 25(14):1754–1760, 2009. doi: 10.1093/bioinformatics/btp324.
- Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. arXiv:2010.08895, 2020.
- T. Lichtenegger and S. Pirker. Recurrence cfd – a novel approach to simulate multiphase flows with strongly separated time scales. *Chemical Engineering Science*, 153:394–410, 2016. doi: 10.1016/j.ces.2016.07.036.
- Julia Ling, Andrew Kurzawski, and Jeremy Templeton. Reynolds averaged turbulence modelling using deep neural networks with embedded invariance. *Journal of Fluid Mechanics*, 807:155–166, 2016. doi: 10.1017/jfm.2016.615.
- Mario Lino, Stathi Fotiadis, Anil A. Bharath, and Chris D. Cantwell. Current and emerging deep-learning methods for the simulation of fluid dynamics. *Proceedings of*

- the Royal Society A: Mathematical, Physical and Engineering Sciences*, 479(2275), 2023. doi: 10.1098/rspa.2023.0058.
- Christopher A. Lipinski, Franco Lombardo, Beryl W. Dominy, and Paul J. Feeney. Experimental and computational approaches to estimate solubility and permeability in drug discovery and development settings. *Advanced Drug Delivery Reviews*, 23(1–3): 3–25, 1997. doi: 10.1016/S0169-409X(96)00423-1.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. arXiv:2210.02747, 2022.
- Phillip Lippe, Bastiaan S. Veeling, Paris Perdikaris, Richard E. Turner, and Johannes Brandstetter. Pde-refiner: Achieving accurate long rollouts with neural pde solvers. arXiv:2308.05732, 2023.
- Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. arXiv:2209.03003, 2022.
- S. Lloyd. Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982. doi: 10.1109/TIT.1982.1056489.
- Duri Long and Brian Magerko. What is ai literacy? competencies and design considerations. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, pages 1–16, 2020. doi: 10.1145/3313831.3376727.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. arXiv:1711.05101, 2017.
- Michael I Love, Wolfgang Huber, and Simon Anders. Moderated estimation of fold change and dispersion for rna-seq data with deseq2. *Genome Biology*, 15(12), 2014. doi: 10.1186/s13059-014-0550-8.
- David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004. doi: 10.1023/B:VISI.0000029664.99615.94.
- Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021. doi: 10.1038/s42256-021-00302-5.
- Scott Lundberg and Su-In Lee. A unified approach to interpreting model predictions. arXiv:1705.07874, 2017.
- Calvin Luo. Understanding diffusion models: A unified perspective. arXiv:2208.11970, 2022.
- Bethany Lusch, J. Nathan Kutz, and Steven L. Brunton. Deep learning for universal linear embeddings of nonlinear dynamics. *Nature Communications*, 9(1), 2018. doi: 10.1038/s41467-018-07210-0.
- Kevin M. Lynch and Frank C. Park. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.
- Stéphane Mallat. Group invariant scattering. *Communications on Pure and Applied*

- Mathematics*, 65(10):1331–1398, 2012. doi: 10.1002/cpa.21413.
- Dimitris G. Manolakis, Vinay K. Ingle, and Stephen M. Kogon. *Statistical and Adaptive Signal Processing*. Artech House, 2005.
- Andreas Mayr, Günter Klambauer, Thomas Unterthiner, and Sepp Hochreiter. Deeptox: Toxicity prediction using deep learning. *Frontiers in Environmental Science*, 3, 2016. doi: 10.3389/fenvs.2015.00080.
- Michael McCabe, Bruno Régaldo-Saint Blancard, Liam Holden Parker, Ruben Ohana, Miles Cranmer, Alberto Bietti, Michael Eickenberg, Siavash Golkar, Geraud Krawezik, Francois Lanusse, Mariel Pettee, Tiberiu Tesileanu, Kyunghyun Cho, and Shirley Ho. Multiple physics pretraining for physical surrogate models. arXiv:2310.02994, 2023.
- Levi D. McClenny and Ulisses M. Braga-Neto. Self-adaptive physics-informed neural networks. *Journal of Computational Physics*, 474:111722, 2023. doi: 10.1016/j.jcp.2022.111722.
- Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133, 1943. doi: 10.1007/BF02478259.
- Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. arXiv:1802.03426, 2018.
- F. R. Menter. Two-equation eddy-viscosity turbulence models for engineering applications. *AIAA Journal*, 32(8):1598–1605, 1994. doi: 10.2514/3.12149.
- Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953. doi: 10.1063/1.1699114.
- Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. arXiv:1301.3781, 2013.
- Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. arXiv:2003.08934, 2020.
- C. Wright Mills. *The Sociological Imagination*. Oxford University Press, 1959.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.
- Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. arXiv:1602.01783, 2016. ICML 2016.
- Christoph Molnar. *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. Independently published, 2 edition, 2022.

- J. J. Monaghan. Smoothed particle hydrodynamics. *Annual Review of Astronomy and Astrophysics*, 30(1):543–574, 1992. doi: 10.1146/annurev.aa.30.090192.002551.
- Masahiro Mori, Karl MacDorman, and Norri Kageki. The uncanny valley [from the field]. *IEEE Robotics & Automation Magazine*, 19(2):98–100, 2012. doi: 10.1109/MR.A.2012.2192811.
- Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and leman go neural: Higher-order graph neural networks. arXiv:1810.02244, 2018.
- Tamara Munzner. *Visualization Analysis and Design*. CRC Press, 2014.
- Clifford Nass and Youngme Moon. Machines and mindlessness: Social responses to computers. *Journal of Social Issues*, 56(1):81–103, 2000. doi: 10.1111/0022-4537.00153.
- Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2 edition, 2006.
- Ruben Ohana, Michael McCabe, Lucas Meyer, Rudy Morel, Fruzsina J. Agocs, Miguel Beneitez, Marsha Berger, Blakesley Burkhart, Keaton Burns, Stuart B. Dalziel, Drummond B. Fielding, Daniel Fortunato, Jared A. Goldberg, Keiya Hirashima, Yan-Fei Jiang, Rich R. Kerswell, Suryanarayana Maddu, Jonah Miller, Payel Mukhopadhyay, Stefan S. Nixon, Jeff Shen, Romain Watteaux, Bruno Régaldo-Saint Blancard, François Rozet, Liam H. Parker, Miles Cranmer, and Shirley Ho. The well: a large-scale collection of diverse physics simulations for machine learning. arXiv:2412.00568, 2024.
- OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic,

- Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeef Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O’Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. Gpt-4 technical report. arXiv:2303.08774, 2023.
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. arXiv:2203.02155, 2022.
- Jeong Joon Park, Peter Florence, Julian Straub, Richard Newcombe, and Steven Lovegrove. Deepsdf: Learning continuous signed distance functions for shape representation. arXiv:1901.05103, 2019.
- Suhas V. Patankar. *Numerical Heat Transfer and Fluid Flow*. Hemisphere, 1980.
- S.V Patankar and D.B Spalding. A calculation procedure for heat, mass and momentum transfer in three-dimensional parabolic flows. *International Journal of Heat and Mass Transfer*, 15(10):1787–1806, 1972. doi: 10.1016/0017-9310(72)90054-3.

- Jaideep Pathak, Shashank Subramanian, Peter Harrington, Sanjeev Raja, Ashesh Chattopadhyay, Morteza Mardani, Thorsten Kurth, David Hall, Zongyi Li, Kamyar Azizzadenesheli, Pedram Hassanzadeh, Karthik Kashinath, and Animashree Anandkumar. Fourcastnet: A global data-driven high-resolution weather model using adaptive fourier neural operators. *arXiv:2202.11214*, 2022.
- Karl Pearson. Liii. on lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11): 559–572, 1901. doi: 10.1080/14786440109462720.
- William Peebles and Saining Xie. Scalable diffusion models with transformers. *arXiv:2212.09748*, 2022.
- Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter W. Battaglia. Learning mesh-based simulation with graph networks. *arXiv:2010.03409*, 2020. International Conference on Learning Representations (ICLR), 2021.
- S. Pirker and T. Lichtenegger. Efficient time-extrapolation of single- and multiphase simulations by transport based recurrence cfd (rcfd). *Chemical Engineering Science*, 188: 65–83, 2018. doi: 10.1016/j.ces.2018.04.059.
- Aske Plaat. *Deep Reinforcement Learning*. Springer, 2022.
- B.T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964. doi: 10.1016/0041-5553(64)90137-5.
- Stephen B. Pope. *Turbulent Flows*. Cambridge University Press, 2000.
- Ryan Poplin, Pi-Chuan Chang, David Alexander, Scott Schwartz, Thomas Colthurst, Alexander Ku, Dan Newburger, Jojo Dijamco, Nam Nguyen, Pegah T Afshar, Sam S Gross, Lizzie Dorfman, Cory Y McLean, and Mark A DePristo. A universal snp and small-indel variant caller using deep neural networks. *Nature Biotechnology*, 36(10): 983–987, 2018. doi: 10.1038/nbt.4235.
- William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, 3 edition, 2007.
- Kristina Preuer, Philipp Renz, Thomas Unterthiner, Sepp Hochreiter, and Günter Klambauer. Fréchet chemnet distance: A metric for generative models for molecules in drug discovery. *Journal of Chemical Information and Modeling*, 58(9):1736–1741, 2018. doi: 10.1021/acs.jcim.8b00234.
- Ilan Price, Alvaro Sanchez-Gonzalez, Ferran Alet, Tom R. Andersson, Andrew El-Kadi, Dominic Masters, Timo Ewalds, Jacklynn Stott, Shakir Mohamed, Peter Battaglia, Remi Lam, and Matthew Willson. Probabilistic weather forecasting with machine learning. *Nature*, 637(8044):84–90, 2025. doi: 10.1038/s41586-024-08252-9.
- Simon J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023.
- Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation. *arXiv:1612.00593*, 2016.
- Jean Rabault, Miroslav Kuchta, Atle Jensen, Ulysse Réglade, and Nicolas Cerardi.

- Artificial neural networks trained through deep reinforcement learning discover control strategies for active flow control. *Journal of Fluid Mechanics*, 865:281–302, 2019. doi: 10.1017/jfm.2019.62.
- L.R. Rabiner. A tutorial on hidden markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989. doi: 10.1109/5.18626.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. arXiv:2103.00020, 2021.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. arXiv:2305.18290, 2023.
- Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred A. Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. arXiv:1806.08734, 2018. ICML 2019.
- M. Raissi, P. Perdikaris, and G.E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.
- Maziar Raissi, Alireza Yazdani, and George Em Karniadakis. Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations. *Science*, 367(6481): 1026–1030, 2020. doi: 10.1126/science.aaw4741.
- Hubert Ramsauer, Bernhard Schöfl, Johannes Lehner, Philipp Seidl, Michael Widrich, Thomas Adler, Lukas Gruber, Markus Holzleitner, Milena Pavlović, Geir Kjetil Sandve, Victor Greiff, David Kreil, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. Hopfield networks is all you need. arXiv:2008.02217, 2020.
- Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.
- Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. arXiv:1506.02640, 2015.
- Byron Reeves and Clifford Nass. *The Media Equation: How People Treat Computers, Television, and New Media Like Real People and Places*. Cambridge University Press, 1996.
- Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. arXiv:1506.01497, 2015.
- Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. Bpr: Bayesian personalized ranking from implicit feedback. arXiv:1205.2618, 2012.
- Danilo Jimenez Rezende and Shakir Mohamed. Variational inference with normalizing flows. arXiv:1505.05770, 2015.
- Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should i trust you?": Explaining the predictions of any classifier. arXiv:1602.04938, 2016.

- Mark A. Richards. *Fundamentals of Radar Signal Processing*. McGraw-Hill, 2 edition, 2014.
- Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951. doi: 10.1214/aoms/1177729586.
- David Rogers and Mathew Hahn. Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50(5):742–754, 2010. doi: 10.1021/ci100050t.
- Hermann Rohling. Radar cfar thresholding in clutter and multiple target situations. *IEEE Transactions on Aerospace and Electronic Systems*, AES-19(4):608–621, 1983. doi: 10.1109/TAES.1983.309350.
- Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. arXiv:2112.10752, 2021.
- Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. arXiv:1505.04597, 2015.
- F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958. doi: 10.1037/h0042519.
- H. H. Rosenbrock. An automatic method for finding the greatest or least value of a function. *The Computer Journal*, 3(3):175–184, 1960. doi: 10.1093/comjnl/3.3.175.
- Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. arXiv:1011.0686, 2010.
- Sam T. Roweis and Lawrence K. Saul. Nonlinear dimensionality reduction by locally linear embedding. *Science*, 290(5500):2323–2326, 2000. doi: 10.1126/science.290.5500.2323.
- Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, 2019. doi: 10.1038/s42256-019-0048-x.
- Samuel H. Rudy, Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Data-driven discovery of partial differential equations. *Science Advances*, 3(4), 2017. doi: 10.1126/sciadv.1602614.
- David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.
- Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2021.
- Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter W. Battaglia. Learning to simulate complex physics with graph networks. arXiv:2002.09405, 2020.
- Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th international conference on World Wide Web*, pages 285–295, 2001. doi: 10.1145/371920.372071.
- Victor Garcia Satorras, Emiel Hoogetboom, and Max Welling. E(n) equivariant graph

- neural networks. arXiv:2102.09844, 2021.
- PETER J. SCHMID. Dynamic mode decomposition of numerical and experimental data. *Journal of Fluid Mechanics*, 656:5–28, 2010. doi: 10.1017/S0022112010001217.
- Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015. doi: 10.1016/j.neunet.2014.09.003.
- John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. Trust region policy optimization. arXiv:1502.05477, 2015a.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. arXiv:1506.02438, 2015b.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.
- Bernhard Schölkopf, Alexander Smola, and Klaus-Robert Müller. Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation*, 10(5):1299–1319, 1998. doi: 10.1162/089976698300017467.
- John R. Searle. Minds, brains, and programs. *Behavioral and Brain Sciences*, 3(3):417–424, 1980. doi: 10.1017/S0140525X00005756.
- Marwin H. S. Segler, Thierry Kogej, Christian Tyrchan, and Mark P. Waller. Generating focused molecule libraries for drug discovery with recurrent neural networks. *ACS Central Science*, 4(1):120–131, 2018a. doi: 10.1021/acscentsci.7b00512.
- Marwin H. S. Segler, Mike Preuss, and Mark P. Waller. Planning chemical syntheses with deep neural networks and symbolic ai. *Nature*, 555(7698):604–610, 2018b. doi: 10.1038/nature25978.
- Philipp Seidl, Philipp Renz, Natalia Dyubankova, Paulo Neves, Jonas Verhoeven, Jörg K. Wegner, Marwin Segler, Sepp Hochreiter, and Günter Klambauer. Improving few- and zero-shot reaction template prediction using modern hopfield networks. *Journal of Chemical Information and Modeling*, 62(9):2111–2120, 2022. doi: 10.1021/acs.jcim.1c01065.
- Philipp Seidl, Andreu Vall, Sepp Hochreiter, and Günter Klambauer. Enhancing activity prediction models in drug discovery with the ability to understand human language. arXiv:2303.03363, 2023.
- Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. arXiv:1610.02391, 2016.
- David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. doi: 10.1038/nature16961.
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew

- Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharmashan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362 (6419):1140–1144, 2018. doi: 10.1126/science.aar6404.
- Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. arXiv:1312.6034, 2013.
- Vincent Sitzmann, Julien N. P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. arXiv:2006.09661, 2020.
- J. SMAGORINSKY. General circulation experiments with the primitive equations. *Monthly Weather Review*, 91(3):99–164, 1963. doi: 10.1175/1520-0493(1963)091<0099:GCEWTP>2.3.CO;2.
- Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. arXiv:1503.03585, 2015.
- Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. arXiv:2010.02502, 2020a.
- Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. arXiv:2011.13456, 2020b.
- Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. arXiv:1703.01365, 2017.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- Richard Szeliski. *Computer Vision: Algorithms and Applications*. Springer, 2 edition, 2022.
- Kunihiko Taira, Steven L. Brunton, Scott T. M. Dawson, Clarence W. Rowley, Tim Colonius, Beverley J. McKeon, Oliver T. Schmidt, Stanislav Gordeyev, Vassilios Theofilis, and Lawrence S. Ukeiley. Modal analysis of fluid flows: An overview. *AIAA Journal*, 55 (12):4013–4041, 2017. doi: 10.2514/1.J056060.
- Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. arXiv:2006.10739, 2020.
- Zachary Teed and Jia Deng. Raft: Recurrent all-pairs field transforms for optical flow. arXiv:2003.12039, 2020.
- Joshua B. Tenenbaum, Vin de Silva, and John C. Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319–2323, 2000. doi: 10.1126/science.290.5500.2319.
- Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics*. MIT Press, 2005.

- N. Thuerey, B. Holzsuh, P. Holl, G. Kohl, M. Lino, Q. Liu, P. Schnell, and F. Trost. Physics-based deep learning. arXiv:2109.05237, 2021.
- Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 58(1):267–288, 1996. doi: 10.1111/j.2517-6161.1996.tb02080.x.
- Michael E. Tipping and Christopher M. Bishop. Probabilistic principal component analysis. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 61(3): 611–622, 1999. doi: 10.1111/1467-9868.00196.
- Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, and Michael M. Bronstein. Understanding over-squashing and bottlenecks on graphs via curvature. arXiv:2111.14522, 2021.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. arXiv:2302.13971, 2023.
- Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024. doi: 10.1038/s41586-023-06747-5.
- A. M. TURING. I.—computing machinery and intelligence. *Mind*, LIX(236):433–460, 1950. doi: 10.1093/mind/LIX.236.433.
- Matthew Turk and Alex Pentland. Eigenfaces for recognition. *Journal of Cognitive Neuroscience*, 3(1):71–86, 1991. doi: 10.1162/jocn.1991.3.1.71.
- Kiwon Um, Robert Brand, Yun, Fei, Philipp Holl, and Nils Thuerey. Solver-in-the-loop: Learning from differentiable physics to interact with iterative pde-solvers. arXiv:2007.00016, 2020.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. arXiv:1706.03762, 2017.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. arXiv:1710.10903, 2017.
- Loup Verlet. Computer "experiments" on classical fluids. i. thermodynamical properties of lennard-jones molecules. *Physical Review*, 159(1):98–103, 1967. doi: 10.1103/PhysRev.159.98.
- Henk Kaarle Versteeg and Weeratunge Malalasekera. *An Introduction to Computational Fluid Dynamics: The Finite Volume Method*. Pearson Education, 2 edition, 2007.
- Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7):1661–1674, 2011. doi: 10.1162/NECO_a_00142.
- Ricardo Vinuesa and Steven L. Brunton. Enhancing computational fluid dynamics with machine learning. *Nature Computational Science*, 2(6):358–366, 2022. doi: 10.1038/s43588-022-00264-7.

- Sifan Wang, Yujun Teng, and Paris Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081, 2021. doi: 10.1137/20M1318043.
- Sifan Wang, Xinling Yu, and Paris Perdikaris. When and why pinns fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 449:110768, 2022. doi: 10.1016/j.jcp.2021.110768.
- Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4): 279–292, 1992. doi: 10.1007/BF00992698.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. arXiv:2201.11903, 2022.
- David Weininger. Smiles, a chemical language and information system. 1. introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36, 1988. doi: 10.1021/ci00057a005.
- Mark Weiser. The computer for the 21st century. *Scientific American*, 265(3):94–104, 1991. doi: 10.1038/scientificamerican0991-94.
- Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992. doi: 10.1007/BF00992696.
- David H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computation*, 8(7):1341–1390, 1996. doi: 10.1162/neco.1996.8.7.1341.
- Yiheng Xie, Towaki Takikawa, Shunsuke Saito, Or Litany, Shiqin Yan, Numair Khan, Federico Tombari, James Tompkin, Vincent Sitzmann, and Srinath Sridhar. Neural fields in visual computing and beyond. arXiv:2111.11426, 2021.
- Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? arXiv:1810.00826, 2018.
- Kevin Yang, Kyle Swanson, Wengong Jin, Connor Coley, Philipp Eiden, Hua Gao, Angel Guzman-Perez, Timothy Hopper, Brian Kelley, Miriam Mathea, Andrew Palmer, Volker Settels, Tommi Jaakkola, Klavs Jensen, and Regina Barzilay. Analyzing learned molecular representations for property prediction. *Journal of Chemical Information and Modeling*, 59(8):3370–3388, 2019. doi: 10.1021/acs.jcim.9b00237.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. arXiv:2210.03629, 2022.
- Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander Smola. Deep sets. arXiv:1703.06114, 2017.
- Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. arXiv:1611.03530, 2016.
- Shoshana Zuboff. *The Age of Surveillance Capitalism: The Fight for a Human Future at the New Frontier of Power*. PublicAffairs, 2019.